

深入浅出
计算机网络

微
视频
版

高 军 陈 君 唐 秀 明 张 剑 / 编 著

清华大学出版社

B站口碑爆棚的“计算机网络微课堂”升级版

深入浅出 计算机网络

高 军 陈 君 唐 秀 明 张 剑 / 编 著



附赠：100+个视频 | 教学大纲 | PPT课件

思维导图 | 配套习题

清华大学出版社

第5章 运输层

5.1 运输层概述

5.2 UDP和TCP的对比

5.3 传输控制协议

5.1 运输层概述

01

进程间基于网络的通信

02

TCP/IP运输层中的两个重要协议

03

运输层端口号

04

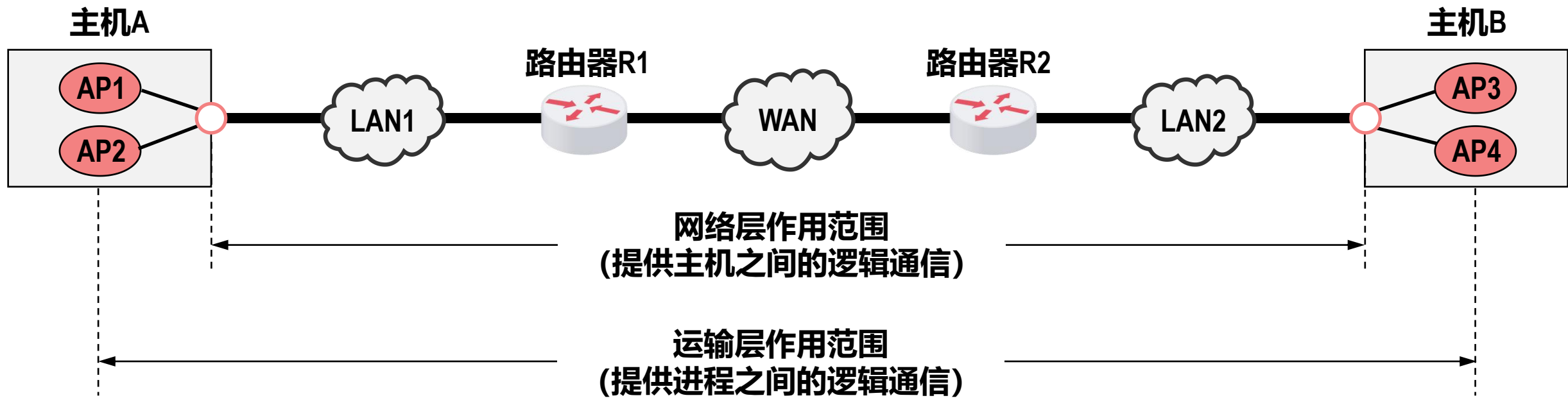
发送方的复用和接收方的分用

05

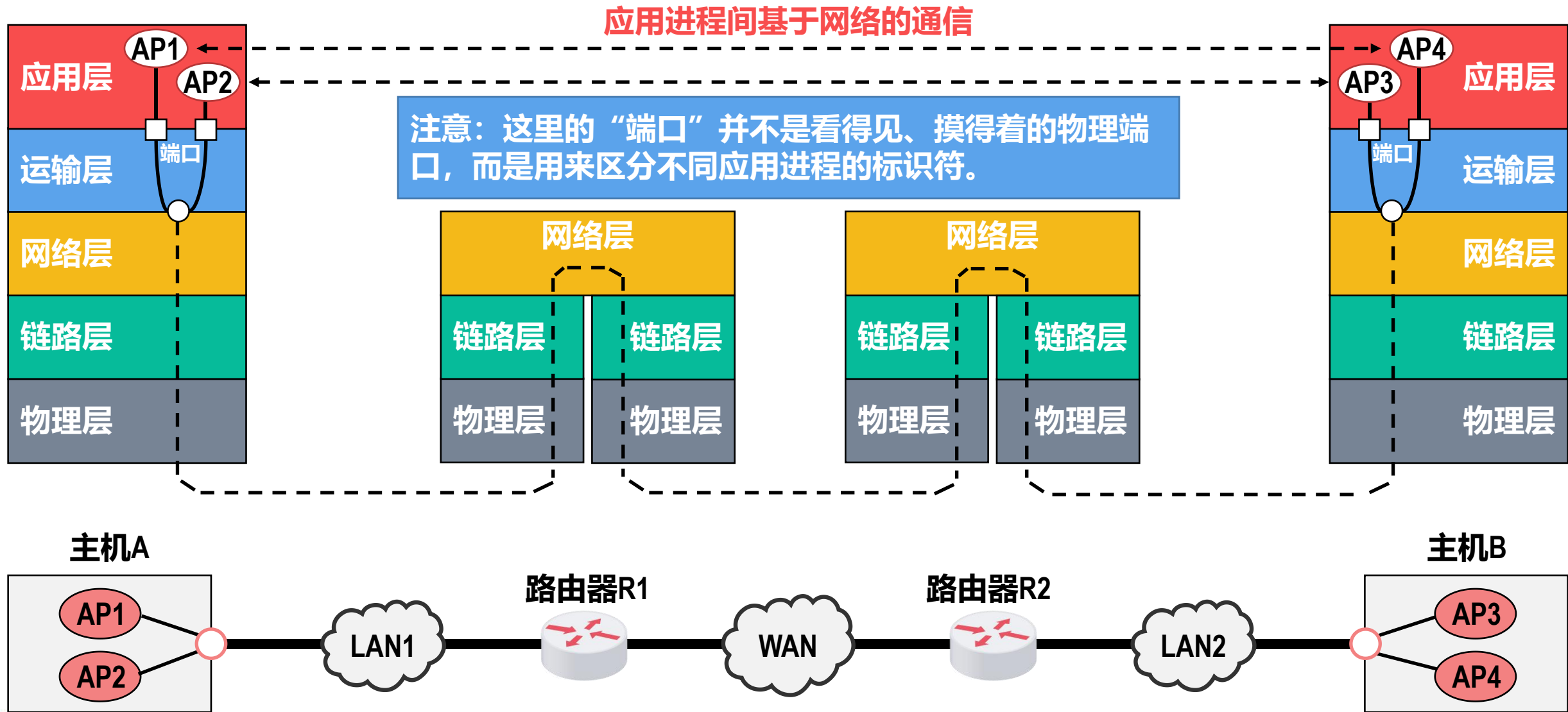
运输层端口号应用举例

01 进程间基于网络的通信

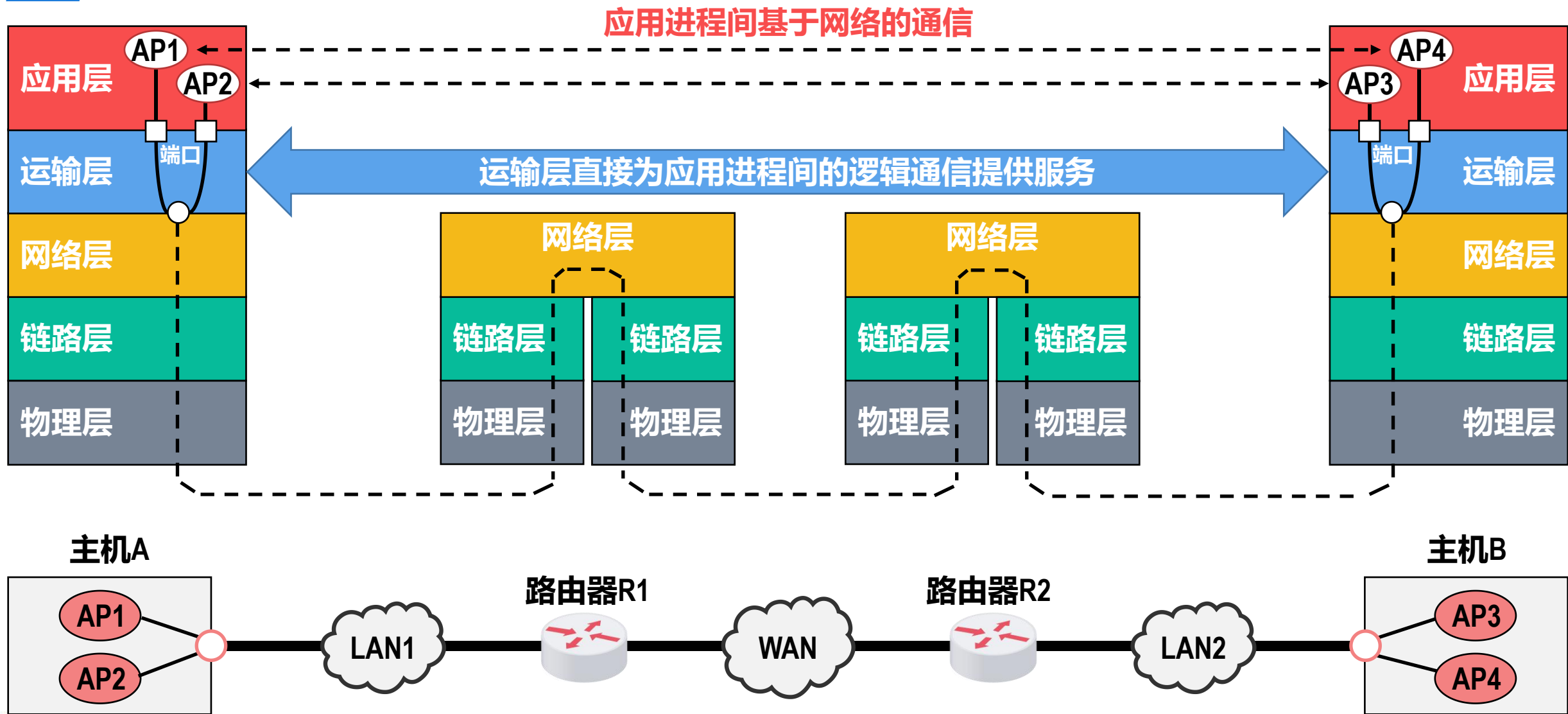
- 第2~4章依次介绍了计算机网络体系结构中的物理层、数据链路层和网络层，它们共同解决了将主机通过异构网络互联起来所面临的问题，实现了主机到主机的通信。
- 然而在计算机网络中实际进行通信的真正实体，是位于通信两端主机中的进程。
- 如何为运行在不同主机上的应用进程提供直接的逻辑通信服务，就是运输层的主要任务。运输层协议又称为端到端协议。



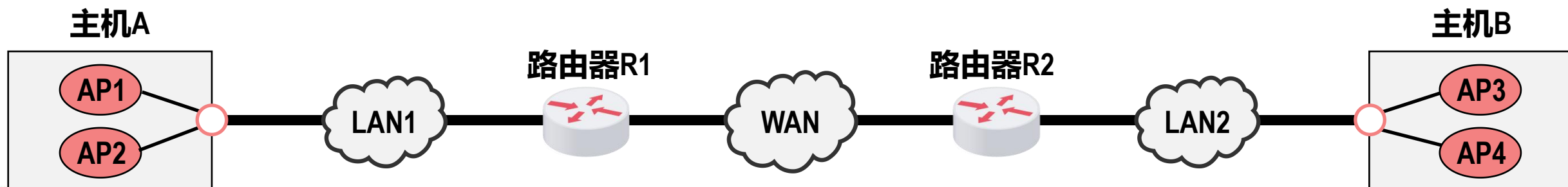
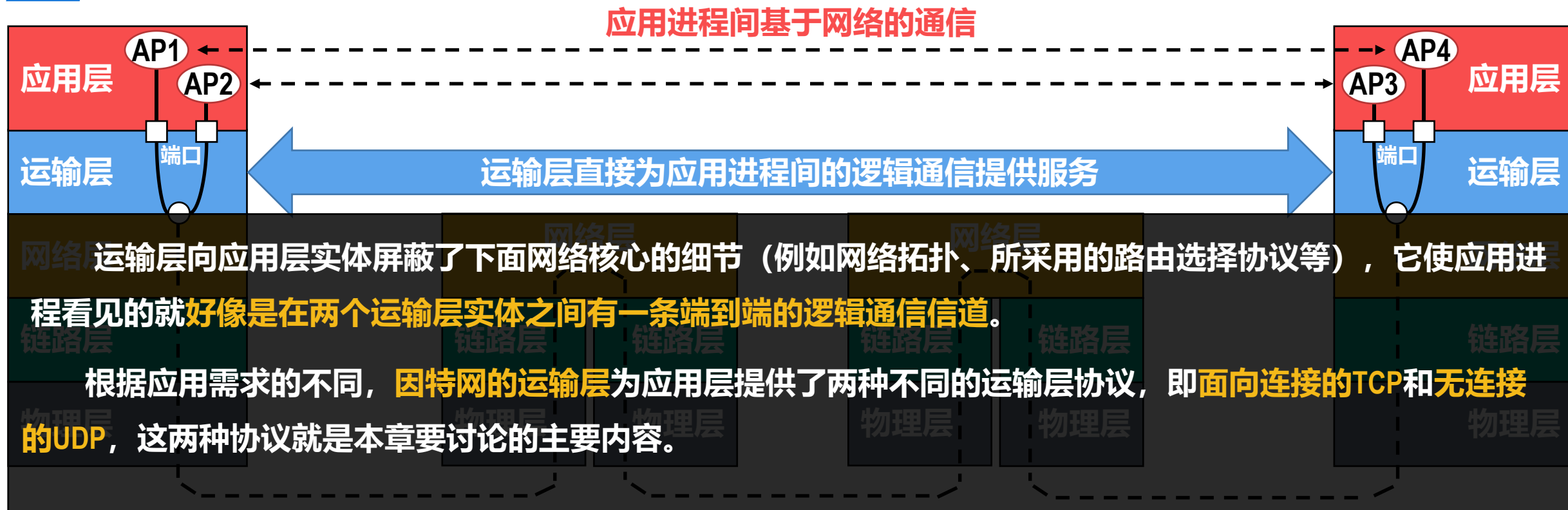
01 进程间基于网络的通信



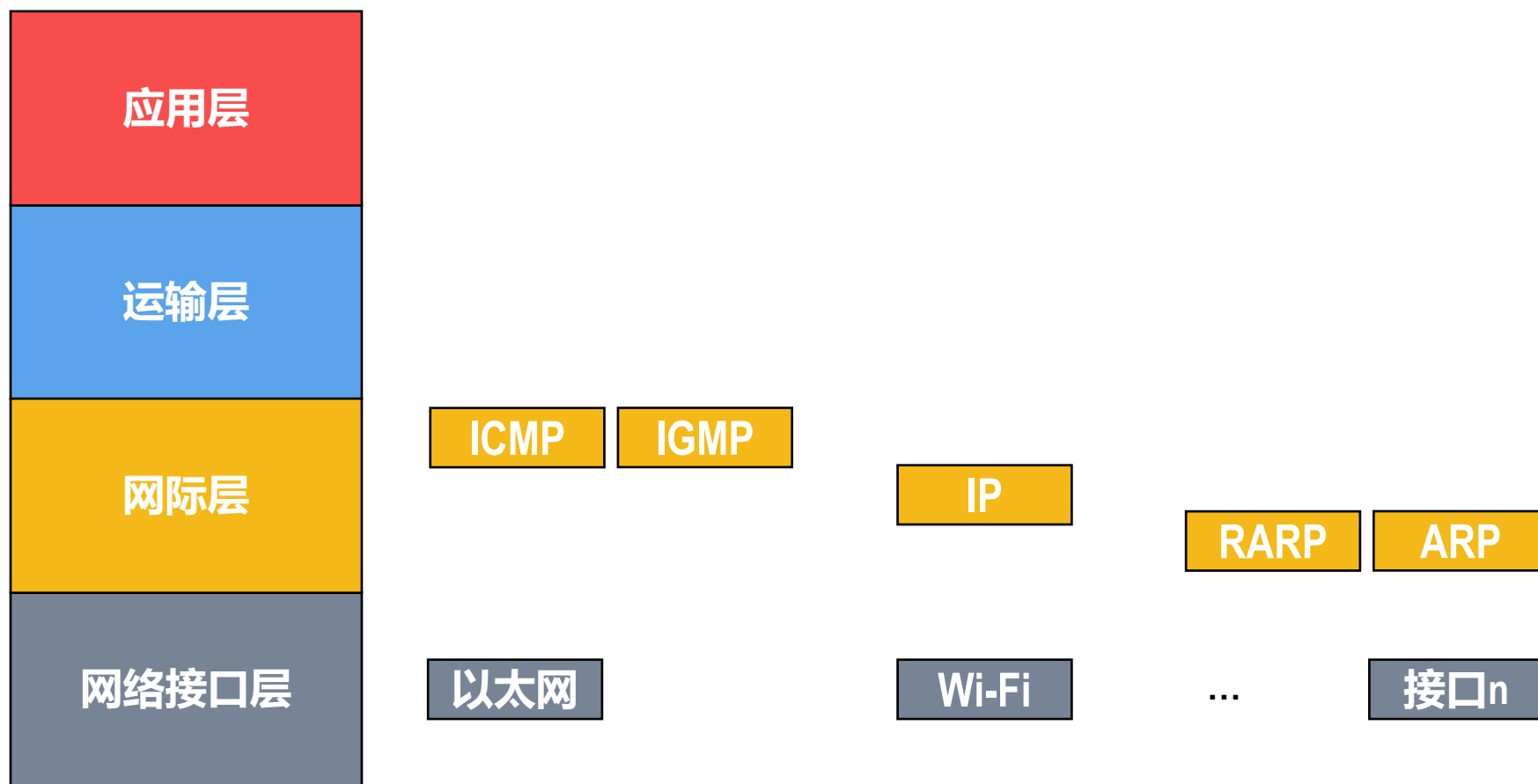
01 进程间基于网络的通信



01 进程间基于网络的通信

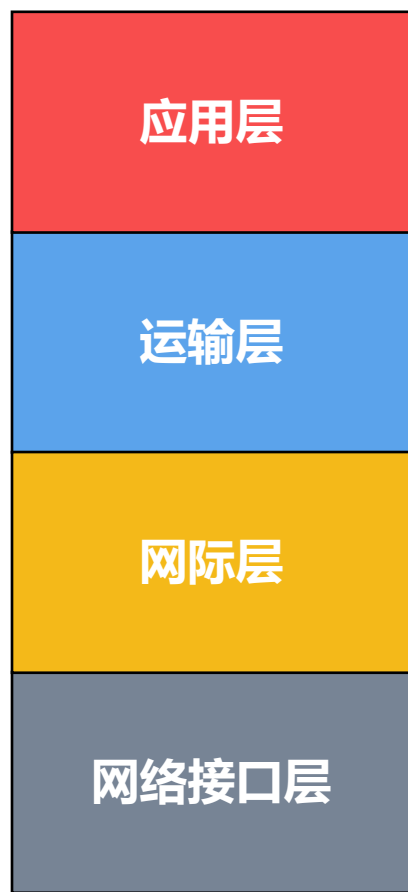


02 TCP/IP运输层中的两个重要协议

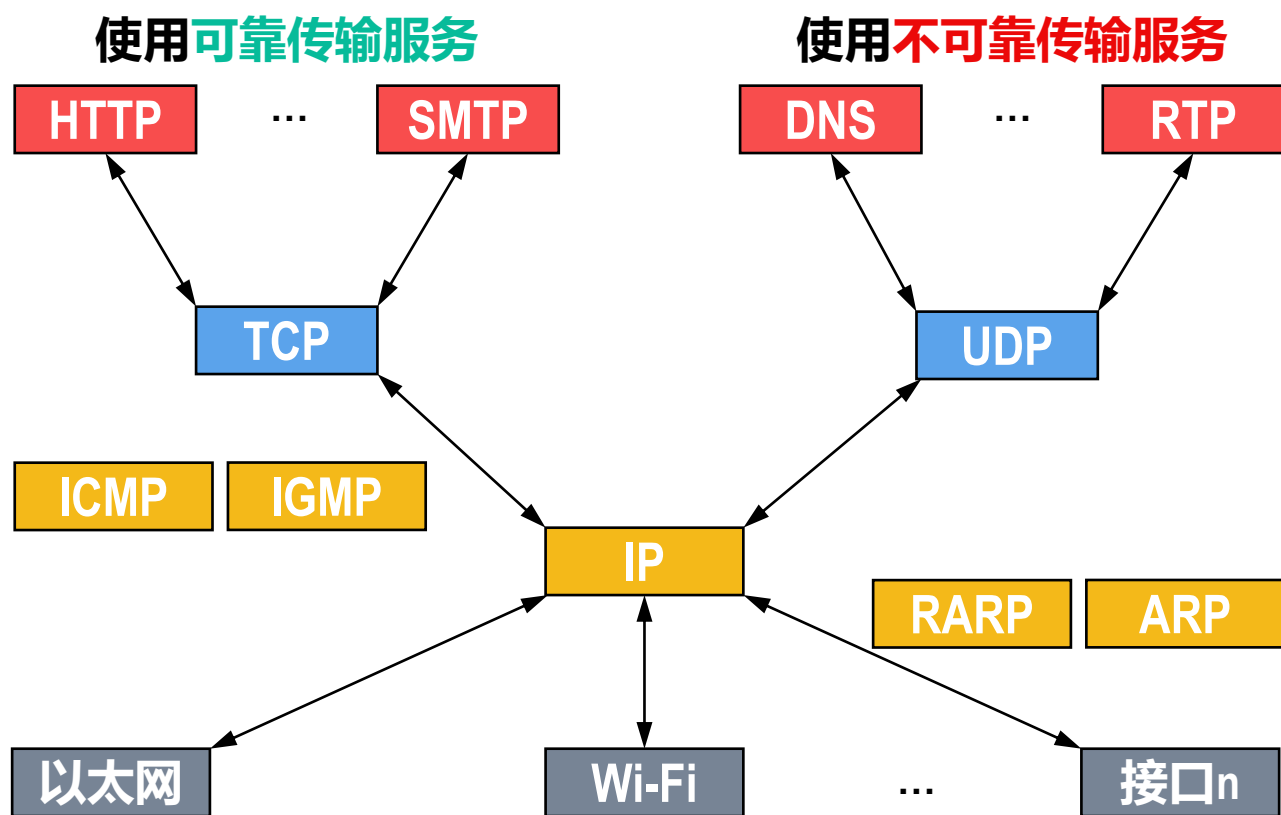


TCP/IP体系结构

02 TCP/IP运输层中的两个重要协议



TCP/IP体系结构



使用**端口号**实现进程间通信

IP向其上层提供**无连接、不可靠**的数据报传输服务

02 TCP/IP运输层中的两个重要协议

TCP

- **传输控制协议**（Transmission Control Protocol, TCP）为其上层提供的是**面向连接的可靠**的数据传输服务。
- 使用TCP通信的双方，在传送数据之前**必须首先建立TCP连接**（逻辑连接，而非物理连接）。数据传输结束后**必须要释放TCP连接**。
- TCP为了实现可靠传输，就必须使用很多措施，例如**TCP连接管理、确认机制、超时重传、流量控制以及拥塞控制**等。
- TCP的**实现复杂**，TCP报文段的**首部比较大，占用处理机资源比较多**。

02 TCP/IP运输层中的两个重要协议

TCP

- **传输控制协议**（Transmission Control Protocol, TCP）为其上层提供的是**面向连接的可靠**的数据传输服务。
- 使用TCP通信的双方，在传送数据之前**必须首先建立TCP连接**（逻辑连接，而非物理连接）。数据传输结束后**必须要释放TCP连接**。
- TCP为了实现可靠传输，就必须使用很多措施，例如**TCP连接管理、确认机制、超时重传、流量控制以及拥塞控制**等。
- TCP的**实现复杂**，TCP报文段的**首部比较大**，**占用处理机资源比较多**。

UDP

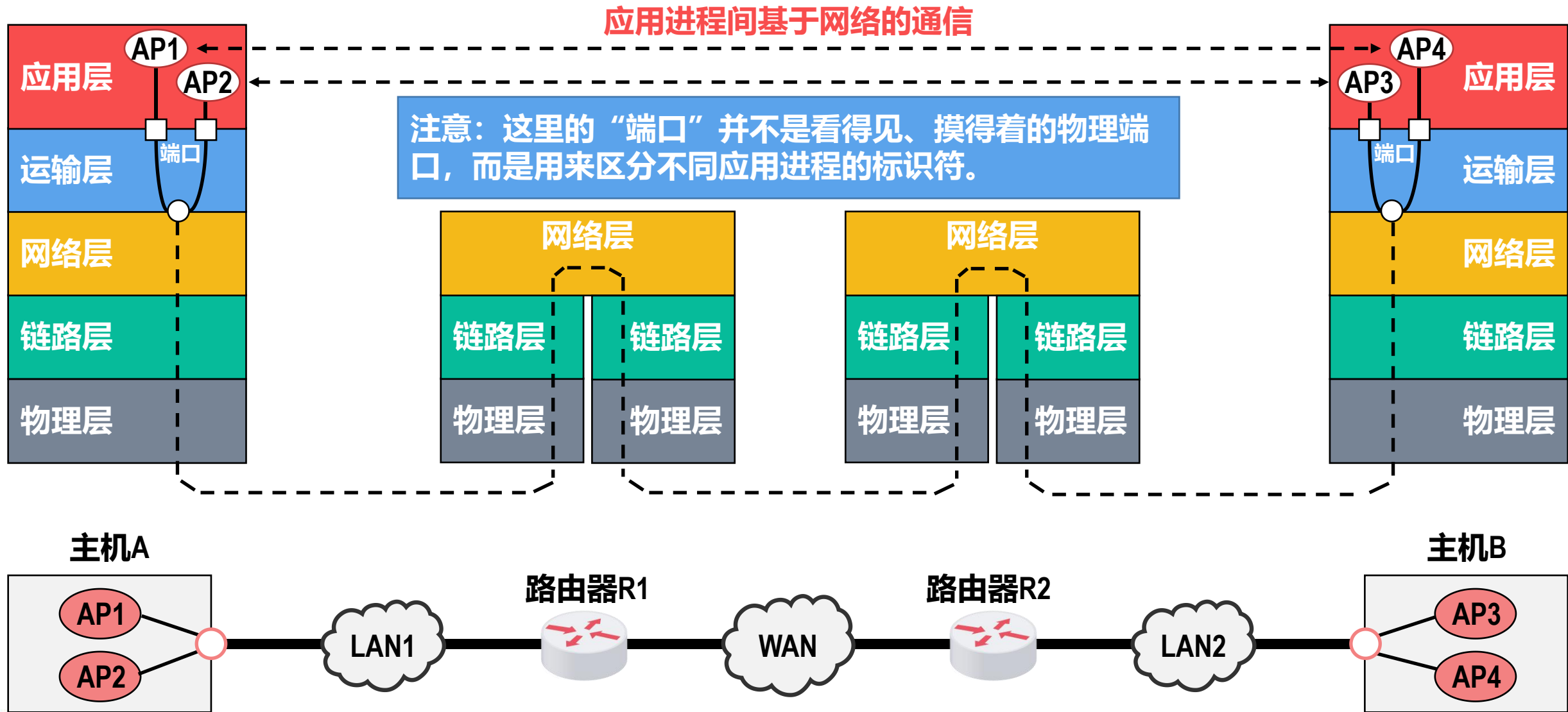
- **用户数据报协议**（User Datagram Protocol, UDP）为其上层提供的是**无连接的不可靠**的数据传输服务。
- 使用UDP通信的双方，在传送数据之前**不需要建立连接**。
- UDP不需要实现可靠传输，因此**不需要使用实现可靠传输的各种机制**。
- UDP的**实现简单**，UDP用户数据报的**首部比较小**。

02 TCP/IP运输层中的两个重要协议

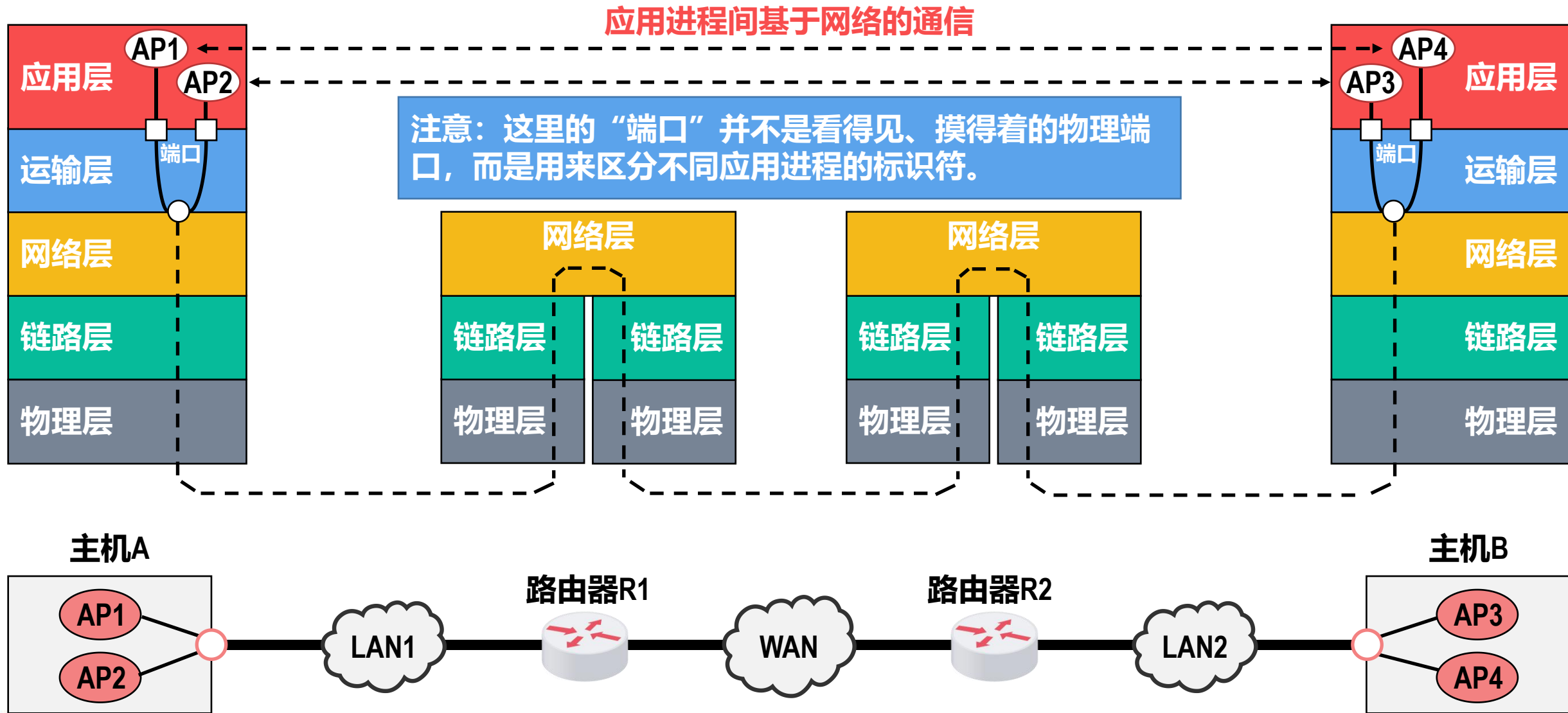
因特网中的一些典型应用所使用的TCP/IP应用层协议和相应的运输层协议

因特网应用	TCP/IP应用层协议	TCP/IP运输层协议
域名解析	域名系统DNS	UDP
文件传送	简单文件传送协议TFTP	UDP
路由选择	路由信息协议RIP	UDP
网络参数配置	动态主机配置协议DHCP	UDP
网络管理	简单网络管理协议SNMP	UDP
远程文件服务器	网络文件系统NFS	UDP
IP电话	专用协议	UDP
流媒体通信	专用协议	UDP
IP多播	网际组管理协议IGMP	UDP
电子邮件	简单邮件传送协议SMTP	TCP
远程终端接入	电传机网络TELNET	TCP
万维网	超文本传送协议HTTP	TCP
文件传送	文件传送协议FTP	TCP

03 运输层端口号



03 运输层端口号



03 运输层端口号

■ 运行在计算机上的进程是使用**进程标识符**（Process Identification, **PID**）来标识的。

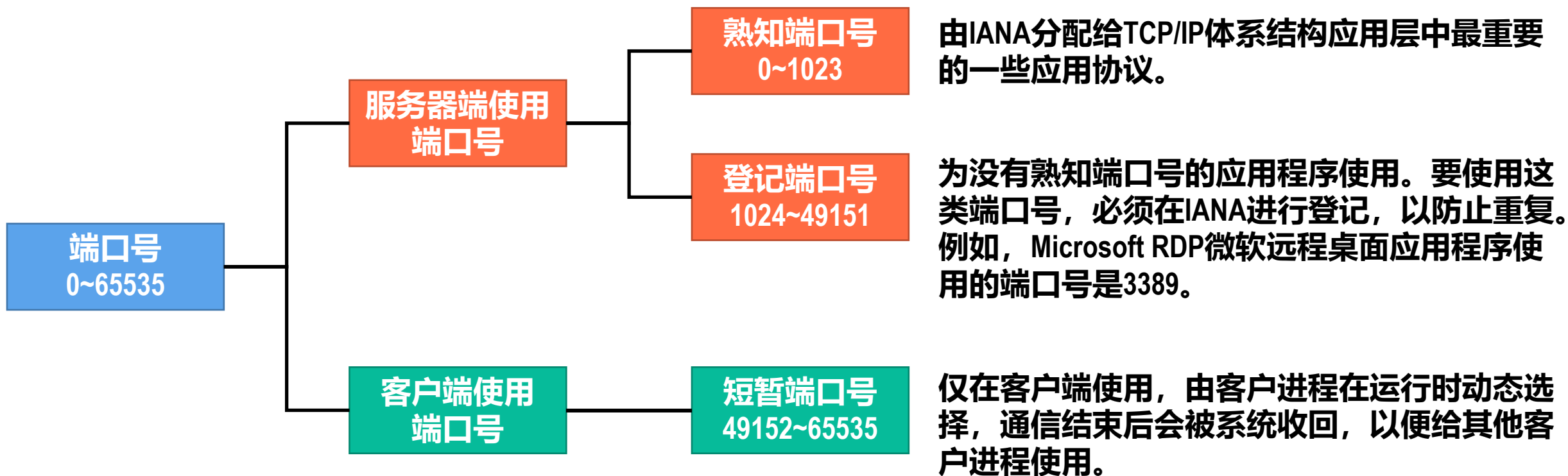
☐ 然而，因特网上的计算机并不是使用统一的操作系统，而**不同操作系统**（Windows、Linux、MacOS）又**使用不同格式的进程标识符**。

☐ 为了使运行不同操作系统的计算机的应用进程之间能够基于网络进行通信，就必须**使用统一的方法对TCP/IP体系的应用进程进行标识**。

■ TCP/IP体系结构的运输层使用**端口号**来标识和区分应用层的不同应用进程。端口号的**长度为16比特**，**取值范围是0~65535**。

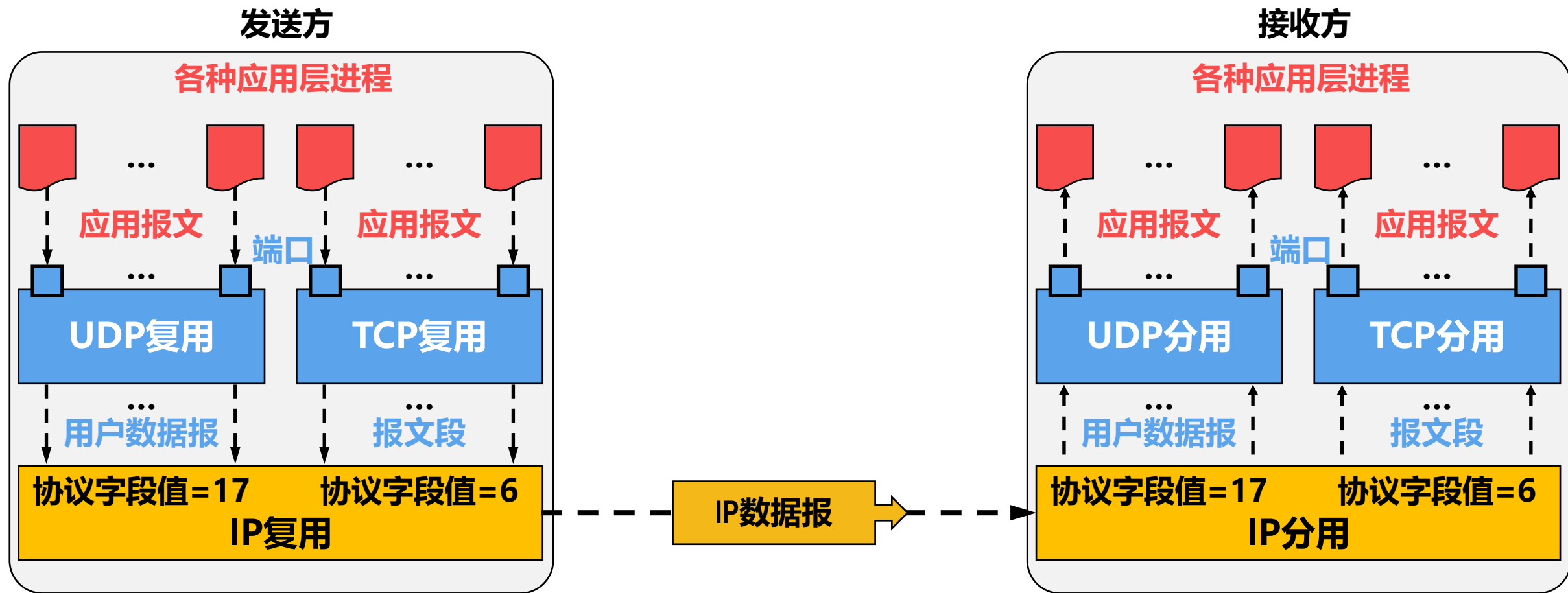
03 运输层端口号

FTP	SMTP	DNS	DHCP	HTTP	BGP	HTTPS	RIP
21/20	25	53	67/68	80	179	443	520

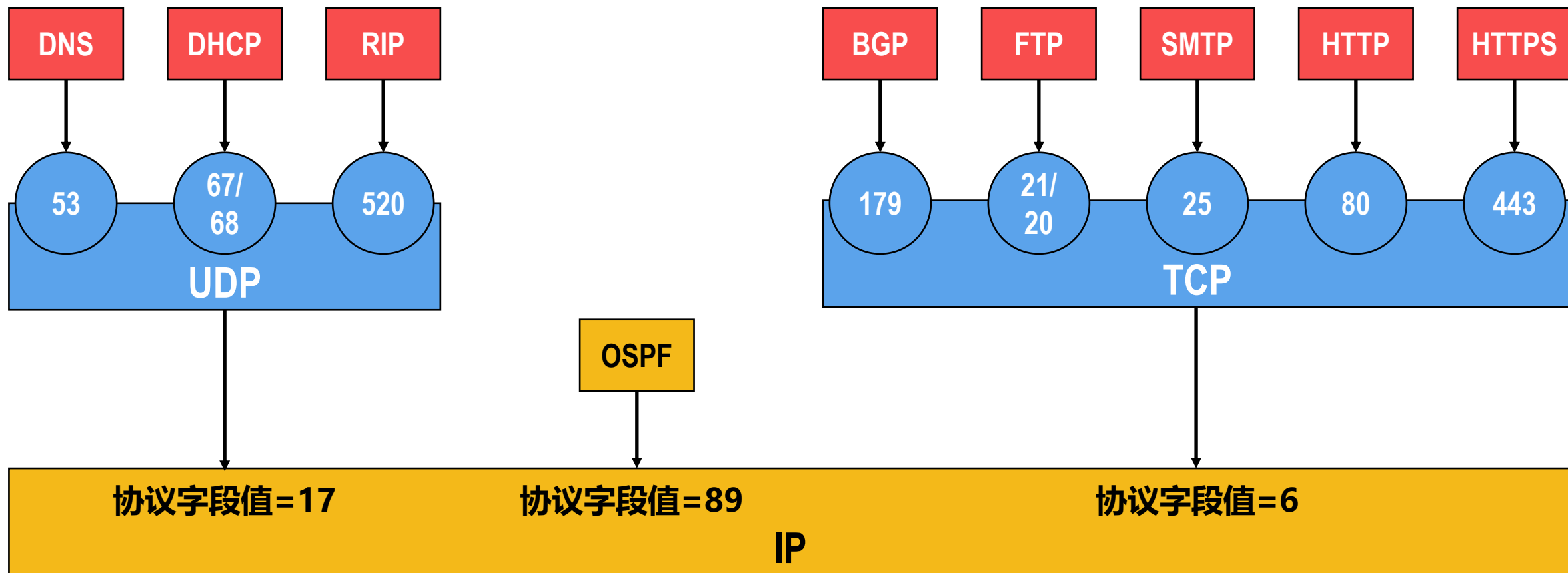


端口号只具有本地意义，即端口号只是为了标识本计算机网络协议栈应用层中的各应用进程。在因特网中，不同计算机中的相同端口号是没有关系的，即相互独立。另外，TCP和UDP端口号之间也是没有关系的。

04 发送方的复用和接收方的分用

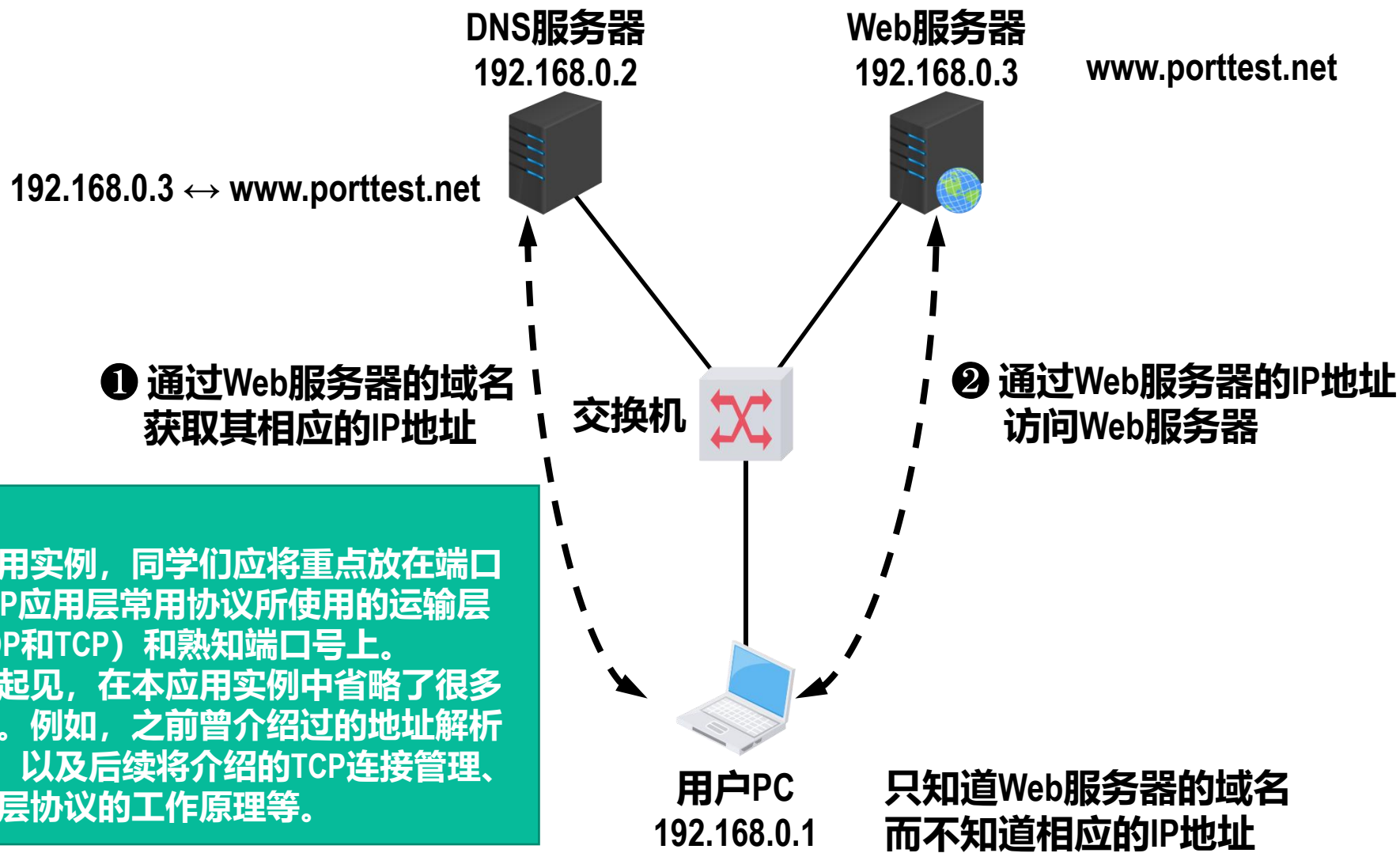


04 发送方的复用和接收方的分用



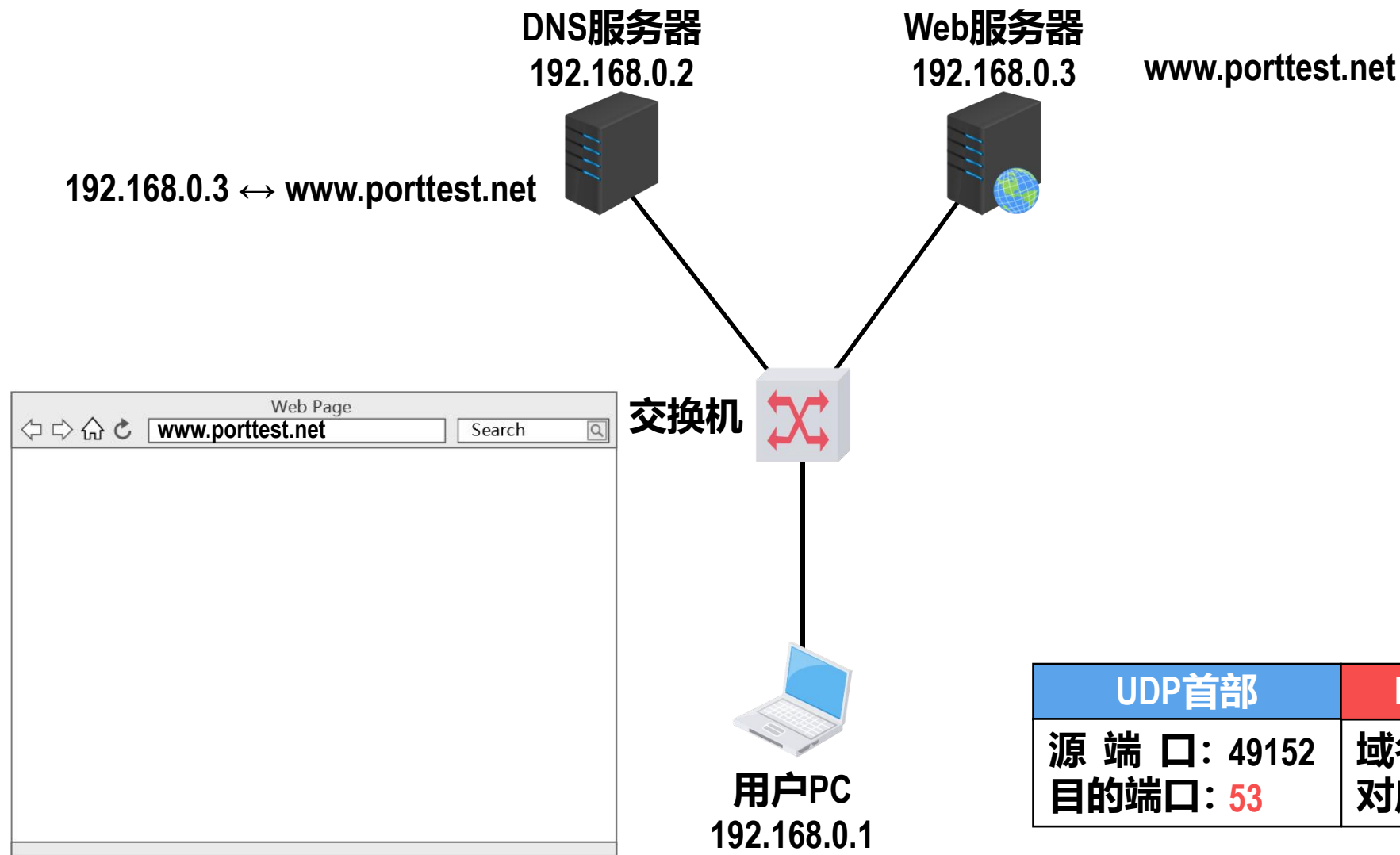
U
5

运输层端口号应用举例

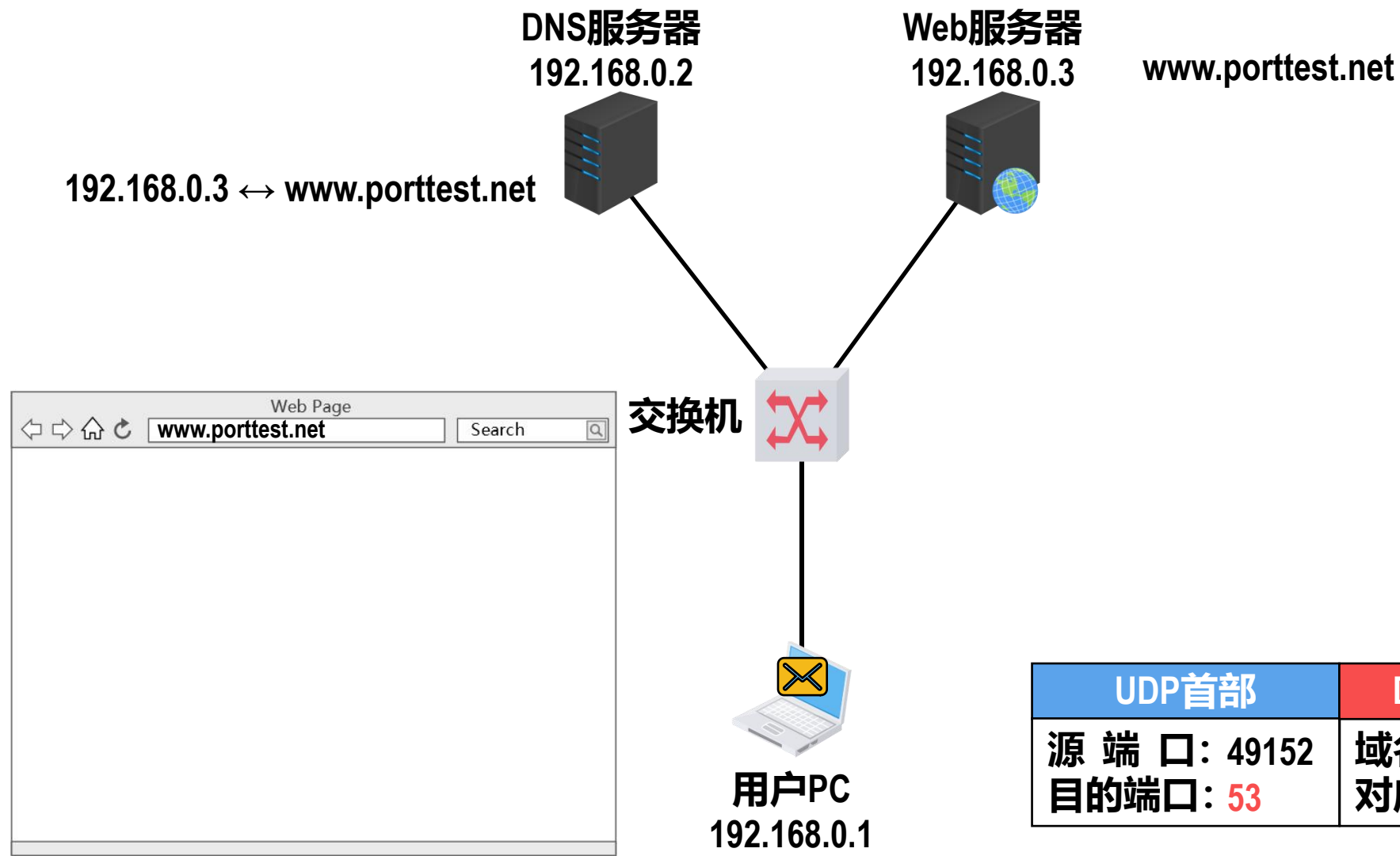


U
5

运输层端口号应用举例



5 运输层端口号应用举例



5 运输层端口号应用举例

UDP首部

DNS查询请求报文

源端口: 49152

目的端口: 53

域名www.porttest.net

对应的IP地址是什么?

192.168.0.3 ↔ www.porttest.net

DNS服务器

192.168.0.2



Web服务器

192.168.0.3



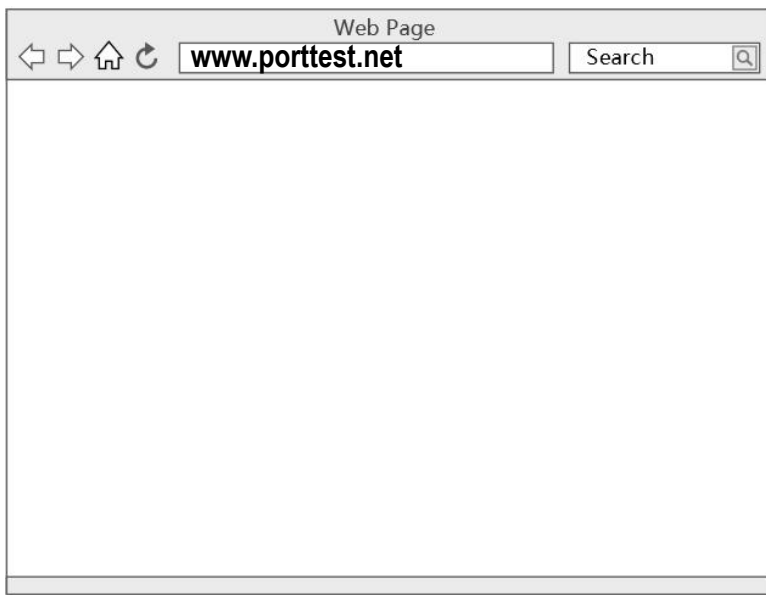
www.porttest.net

交换机



用户PC

192.168.0.1



5 运输层端口号应用举例

UDP首部

DNS查询请求报文

源端口: 49152

目的端口: 53

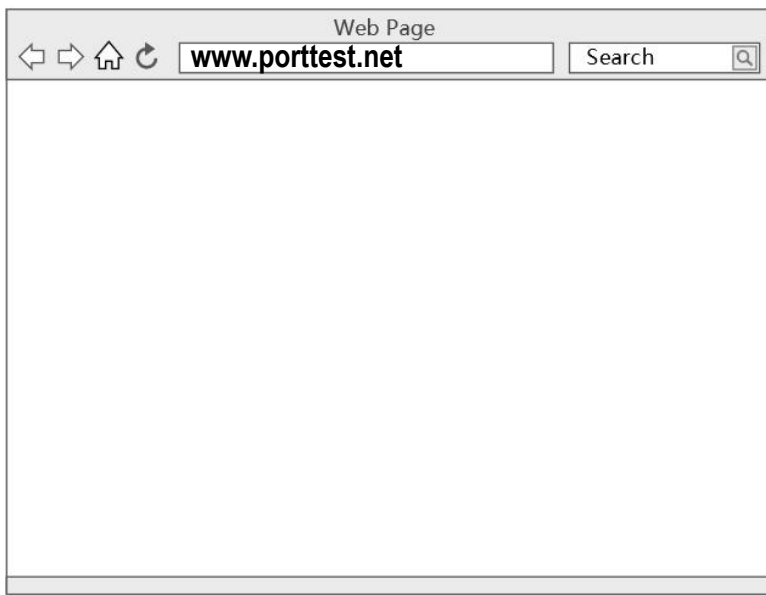
域名www.porttest.net
对应的IP地址是什么?

① 2.168.0.3 ↔ www.porttest.net

DNS服务器
192.168.0.2Web服务器
192.168.0.3

www.porttest.net

交换机

用户PC
192.168.0.1

5 运输层端口号应用举例

UDP首部

源 端 口: 49152
目的端口: 53

DNS查询请求报文

域名www.porttest.net
对应的IP地址是什么?

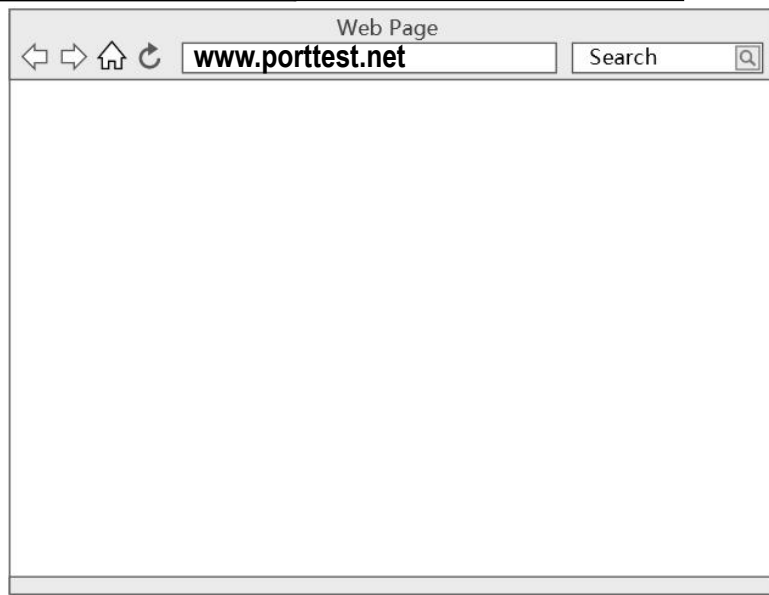
192.168.0.3 ↔ www.porttest.net

DNS响应报文

域名www.porttest.net
对应的IP地址192.168.0.3

UDP首部

源 端 口: 53
目的端口: 49152



DNS服务器
192.168.0.2



Web服务器
192.168.0.3



www.porttest.net

交换机



用户PC
192.168.0.1

U5

运输层端口号应用举例

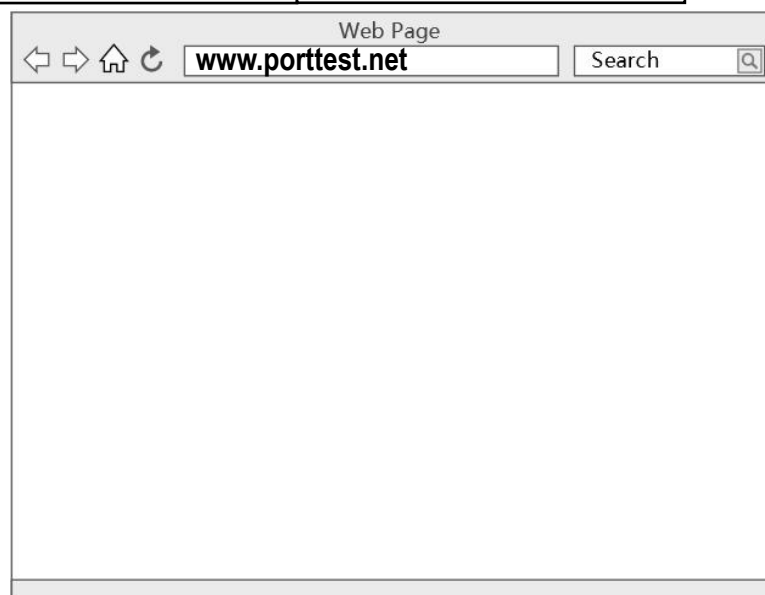
DNS服务器
192.168.0.2Web服务器
192.168.0.3

www.porttest.net

192.168.0.3 ↔ www.porttest.net

DNS响应报文

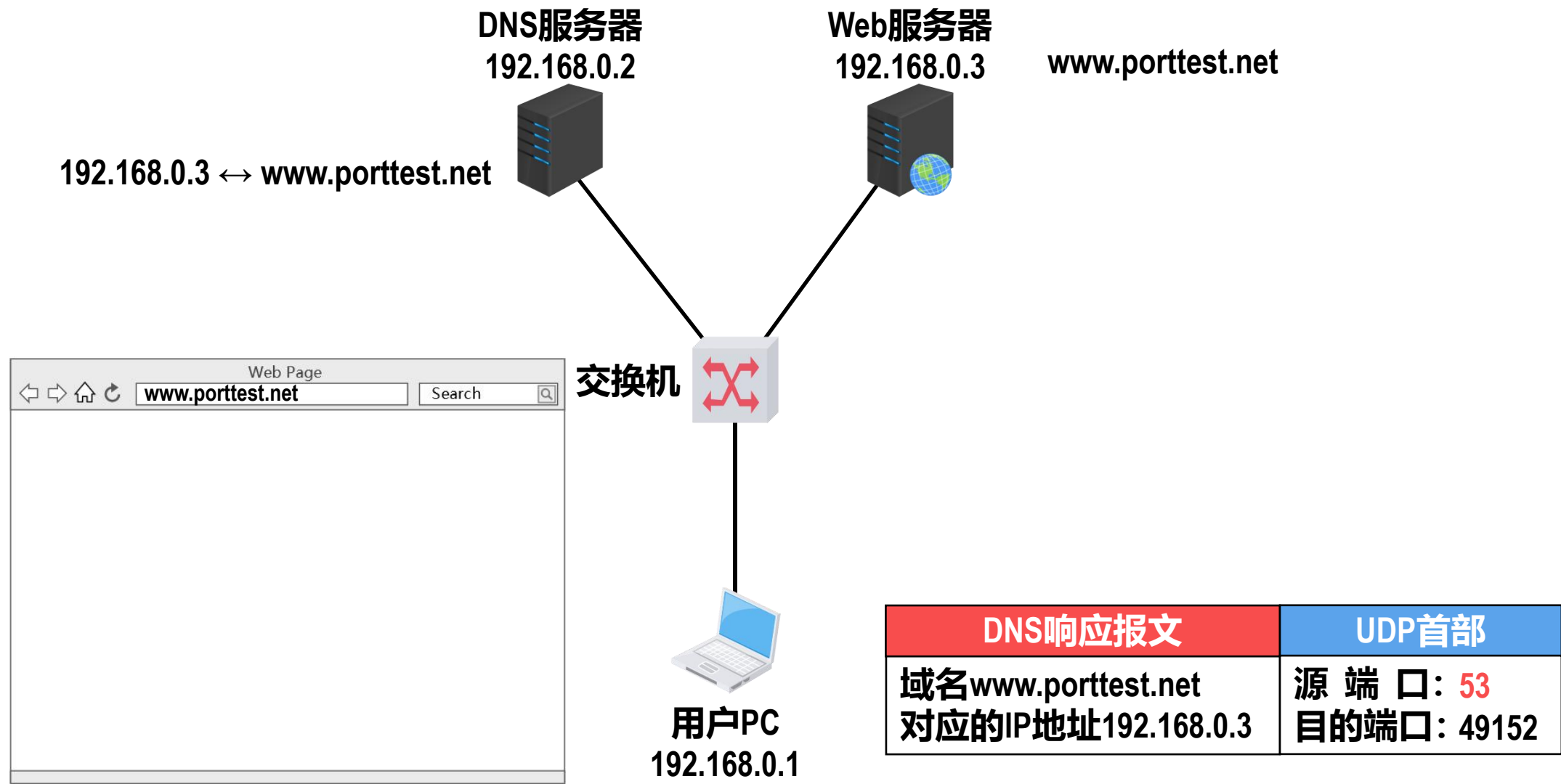
UDP首部

域名www.porttest.net
对应的IP地址192.168.0.3源端口: 53
目的端口: 49152

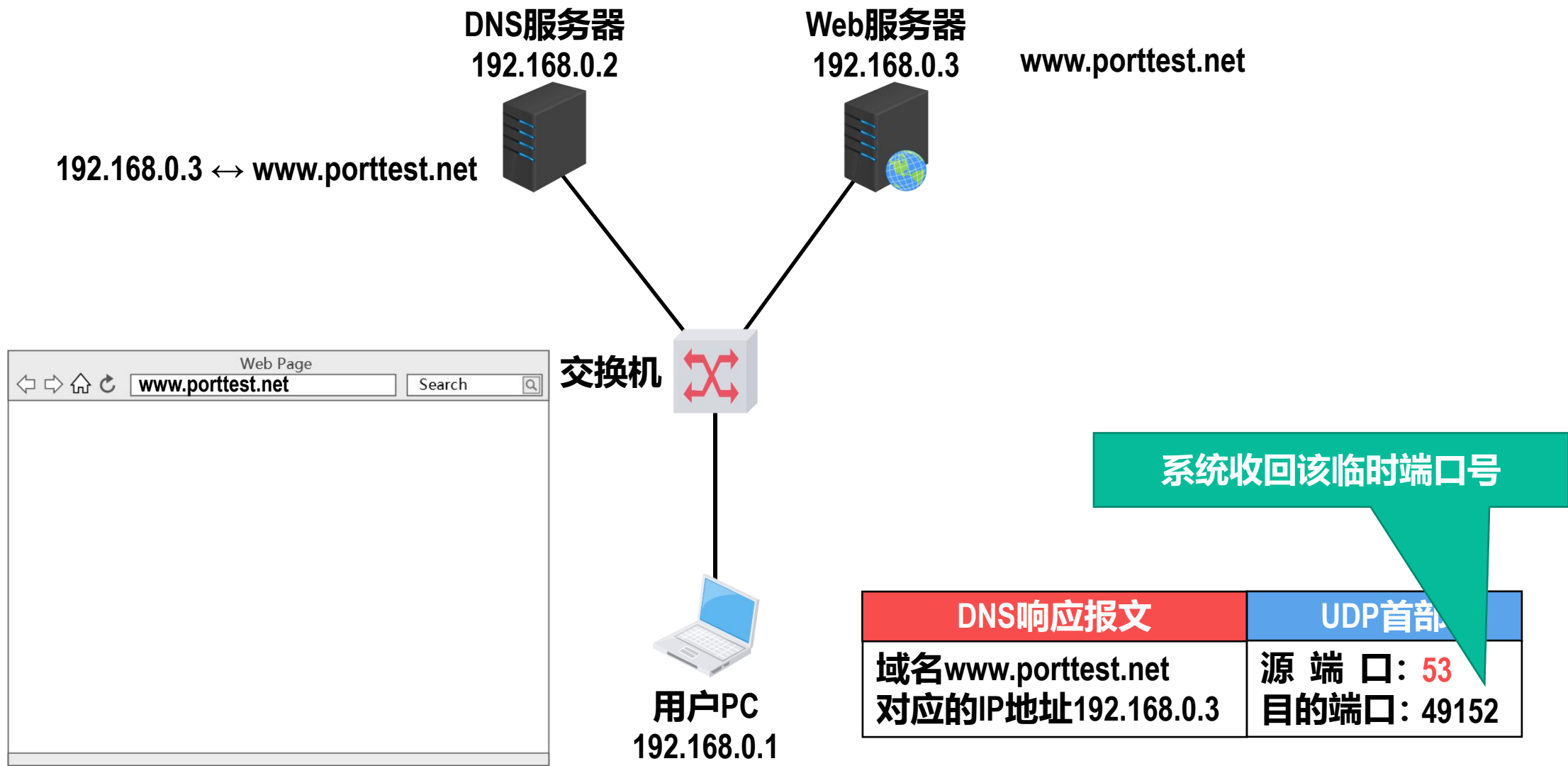
交换机

用户PC
192.168.0.1

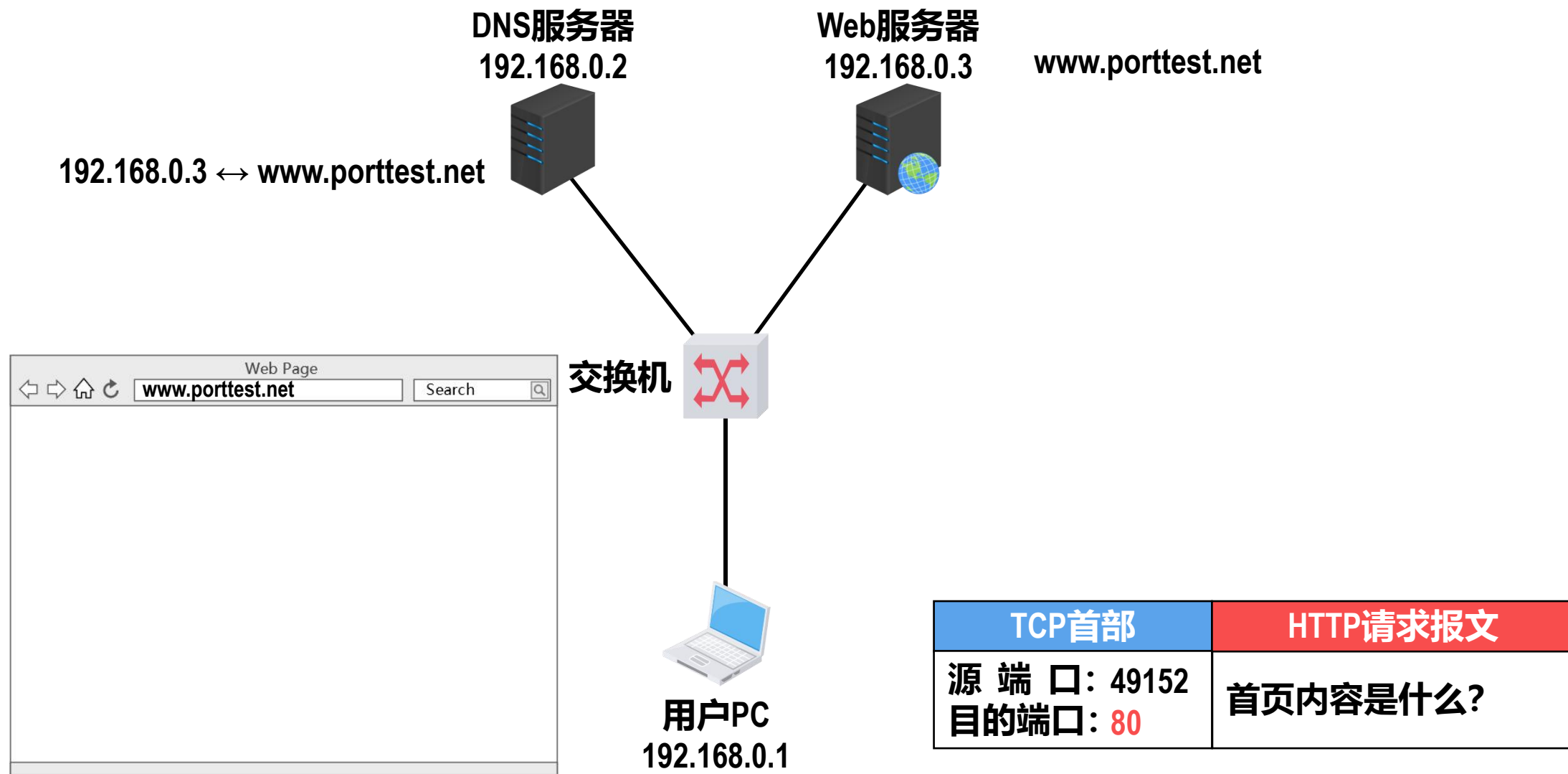
5 运输层端口号应用举例



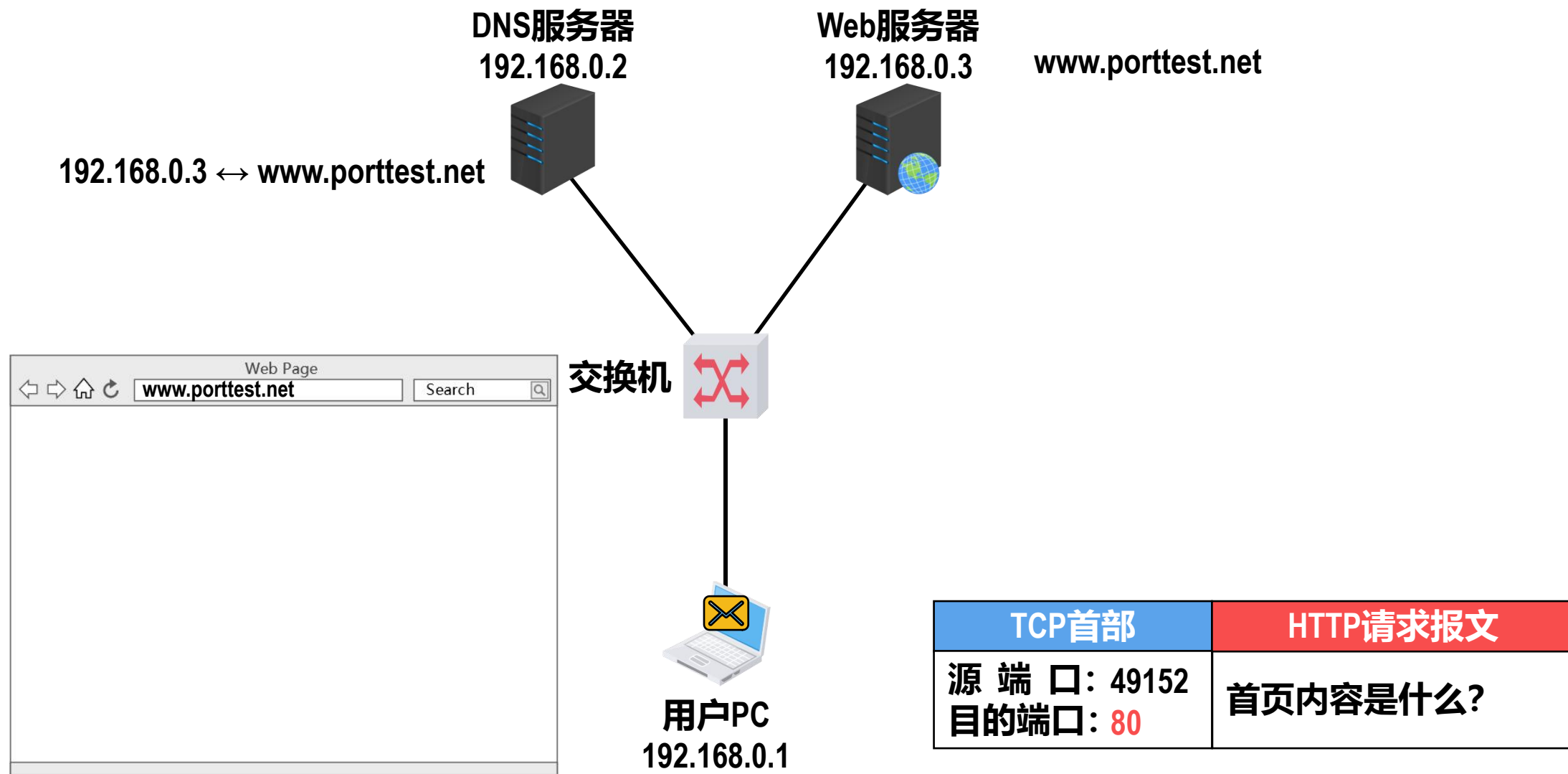
5 运输层端口号应用举例



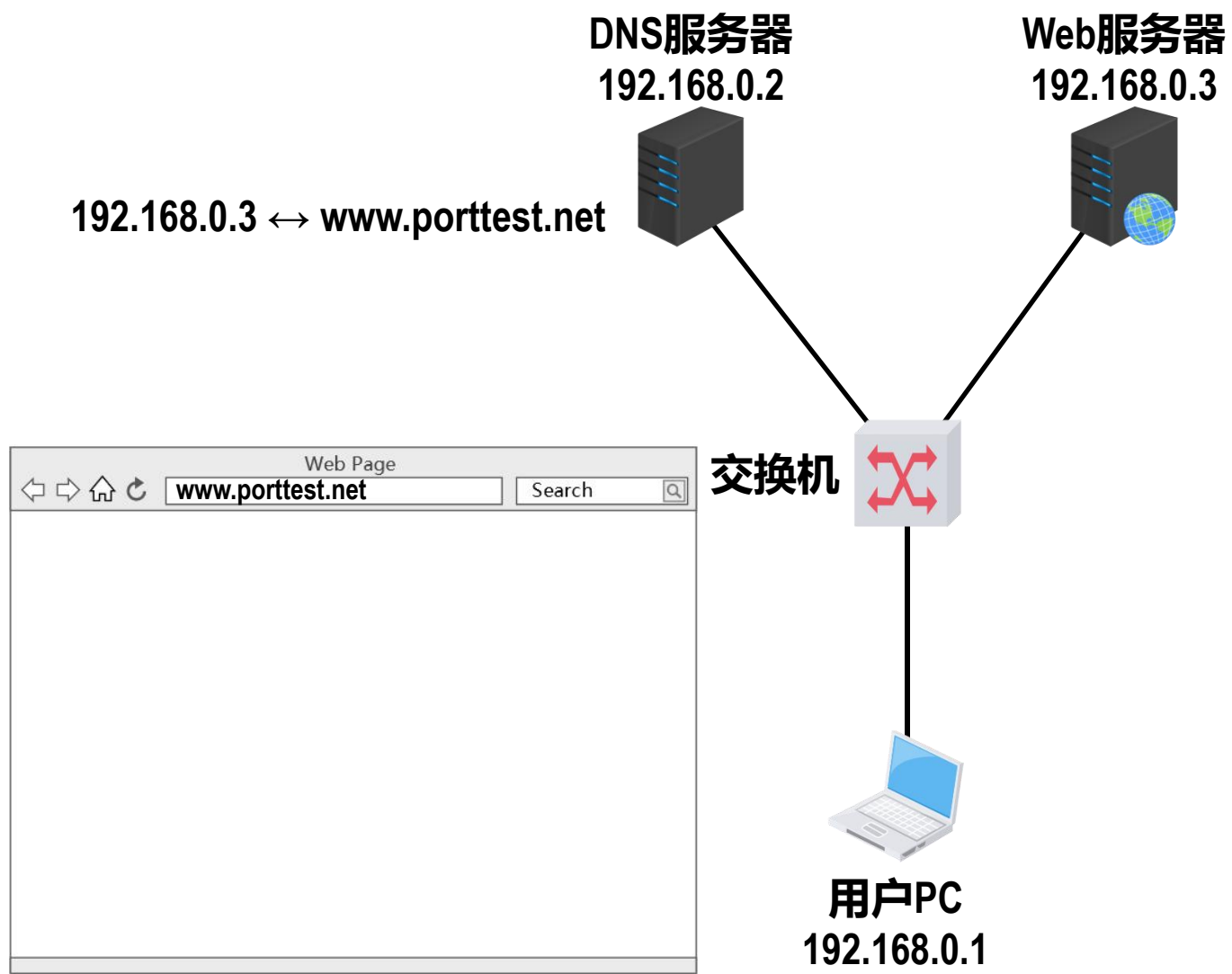
5 运输层端口号应用举例



5 运输层端口号应用举例

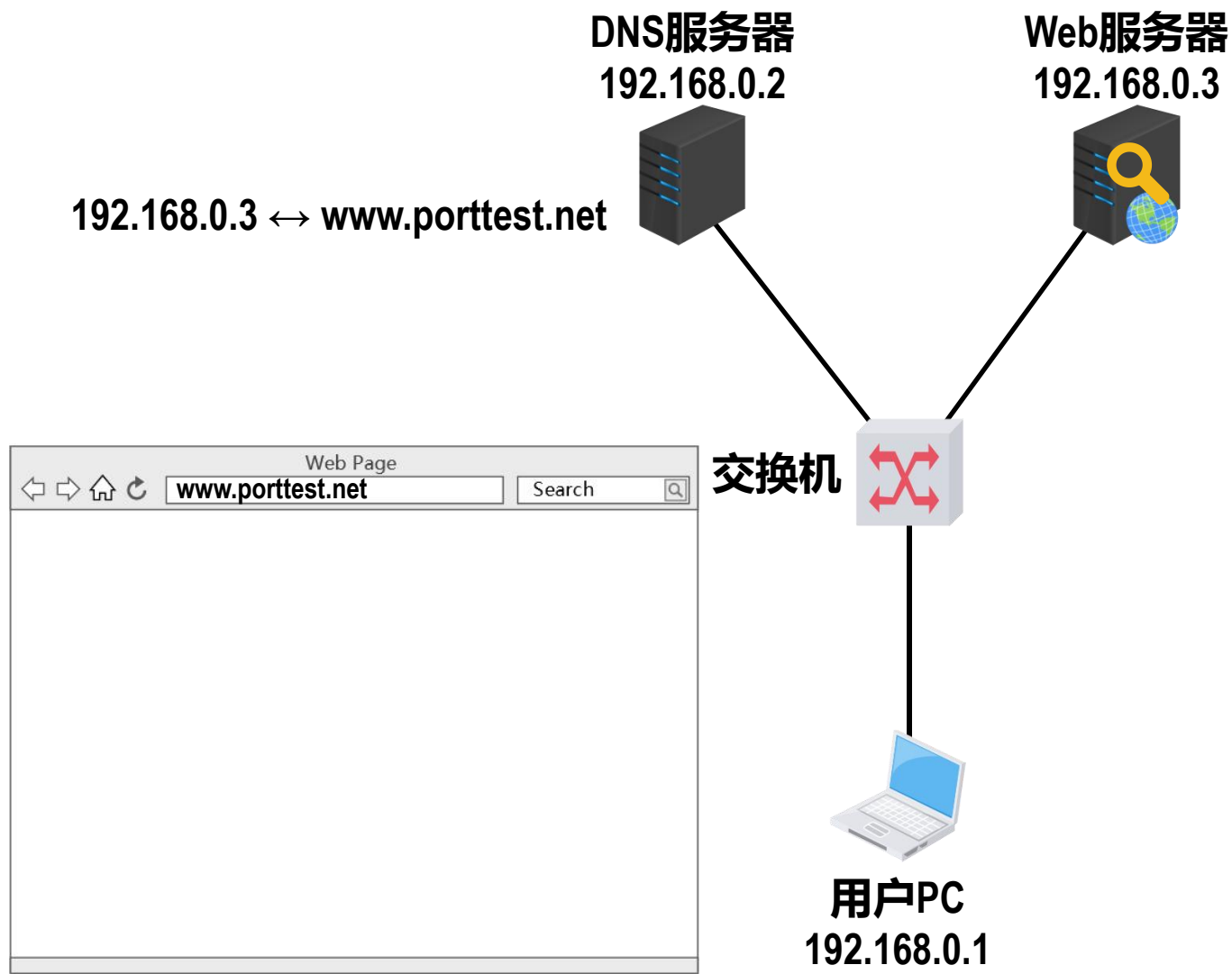


U5 运输层端口号应用举例



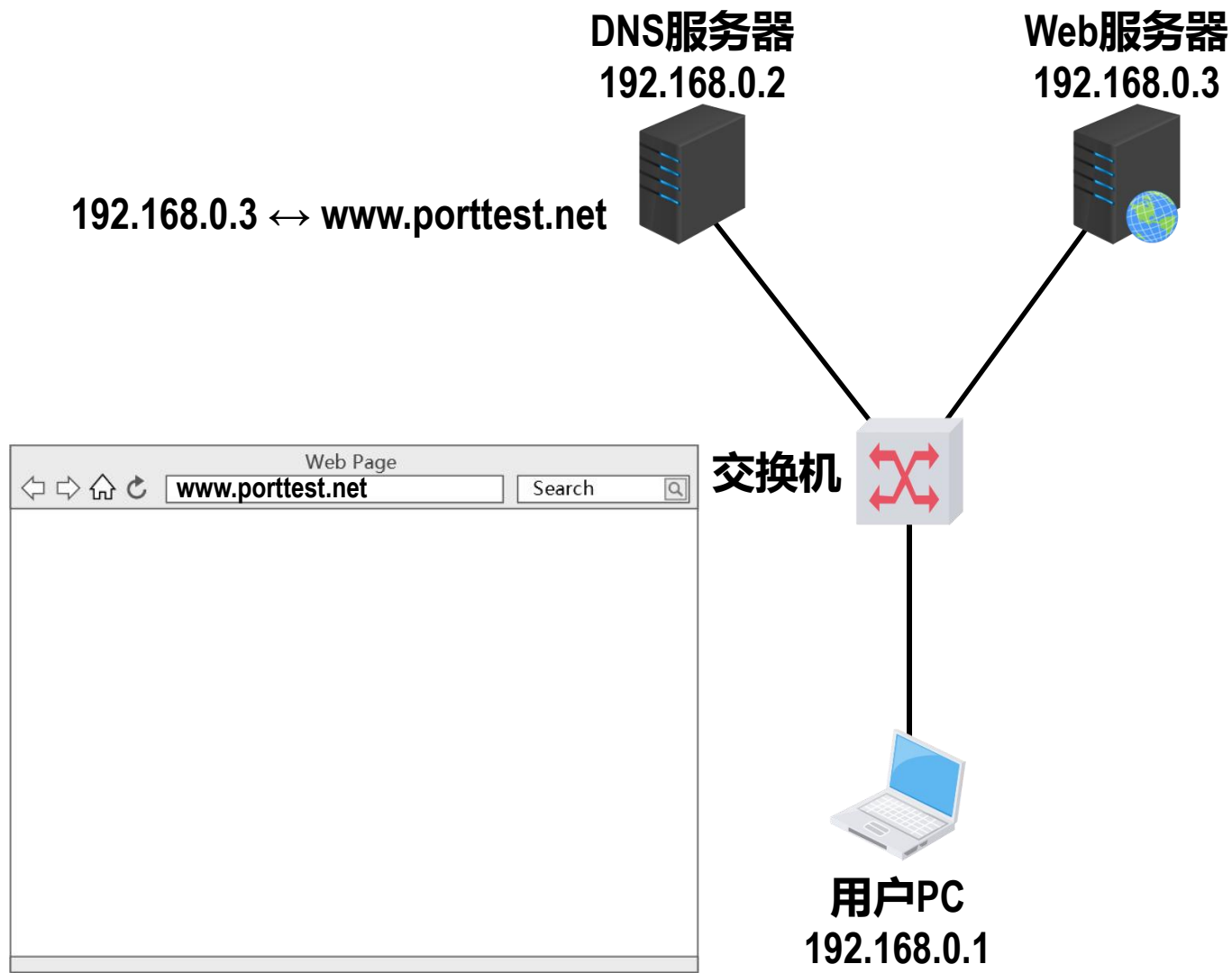
HTTP请求报文	TCP首部
首页内容是什么?	源端口: 49152 目的端口: 80

5 运输层端口号应用举例



HTTP请求报文	TCP首部
首页内容是什么?	源端口: 49152 目的端口: 80

5 运输层端口号应用举例

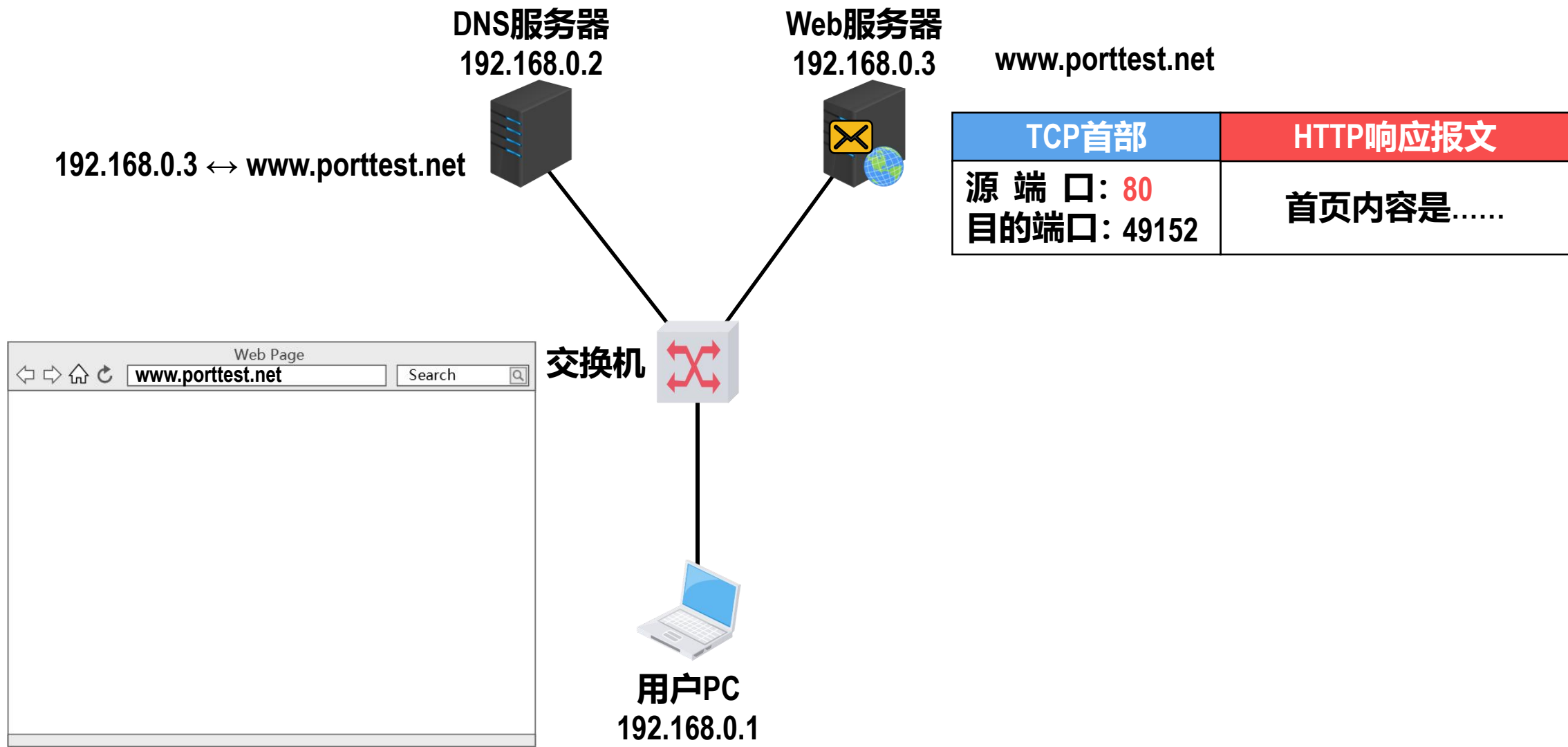


HTTP请求报文	TCP首部
首页内容是什么?	源 端 口: 49152 目的端口: 80

TCP首部	HTTP响应报文
源 端 口: 80 目的端口: 49152	首页内容是.....

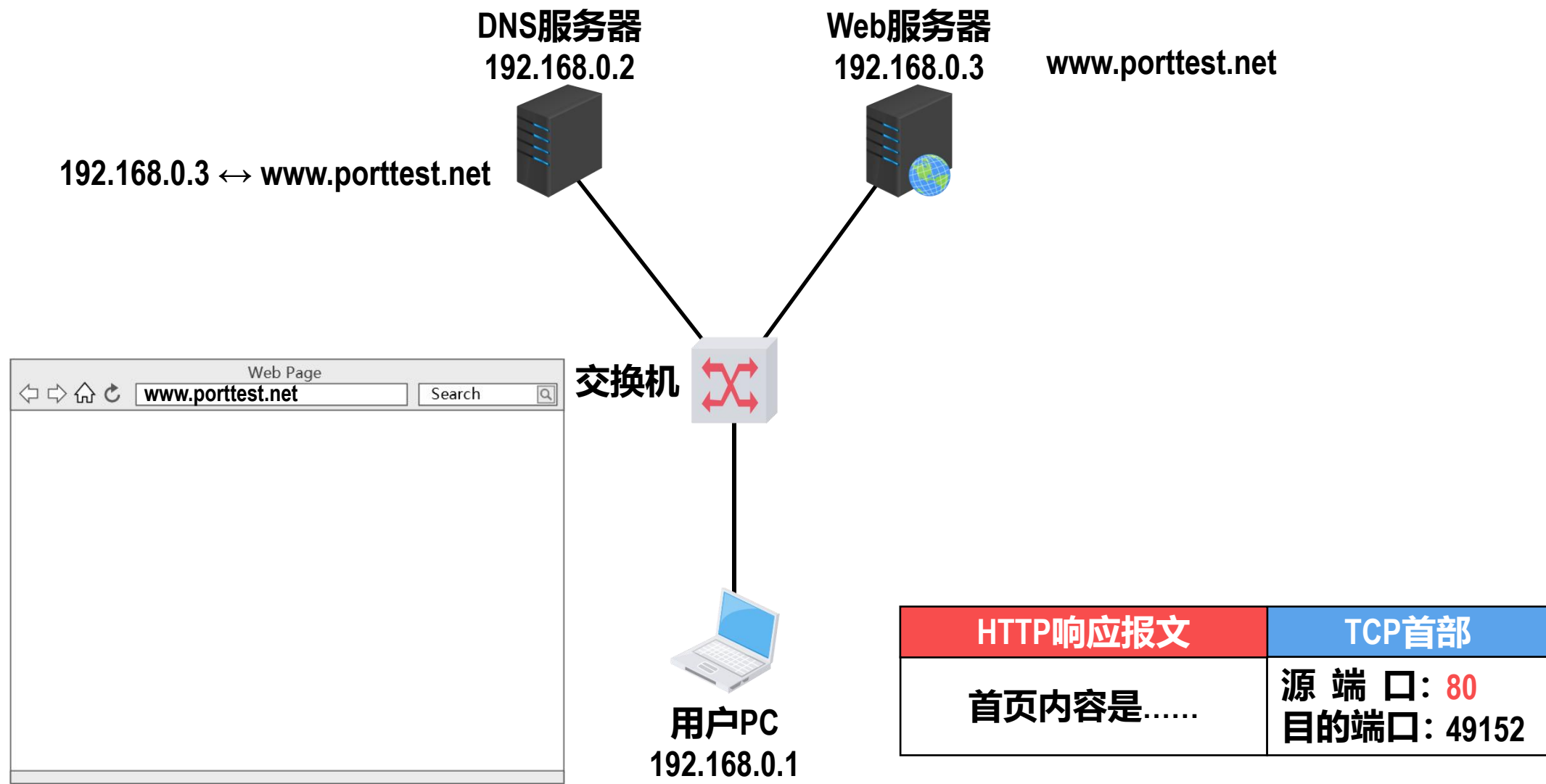
U5

运输层端口号应用举例

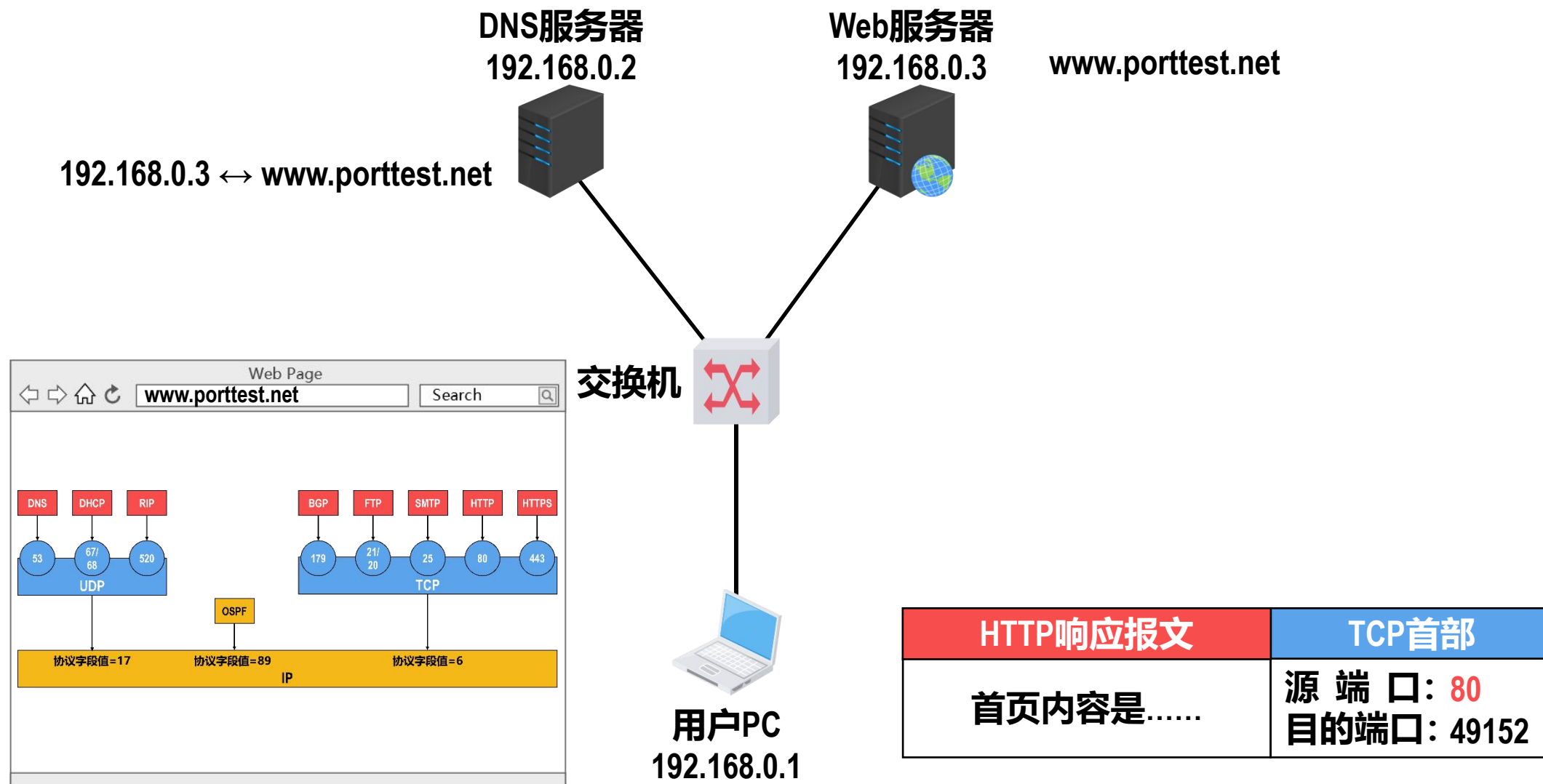


U
5

运输层端口号应用举例



5 运输层端口号应用举例



5.2 UDP和TCP的对比

01

无连接的UDP和面向连接的TCP

02

UDP和TCP对单播、多播和广播的支持情况

03

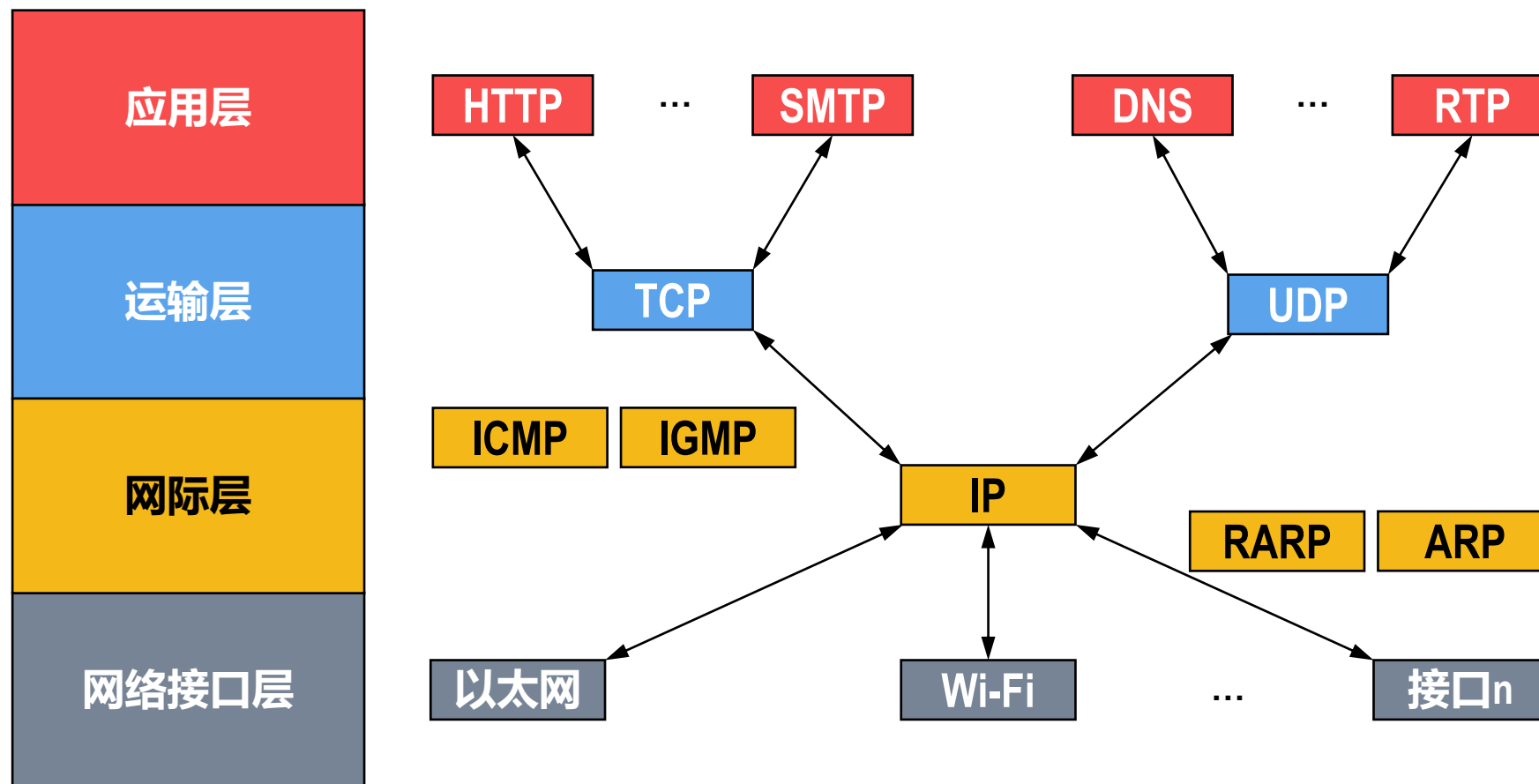
UDP和TCP对应用层报文的处理

04

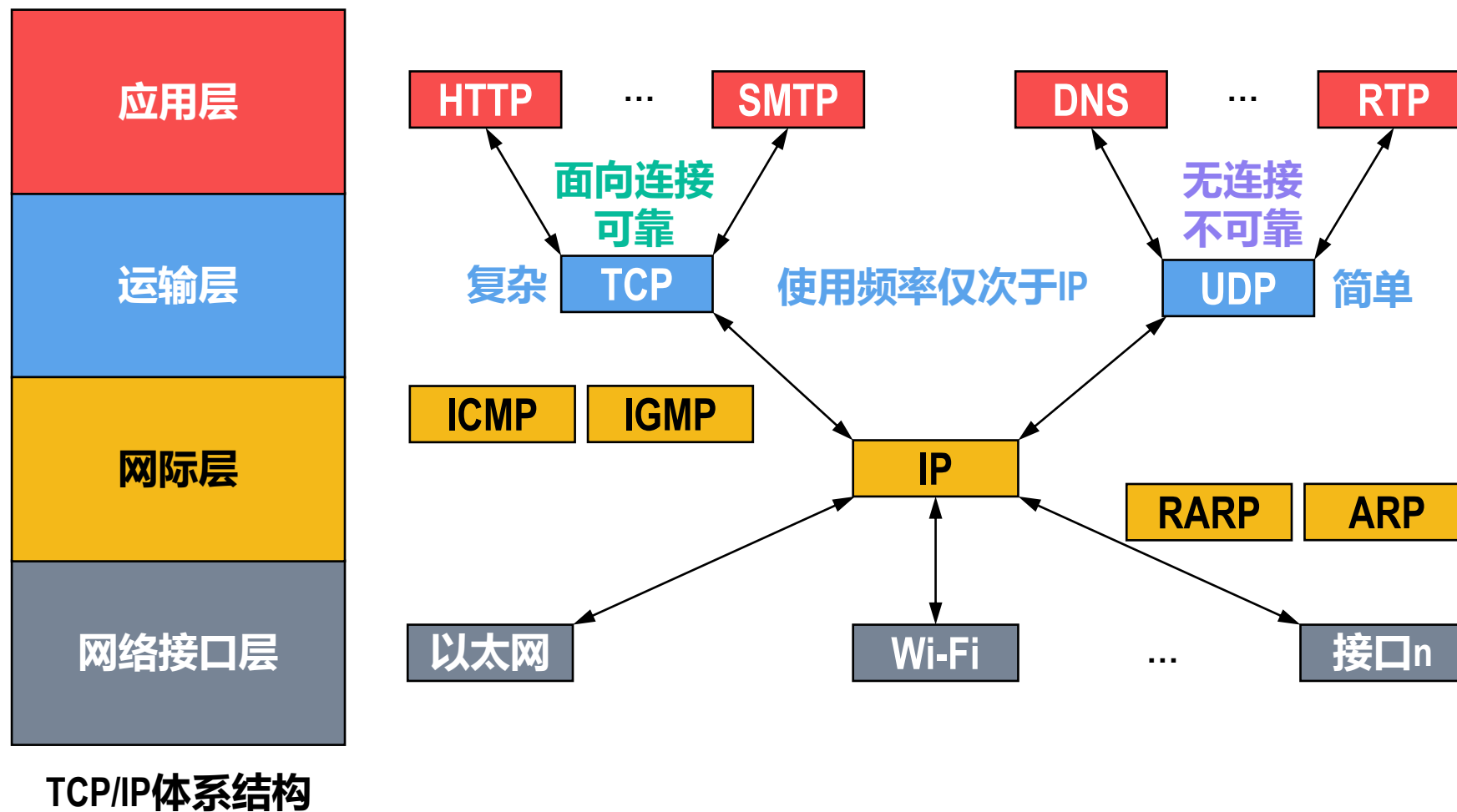
UDP和TCP对数据传输可靠性的支持情况

05

UDP首部和TCP首部的对比

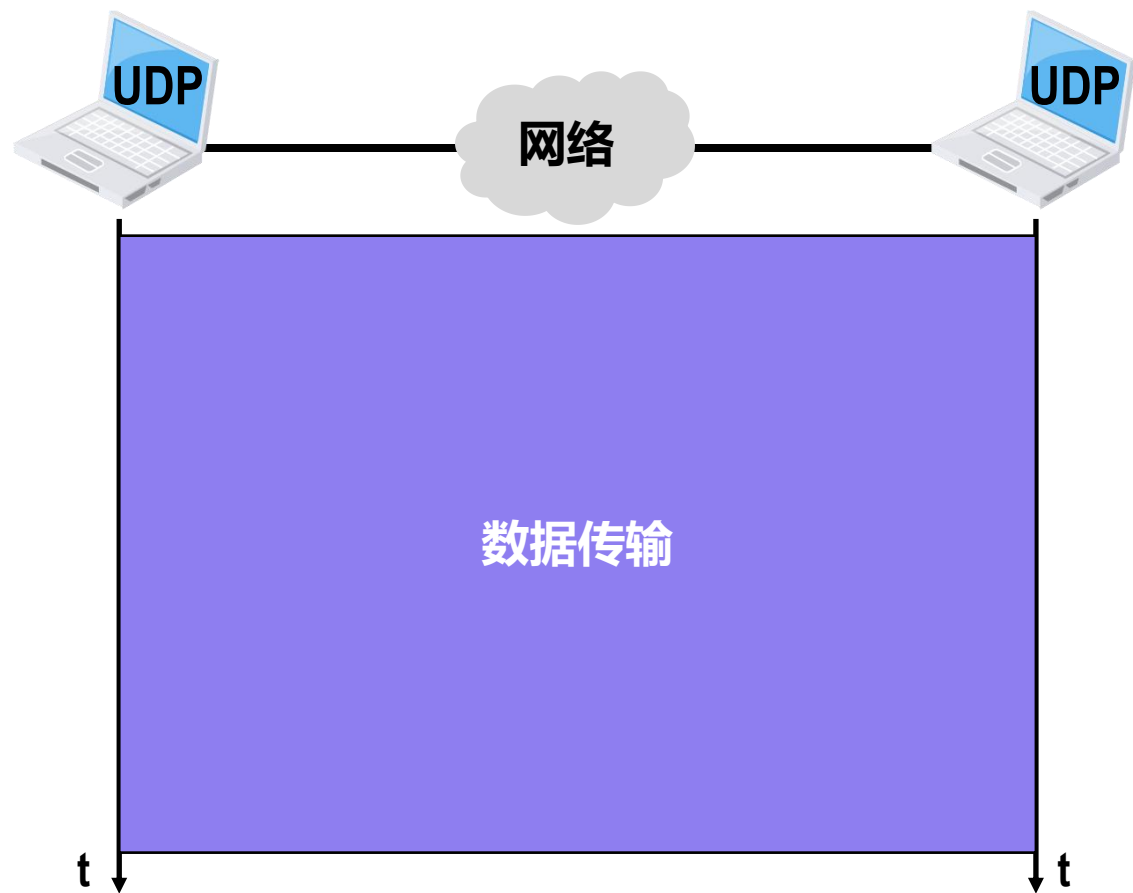


TCP/IP体系结构

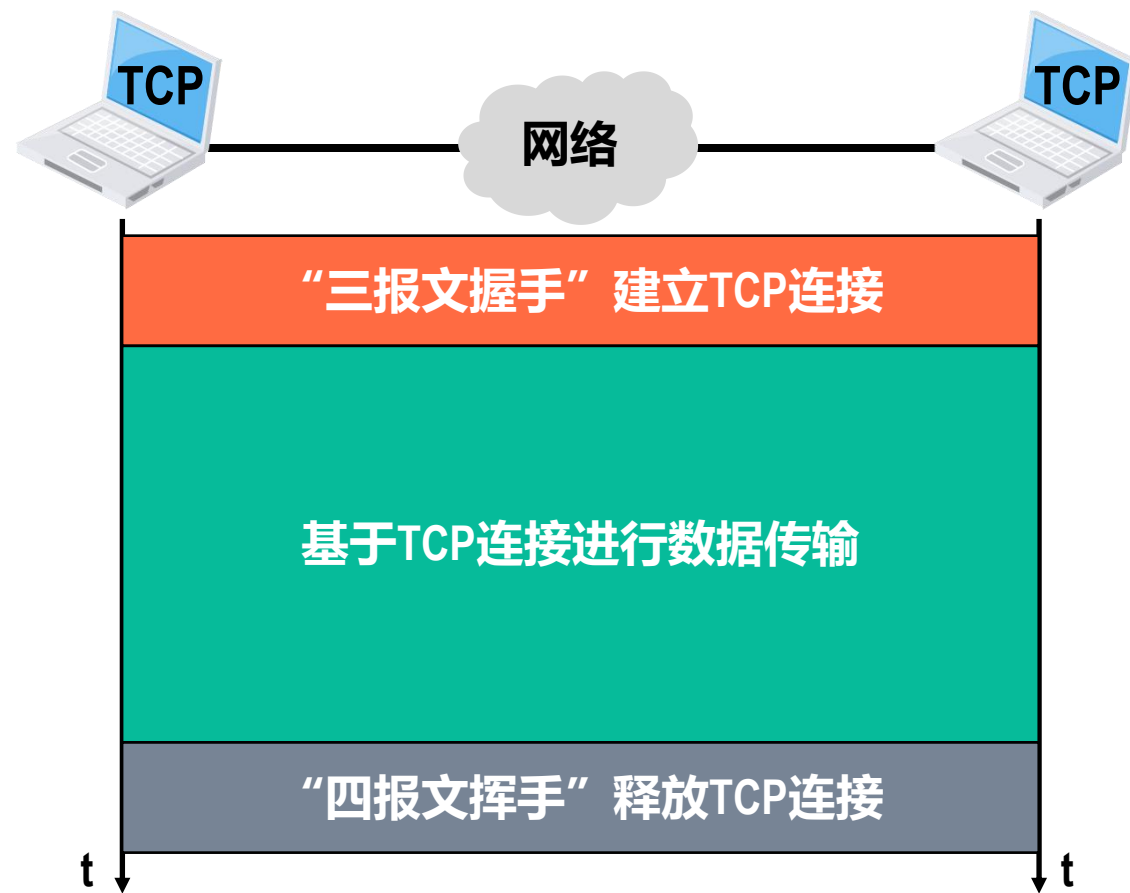


01 无连接的UDP和面向连接的TCP —— “连接”指逻辑连接关系，而不是物理连接。

用户数据报协议UDP (User Datagram Protocol)



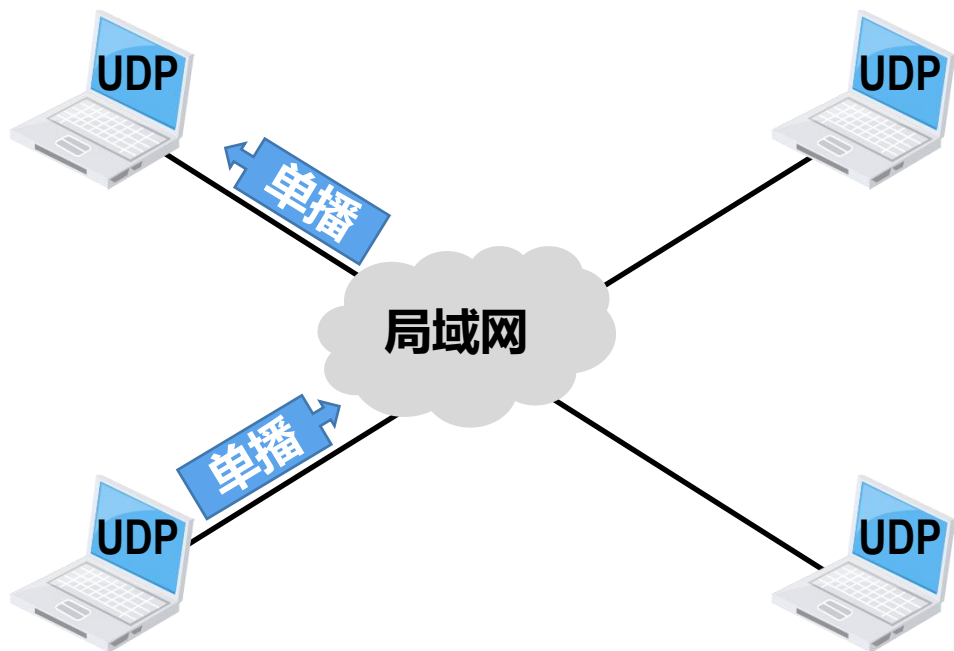
传输控制协议TCP (Transmission Control Protocol)



02 UDP和TCP对单播、多播和广播的支持情况

用户数据报协议UDP
(User Datagram Protocol)

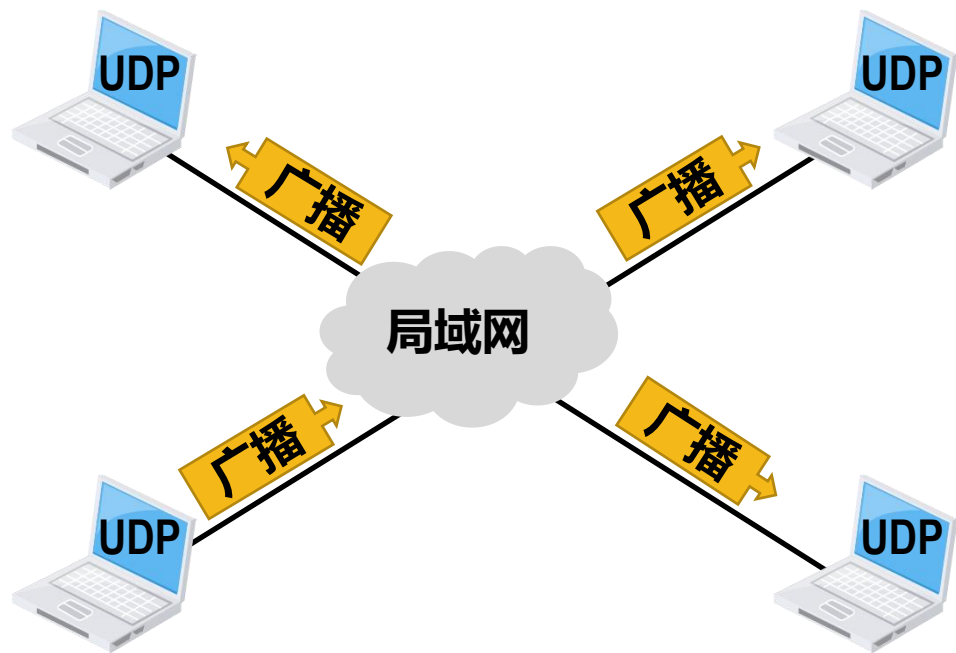
传输控制协议TCP
(Transmission Control Protocol)



02 UDP和TCP对单播、多播和广播的支持情况

用户数据报协议UDP
(User Datagram Protocol)

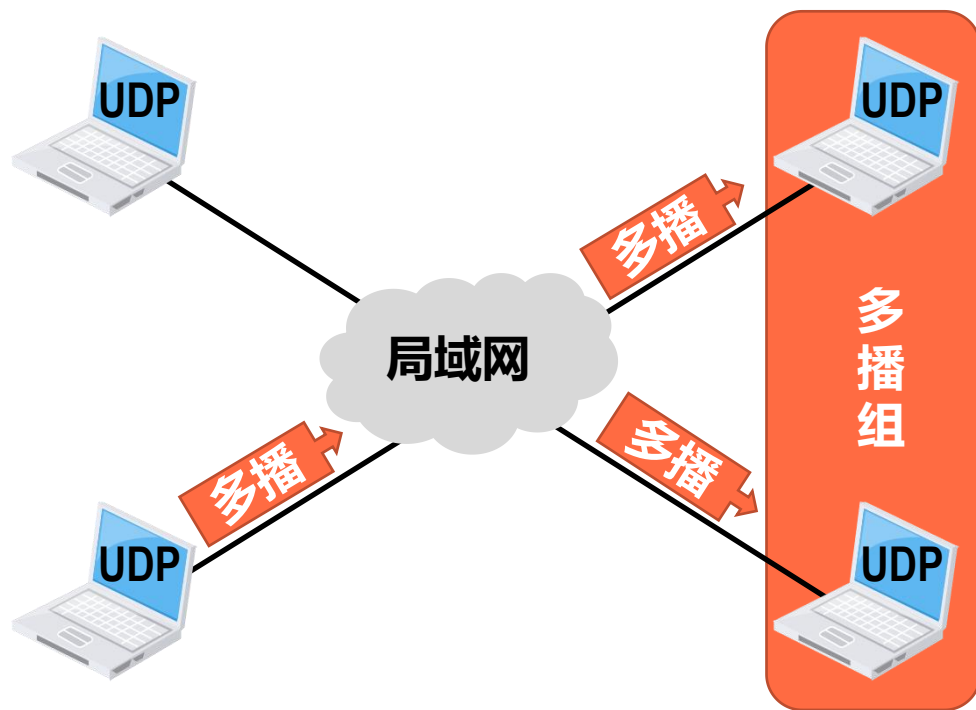
传输控制协议TCP
(Transmission Control Protocol)



02 UDP和TCP对单播、多播和广播的支持情况

用户数据报协议UDP
(User Datagram Protocol)

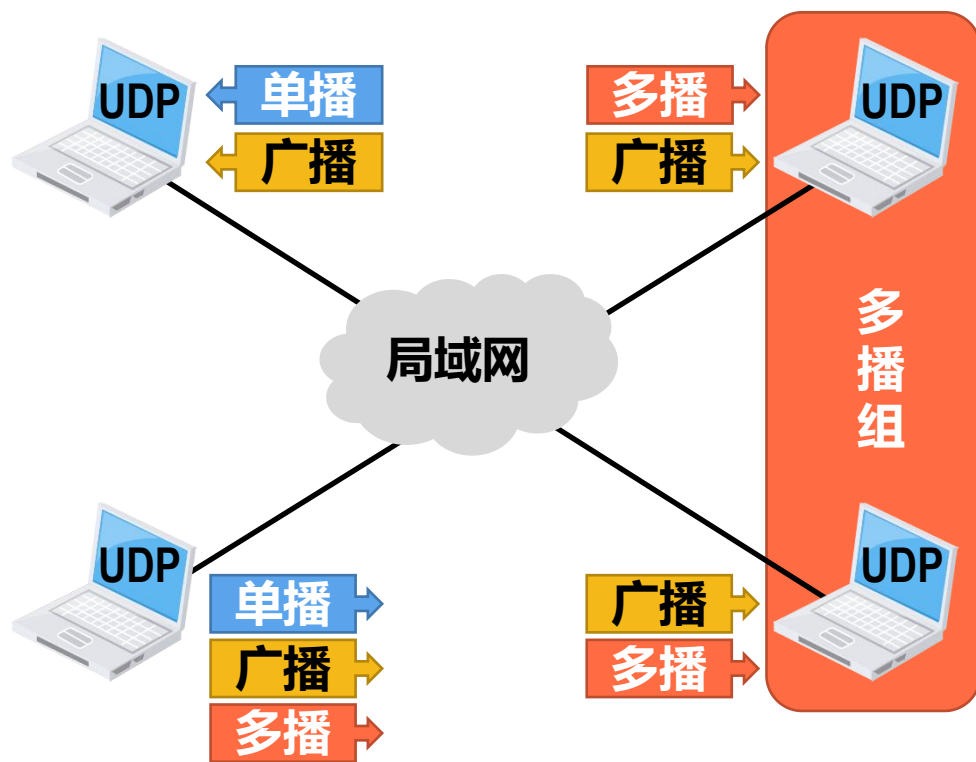
传输控制协议TCP
(Transmission Control Protocol)



02 UDP和TCP对单播、多播和广播的支持情况

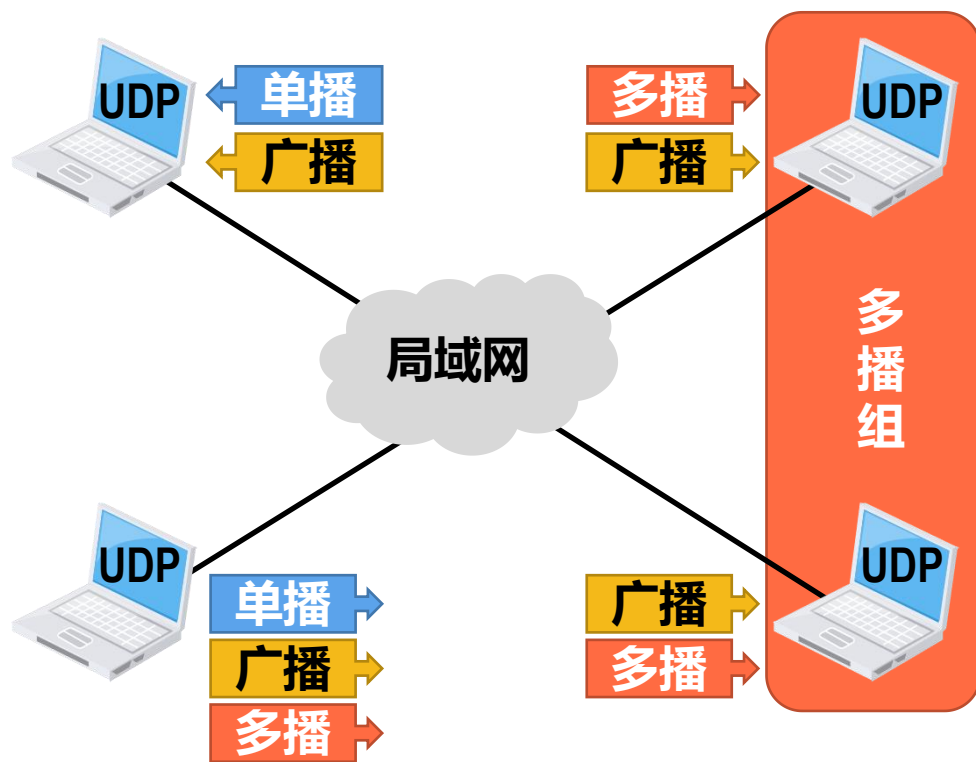
用户数据报协议UDP
(User Datagram Protocol)

传输控制协议TCP
(Transmission Control Protocol)

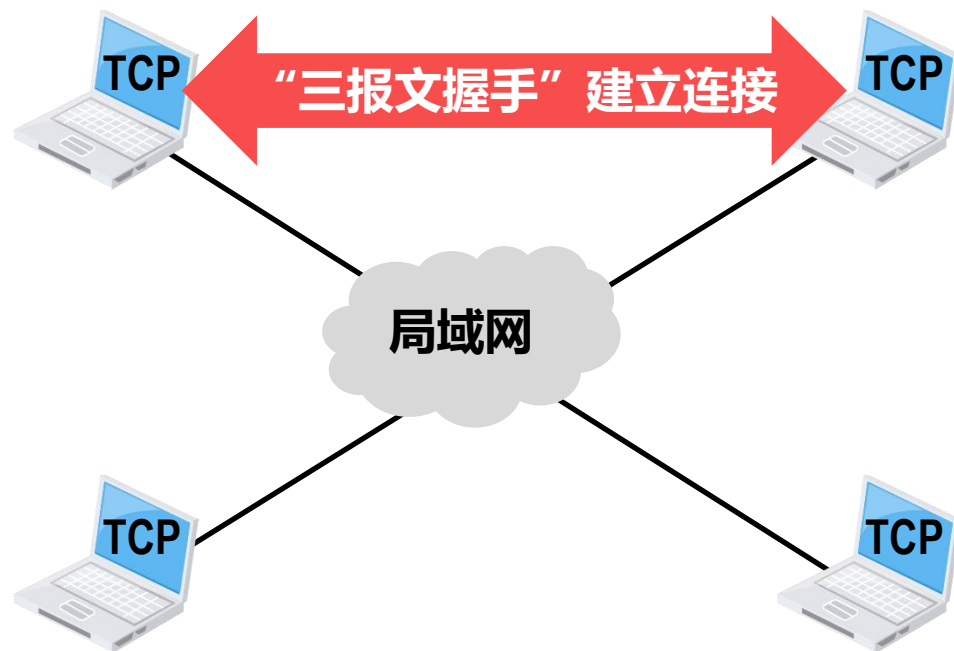


UDP支持单播、多播和广播

02 UDP和TCP对单播、多播和广播的支持情况

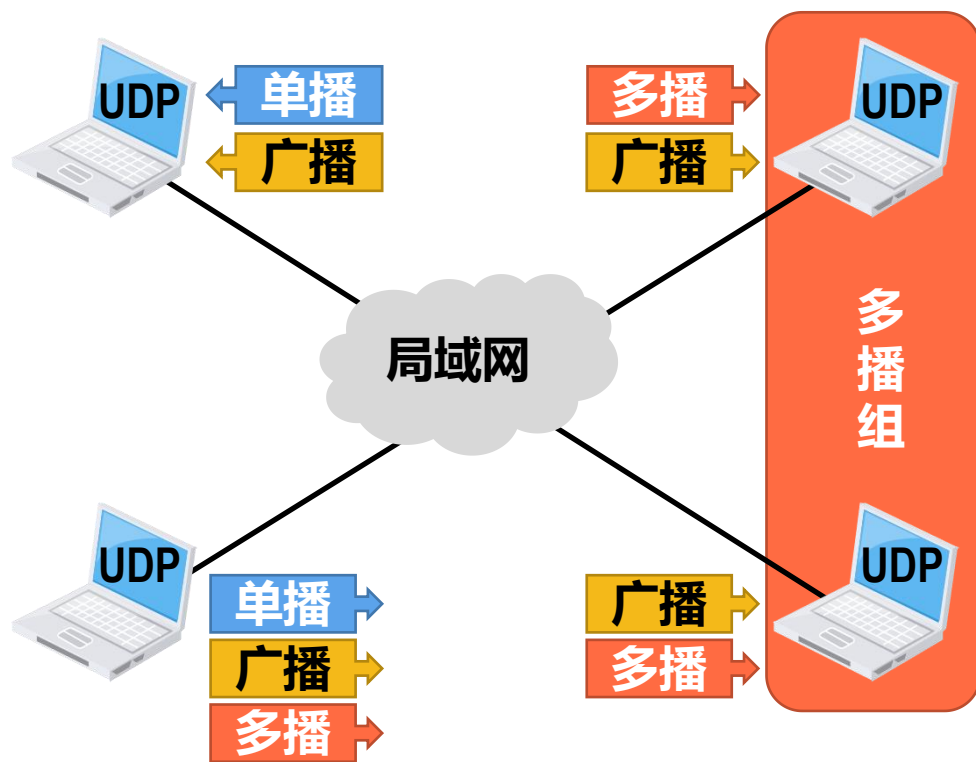
用户数据报协议UDP
(User Datagram Protocol)

UDP支持单播、多播和广播

传输控制协议TCP
(Transmission Control Protocol)

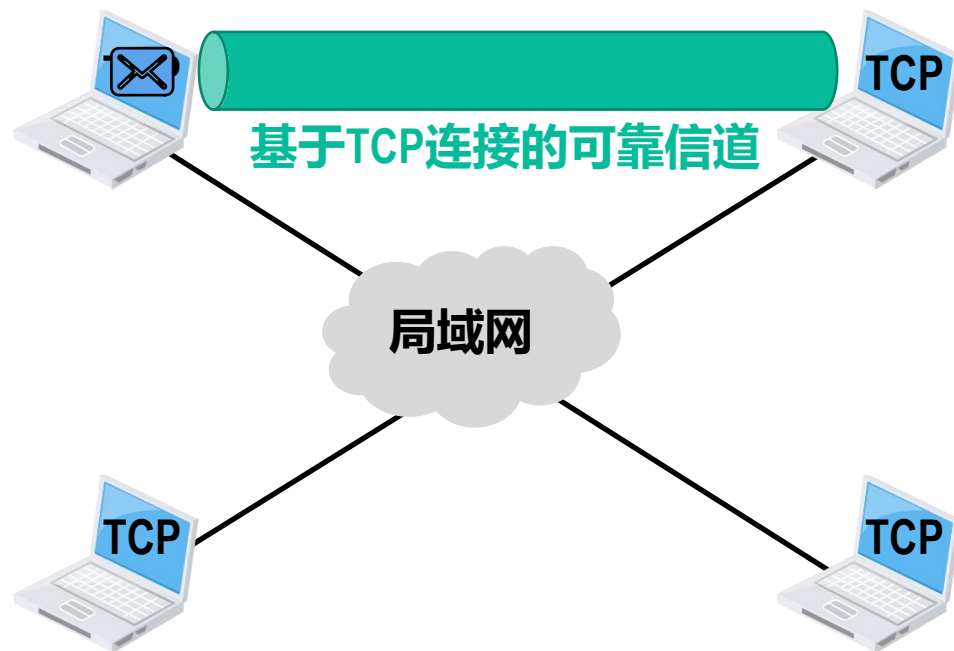
02 UDP和TCP对单播、多播和广播的支持情况

用户数据报协议UDP
(User Datagram Protocol)

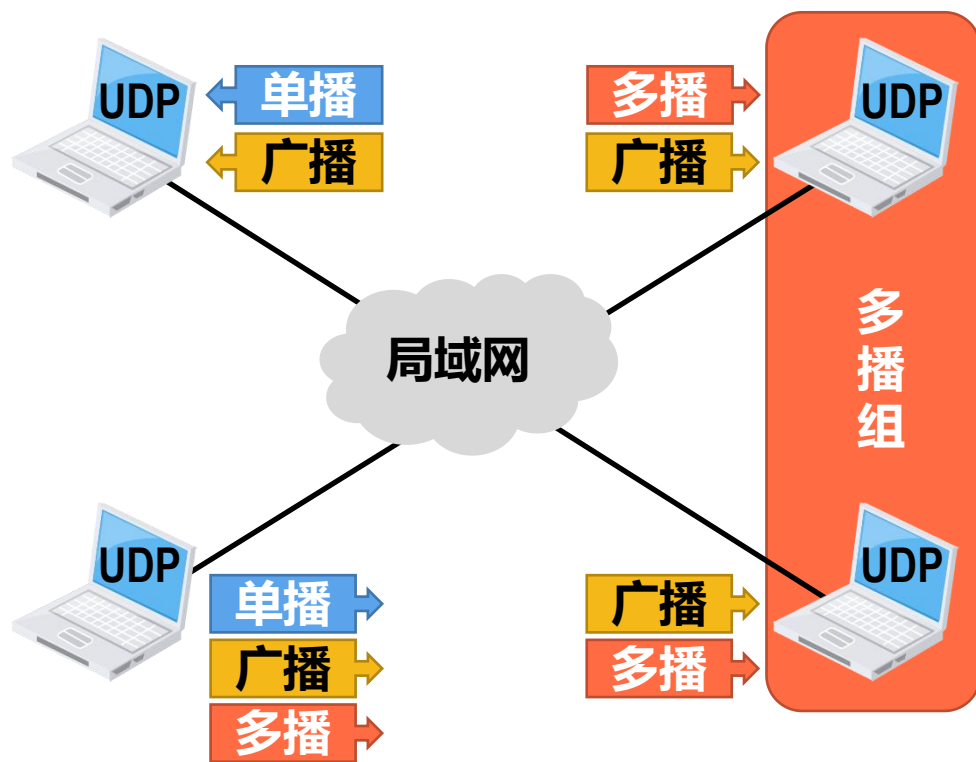


UDP支持单播、多播和广播

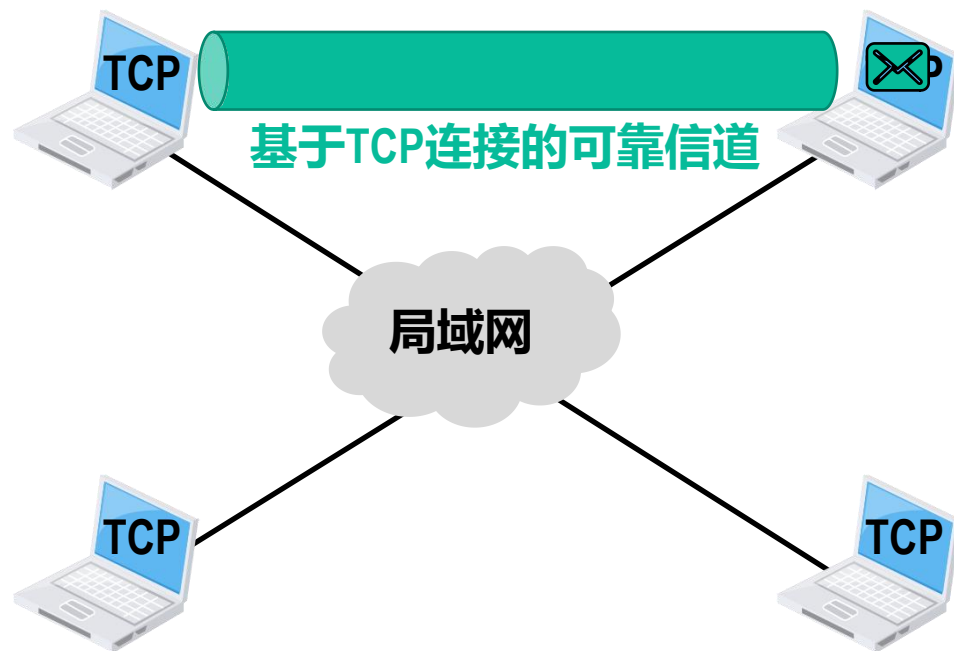
传输控制协议TCP
(Transmission Control Protocol)



02 UDP和TCP对单播、多播和广播的支持情况

用户数据报协议UDP
(User Datagram Protocol)

UDP支持单播、多播和广播

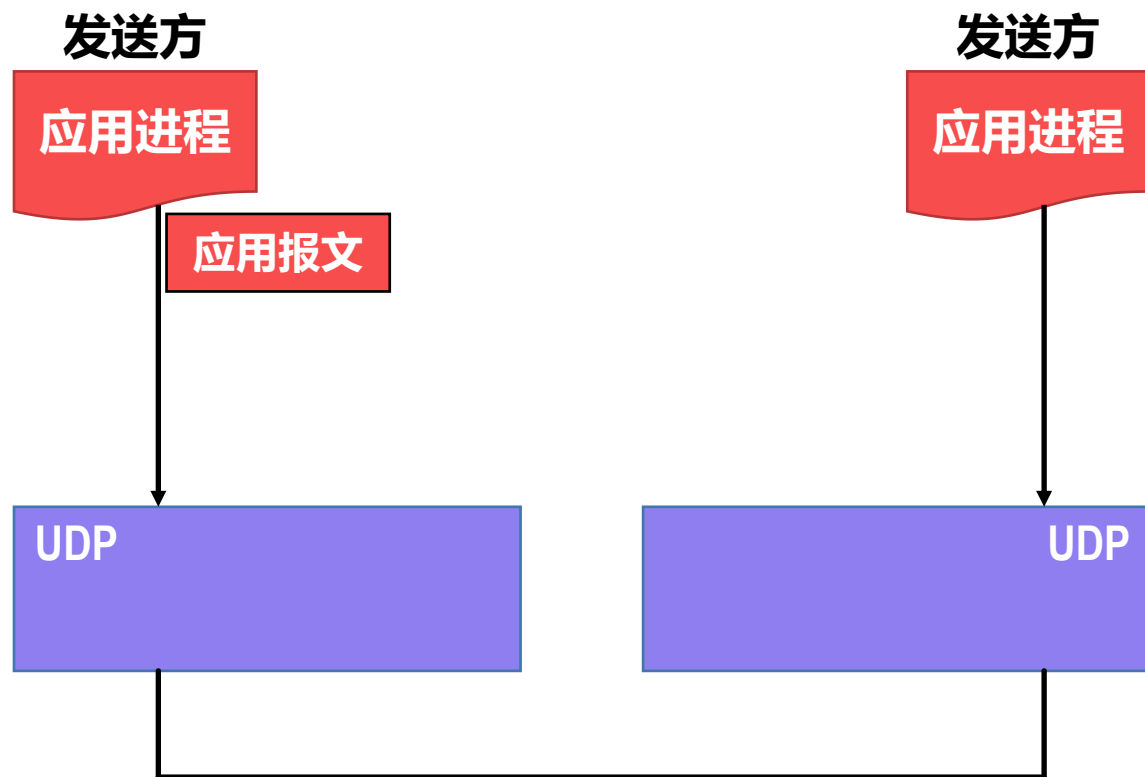
传输控制协议TCP
(Transmission Control Protocol)

TCP仅支持单播

03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

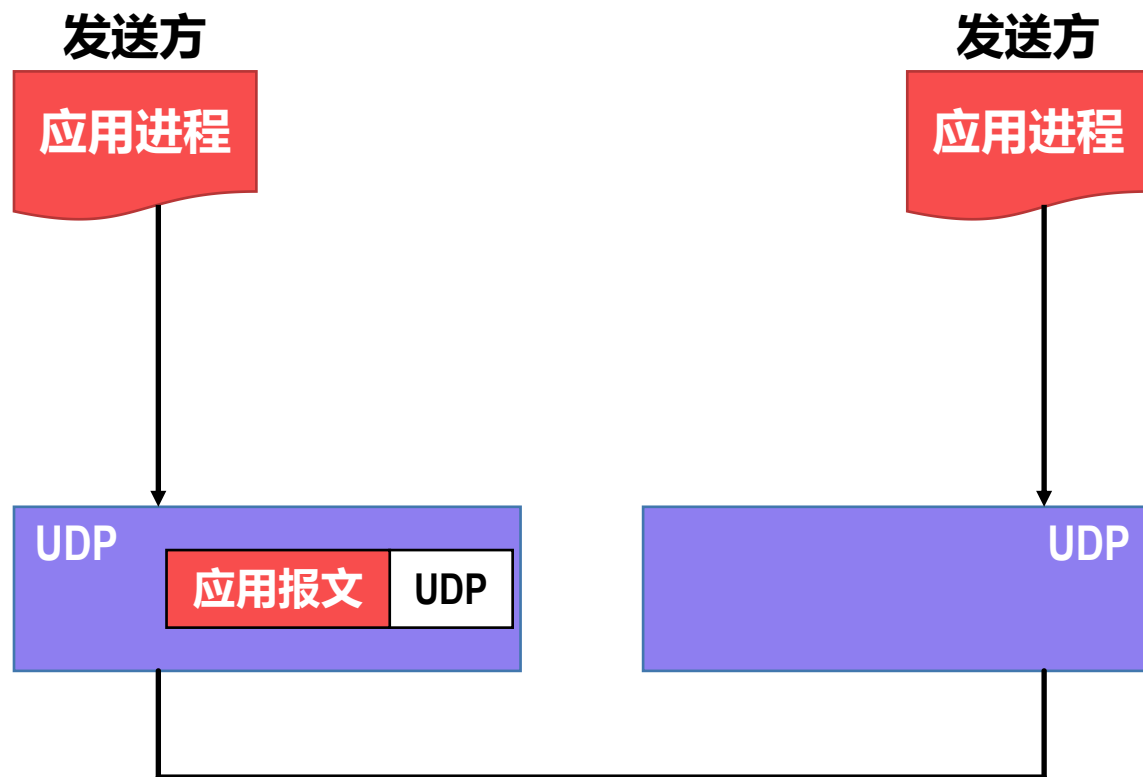
传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

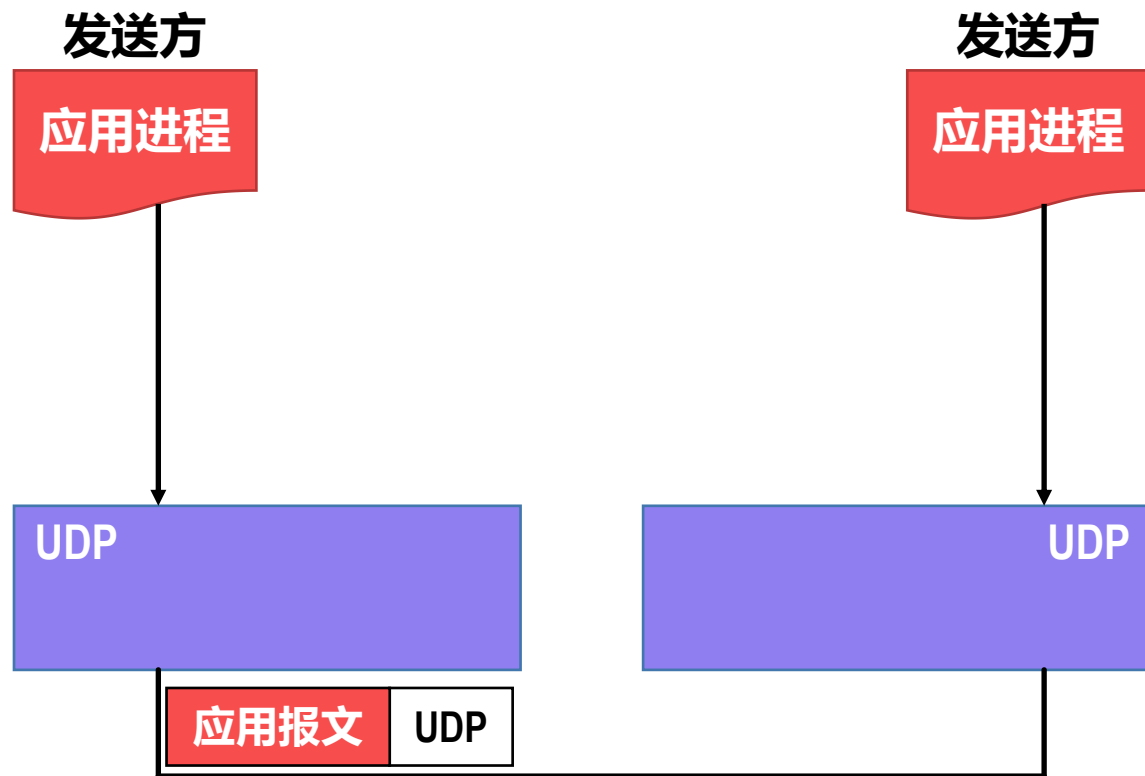
传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

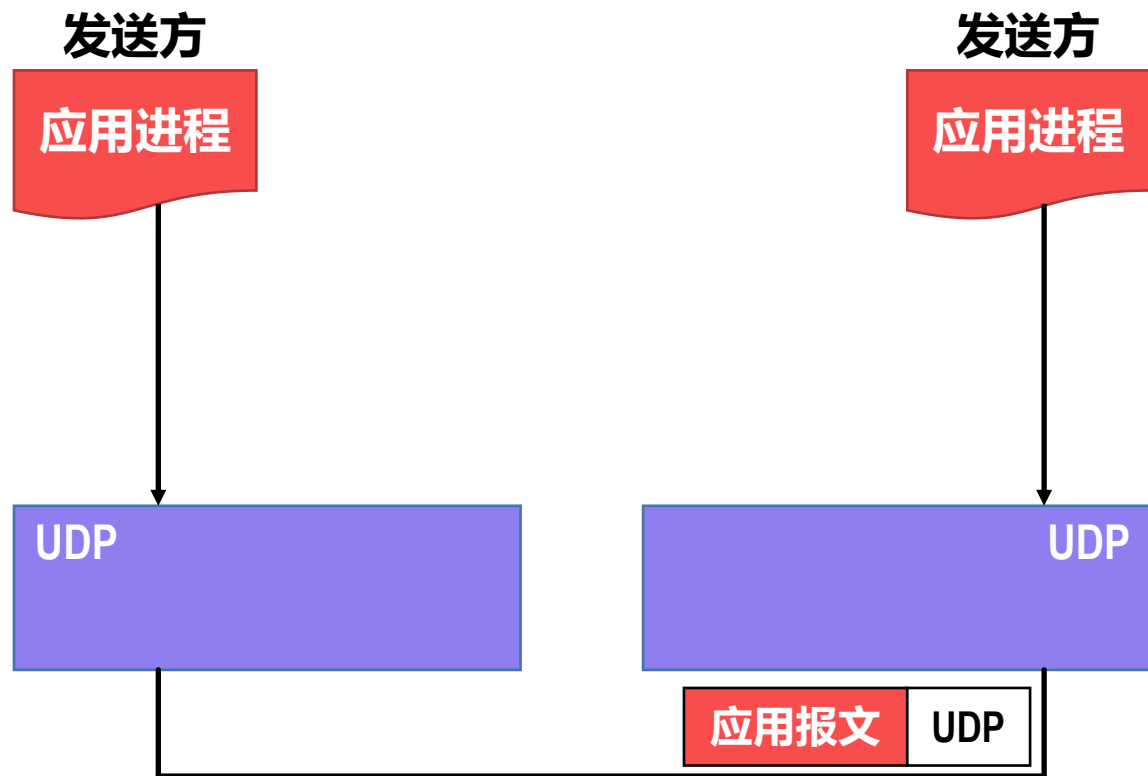
传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

发送方

应用进程

UDP

传输控制协议TCP
(Transmission Control Protocol)

发送方

应用进程

UDP

应用报文

UDP

03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

发送方

应用进程

UDP

传输控制协议TCP
(Transmission Control Protocol)

发送方

应用进程

UDP

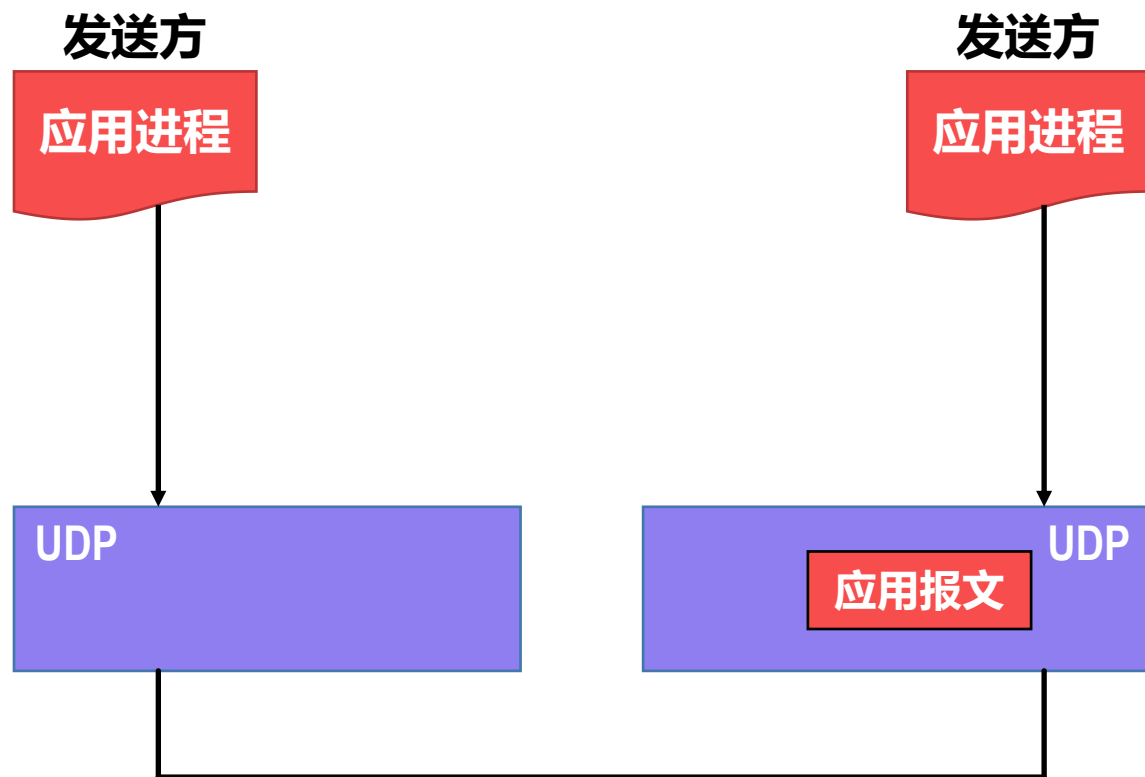
应用报文

UDP

03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

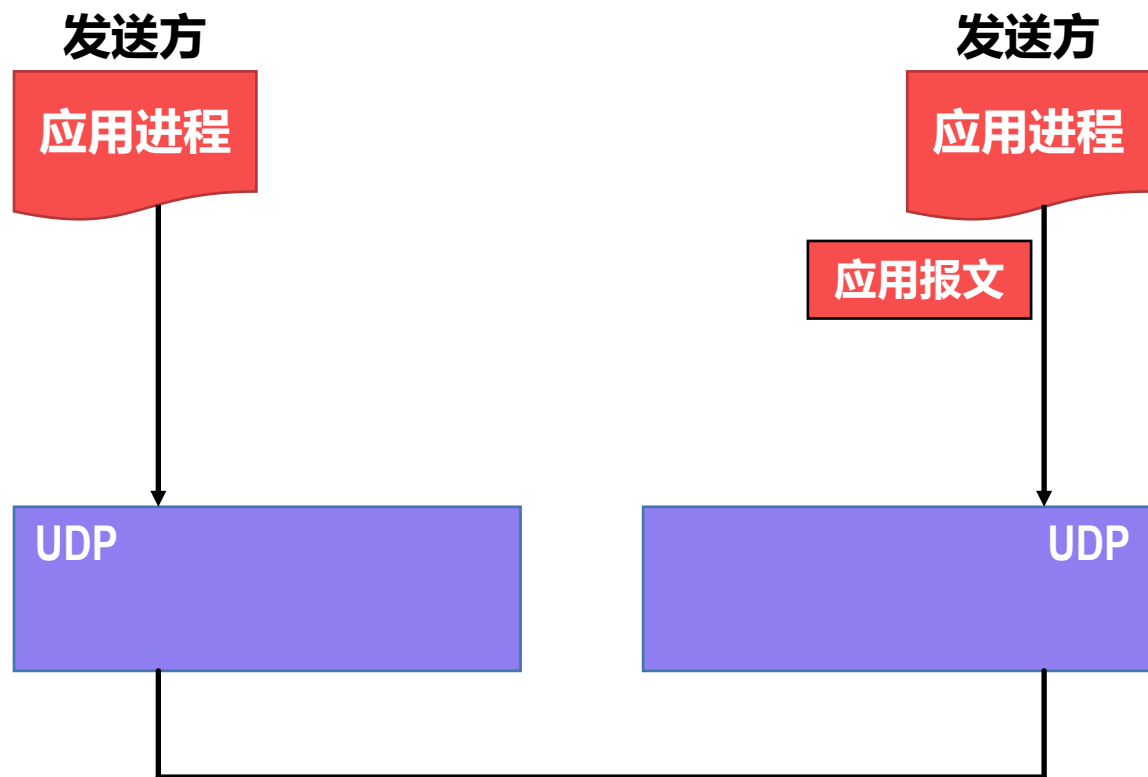
传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

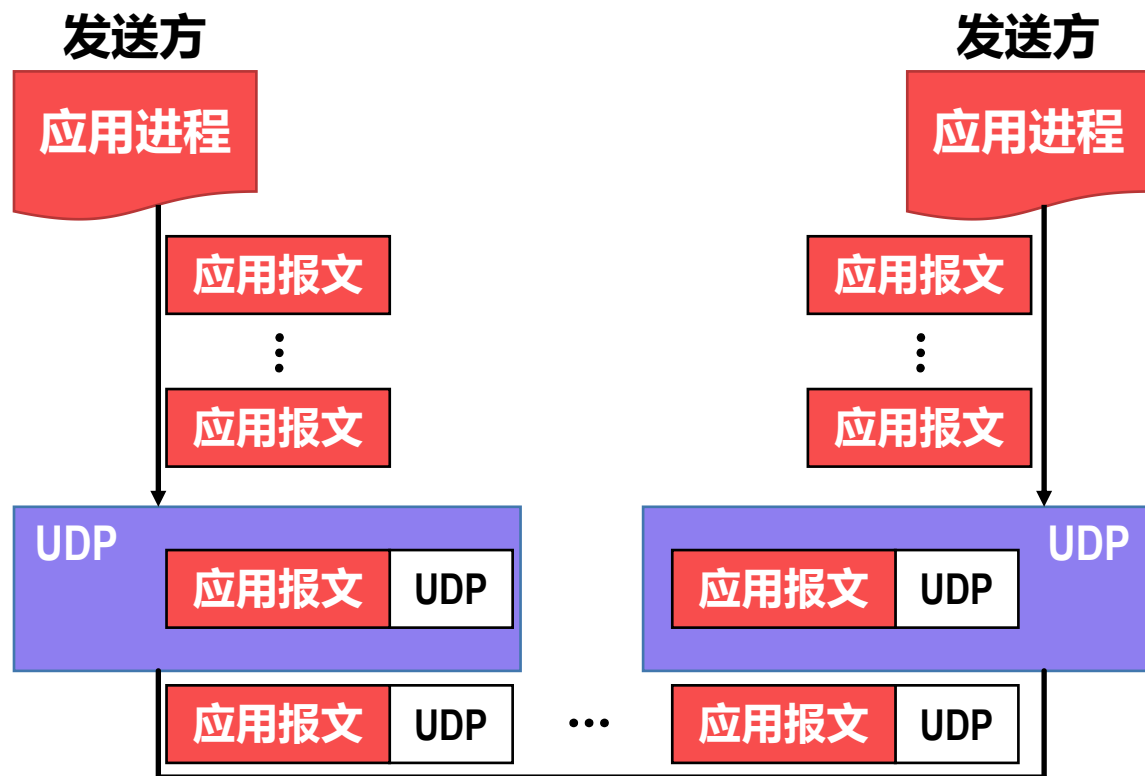
用户数据报协议UDP
(User Datagram Protocol)

传输控制协议TCP
(Transmission Control Protocol)



03 UDP和TCP应用层报文的处理

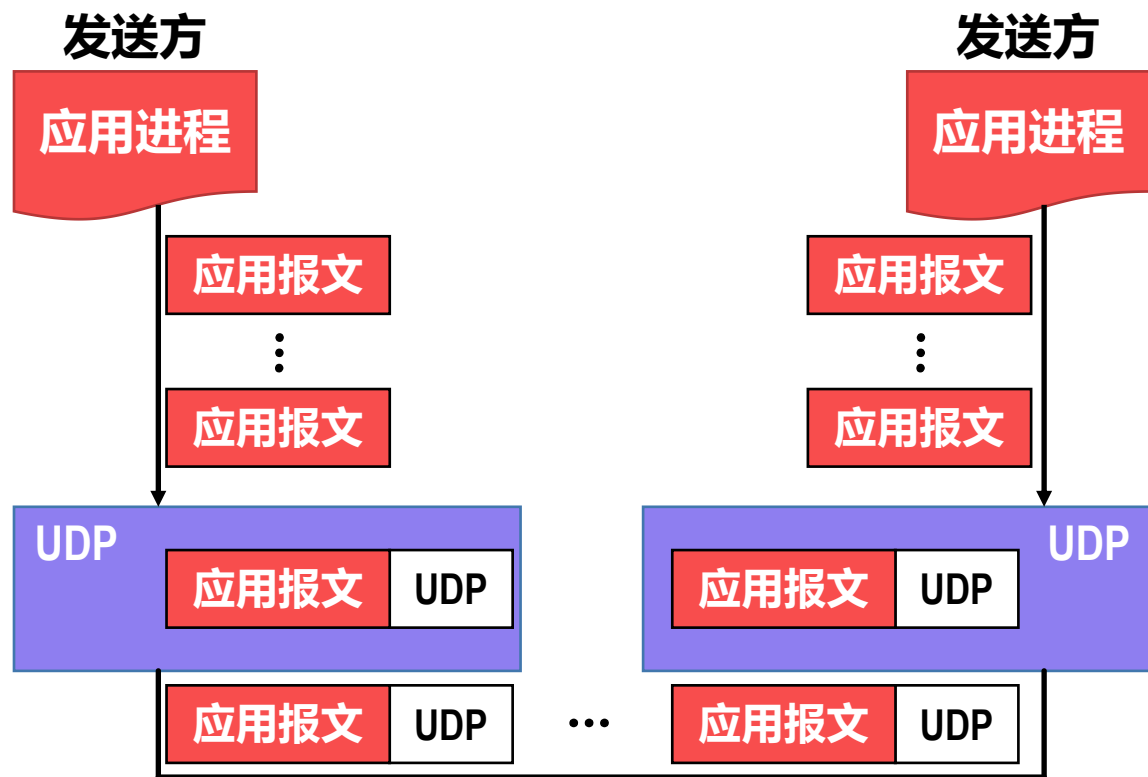
用户数据报协议UDP
(User Datagram Protocol)



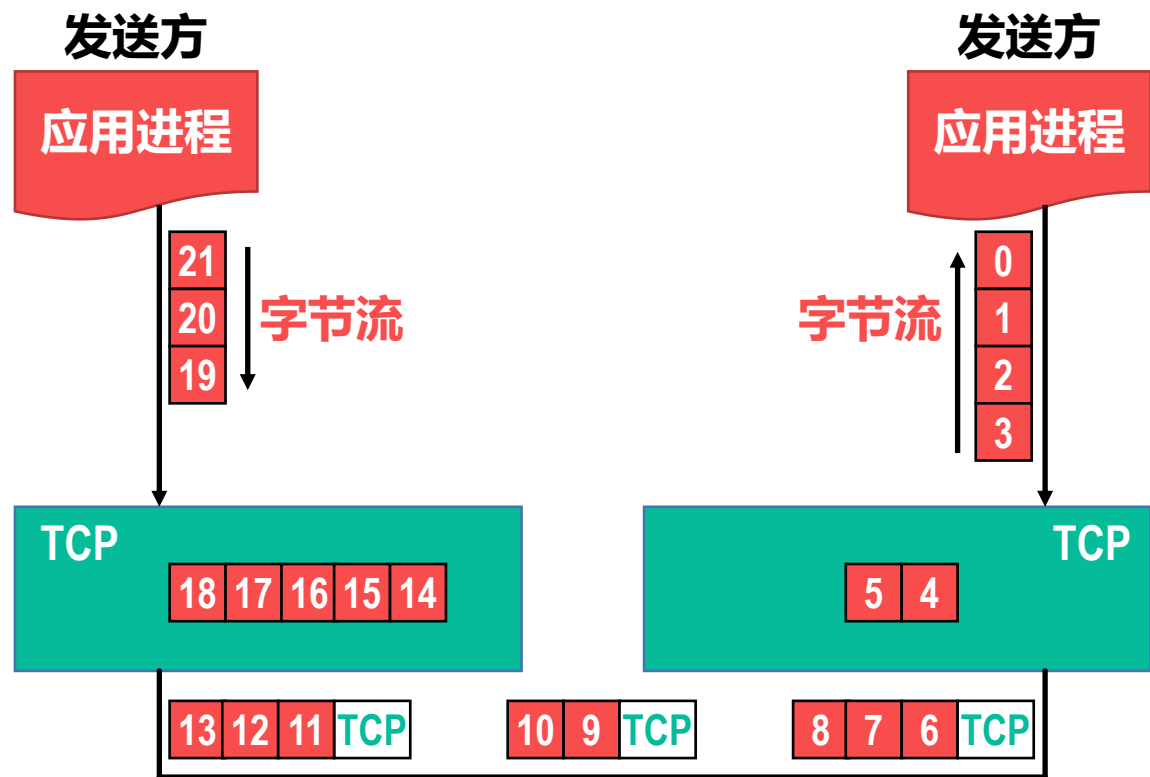
UDP是面向应用报文的

传输控制协议TCP
(Transmission Control Protocol)

03 UDP和TCP应用层报文的处理

用户数据报协议UDP
(User Datagram Protocol)

UDP是面向应用报文的

传输控制协议TCP
(Transmission Control Protocol)

TCP是面向字节流的

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)

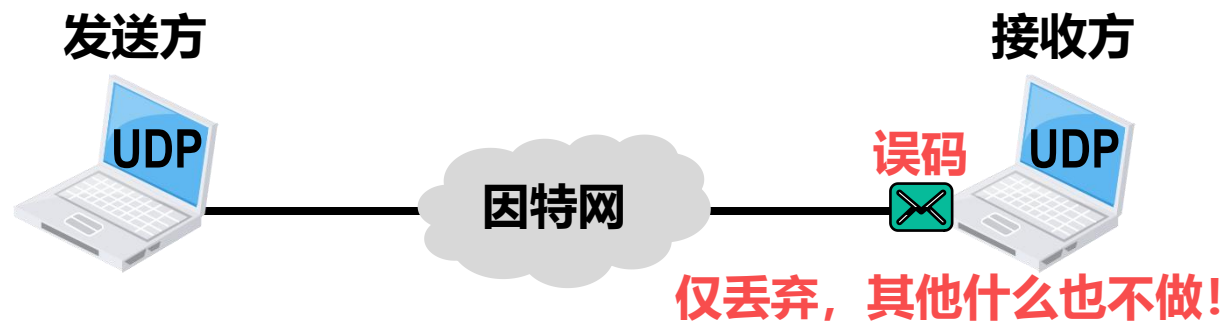


应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

传输控制协议TCP
(Transmission Control Protocol)

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)

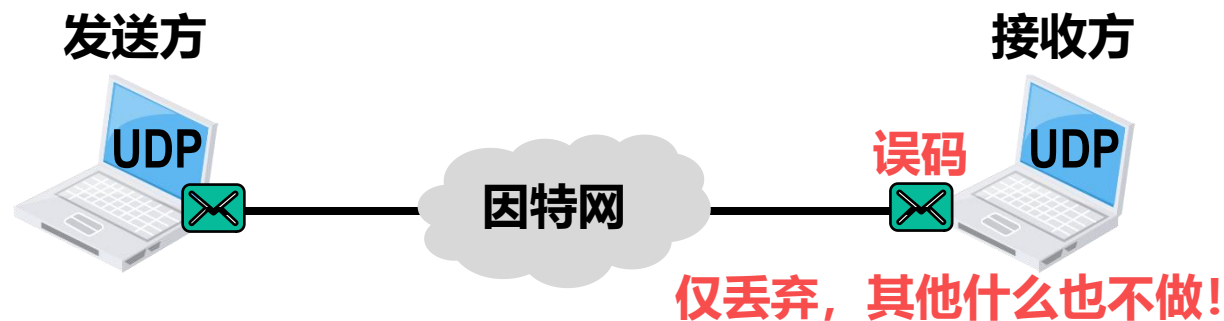


应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

传输控制协议TCP
(Transmission Control Protocol)

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)



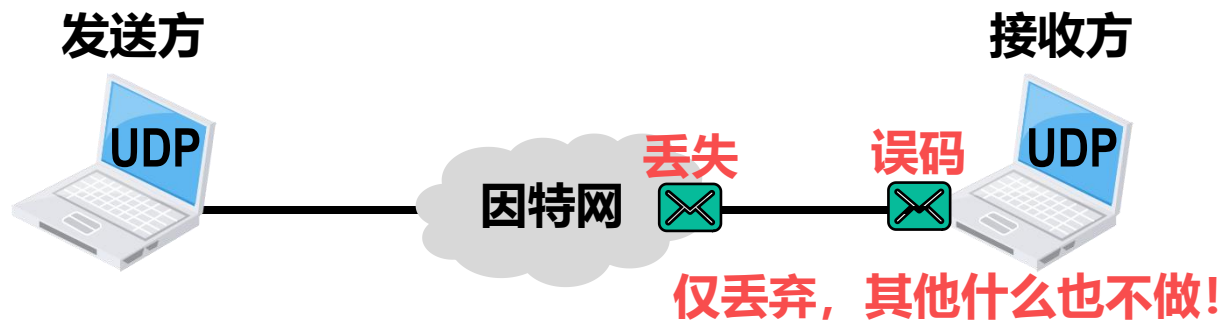
传输控制协议TCP
(Transmission Control Protocol)

应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)

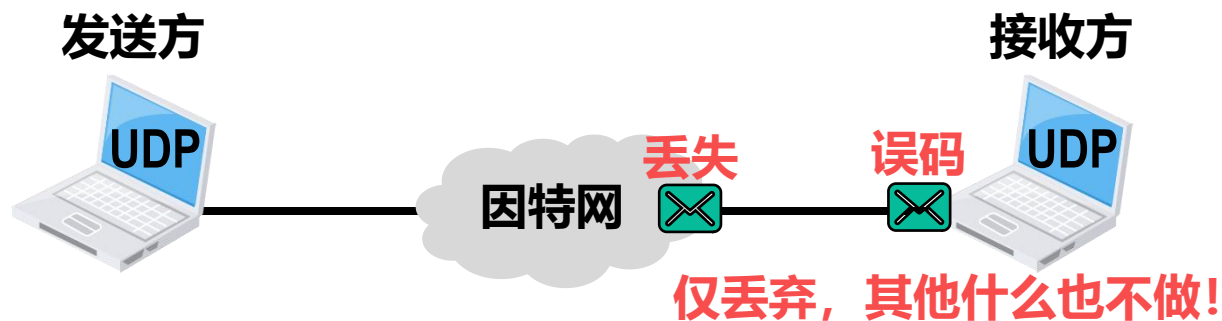
传输控制协议TCP
(Transmission Control Protocol)



应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

UDP向上层提供**无连接不可靠传输服务**
(适用于IP电话、视频会议等**实时应用**)

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)

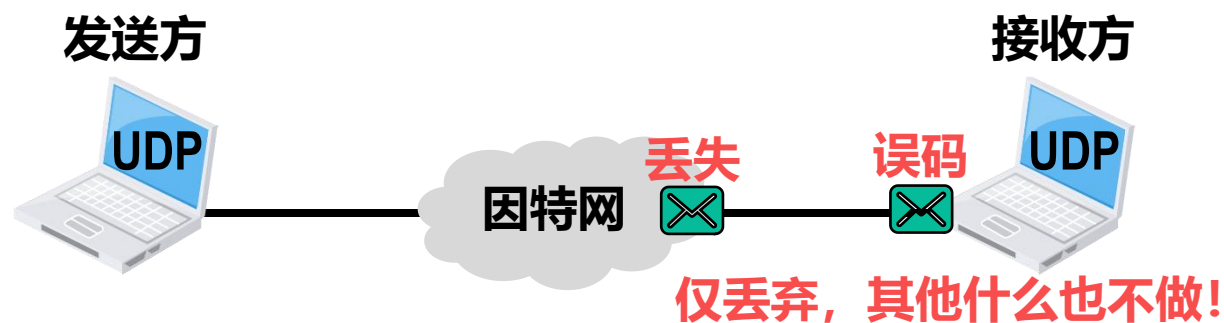
应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

UDP向上层提供**无连接不可靠传输服务**
(适用于IP电话、视频会议等**实时应用**)

传输控制协议TCP
(Transmission Control Protocol)

应用层		应用层
运输层		运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

04 UDP和TCP对数据传输可靠性的支持情况

用户数据报协议UDP
(User Datagram Protocol)

应用层		应用层
运输层	无连接不可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

UDP向上层提供**无连接不可靠传输服务**
(适用于IP电话、视频会议等**实时应用**)

传输控制协议TCP
(Transmission Control Protocol)

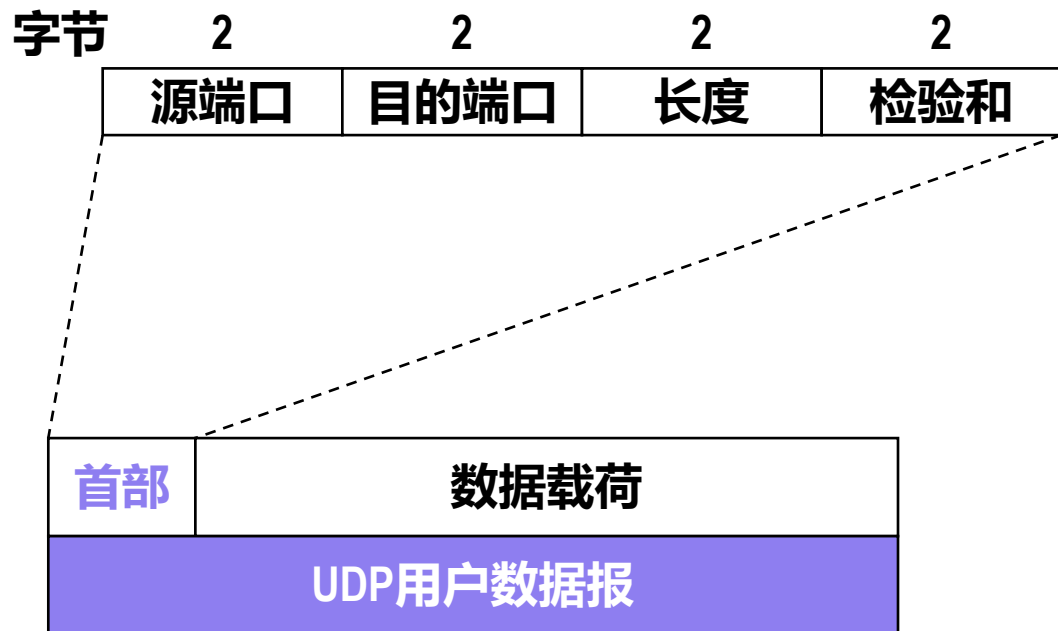
应用层		应用层
运输层	面向连接可靠传输服务	运输层
网际层	无连接不可靠传输服务	网际层
网络接口层		网络接口层

TCP向上层提供**面向连接的可靠传输服务**
(适用于**要求可靠传输的应用**，例如文件传输)

05 UDP首部和TCP首部的对比

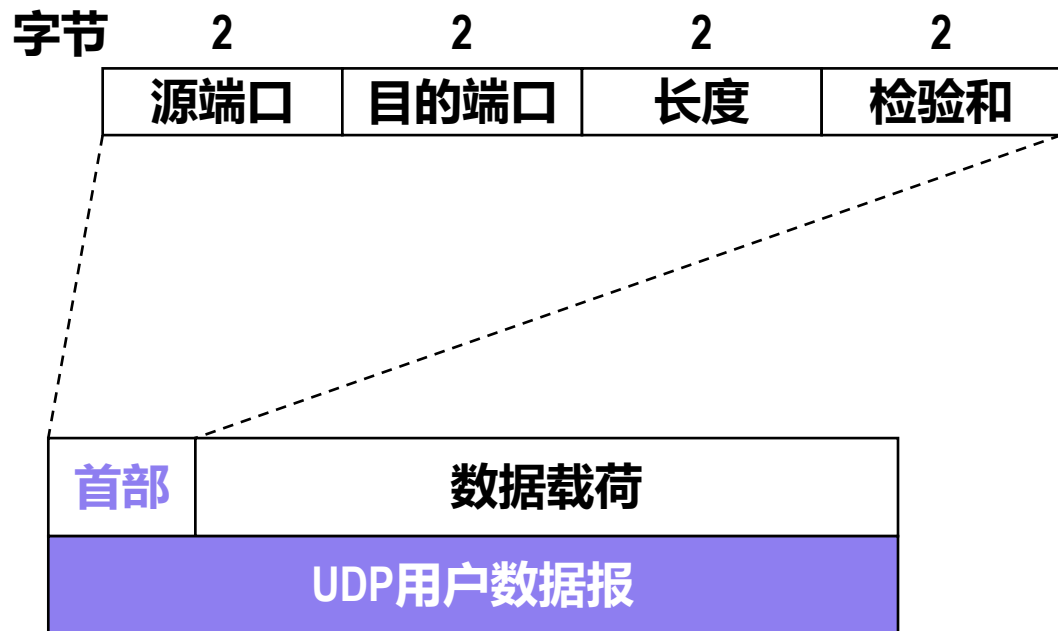
用户数据报协议UDP
(User Datagram Protocol)

传输控制协议TCP
(Transmission Control Protocol)

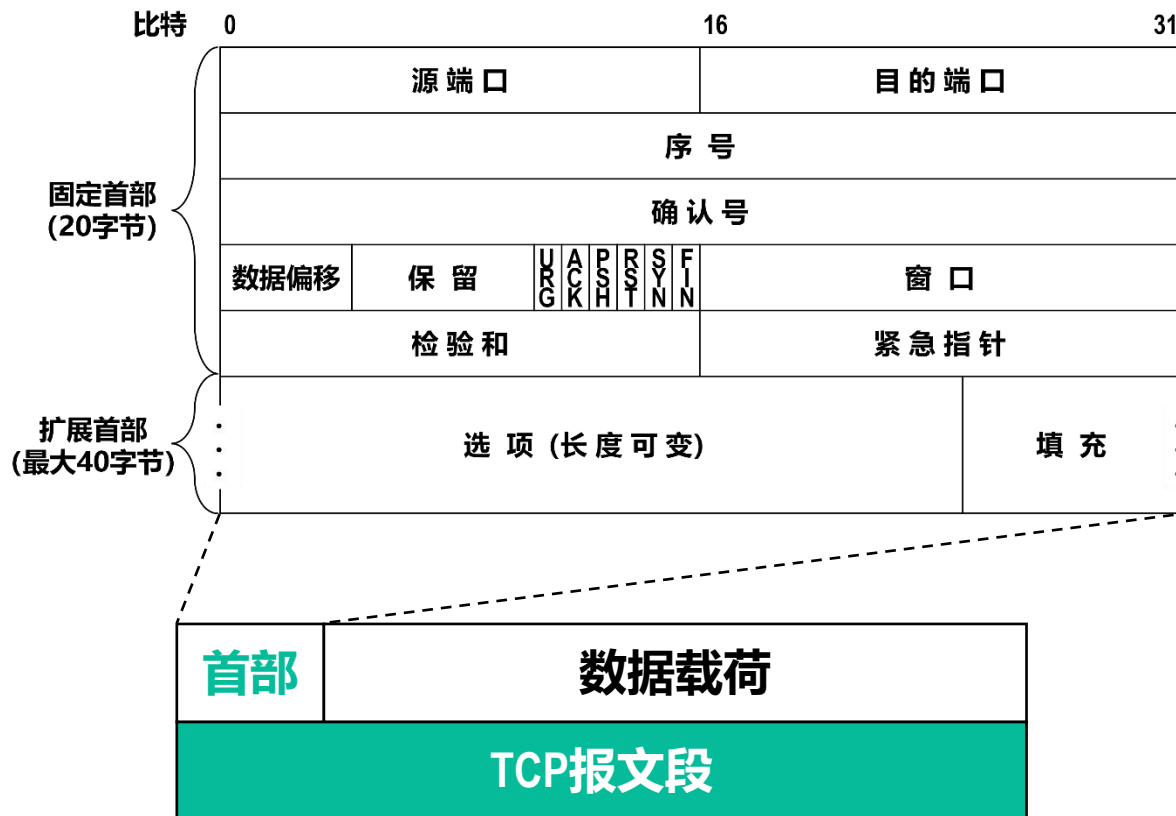


UDP用户数据报首部仅8字节

05 UDP首部和TCP首部的对比

用户数据报协议UDP
(User Datagram Protocol)

UDP用户数据报首部仅8字节

传输控制协议TCP
(Transmission Control Protocol)

TCP报文段首部最小20字节, 最大60字节

用户数据报协议UDP (User Datagram Protocol)

- ☐ 无连接
- ☐ 支持“一对一”、“一对多”、“多对一”和“多对多”交互通信。
- ☐ 面向应用报文
- ☐ 尽最大努力交付，即不可靠；不使用流量控制和拥塞控制。
- ☐ 首部开销小，仅8字节。

传输控制协议TCP (Transmission Control Protocol)

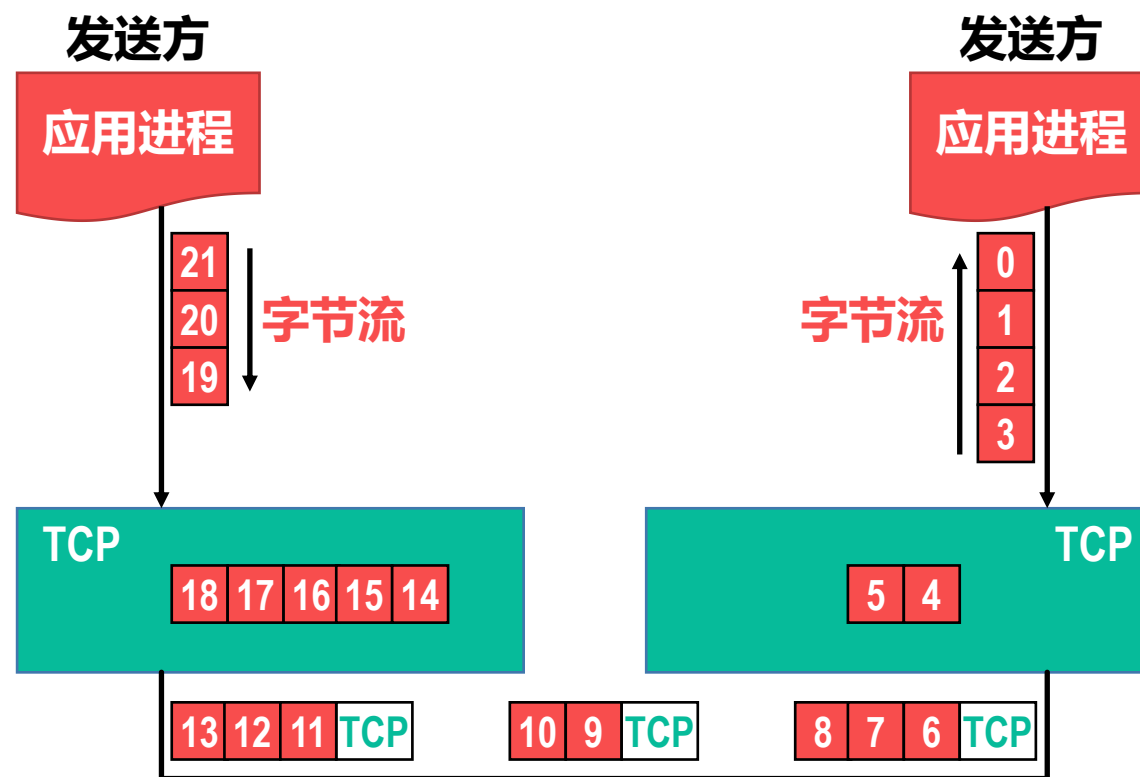
- ☐ 面向连接
- ☐ 每一条TCP连接只能有两个端点EP，只能是一对一通信。
- ☐ 面向字节流
- ☐ 可靠传输，使用流量控制和拥塞控制。
- ☐ 首部最小20字节，最大60字节。

5.3.1 TCP报文段的首部格式

01

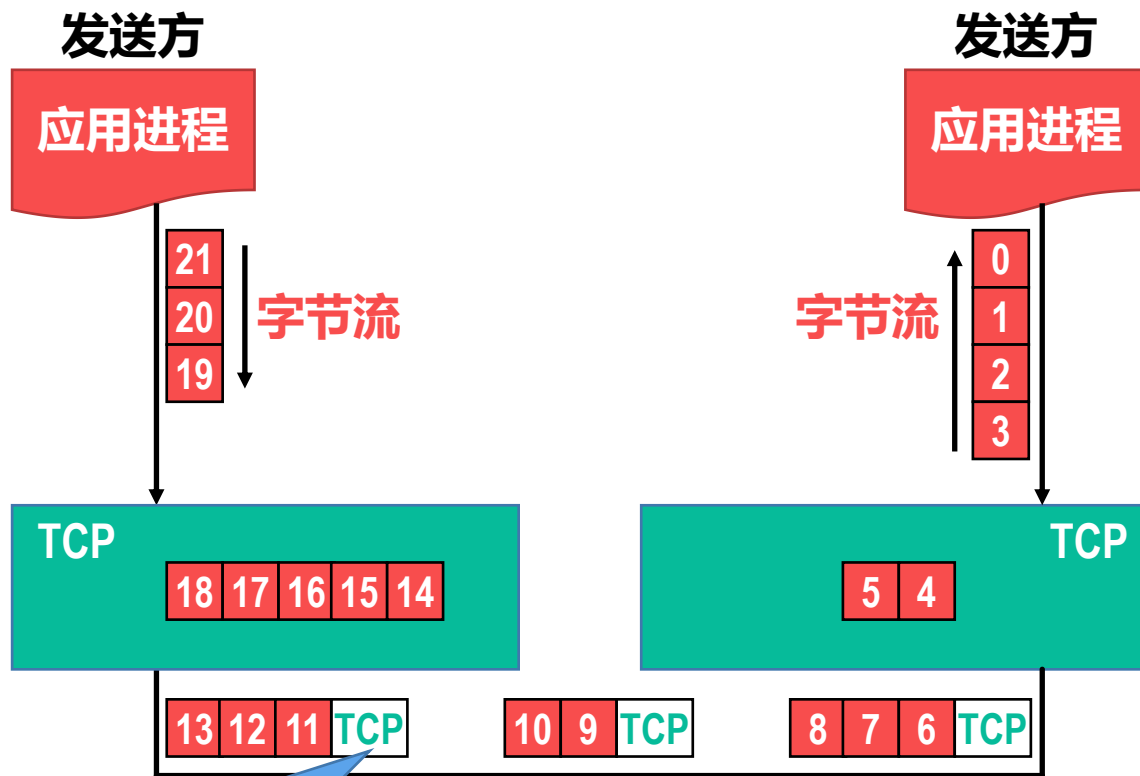
TCP报文段的首部格式

01 TCP报文段的首部格式



TCP是面向字节流的

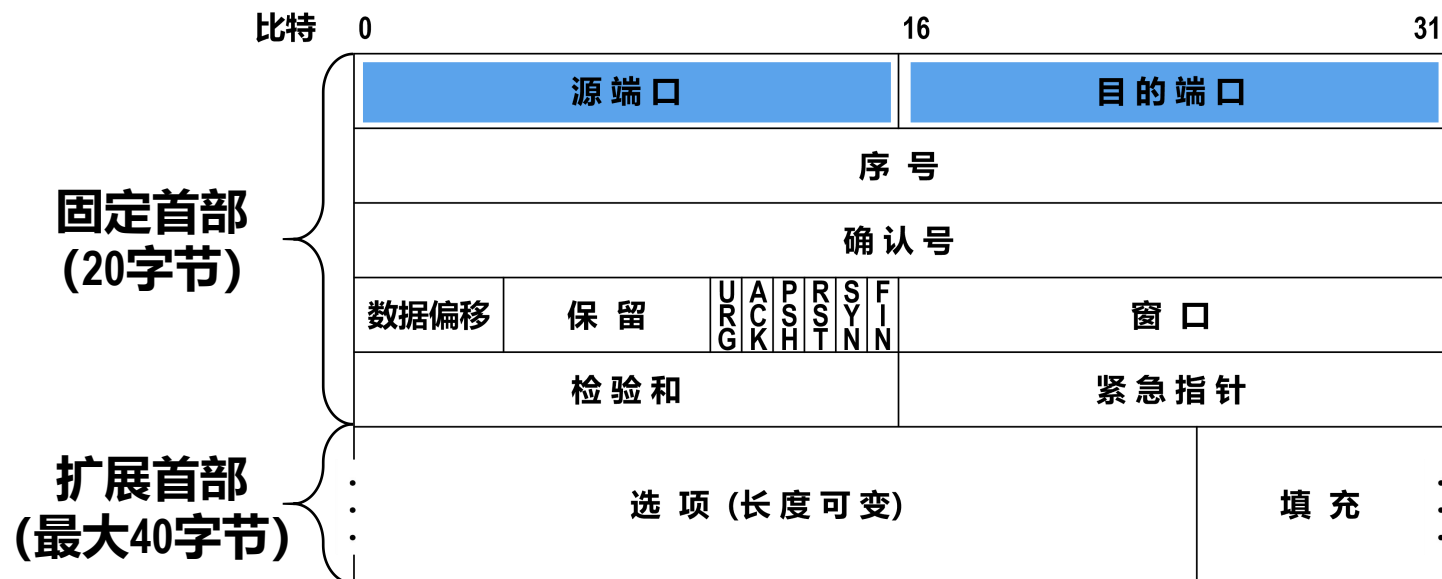
01 TCP报文段的首部格式



TCP的全部功能需要依靠其首部中的各字段来实现。

TCP是面向字节流的

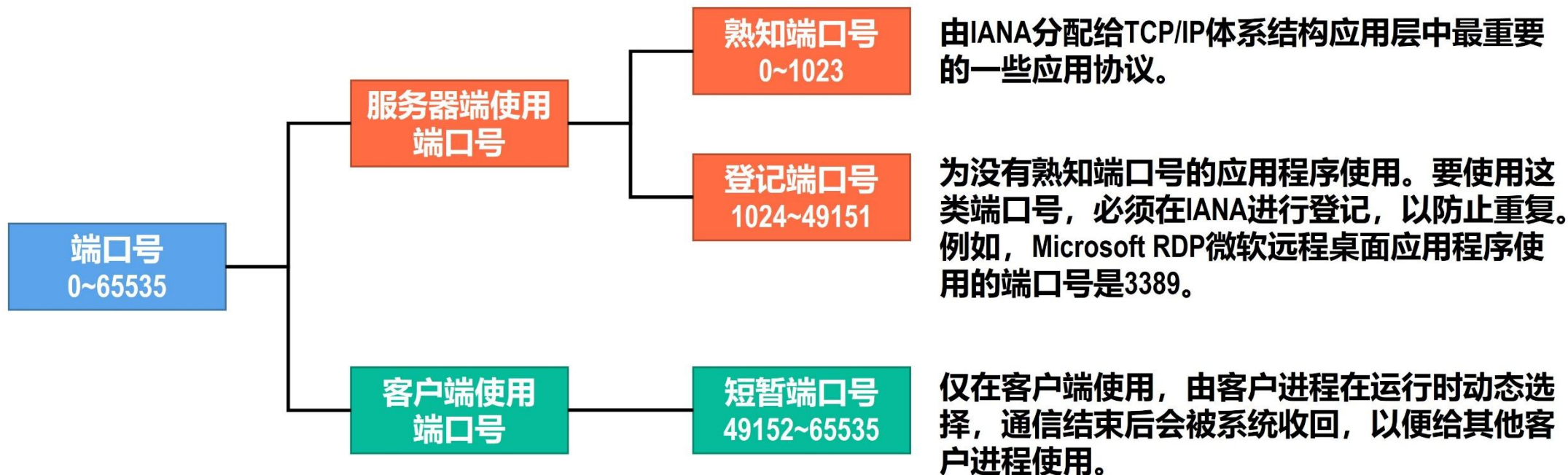
01 TCP报文段的首部格式



01 TCP报文段的首部格式

5.1.3 运输层端口号、复用与分用的概念

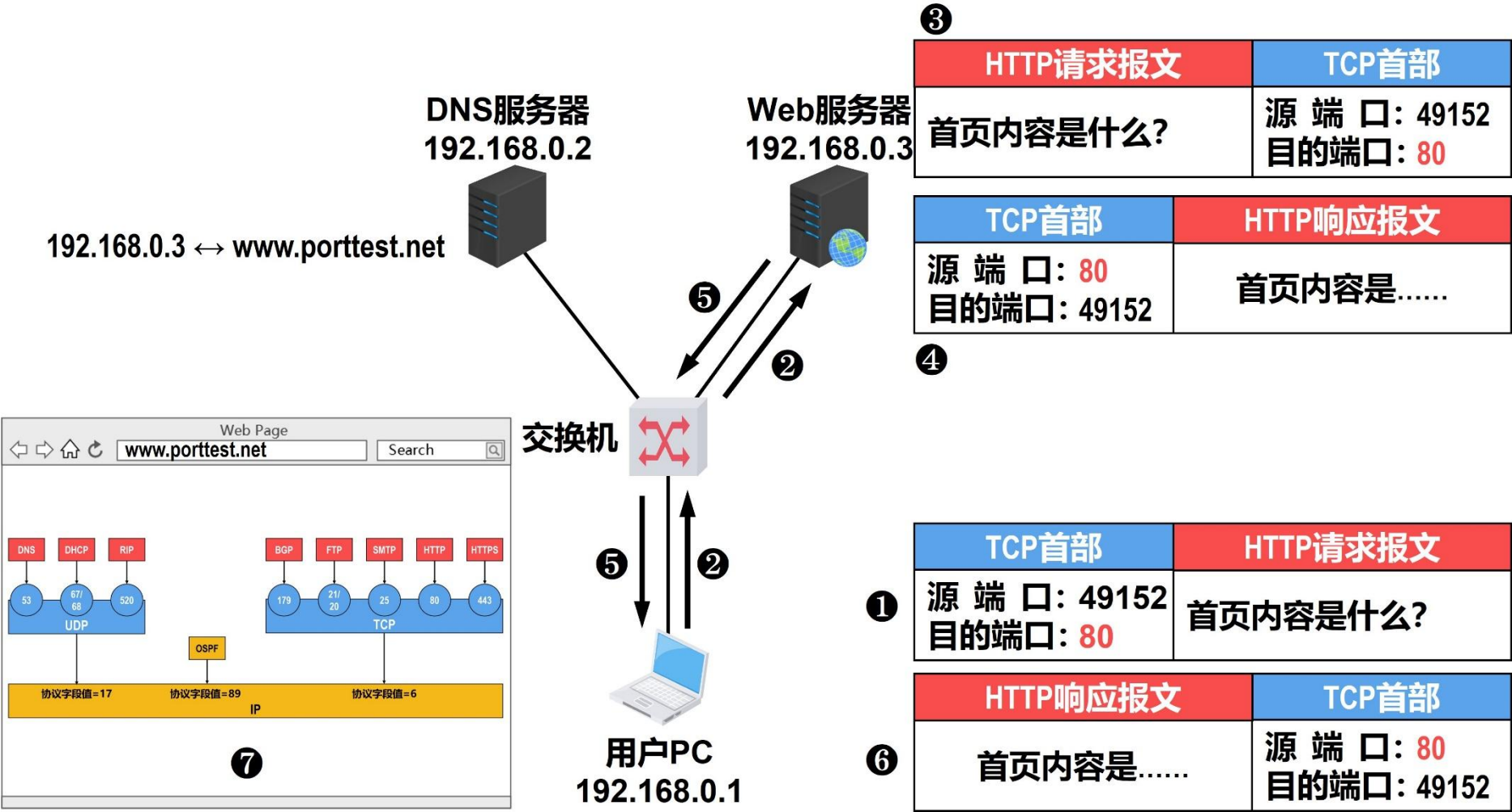
FTP	SMTP	DNS	DHCP	HTTP	BGP	HTTPS	RIP
21/20	25	53	67/68	80	179	443	520



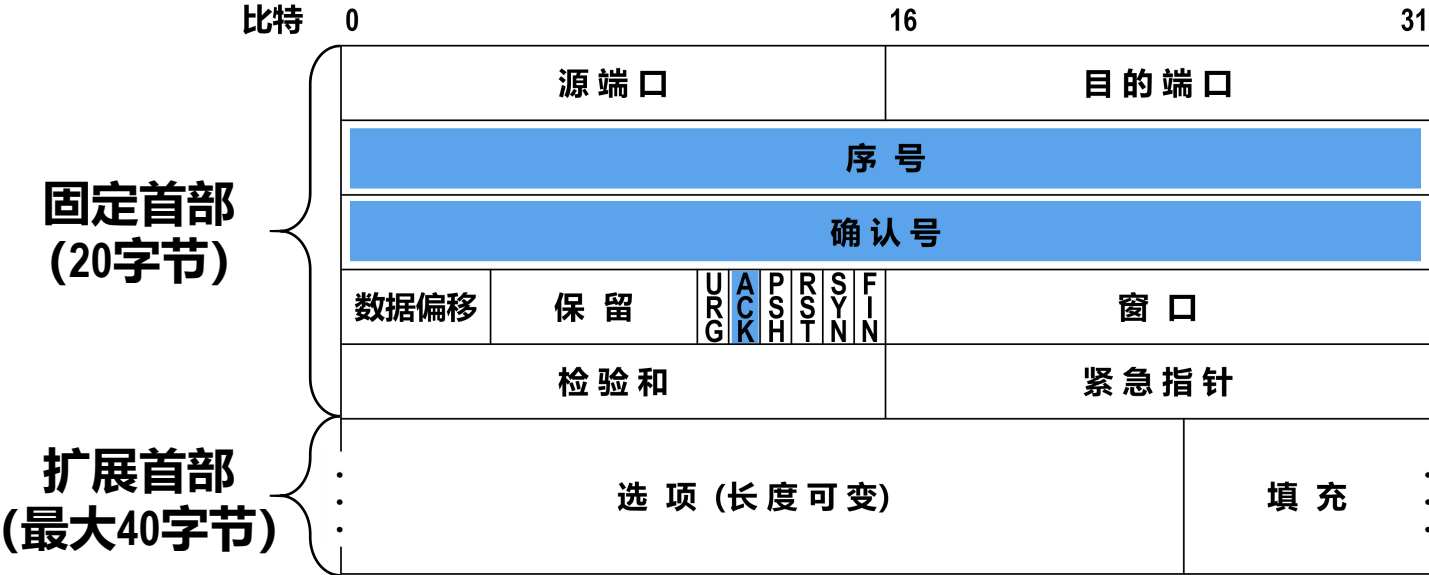
端口号只具有本地意义，即端口号只是为了标识本计算机网络协议栈应用层中的各应用进程。在因特网中，不同计算机中的相同端口号是没有关系的，即相互独立。另外，TCP和UDP端口号之间也是没有关系的。

01 TCP报文段的首部格式

5.1.3 运输层端口号、复用与分用的概念



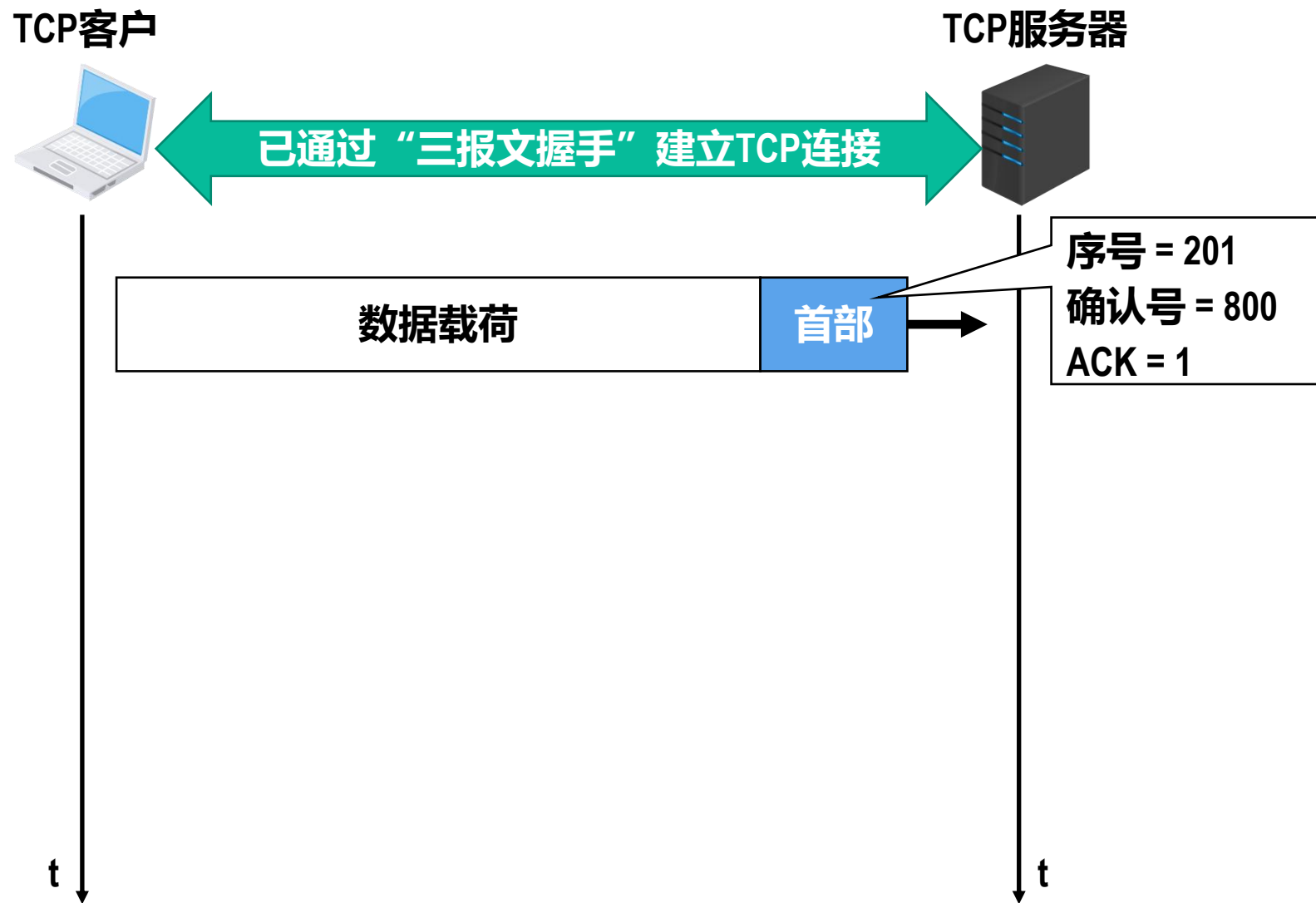
01 TCP报文段的首部格式



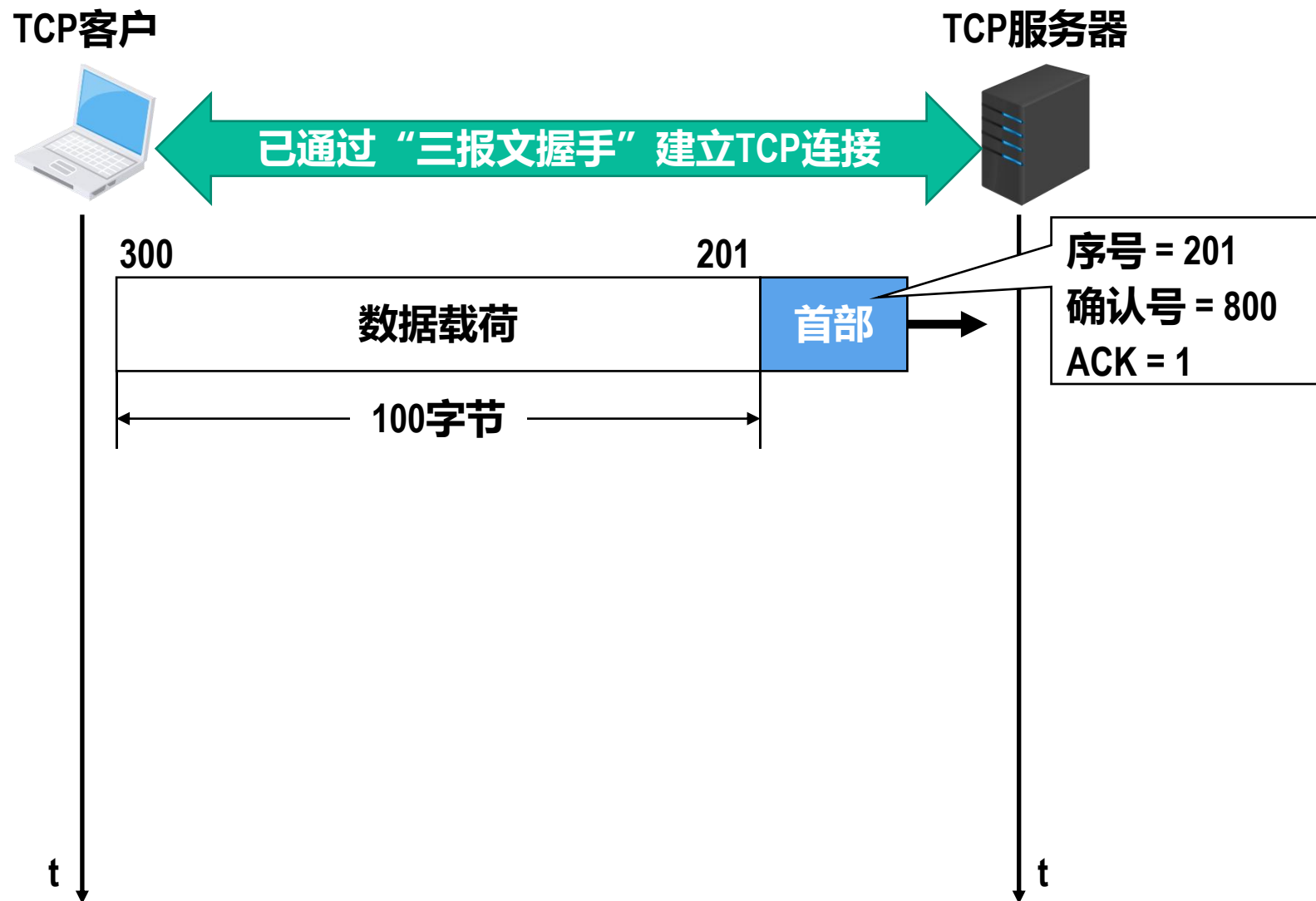
TCP实现可靠传输的
相关字段

- 序号** 占32比特，取值范围0~2³²-1。当序号增加到最后一个时，下一个序号又回到0。用来指出本TCP报文段数据载荷的第一个字节的序号。
- 确认号** 占32比特，取值范围0~2³²-1。当确认号增加到最后一个时，下一个确认号又回到0。用来指出期望收到对方下一个TCP报文段的数据载荷的第一个字节的序号，同时也是对之前收到的所有数据的确认。
- 确认标志位ACK** 只有当ACK取值为1时，确认号字段才有效。ACK取值为0时，确认号字段无效。TCP规定：在TCP连接建立后，所有传送的TCP报文段都必须把ACK置1。

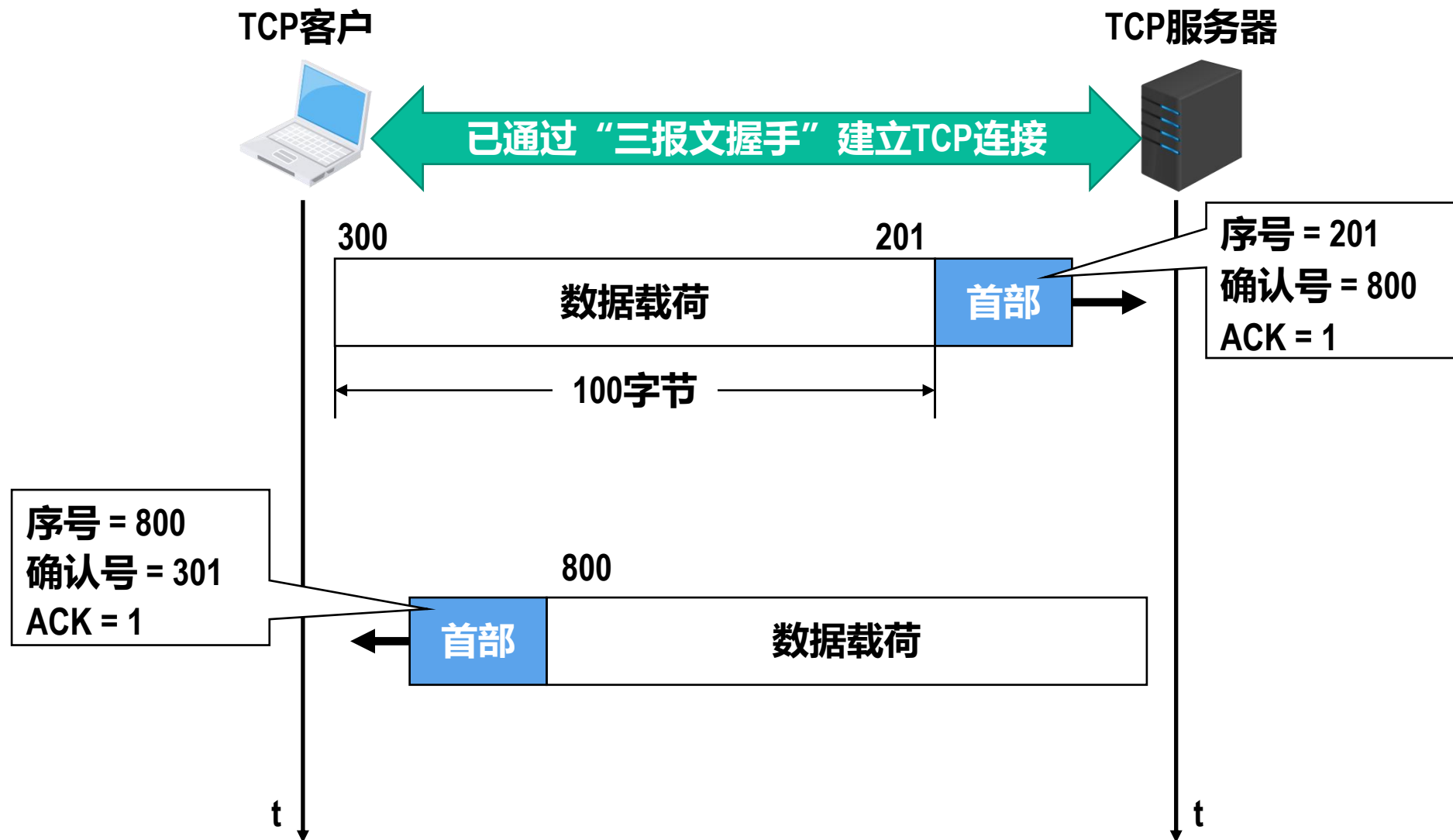
01 TCP报文段的首部格式



01 TCP报文段的首部格式



01 TCP报文段的首部格式



01 TCP报文段的首部格式

【2009年 题38】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了两个连续的TCP段，分别包含300字节和500字节的有效载荷，第一个段的序列号为200，主机乙正确接收到两个段后，发送给主机甲的确认序列号是（**D**）。

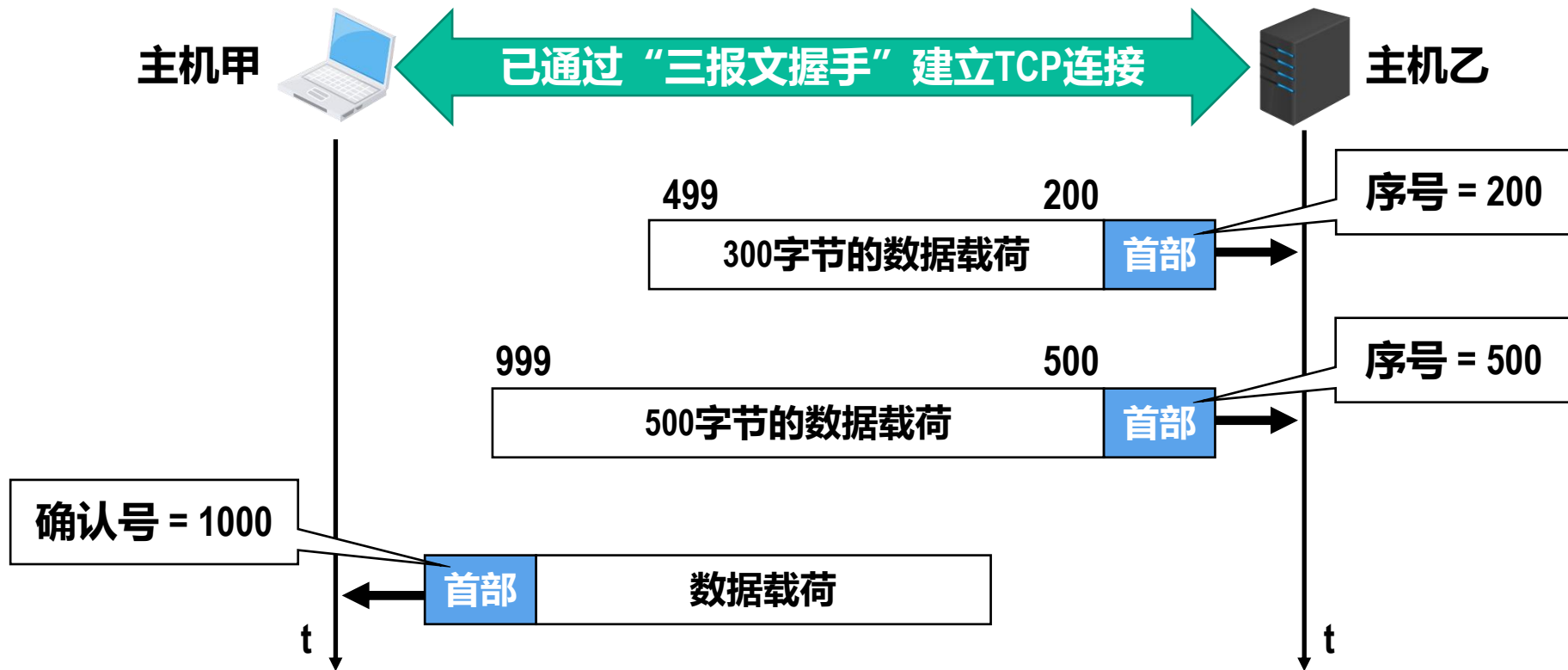
A. 500

B. 700

C. 800

D. 1000

解析



01 TCP报文段的首部格式

【2013年 题39】主机甲与主机乙之间已建立一个TCP连接，双方持续有数据传输，且数据无差错与丢失。若甲收到1个来自乙的TCP段，该段的序号是1913、确认序号为2046、有效载荷为100字节，则甲立即发送给乙的TCP段的序号和确认序号分别是（**B**）。

A. 2046、2012

B. 2046、2013

C. 2047、2012

D. 2047、2013

解析



01 TCP报文段的首部格式

【2013年 题39】主机甲与主机乙之间已建立一个TCP连接，双方持续有数据传输，且数据无差错与丢失。若甲收到1个来自乙的TCP段，该段的序号是1913、确认序号为2046、有效载荷为100字节，则甲立即发送给乙的TCP段的序号和确认序号分别是（**B**）。

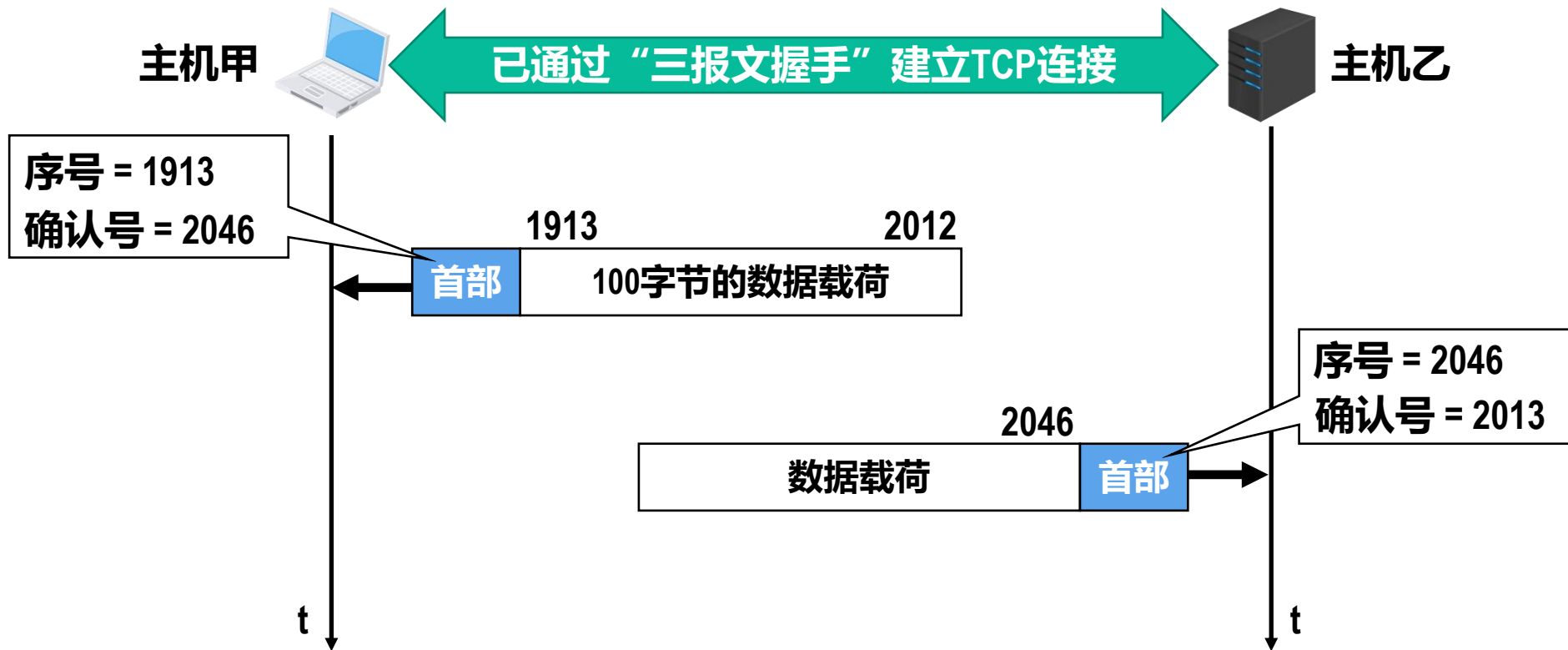
A. 2046、2012

B. 2046、2013

C. 2047、2012

D. 2047、2013

解析



01 TCP报文段的首部格式

【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1和第3个段，则主机乙发送给主机甲的确认序号是（**B**）。

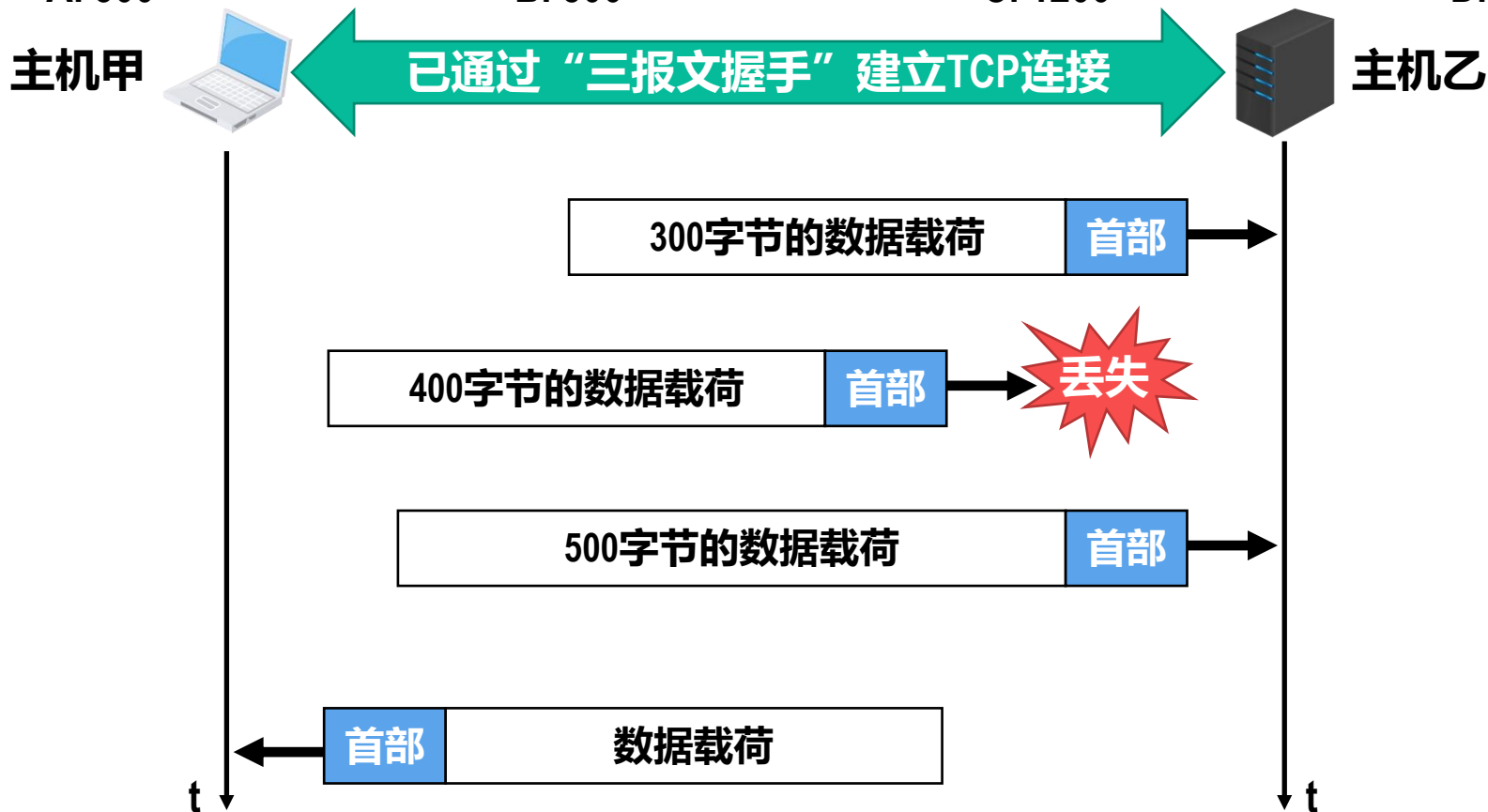
A. 300

B. 500

C. 1200

D. 1400

解析



01 TCP报文段的首部格式

【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1和第3个段，则主机乙发送给主机甲的确认序号是（**B**）。

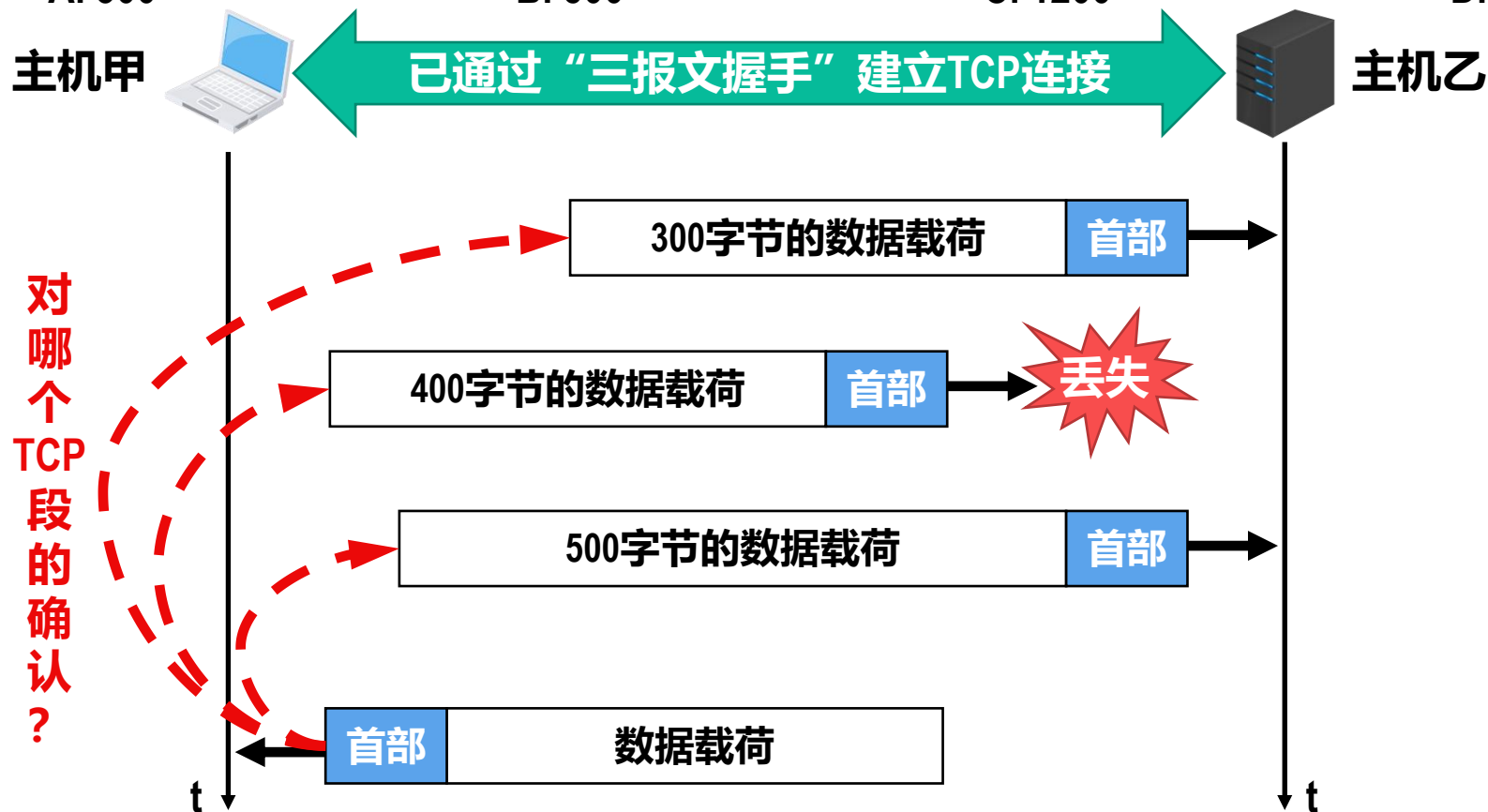
A. 300

B. 500

C. 1200

D. 1400

解析



01 TCP报文段的首部格式

【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1和第3个段，则主机乙发送给主机甲的确认序号是（**B**）。

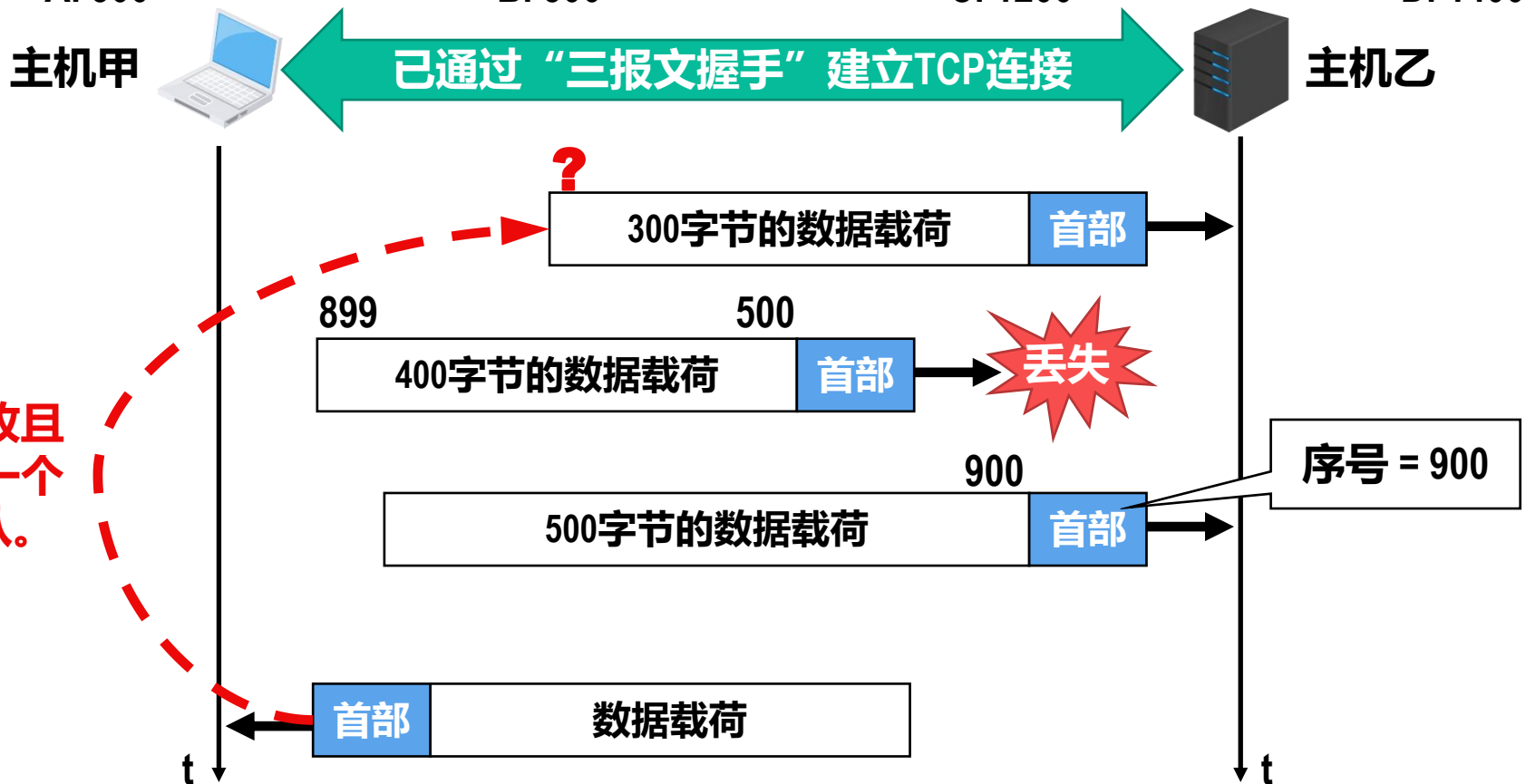
A. 300

B. 500

C. 1200

D. 1400

解析



这是对已正确接收且按序到达的最后一个TCP报文段的确认。

01 TCP报文段的首部格式

【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1和第3个段，则主机乙发送给主机甲的确认序号是（**B**）。

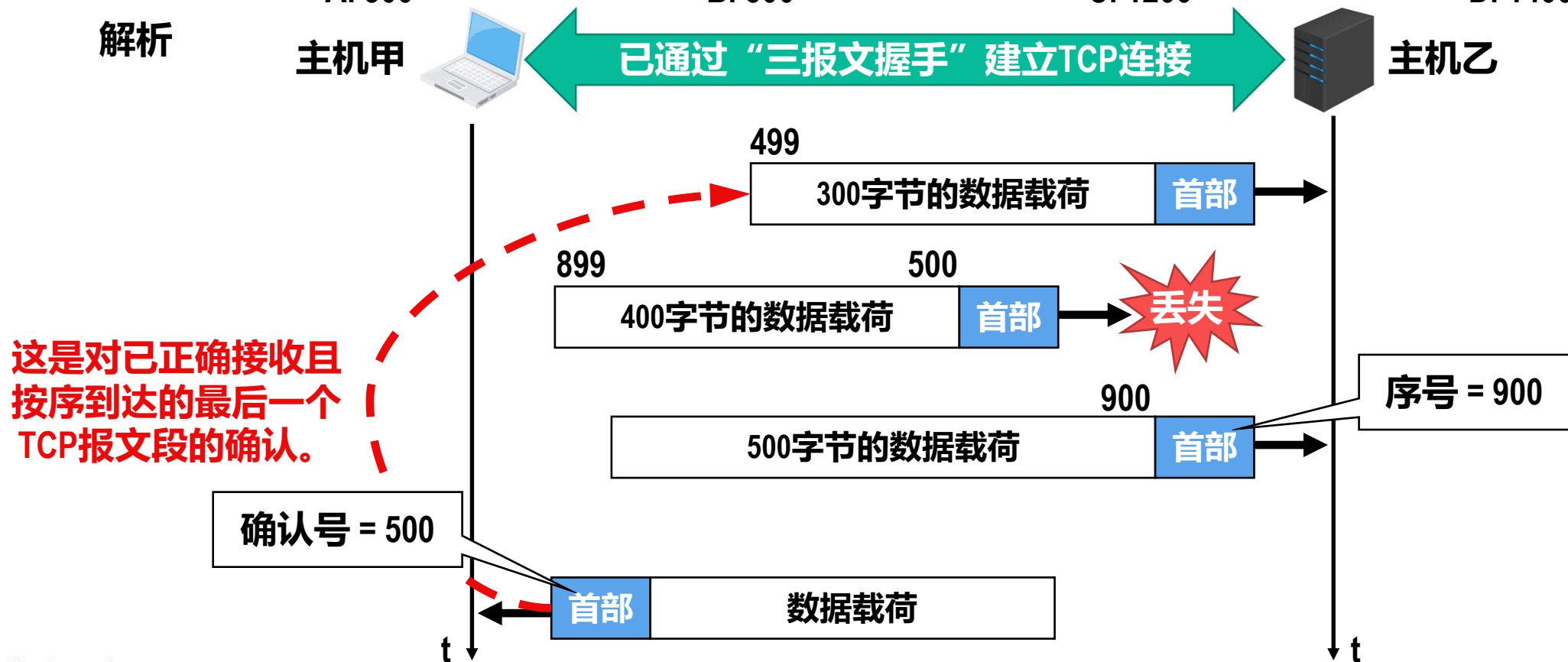
A. 300

B. 500

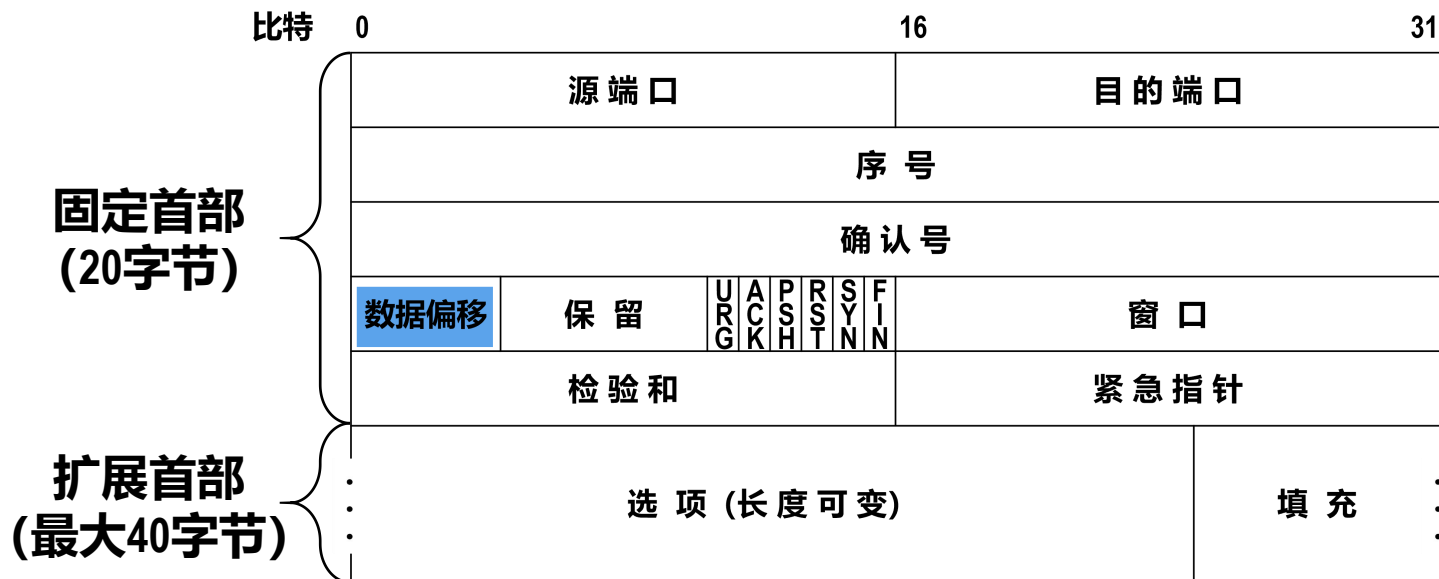
C. 1200

D. 1400

解析



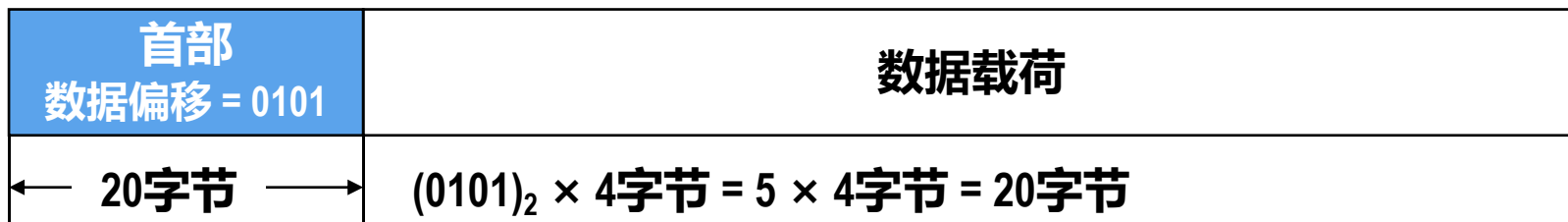
01 TCP报文段的首部格式



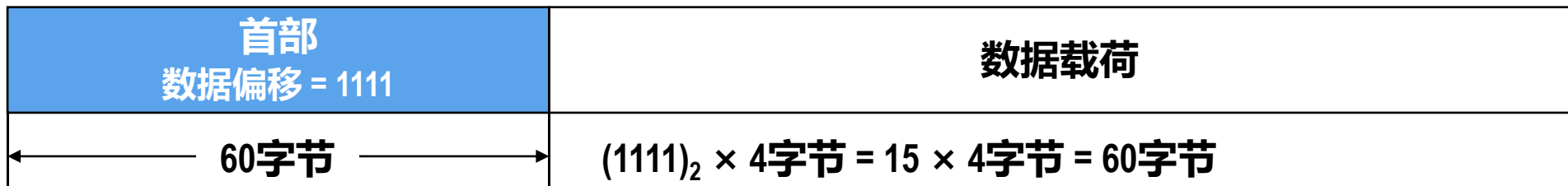
数据偏移

- 占4比特，该字段的取值以4字节为单位。
- 指出TCP报文段的数据载荷部分的起始处距离TCP报文段的起始处有多远，这实际上指出了TCP报文段的首部长度的。

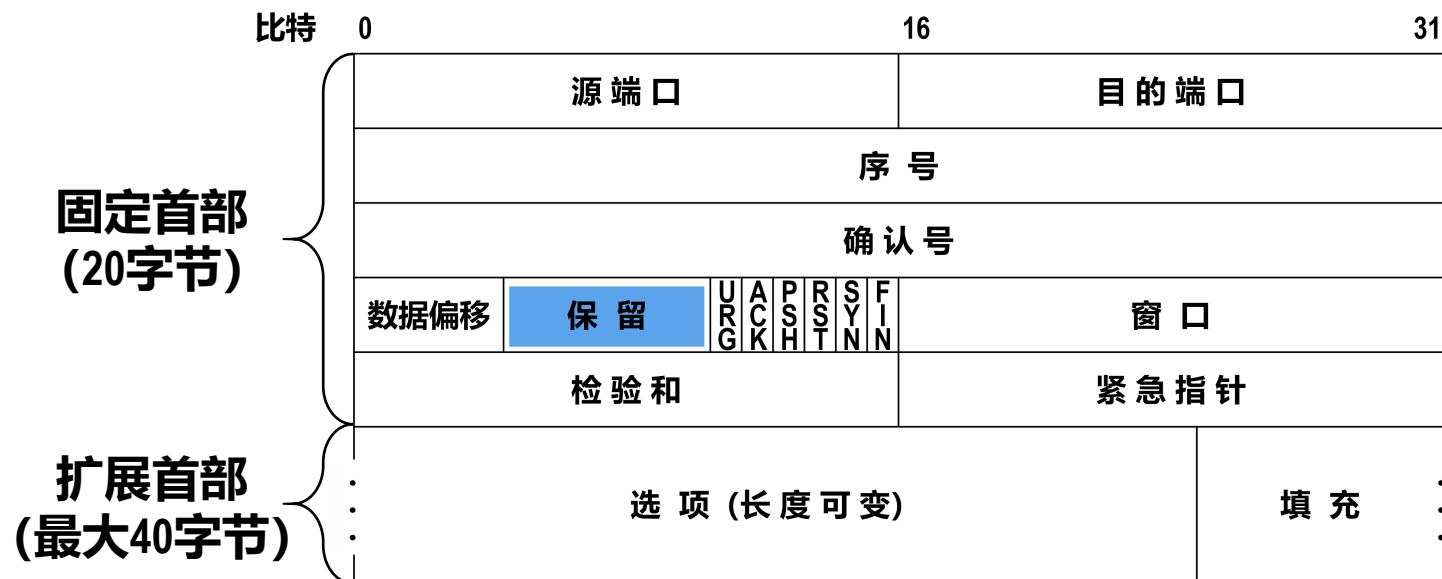
某个TCP报文段



某个TCP报文段



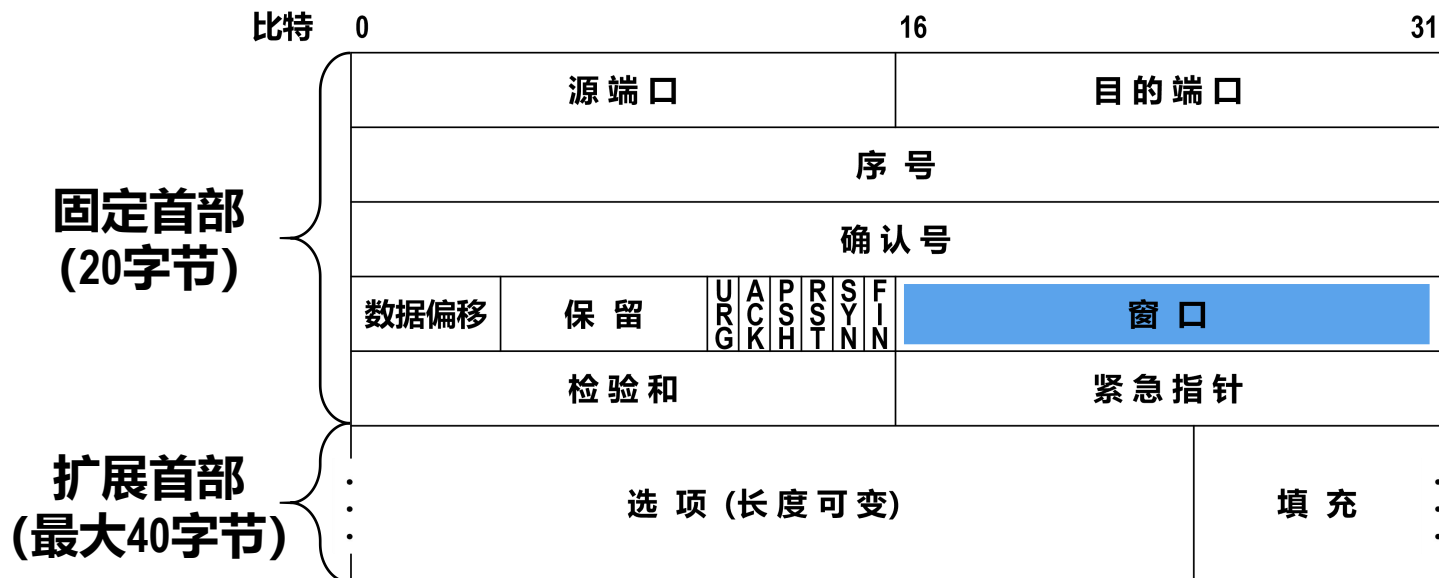
01 TCP报文段的首部格式



保留

- ☐ 占6比特
- ☐ 保留为今后使用
- ☐ 目前应置为0

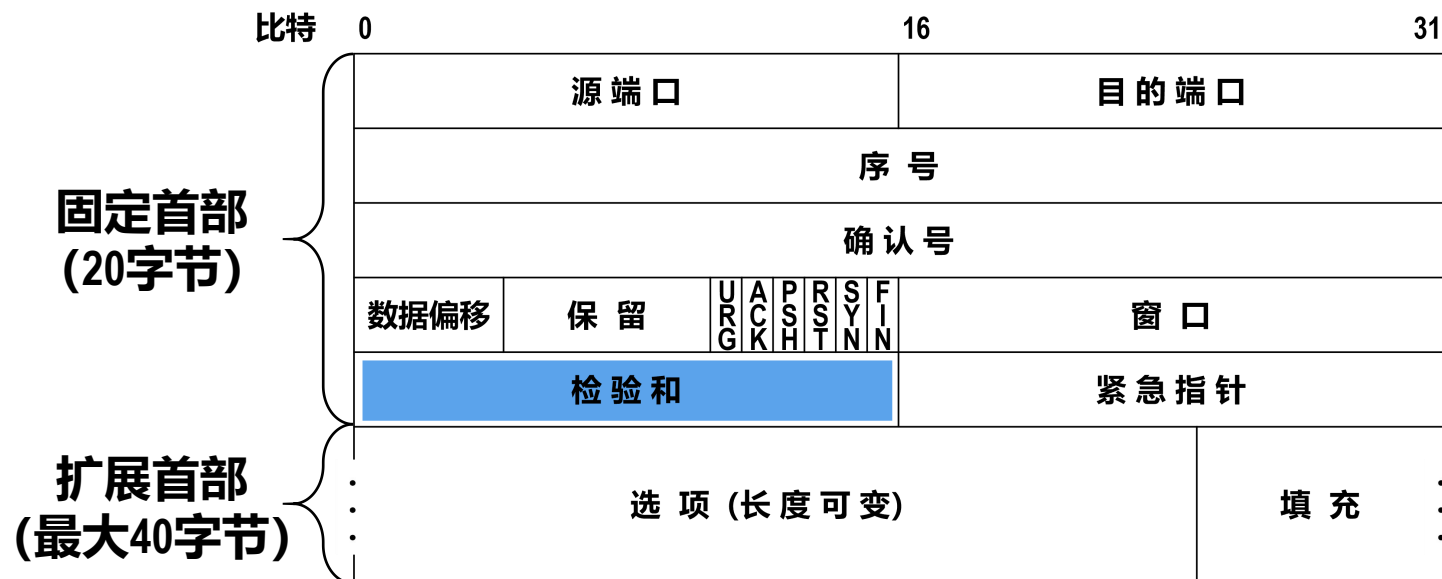
01 TCP报文段的首部格式



窗口

- 占16比特，该字段的取值以字节为单位。
- 指出发送本报文段的一方的接收窗口的大小，即接收缓存的可用空间大小，这用来表征接收方的接收能力。
- 在计算机网络中，经常用接收方的接收能力的大小来控制发送方的数据发送量，这就是所谓的流量控制。

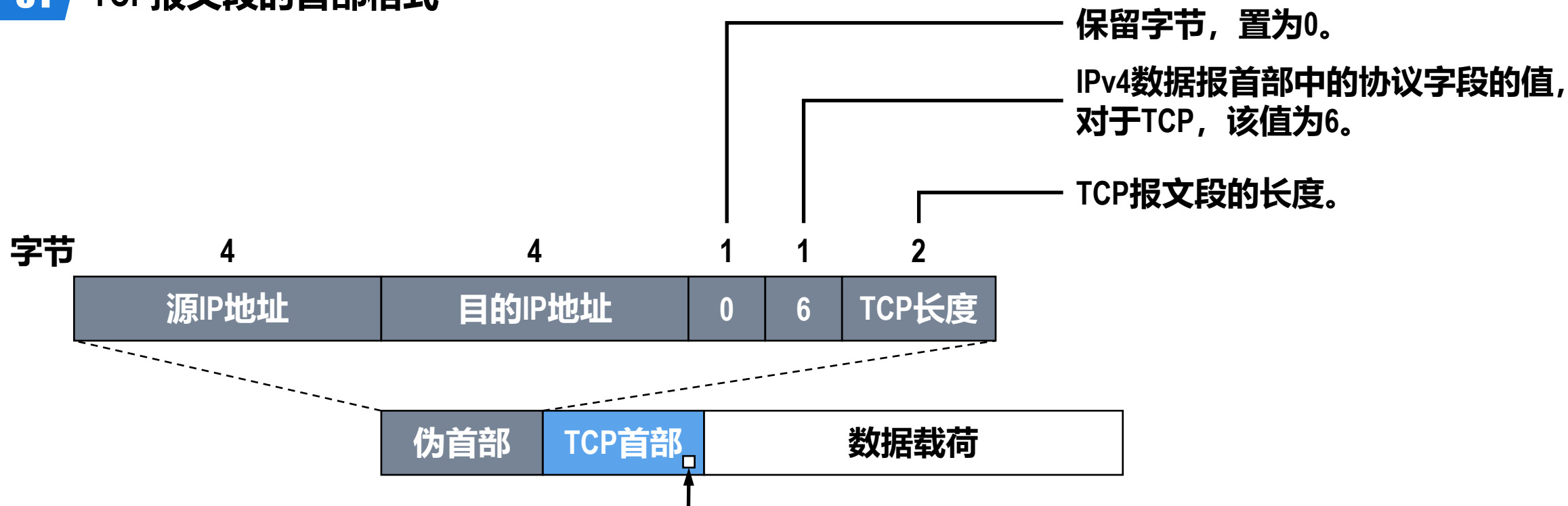
01 TCP报文段的首部格式



检验和

- 占16比特
- 用来检查整个TCP报文段在传输过程中是否出现了误码。

01 TCP报文段的首部格式

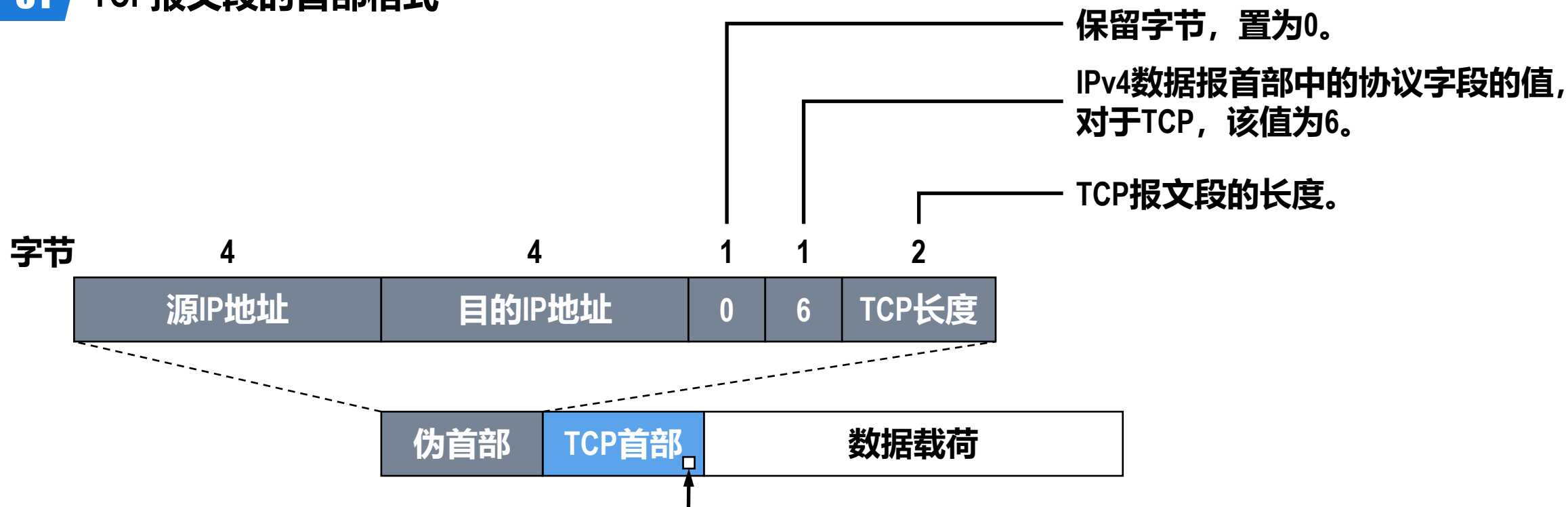


发送方检验和计算方法

- ① 将TCP首部中检验和字段的值置为0。
- ② 将伪首部、TCP首部以及数据载荷这三部分划分成若干个2字节的字。若伪首部、TCP首部以及数据载荷这三部分的总长度不是偶数个字节, 则在最后添加1个“全0”字节。
- ③ 对划分出的全部2字节的字进行反码算数运算求和, 并将求和结果取反码。
- ④ 将步骤③得到的结果写入TCP首部中的检验和字段。

与IPv4数据报首部检验和的计算方法 (4.2.7节) 类似, 差别仅在于前者只对首部进行检验。

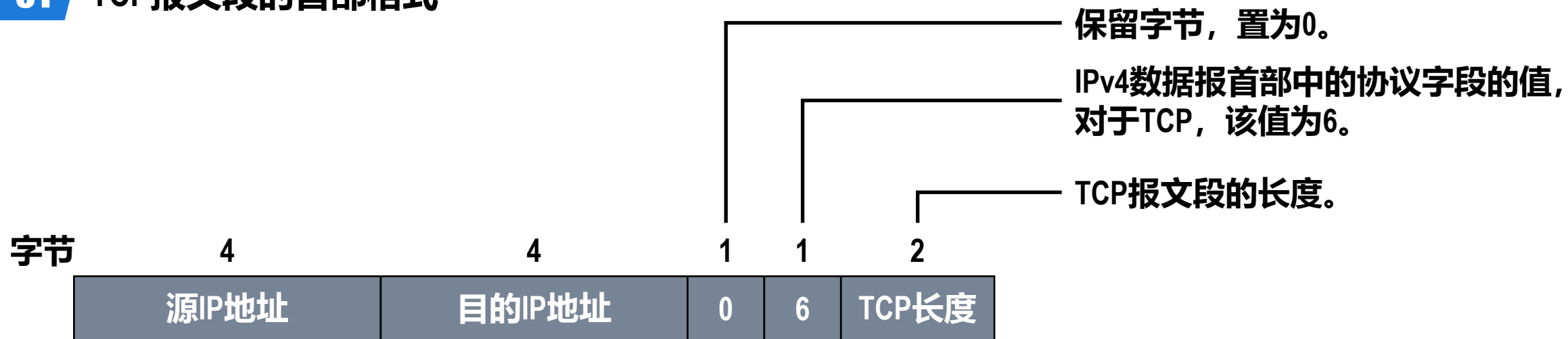
01 TCP报文段的首部格式



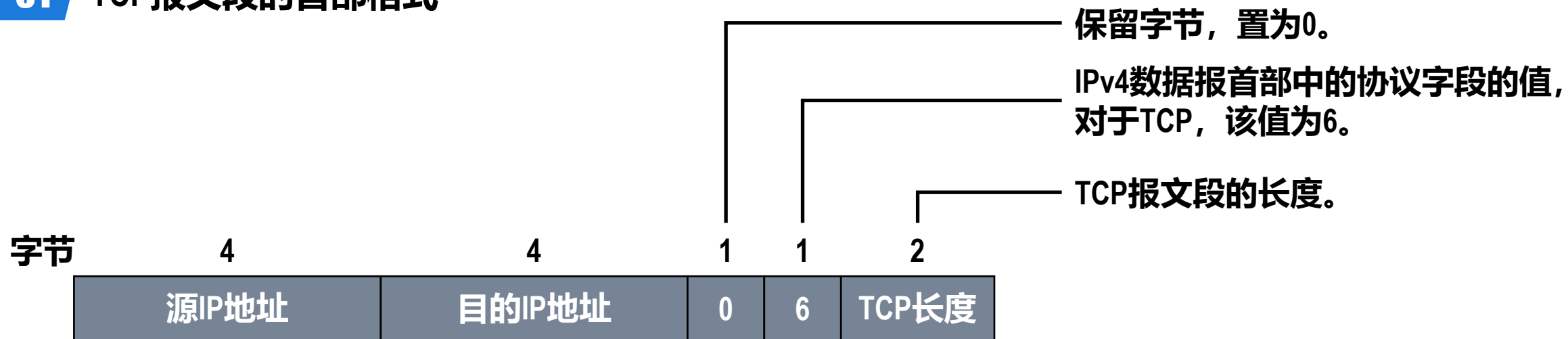
接收方通过检验和字段的值判断TCP报文段在传输过程中是否误码的方法

- ① 给接收到的TCP报文段前面添加一个12字节的伪首部。
- ② 将伪首部、TCP首部以及数据载荷这三部分划分成若干个2字节的字。
- ③ 对划分出的全部2字节的字进行反码算数运算求和，并将求和结果取反码。
- ④ 若步骤③得到的结果为0，则表示TCP报文段在传输过程中没有误码。否则，表示出现了误码。

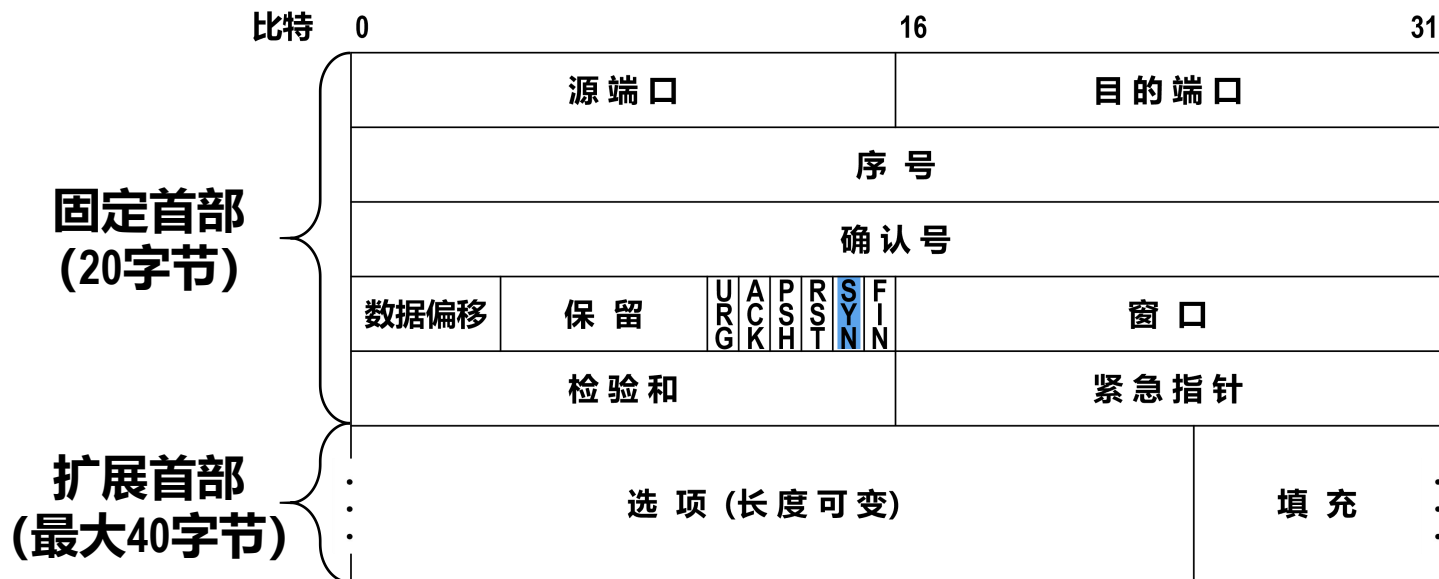
01 TCP报文段的首部格式



01 TCP报文段的首部格式



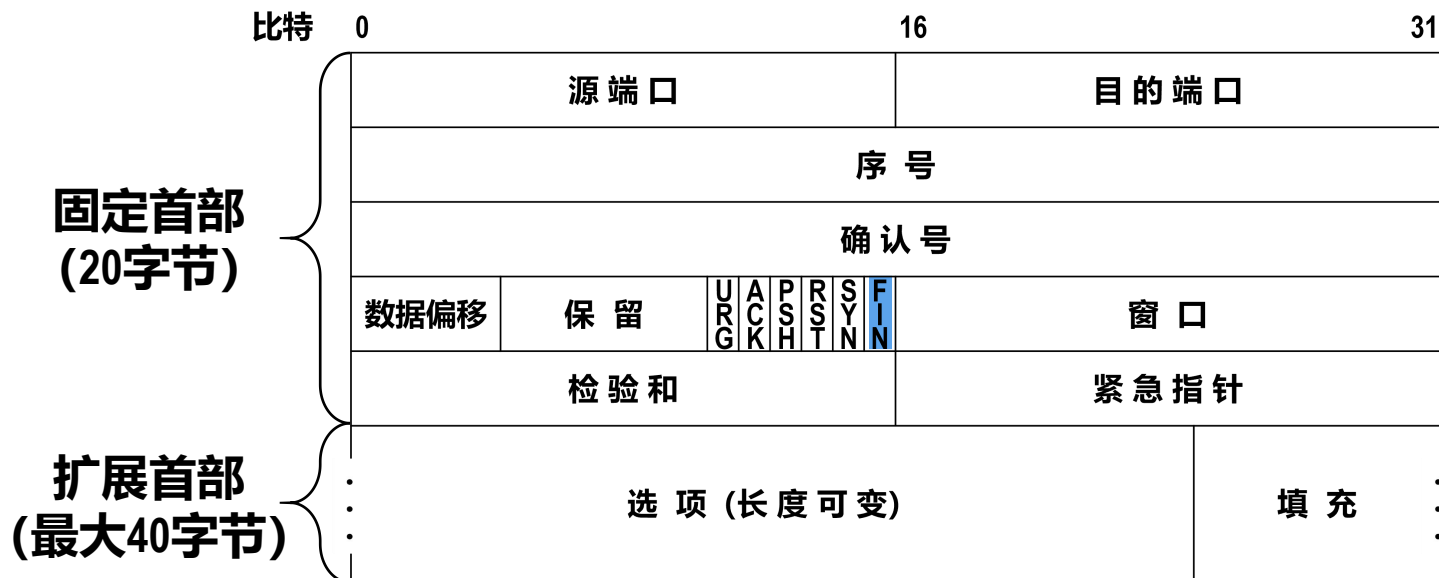
01 TCP报文段的首部格式



同步标志位SYN

- 用于TCP “三报文握手” **建立连接**。
- 当**SYN=1且ACK=0**时，表明这是一个**TCP连接请求报文段**。
- 对方若**同意建立连接**，则应在**响应的**TCP报文段的首部中使**SYN=1且ACK=1**。
- 综上所述，SYN为1的TCP报文段要么是一个连接请求报文段，要么是一个连接响应报文段。

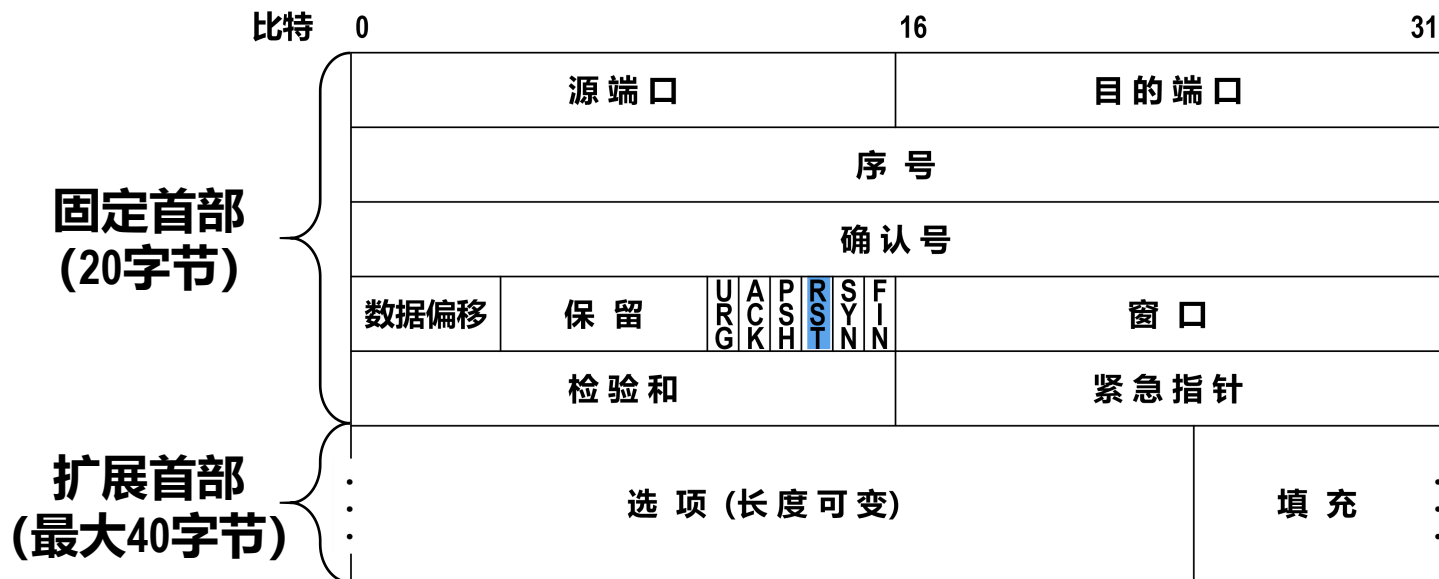
01 TCP报文段的首部格式



终止标志位FIN

- 用于TCP “四报文挥手” **释放连接**。
- 当**FIN=1**时，表明此TCP报文段的发送方已经将全部数据发送完毕，现在**要求释放TCP连接**。

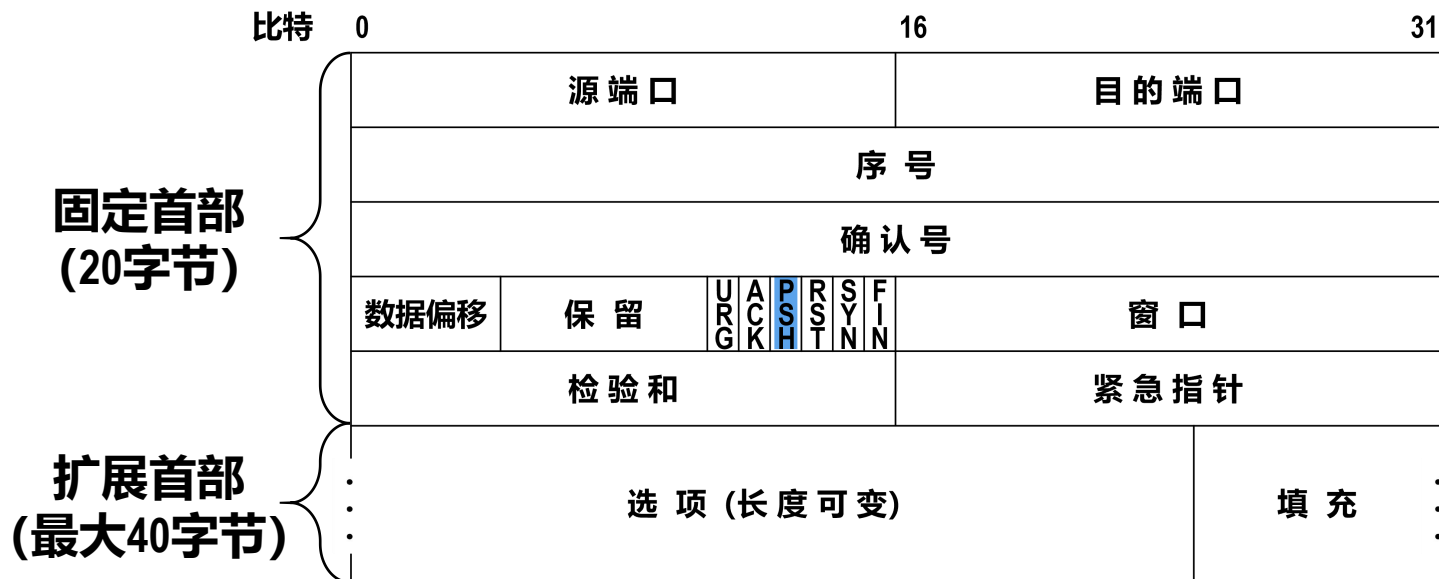
01 TCP报文段的首部格式



复位标志位RST

- 用于**复位TCP连接**。
- 当**RST=1**时，表明**TCP连接中出现严重差错，必须释放连接，然后再重新建立连接**。
- RST置1还用来**拒绝一个非法的TCP报文段或拒绝打开一个TCP连接**。

01 TCP报文段的首部格式

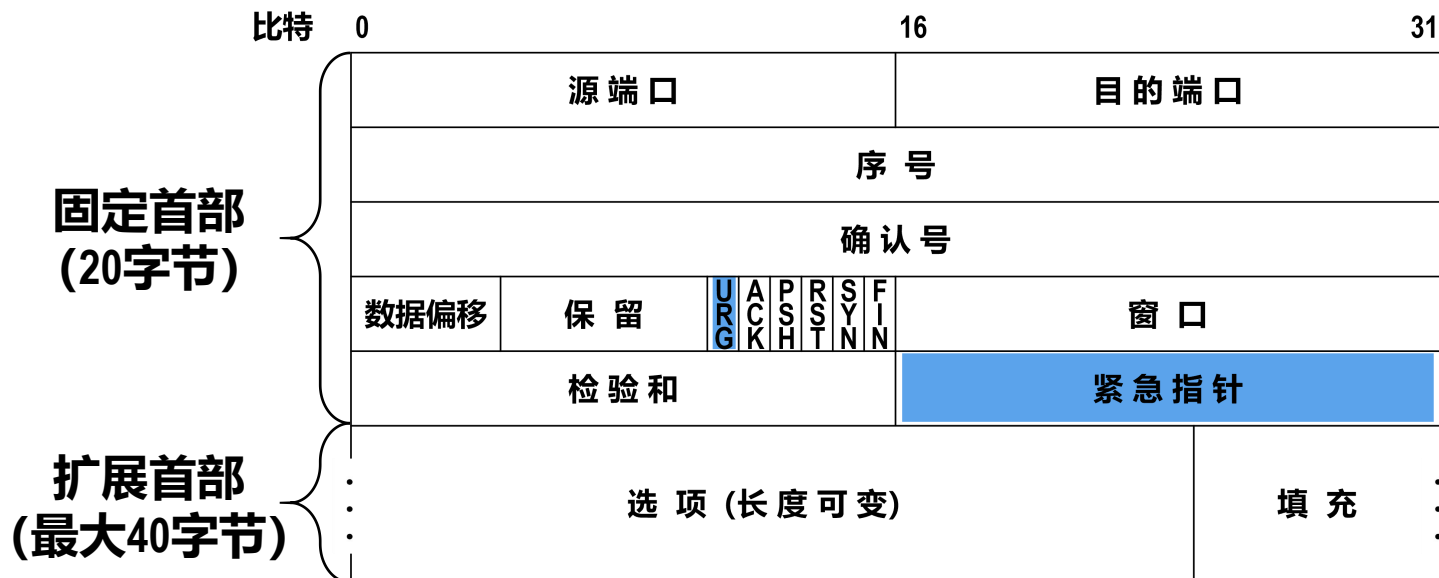


- 出于效率的考虑，TCP的发送方可能会延迟发送数据，而TCP的接收方可能会延迟向应用进程交付数据。这样可以一次处理更多的数据。
- 但是当两个应用进程进行交互式通信时，有时在一端的应用进程希望在键入一个命令后立即就能够收到对方的响应。在这种情况下，应用进程可以通知TCP使用推送（PUSH）操作。

推送标志位PSH

- 发送方TCP把PSH置1，并立即创建一个TCP报文段发送出去，而不需要积累到足够多的数据再发送。
- 接收方TCP收到PSH为1的TCP报文段，就尽快地交付给应用进程，而不再等到接收到足够多的数据才向上交付。

01 TCP报文段的首部格式



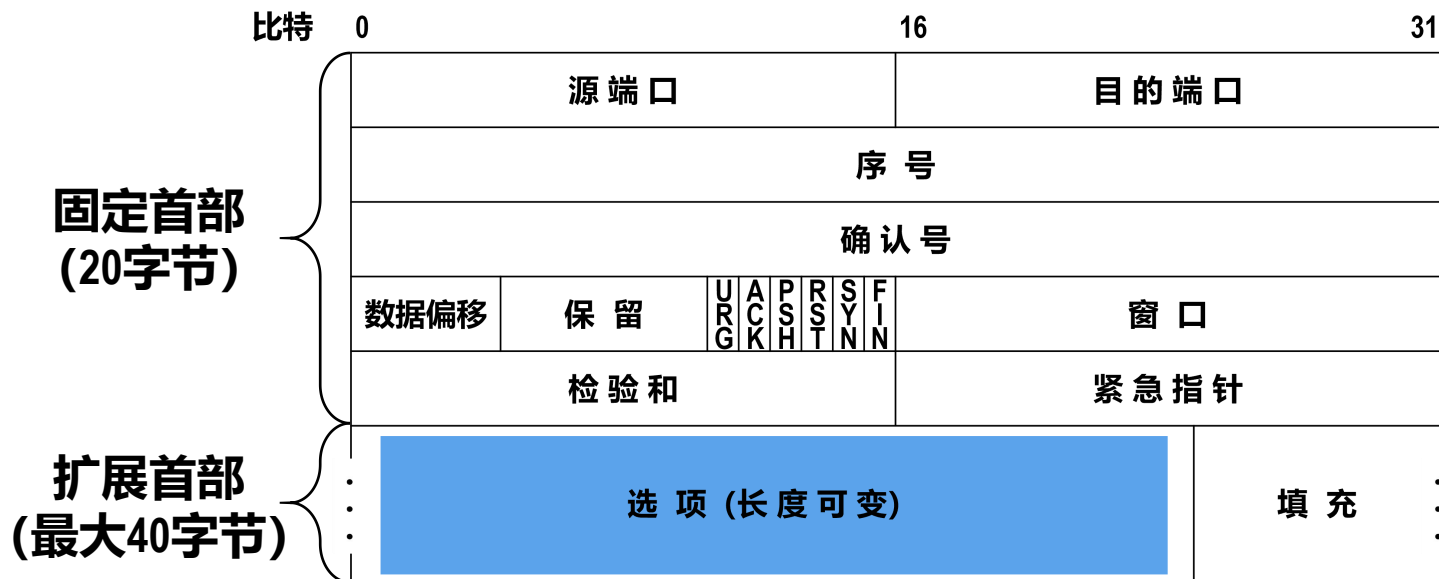
紧急标志位URG

- 当URG=1时，紧急指针字段有效。
- 当URG=0时，紧急指针字段无效。

紧急指针

- 占16比特，以字节为单位，用来指明紧急数据的长度。
- 当**发送方有紧急数据**时，可将紧急数据“**插队**”到发送缓存的最前面，并立刻封装到一个TCP报文段中进行发送。**紧急指针会指出本报文段数据载荷部分包含了多长的紧急数据**，紧急数据之后是普通数据。
- **接收方收到紧急标志位为1的TCP报文段**，会按照紧急指针字段的值从报文段数据载荷中取出**紧急数据**并直接上交应用进程，而**不必在接收缓存中排队**。

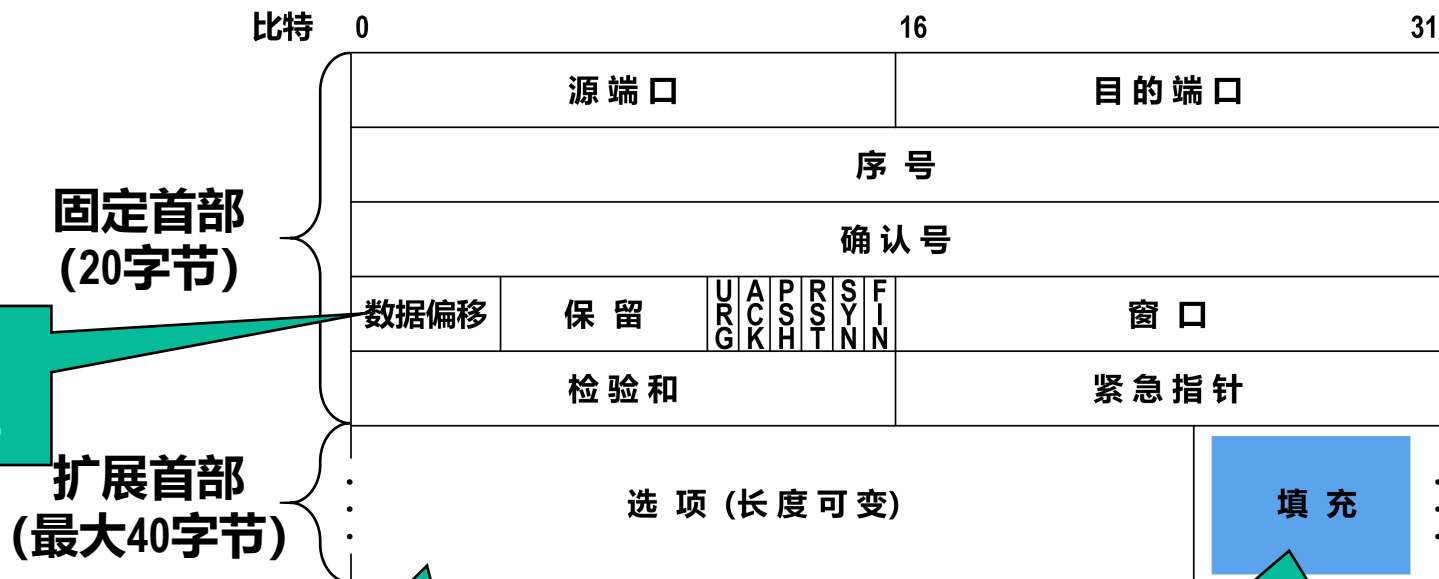
01 TCP报文段的首部格式



选项 (长度可变, 最大40字节)

- ❑ 最大报文段长度MSS选项: 指出TCP报文段数据载荷部分的最大长度, 而不是整个TCP报文段的长度。
- ❑ 窗口扩大选项: 用来扩大窗口, 提高吞吐率。
- ❑ 时间戳选项:
 - 用于计算往返时间RTT
 - 用于处理序号超范围的情况, 又称为防止序号绕回PAWS。
- ❑ 选择确认选项: 用来实现选择确认功能。

01 TCP报文段的首部格式



首部长度的单位以4字节为单位

长度可变 (0~40字节)

若选项字段的长度加上20字节固定首部的长度不能被4字节整除时，需要填充相应数量的比特0，以确保首部长度的单位以4字节为单位。

与IPv4数据报首部中的填充字段作用一样

似曾相识



5.3.2 TCP的运输连接管理

01

“三报文握手” 建立TCP连接

02

“四报文挥手” 释放TCP连接

01 “三报文握手” 建立TCP连接

■ TCP是面向连接的协议，它基于运输连接来传送TCP报文段。

☐ TCP运输连接的建立和释放，是每一次面向连接的通信中必不可少的过程。

■ TCP运输连接有以下三个阶段：

- ① 通过“三报文握手”来建立TCP连接。
- ② 基于已建立的TCP连接进行可靠的数据传输。
- ③ 在数据传输结束后，还要通过“四报文挥手”来释放TCP连接。

01 “三报文握手” 建立TCP连接

TCP运输连接有以下三个阶段：

- ① 通过 “三报文握手” 来建立TCP连接。
- ② 基于已建立的TCP连接进行可靠的数据传输。
- ③ 在数据传输结束后，还要通过 “四报文挥手” 来释放TCP连接。



01 “三报文握手” 建立TCP连接

■ “三报文握手” 建立TCP连接的目的在于解决以下三个主要问题：

- ☐ 使TCP双方能够确知对方的存在。
- ☐ 使TCP双方能够协商一些参数（例如最大报文段长度、最大窗口大小、时间戳选项等）。
- ☐ 使TCP双方能够对运输实体资源进行分配和初始化。运输实体资源包括缓存大小、各状态变量、连接表中的项目等。

01 “三报文握手” 建立TCP连接



传输控制块TCB

TCP连接表
指向发送和接收缓存的指针
指向重传队列的指针
当前的发送和接收序号
⋮

01 “三报文握手” 建立TCP连接

传输控制块TCB

TCP连接表
指向发送和接收缓存的指针
指向重传队列的指针
当前的发送和接收序号
⋮

TCP客户



CLOSED
关闭



建立TCP连接

TCP服务器

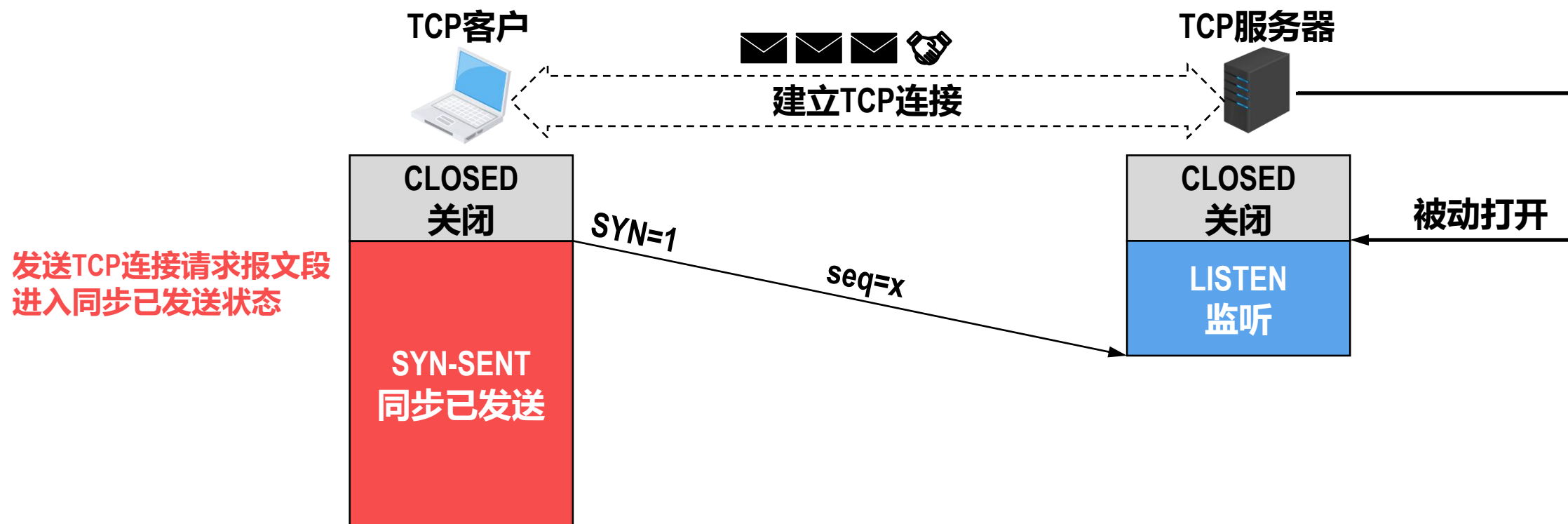


CLOSED
关闭

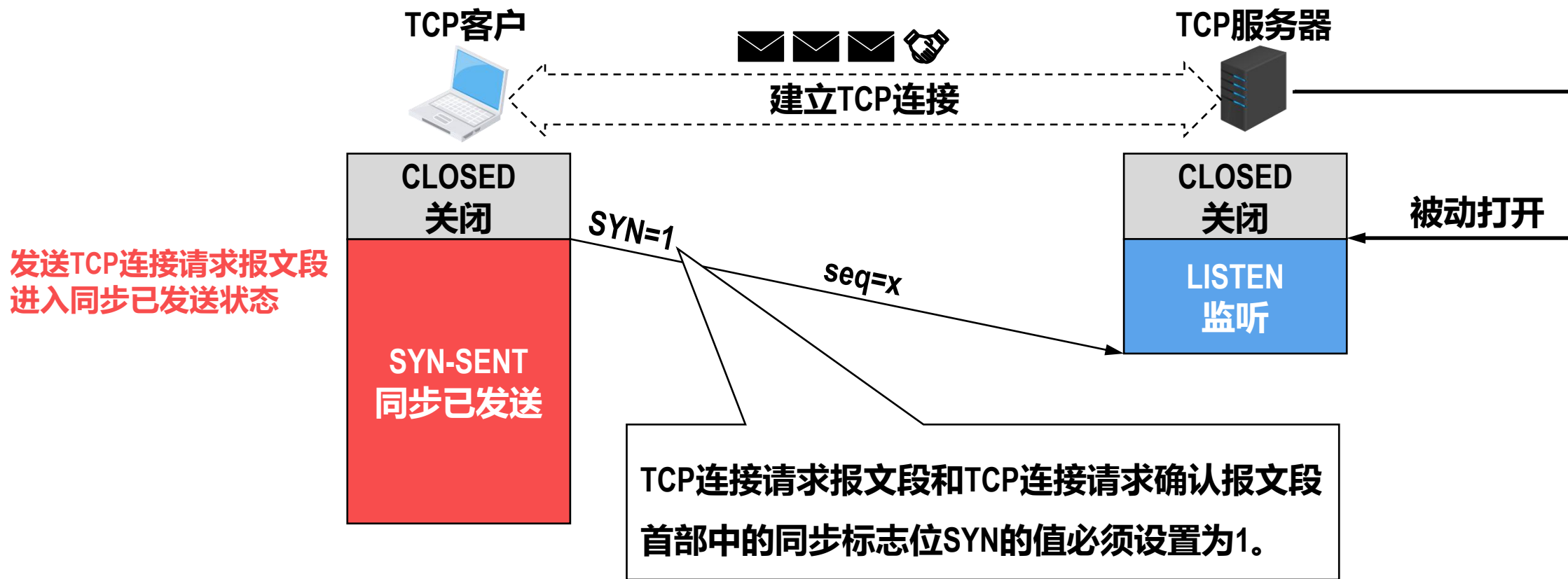
LISTEN
监听

被动打开

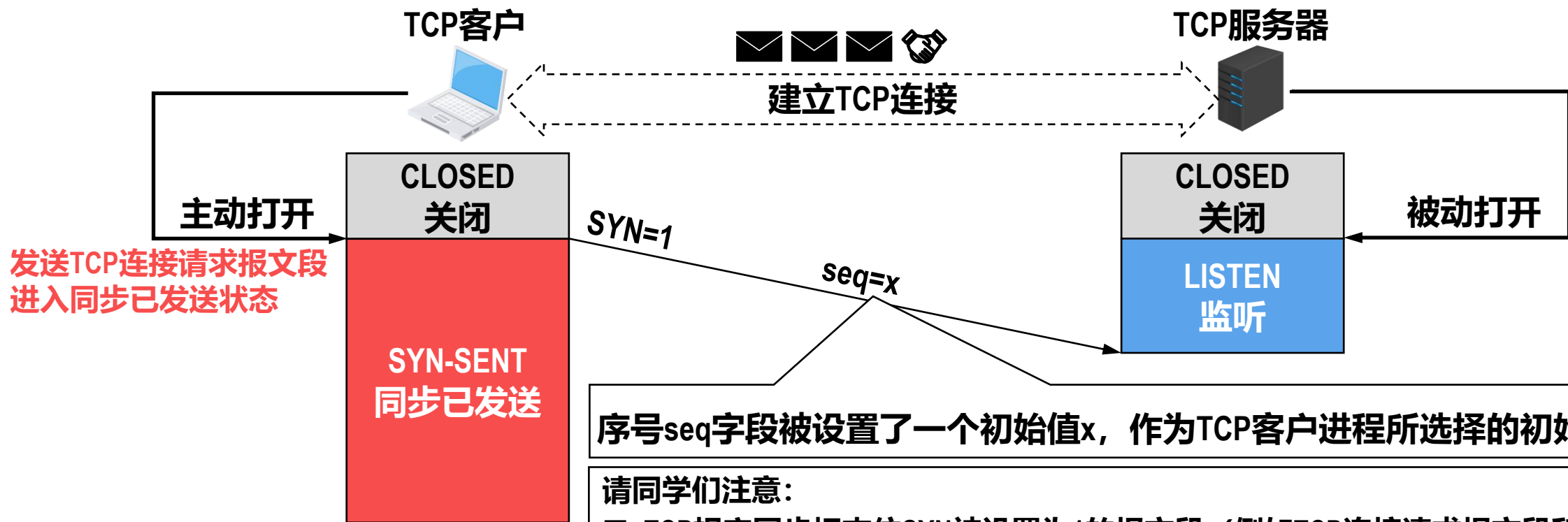
01 “三报文握手” 建立TCP连接



01 “三报文握手” 建立TCP连接



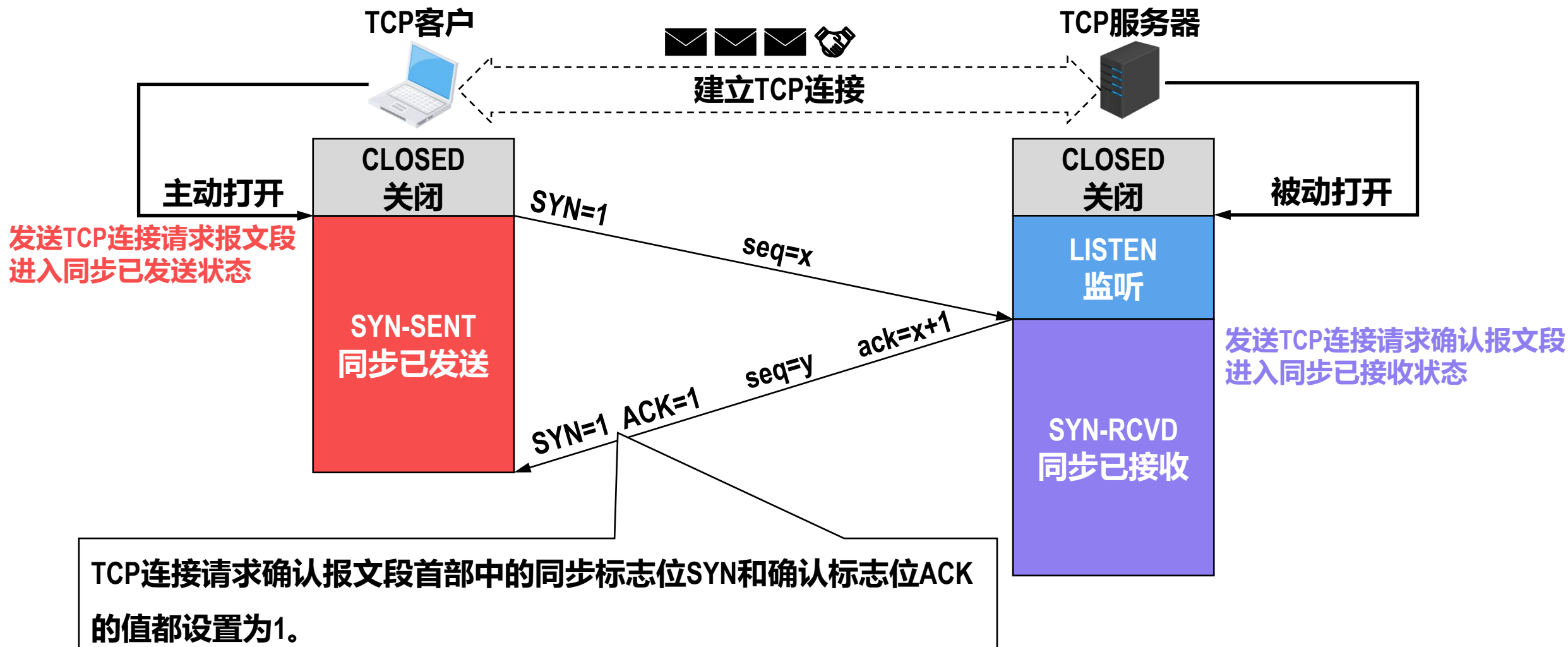
01 “三报文握手” 建立TCP连接



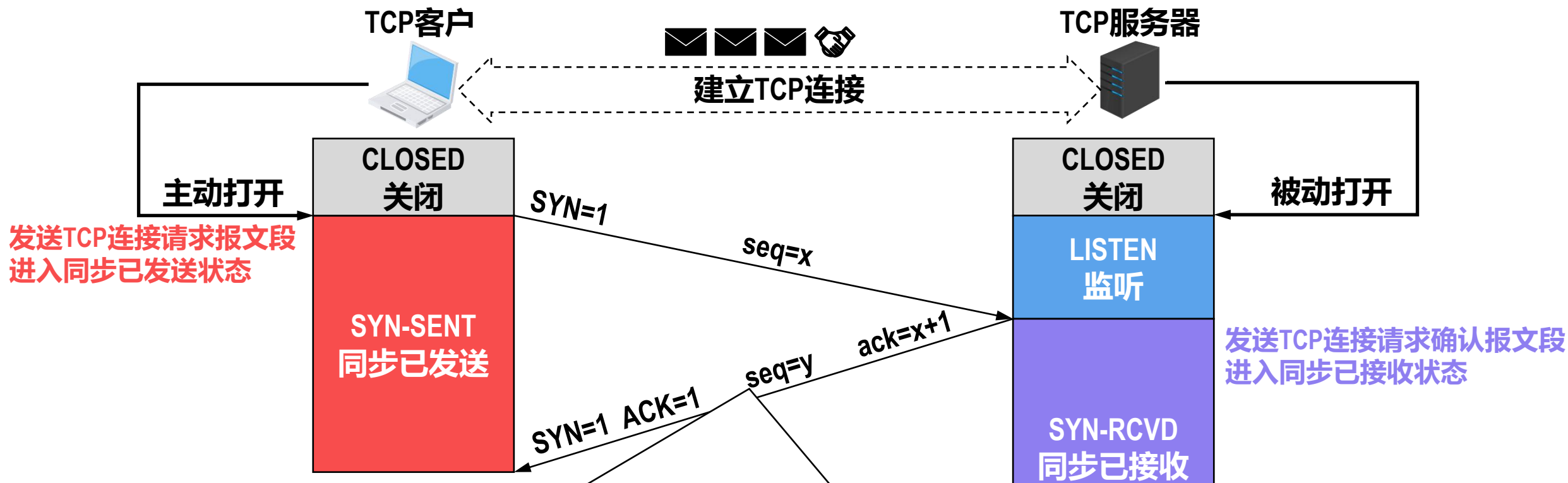
请同学们注意：

- TCP规定同步标志位SYN被设置为1的报文段（例如TCP连接请求报文段和TCP连接请求确认报文段）不能携带数据，但要消耗掉一个序号。
- 按上述规定，TCP连接请求报文段不能携带数据（即没有数据载荷），但是会消耗掉序号x。
- 因此，TCP客户进程下一次发送的TCP报文段的数据载荷的第一个字节的序号为x+1。

01 “三报文握手” 建立TCP连接



01 “三报文握手” 建立TCP连接

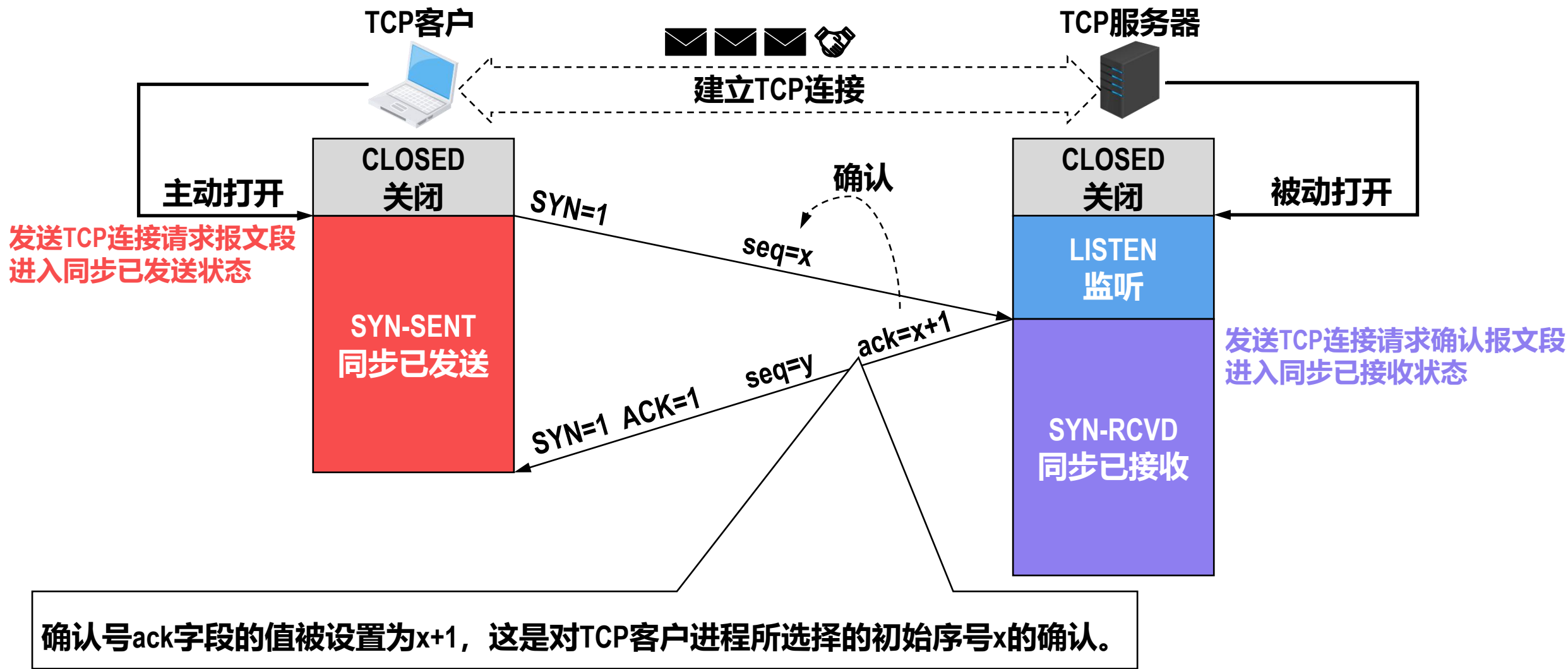


序号seq字段被设置了一个初始值y，作为TCP服务器进程所选择的初始序号。

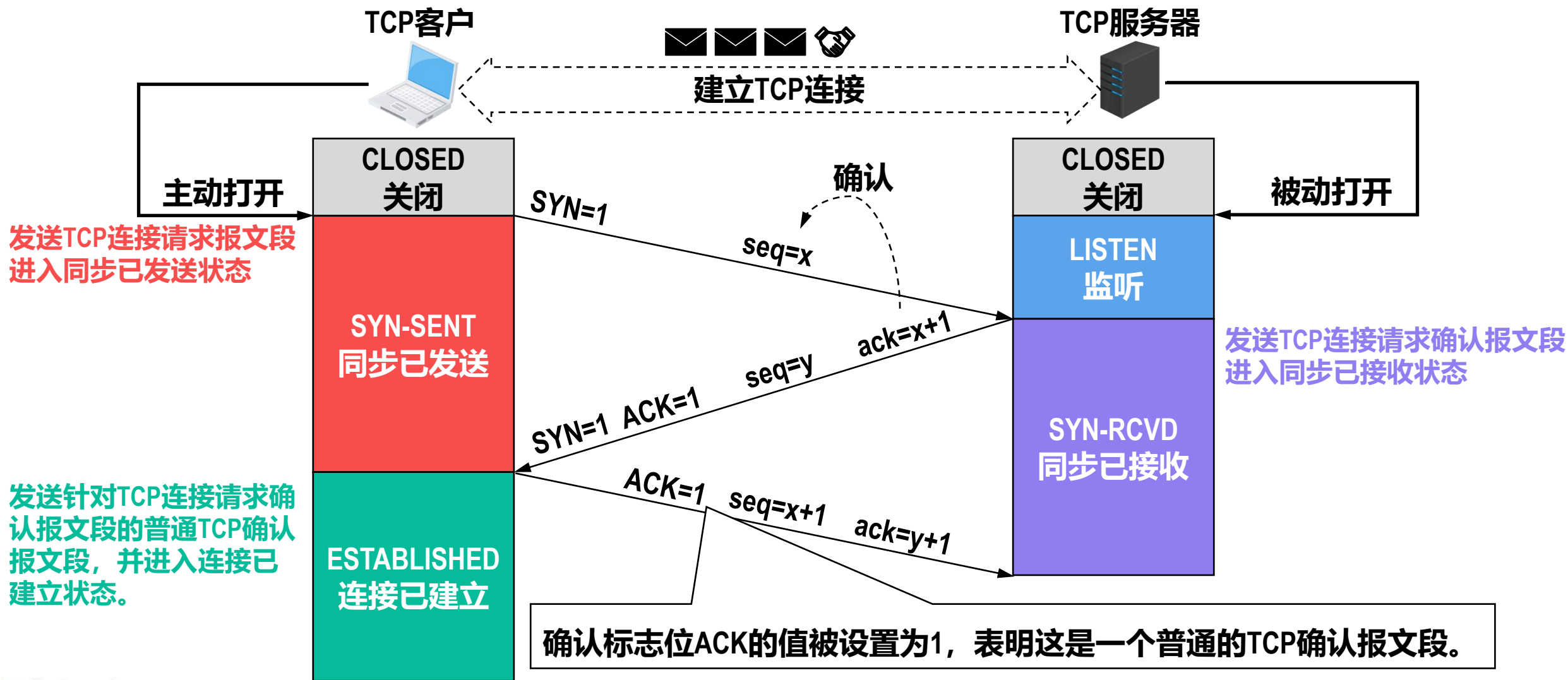
请同学们注意：

- TCP规定同步标志位SYN被设置为1的报文段（例如TCP连接请求报文段和TCP连接请求确认报文段）不能携带数据，但要消耗掉一个序号。

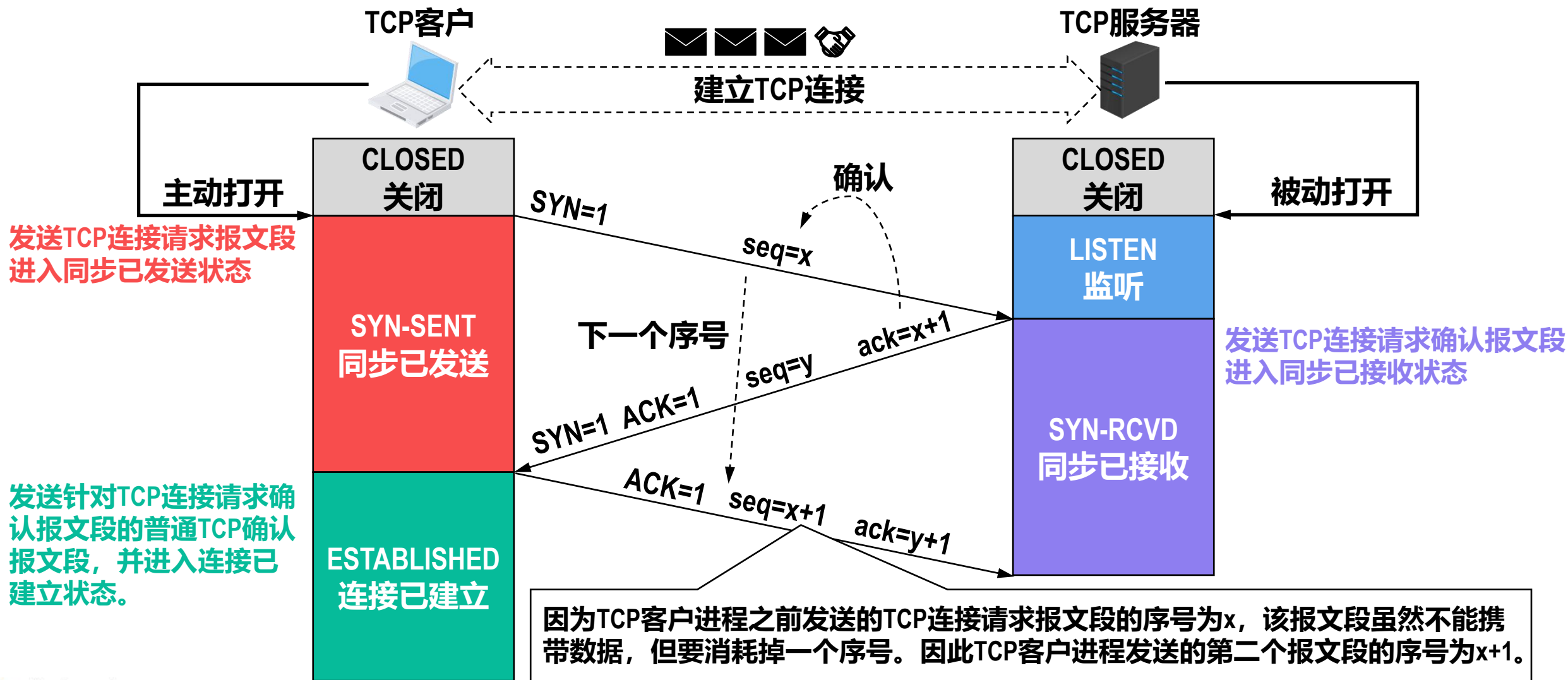
01 “三报文握手” 建立TCP连接



01 “三报文握手” 建立TCP连接



01 “三报文握手” 建立TCP连接

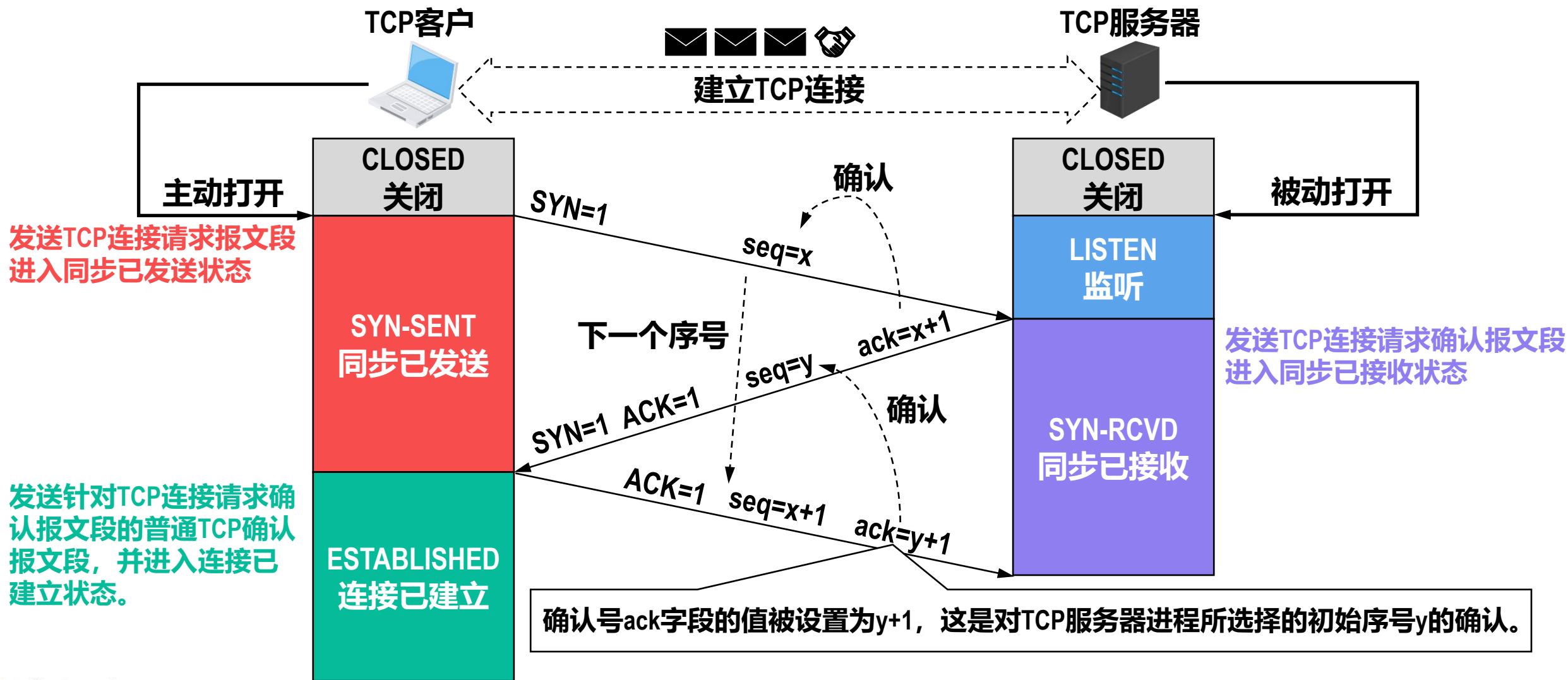


01

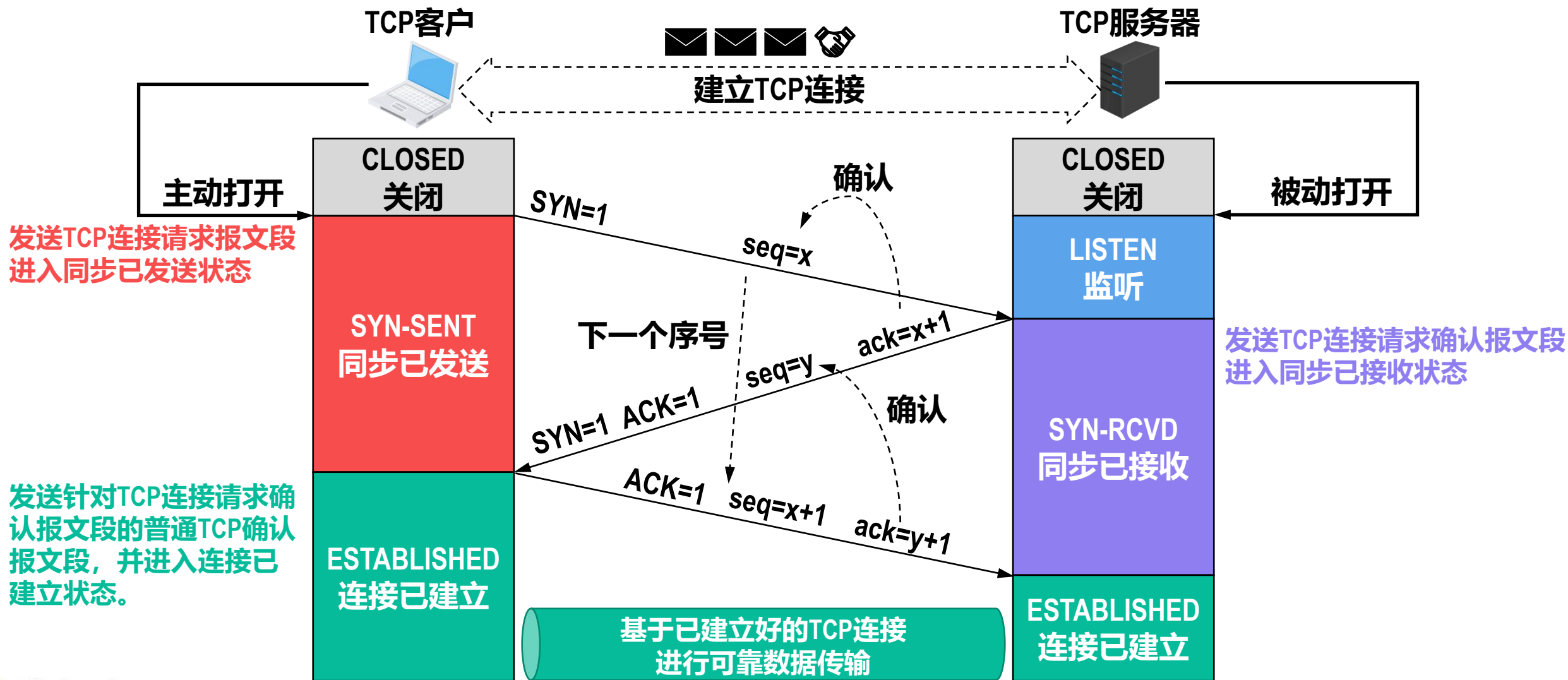


- TCP规定普通的TCP确认报文段可以携带数据，但如果不携带数据，则不消耗序号。
- 如果该报文段不携带数据，则TCP客户进程要发送的下一个数据报文段的序号仍为x+1。

01 “三报文握手” 建立TCP连接



01 “三报文握手” 建立TCP连接



01

“三报文握手” 建立TCP连接

【2011年 题39】主机甲向主机乙发送一个 (SYN=1, seq=11220) 的TCP段, 期望与主机乙建立TCP连接, 若主机乙接受该连接请求, 则主机乙向主机甲发送的正确的TCP段可能是 (C)。

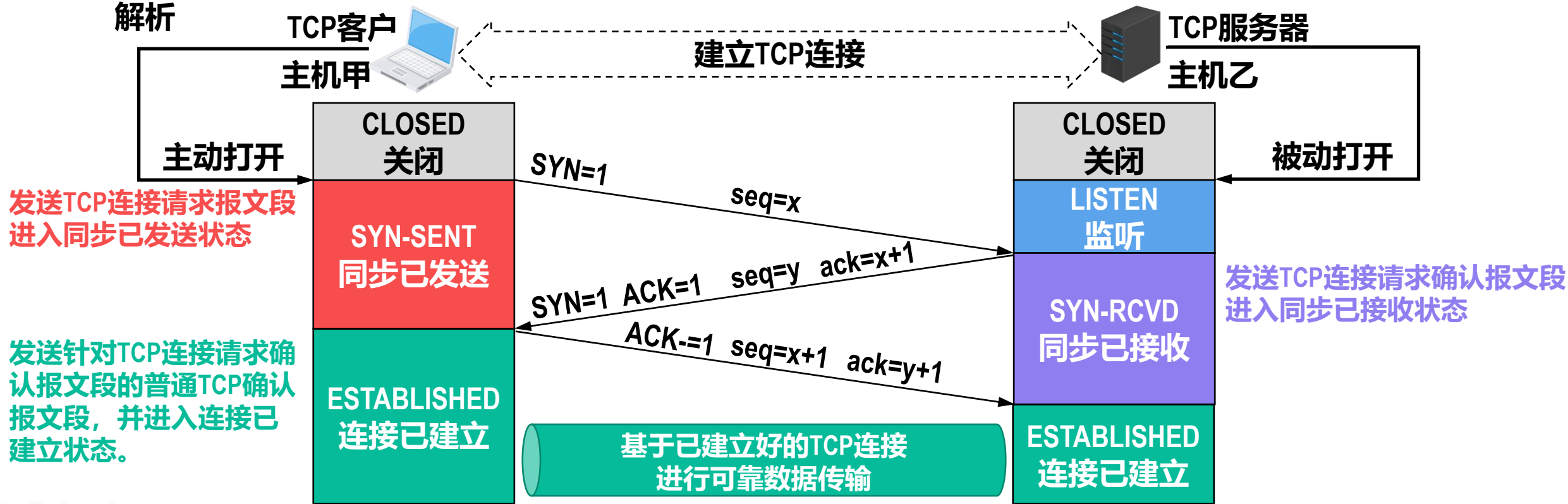
A. (SYN=0, ACK=0, seq=11221, ack=11221)

B. (SYN=1, ACK=1, seq=11220, ack=11220)

C. (SYN=1, ACK=1, seq=11221, ack=11221)

D. (SYN=0, ACK=0, seq=11220, ack=11220)

解析



01

“三报文握手” 建立TCP连接

【2011年 题39】主机甲向主机乙发送一个 (SYN=1, seq=11220) 的TCP段, 期望与主机乙建立TCP连接, 若主机乙接受该连接请求, 则主机乙向主机甲发送的正确的TCP段可能是 (C)。

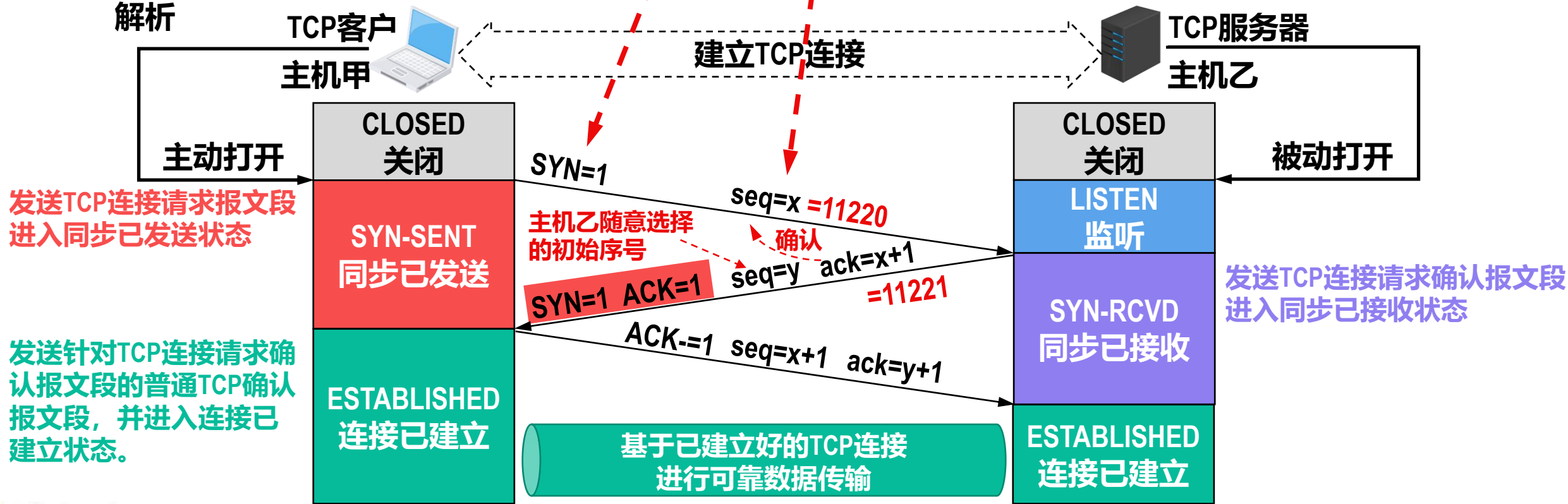
A. (SYN=0, ACK=0, seq=11221, ack=11221)

B. (SYN=1, ACK=1, seq=11220, ack=11220)

C. (SYN=1, ACK=1, seq=11221, ack=11221)

D. (SYN=0, ACK=0, seq=11220, ack=11220)

解析



01 “三报文握手” 建立TCP连接

【2019年 题39】若主机甲主动发起一个与主机乙的TCP连接，甲、乙选择的初始序列号分别为2018和2046，则第三次握手TCP段的确认序列号是（**D**）。

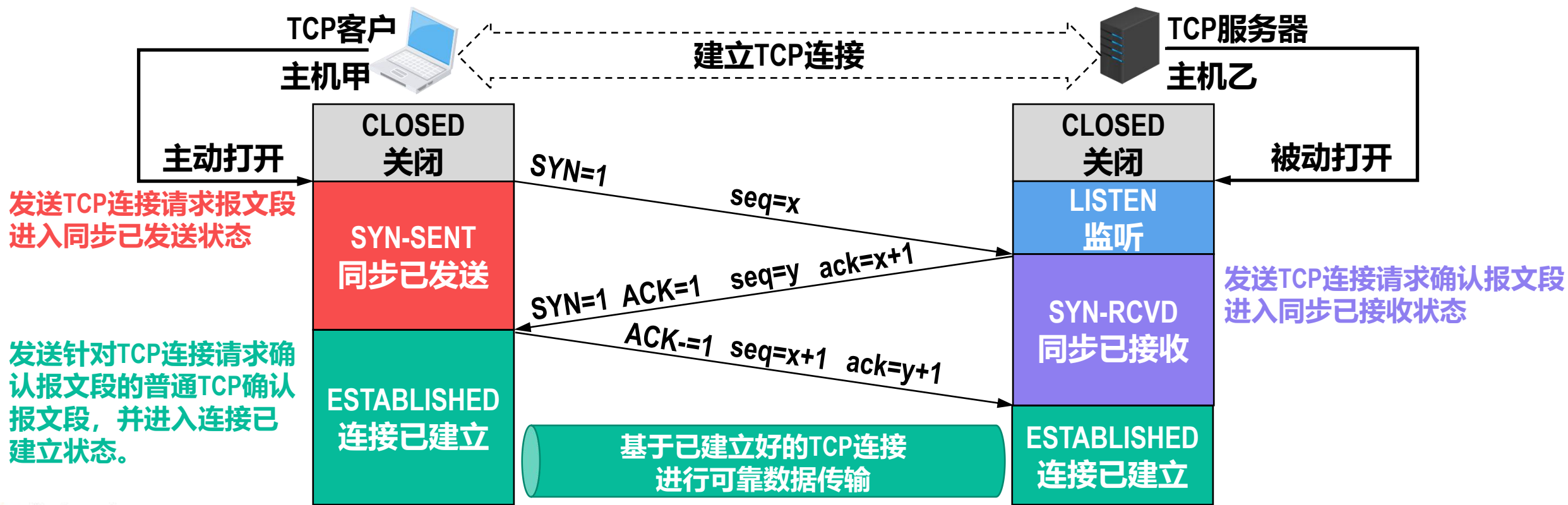
A. 2018

B. 2019

C. 2046

D. 2047

解析



01 “三报文握手” 建立TCP连接

【2019年 题39】若主机甲主动发起一个与主机乙的TCP连接，甲、乙选择的初始序列号分别为2018和2046，则第三次握手TCP段的确认序列号是（**D**）。

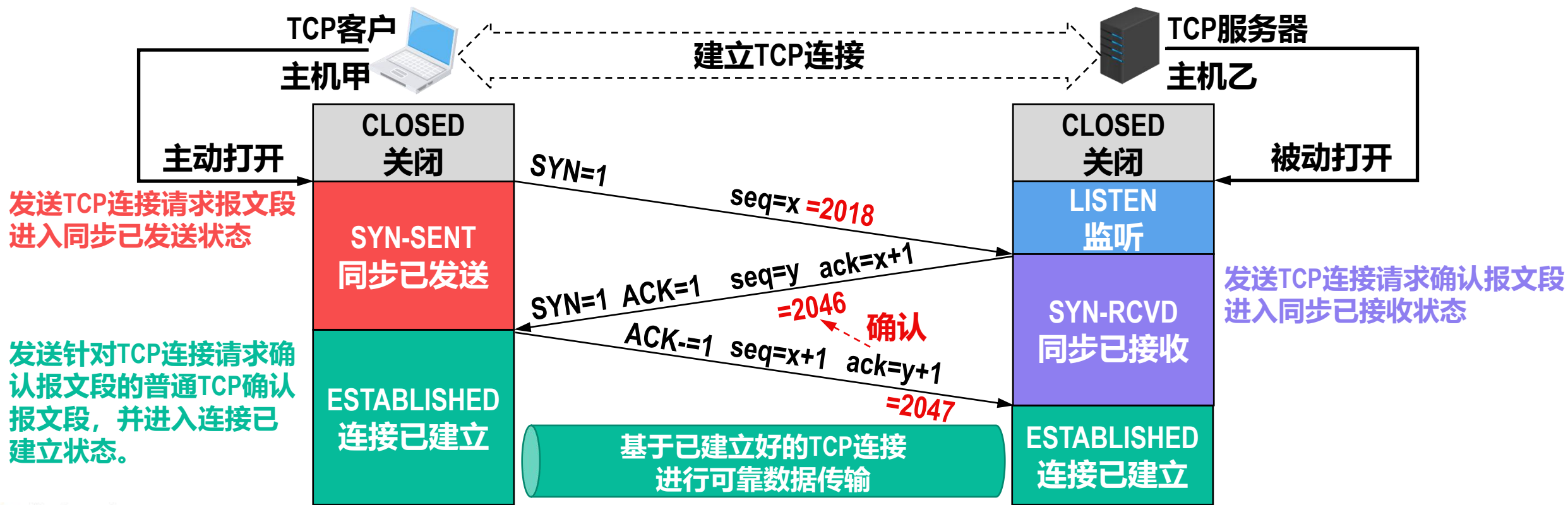
A. 2018

B. 2019

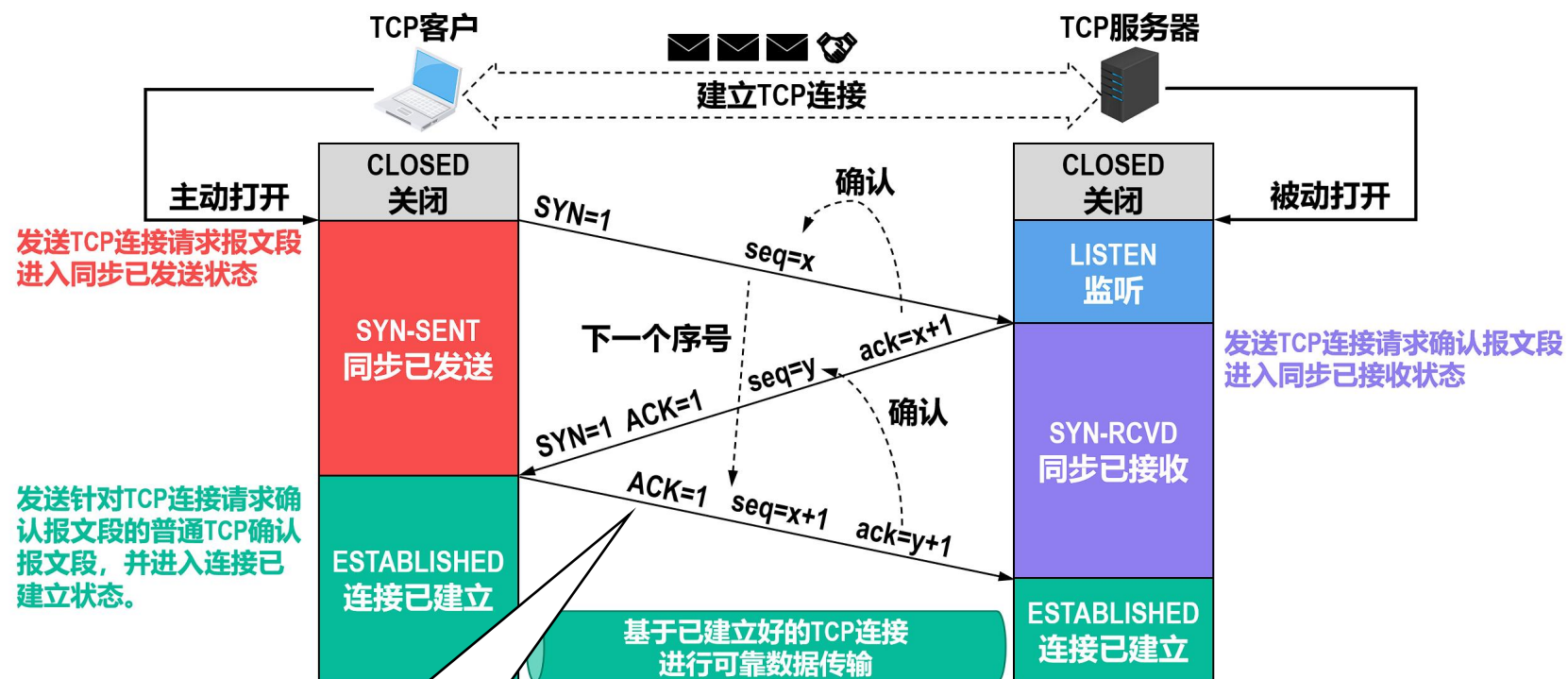
C. 2046

D. 2047

解析



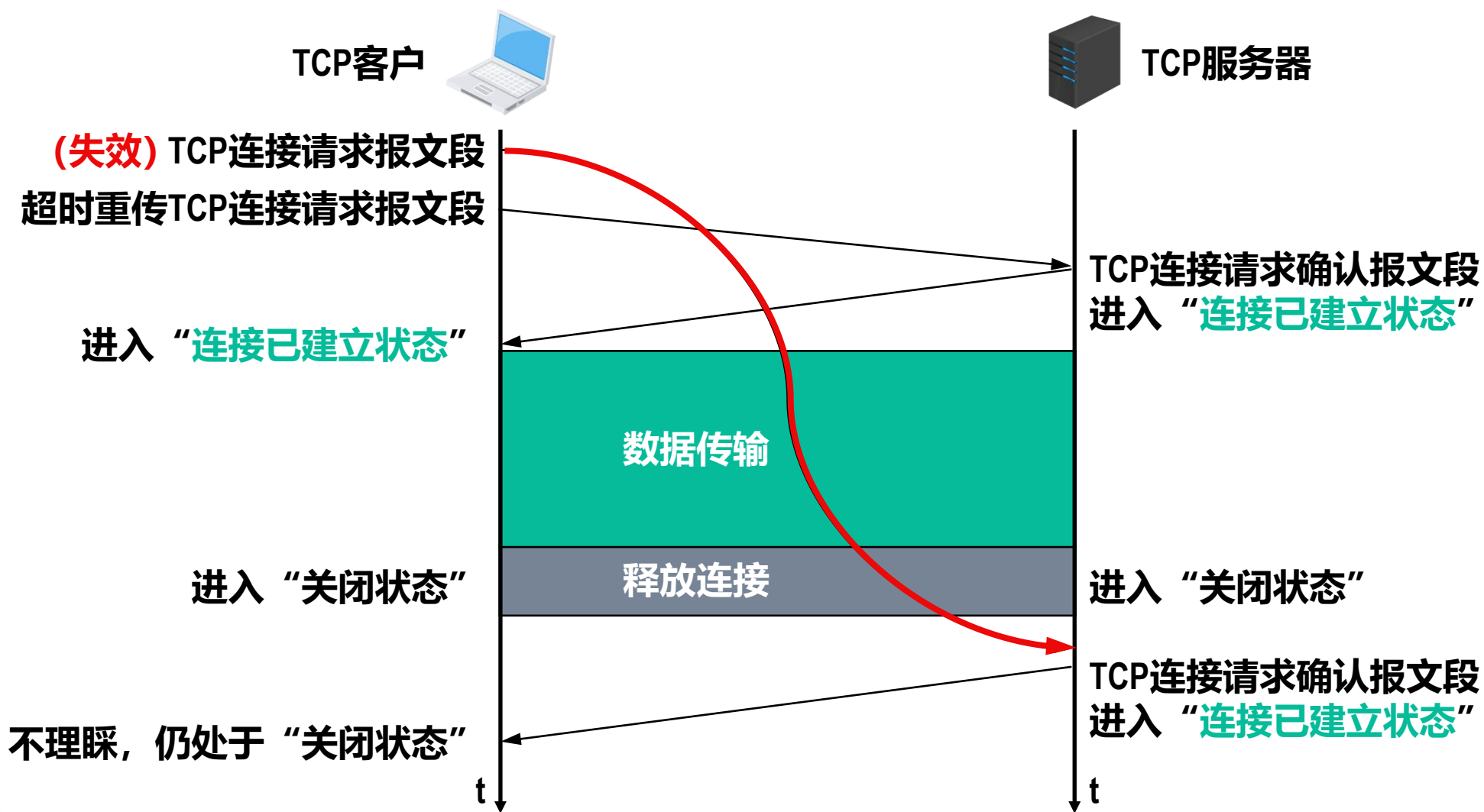
01 “三报文握手” 建立TCP连接



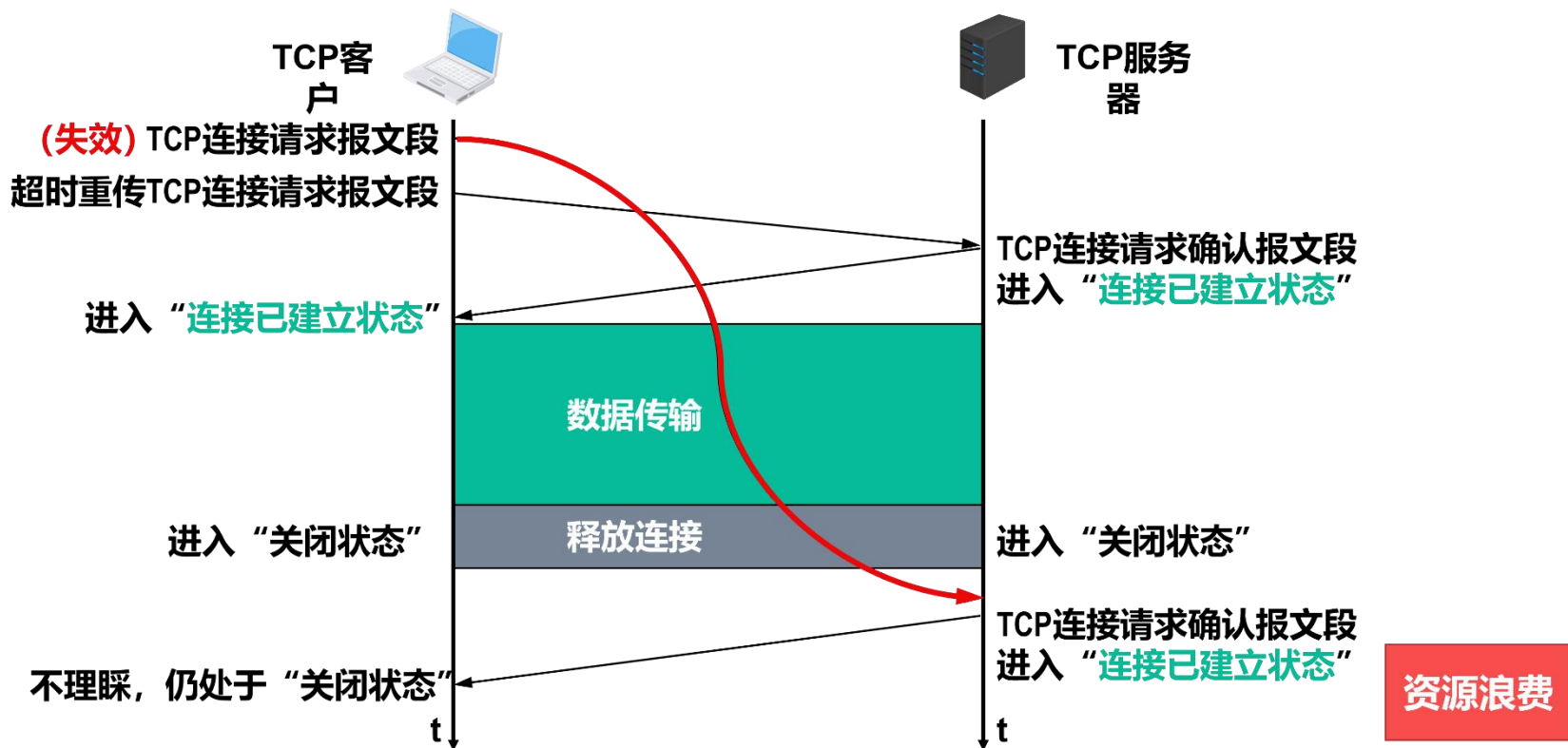
第三个TCP报文段
是否多余？



01 “三报文握手”建立TCP连接 —— 使用“三报文握手”而不是“两报文握手”建立TCP连接的原因

**资源浪费**

01 “三报文握手”建立TCP连接 —— 使用“三报文握手”而不是“两报文握手”建立TCP连接的原因



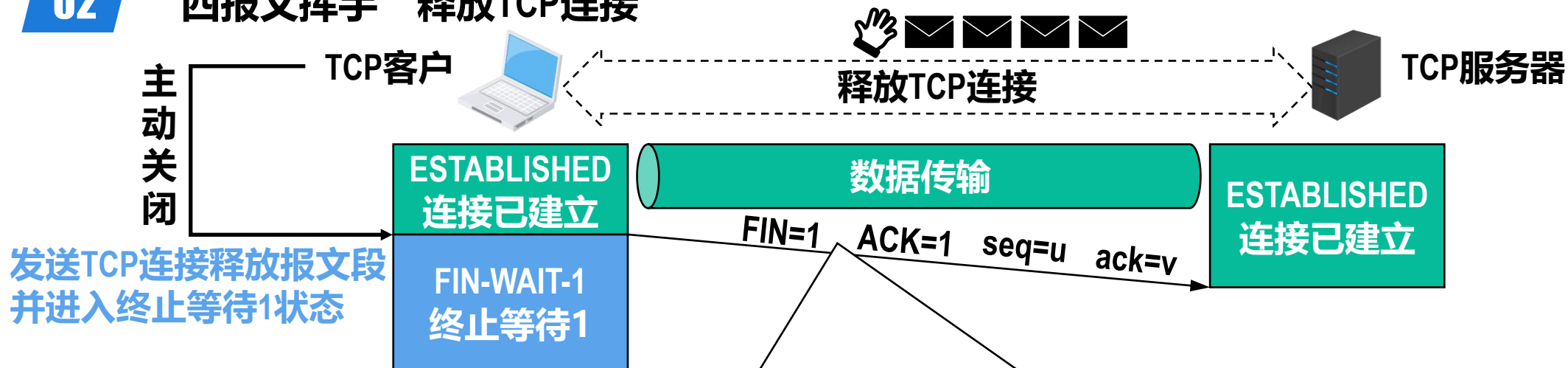
采用“三报文握手”而不是“两报文握手”来建立TCP连接，是为了防止已失效的TCP连接请求报文段突然又传送到了TCP服务器进程，因而导致错误。

02

“四报文挥手” 释放TCP连接

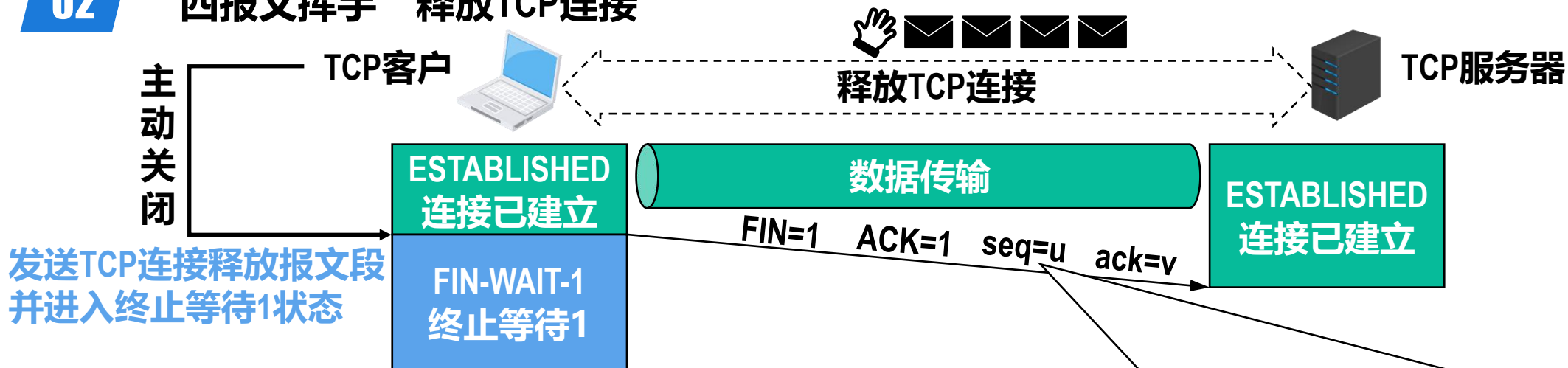


02 “四报文挥手” 释放TCP连接



TCP连接释放报文段首部中的终止标志位FIN和确认标志位ACK的值都被设置为1。表明这是一个TCP连接释放报文段，同时也对之前收到的TCP报文段进行确认。

02 “四报文挥手” 释放TCP连接

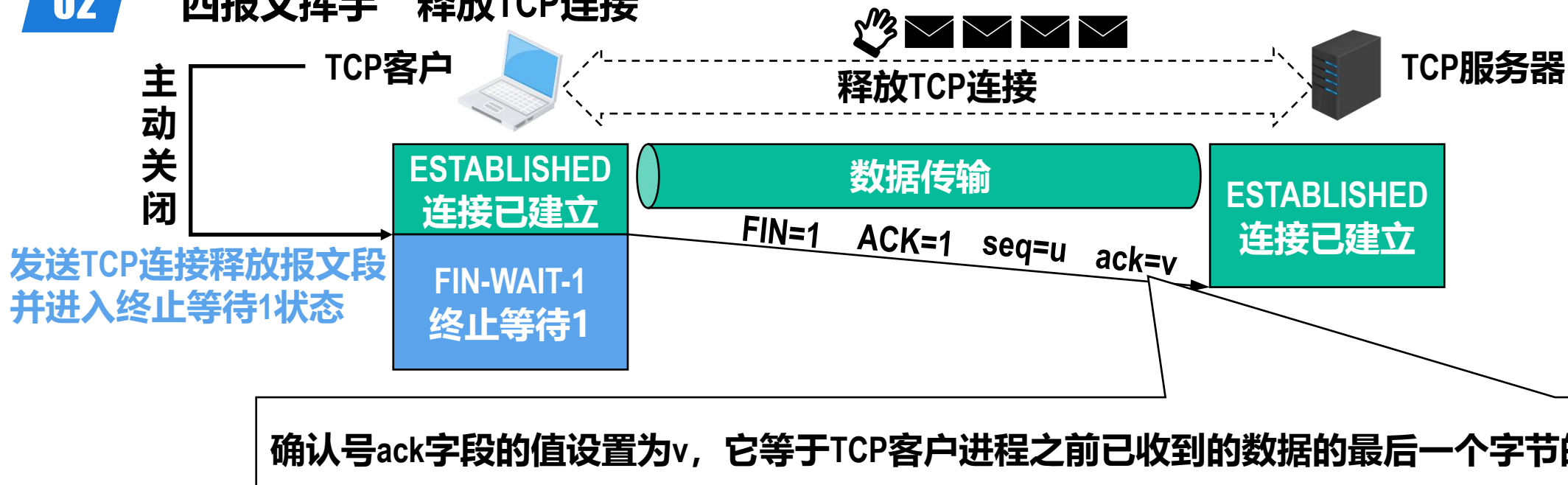


序号seq字段的值设置为u，它等于TCP客户进程之前已经传送过的数据的最后一个字节的序号加1。

请同学们注意：

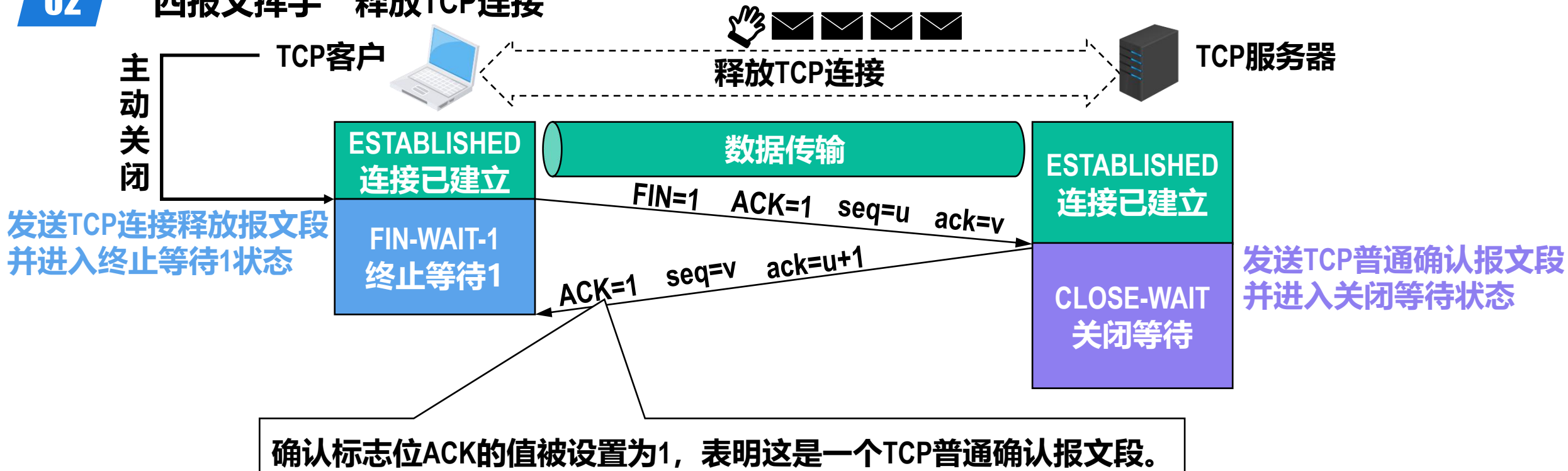
□ TCP规定终止标志位FIN等于1的TCP报文段即使不携带数据，也要消耗掉一个序号。

02 “四报文挥手” 释放TCP连接



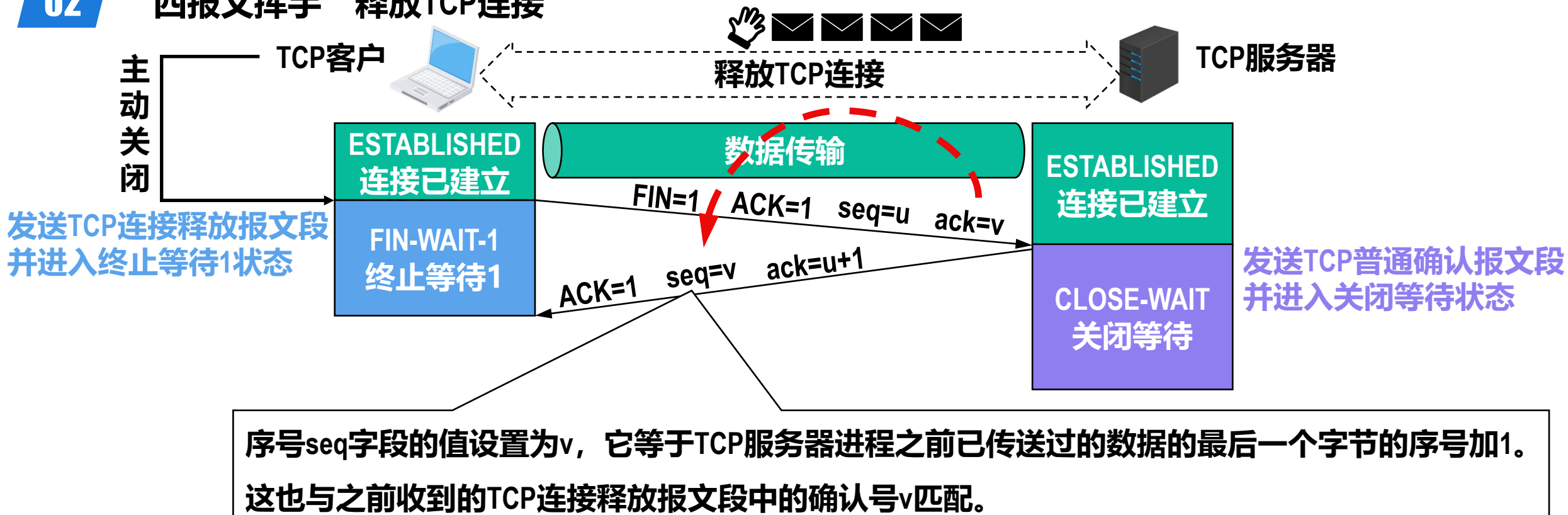
02

“四报文挥手” 释放TCP连接

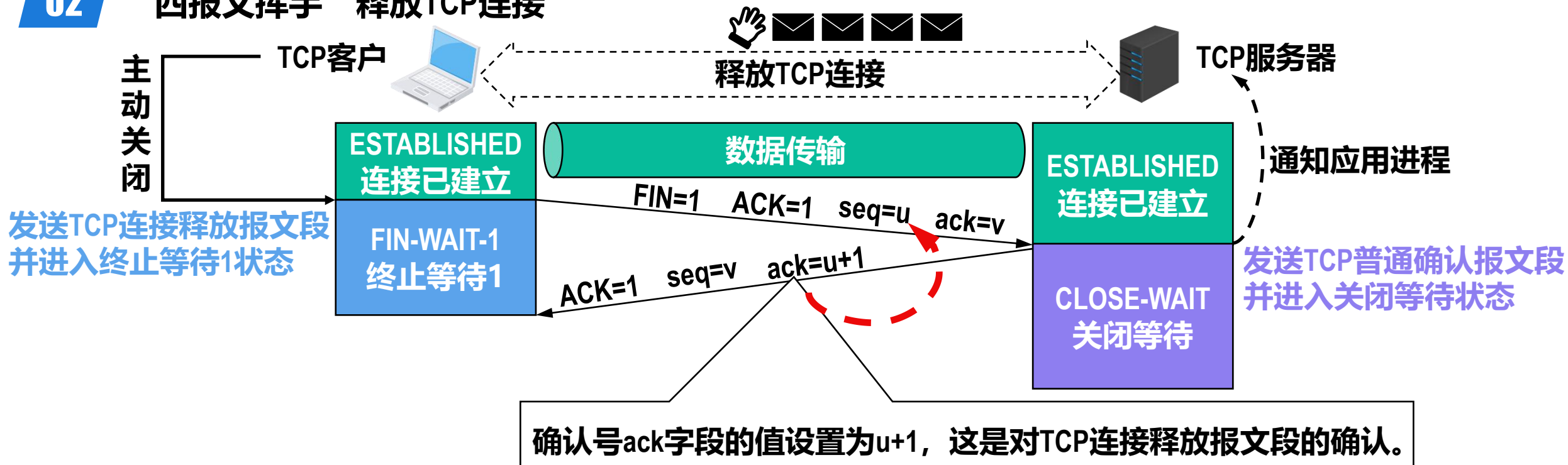


02

“四报文挥手” 释放TCP连接



02 “四报文挥手” 释放TCP连接



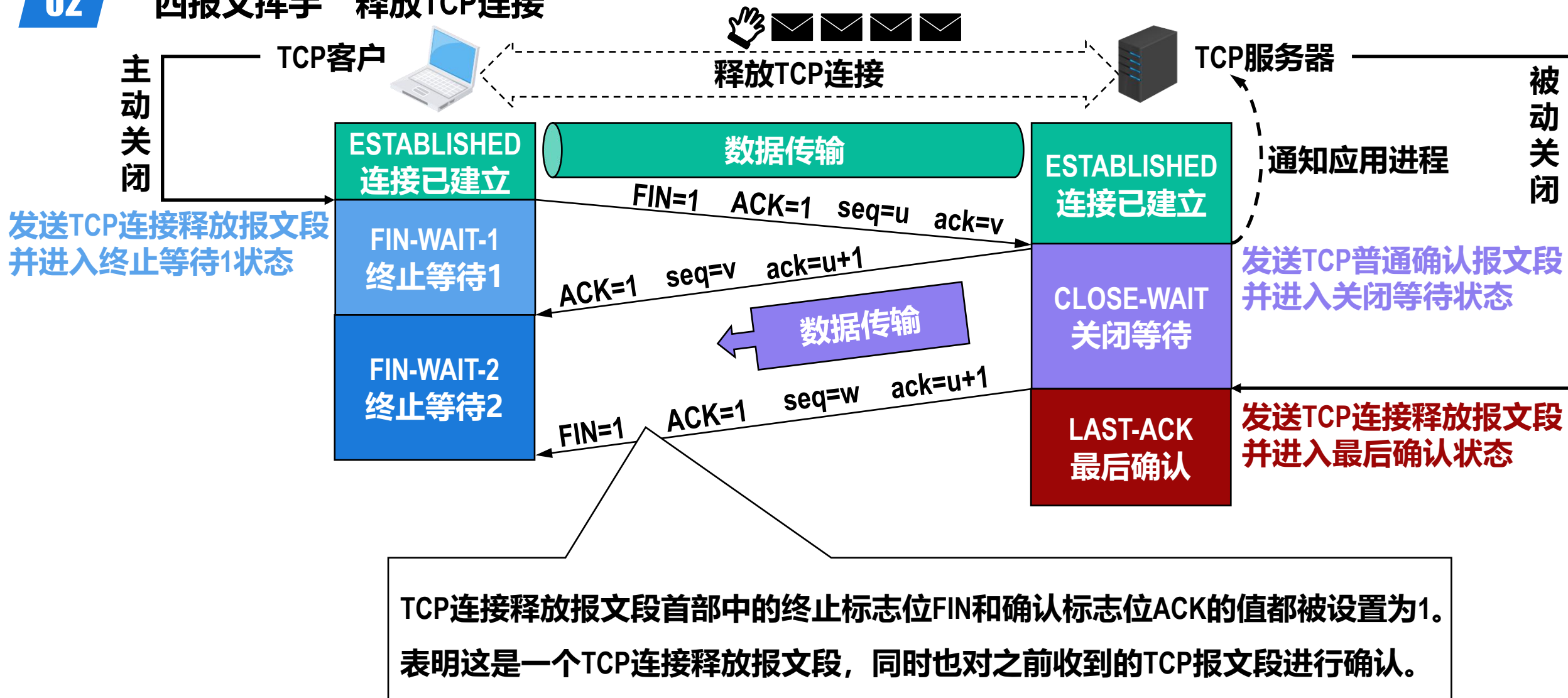
02



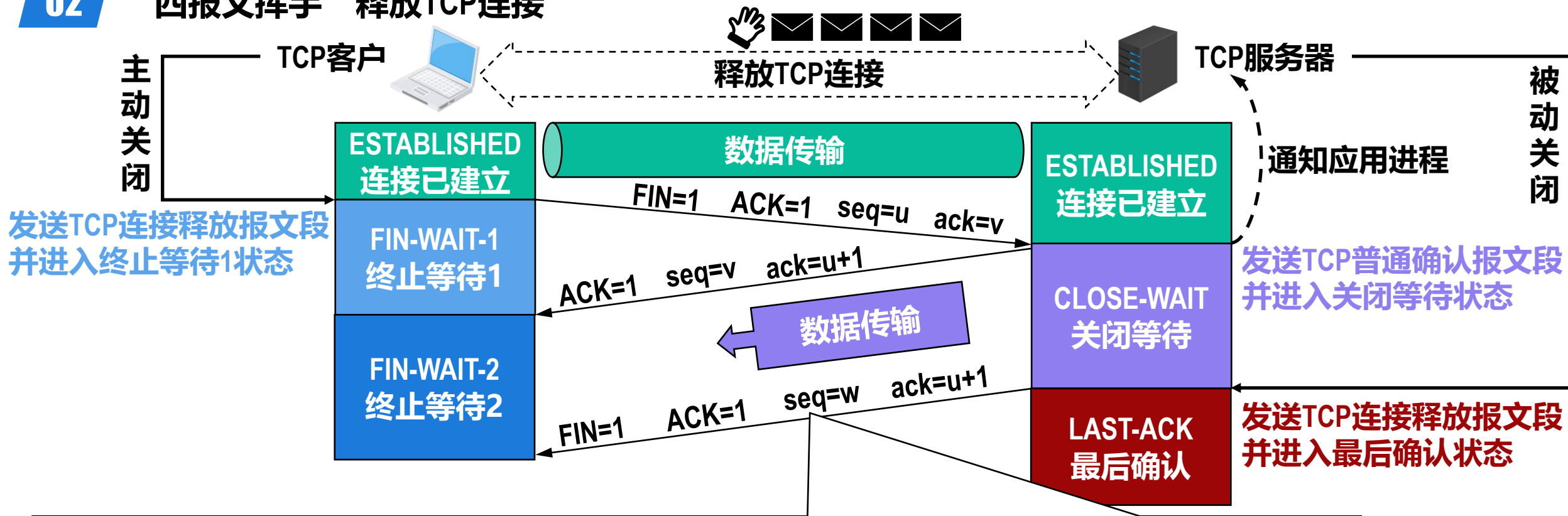
- 半关闭状态可能会持续一段时间。**

02

“四报文挥手” 释放TCP连接

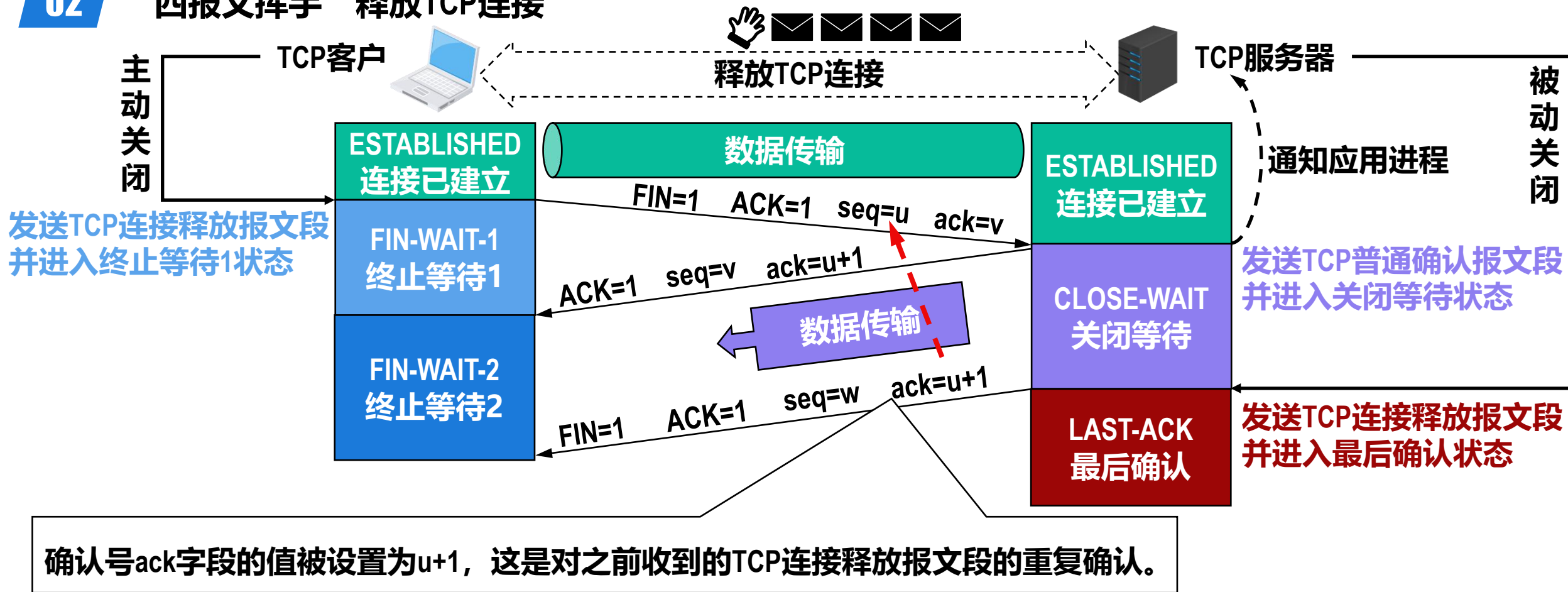


02 “四报文挥手” 释放TCP连接

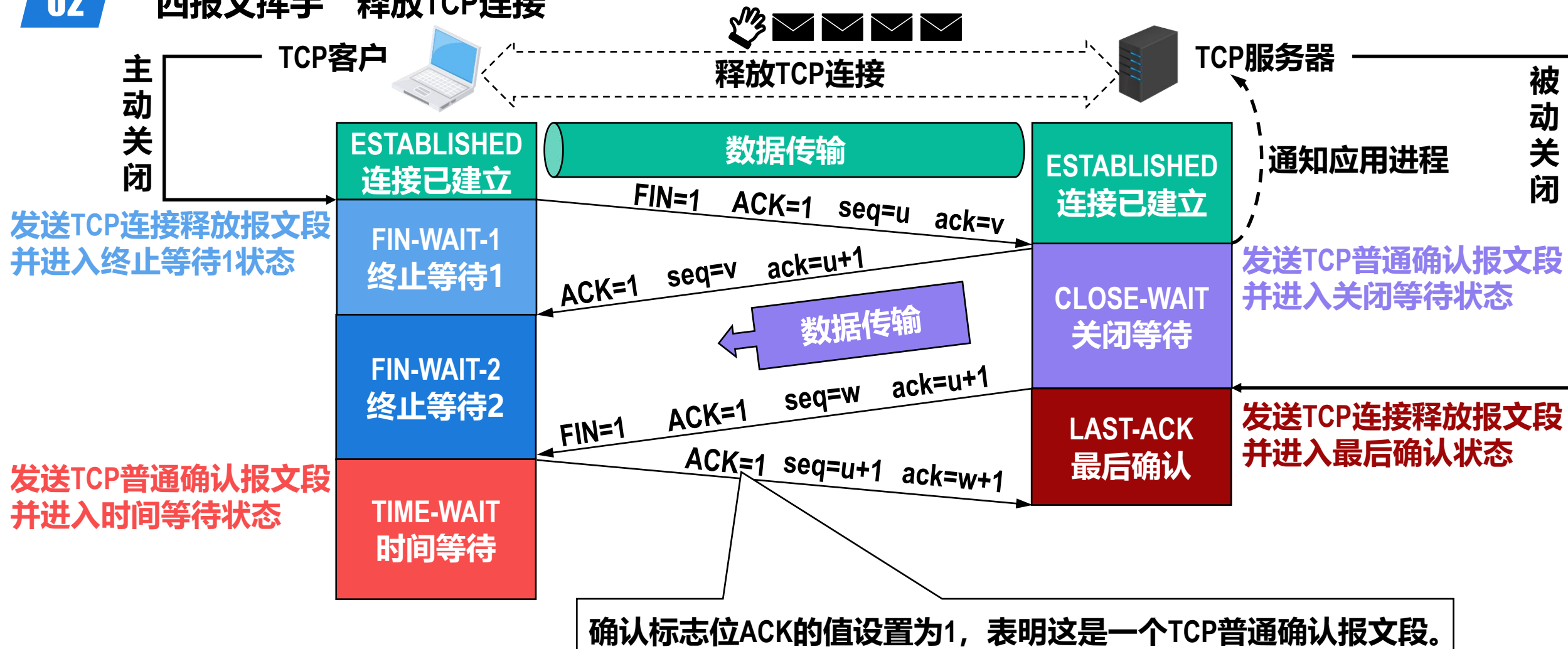


序号seq字段的值假定被设置为w，这是因为在半关闭状态下TCP服务器进程可能又发送了一些数据。

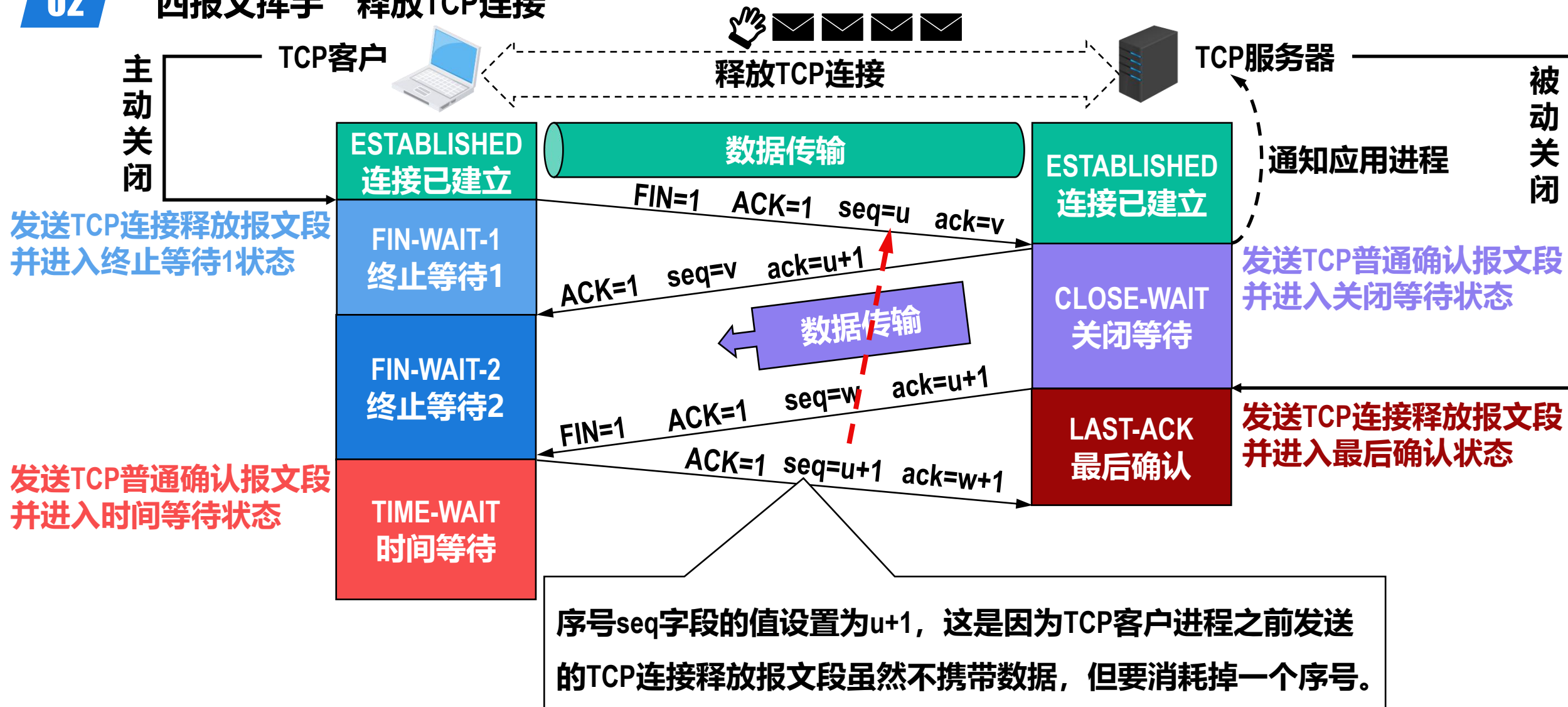
02 “四报文挥手” 释放TCP连接



02 “四报文挥手” 释放TCP连接

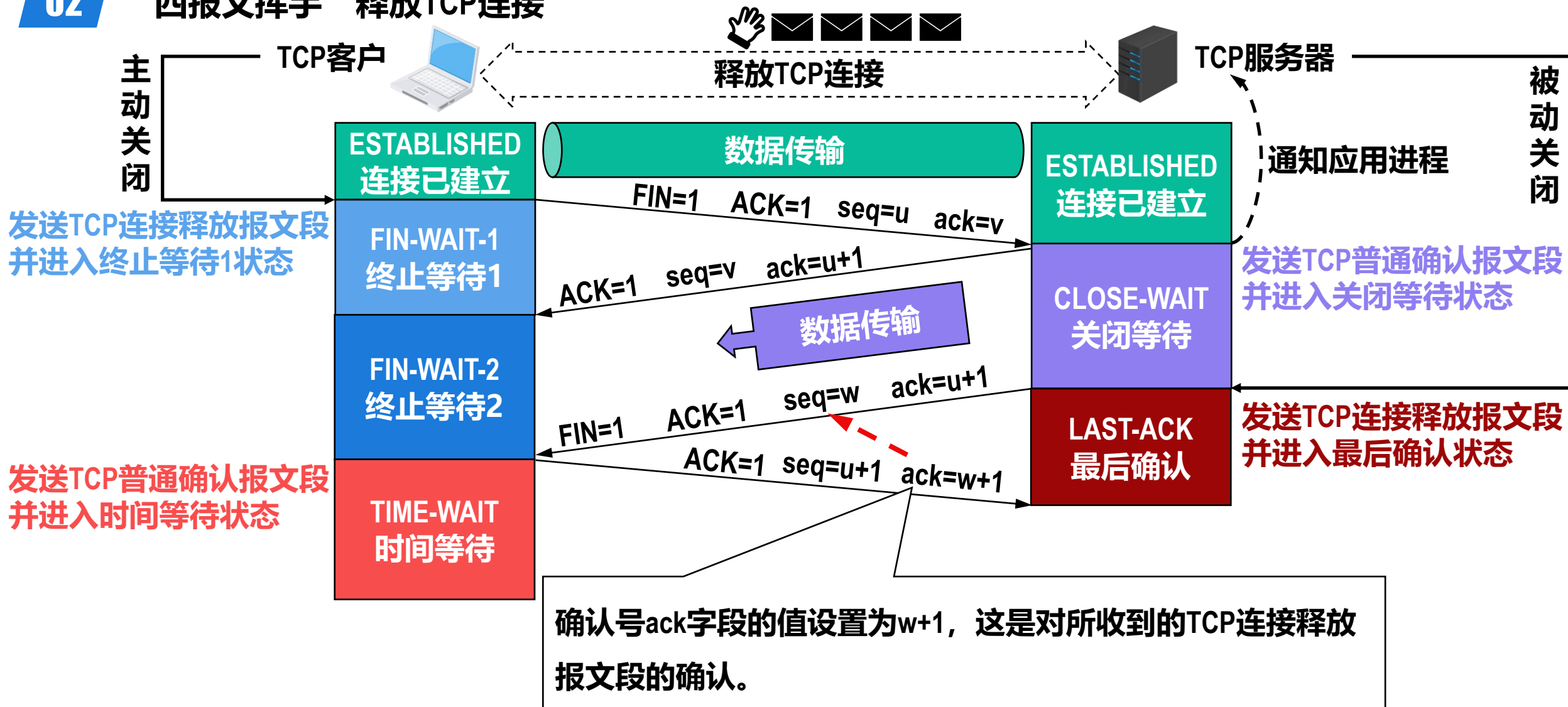


02 “四报文挥手” 释放TCP连接



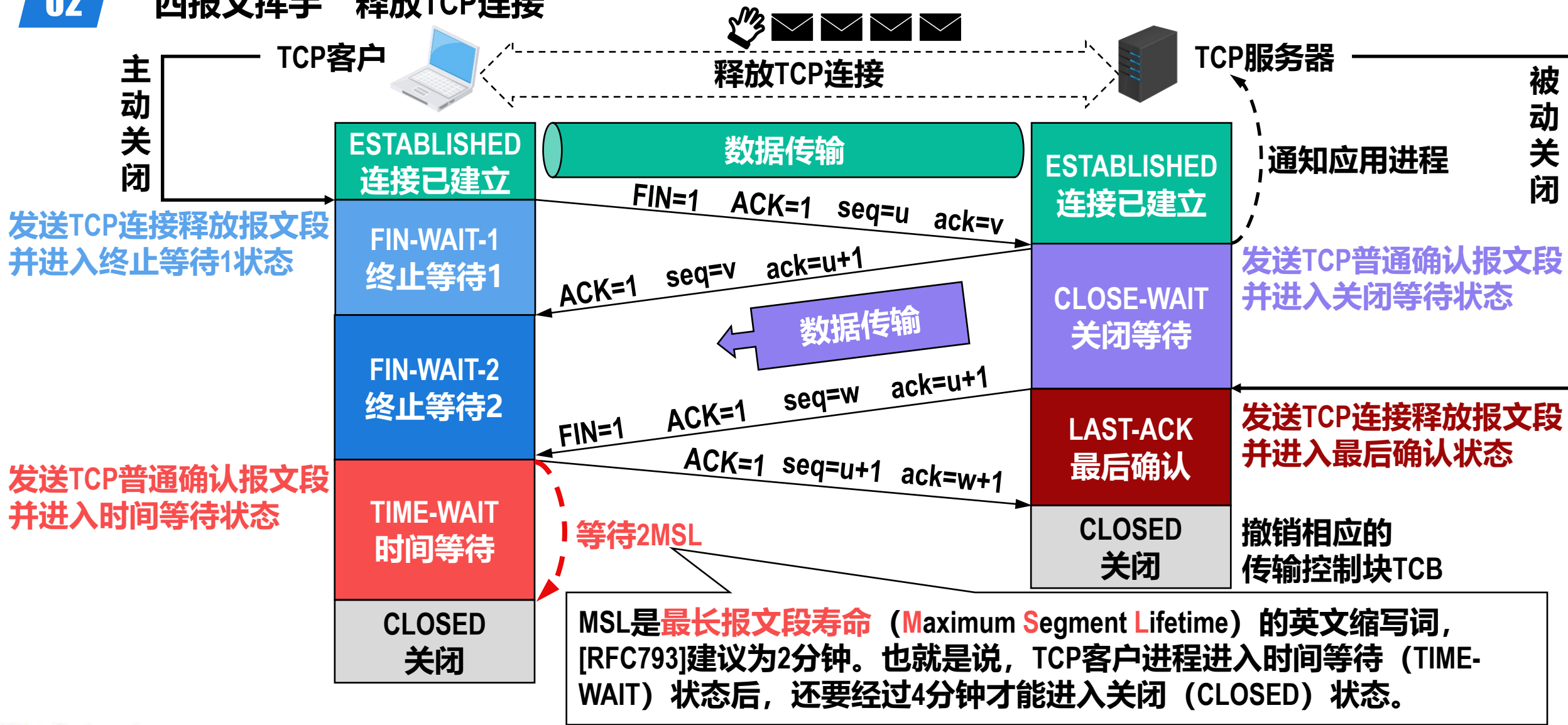
02

“四报文挥手” 释放TCP连接

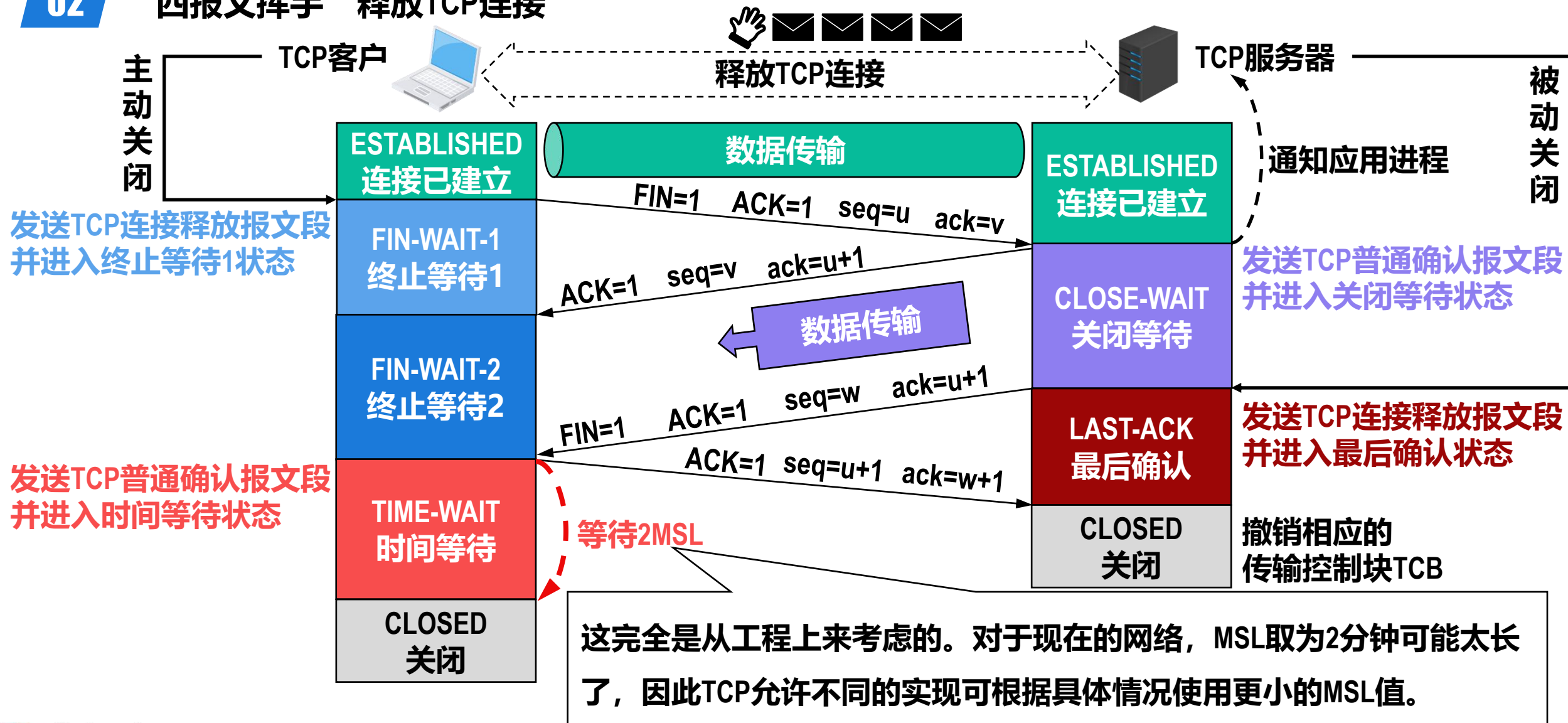


02

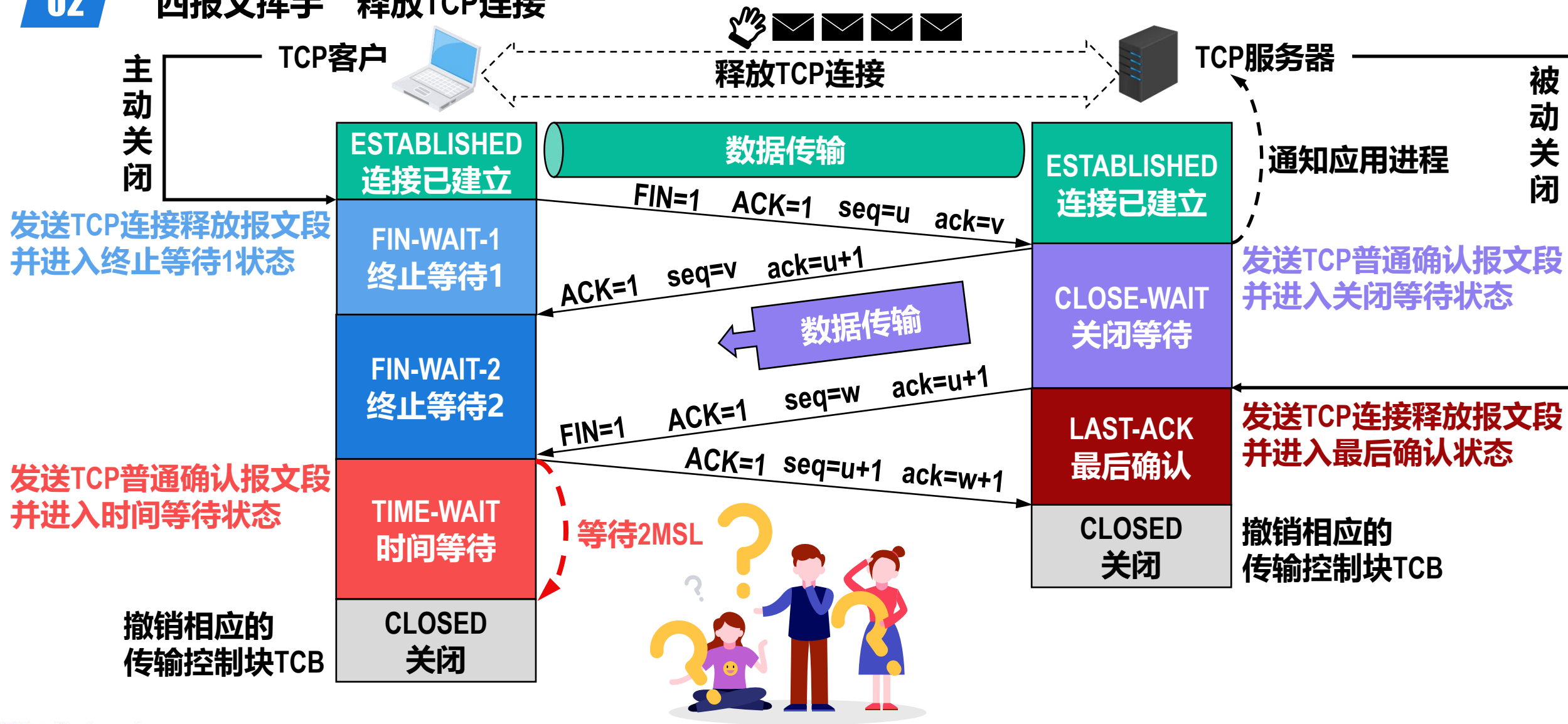
“四报文挥手” 释放TCP连接



02 “四报文挥手” 释放TCP连接



02 “四报文挥手” 释放TCP连接

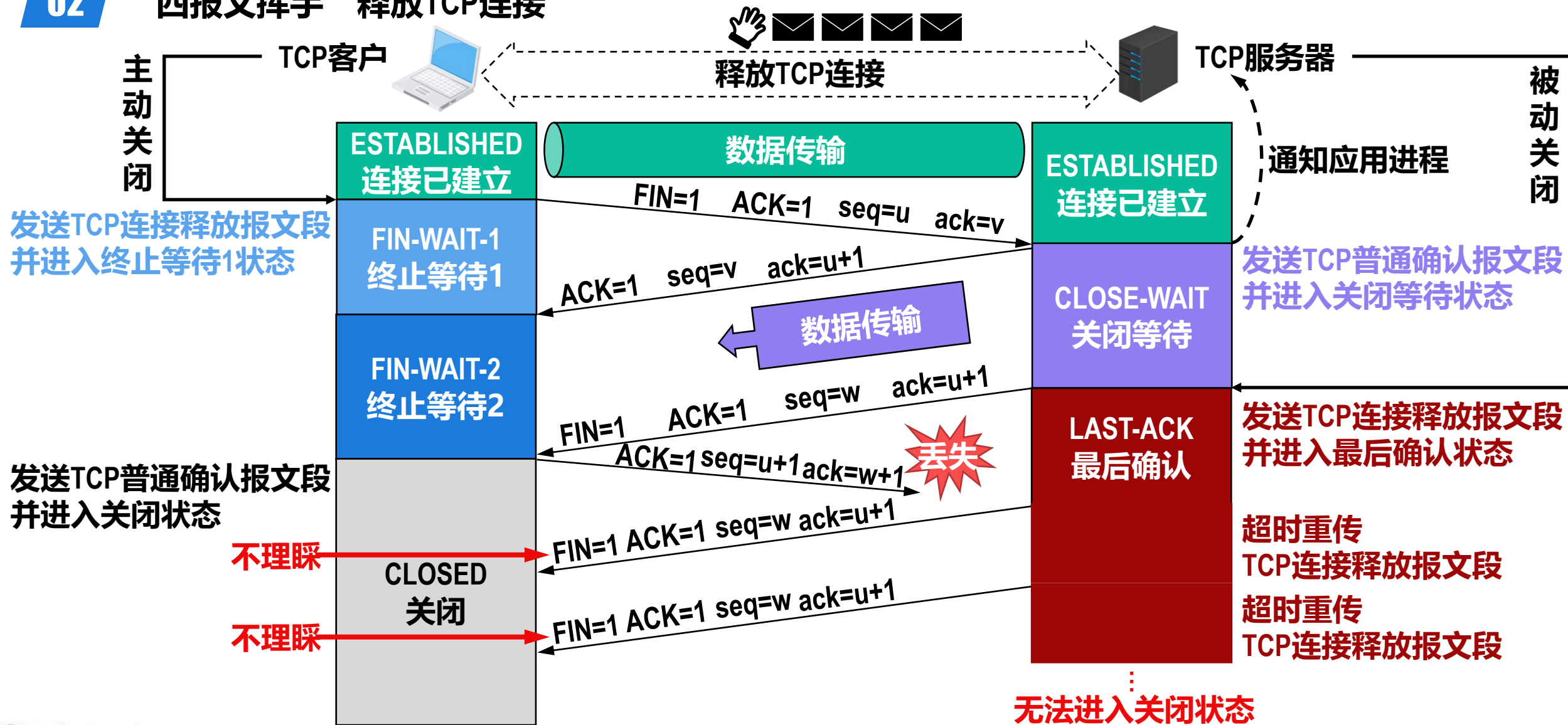


02 “四报文挥手” 释放TCP连接



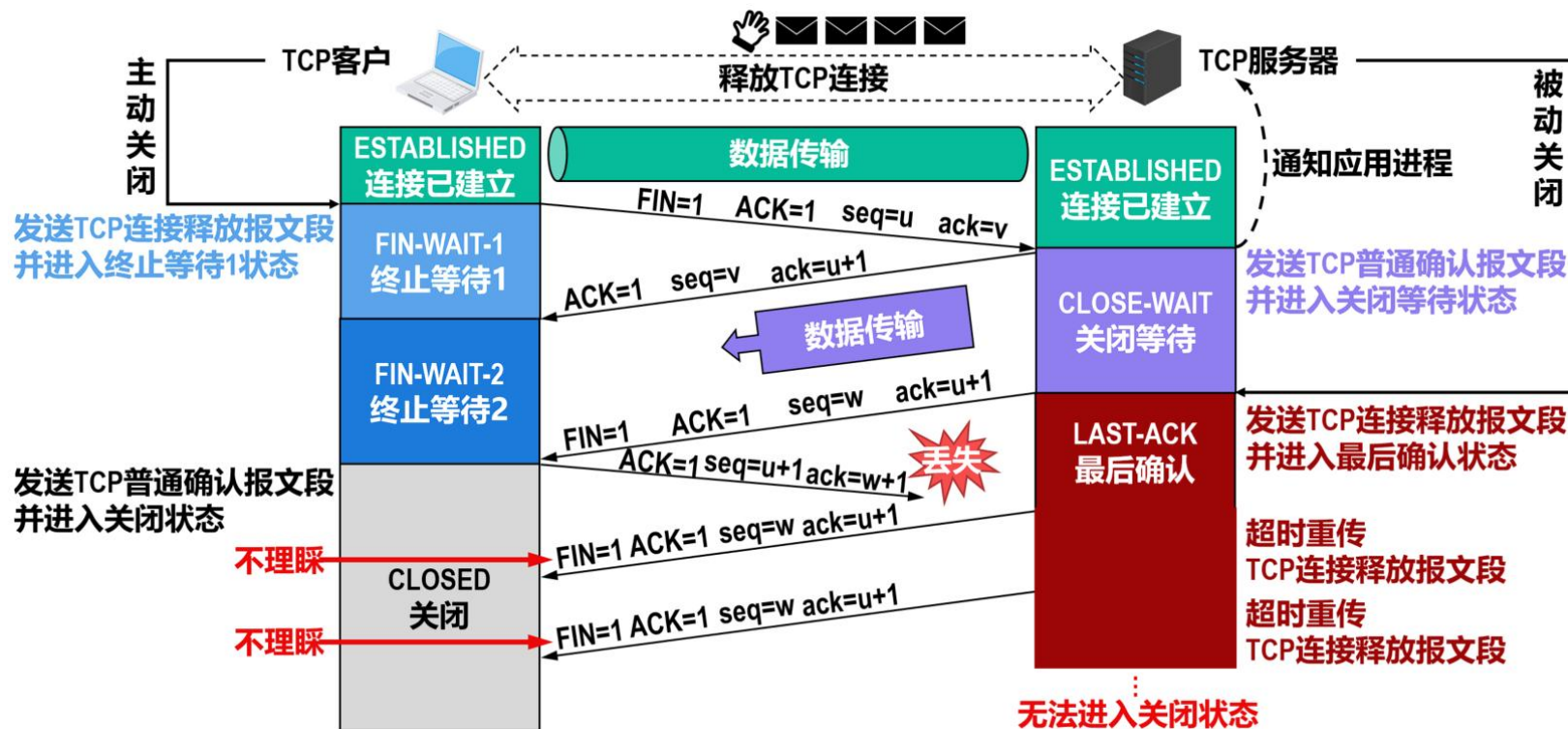
02

“四报文挥手”释放TCP连接



02

“四报文挥手” 释放TCP连接



- 处于时间等待 (TIME-WAIT) 状态后要经过2MSL时长, 可以**确保TCP服务器进程能够收到最后一个TCP确认报文段而进入关闭 (CLOSED) 状态。**
- 另外, TCP客户进程在发送完最后一个TCP确认报文段后, 再经过2MSL时长, 就可以**使本次连接持续时间内所产生出的所有报文段都从网络中消失。**这样就可以使下一个**新的TCP连接中不会出现旧连接中的报文段。**

02 “四报文挥手” 释放TCP连接

【2020年 题39】若主机甲与主机乙建立TCP连接时发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（C）。

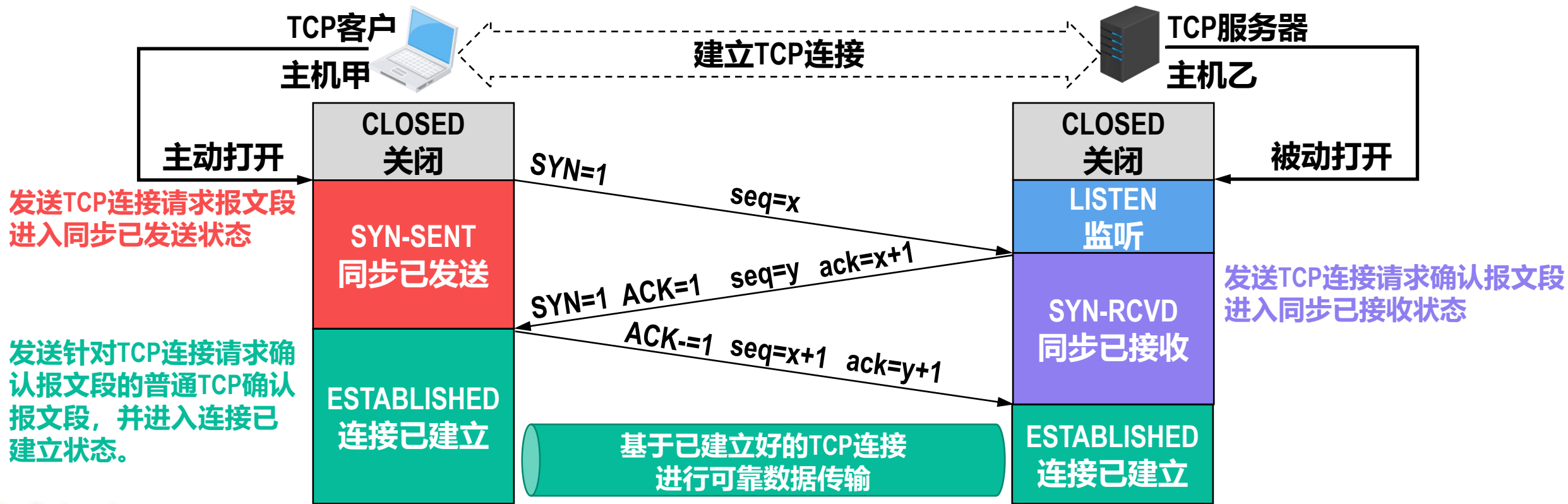
解析

A. 4002

B. 4001

C. 4000

D. 3999



02 “四报文挥手” 释放TCP连接

【2020年 题39】若主机甲与主机乙建立TCP连接时发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（C）。

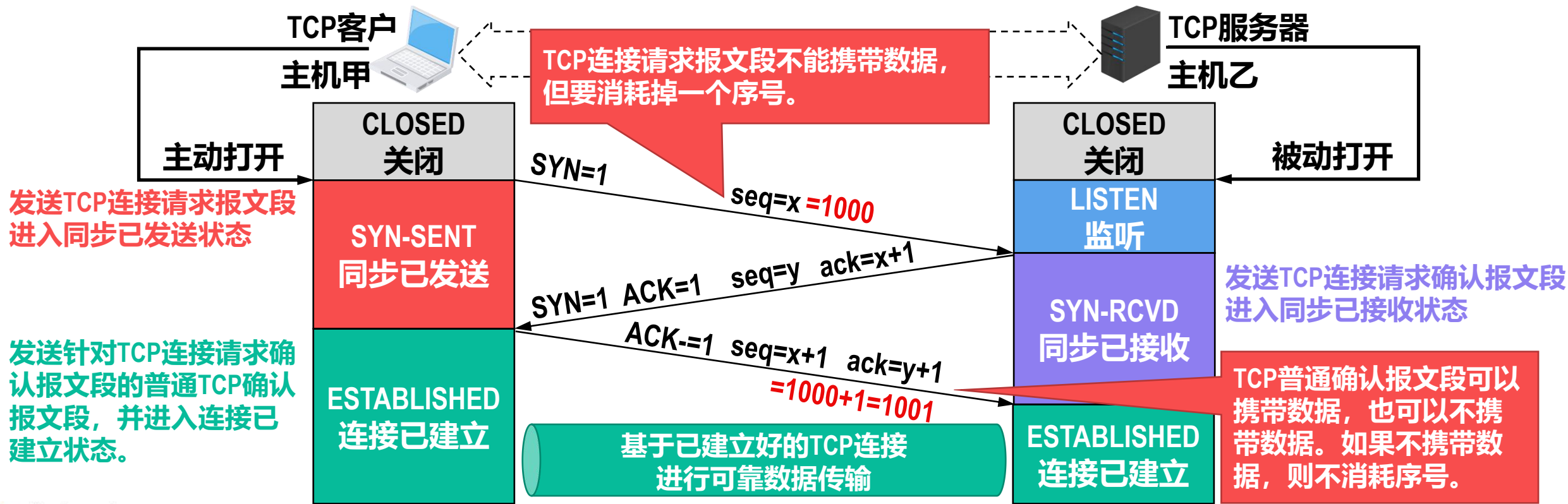
A. 4002

B. 4001

C. 4000

D. 3999

解析



02

“四报文挥手” 释放TCP连接

【2020年 题39】若主机甲与主机乙建立TCP连接时发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（C）。

解析

A. 4002

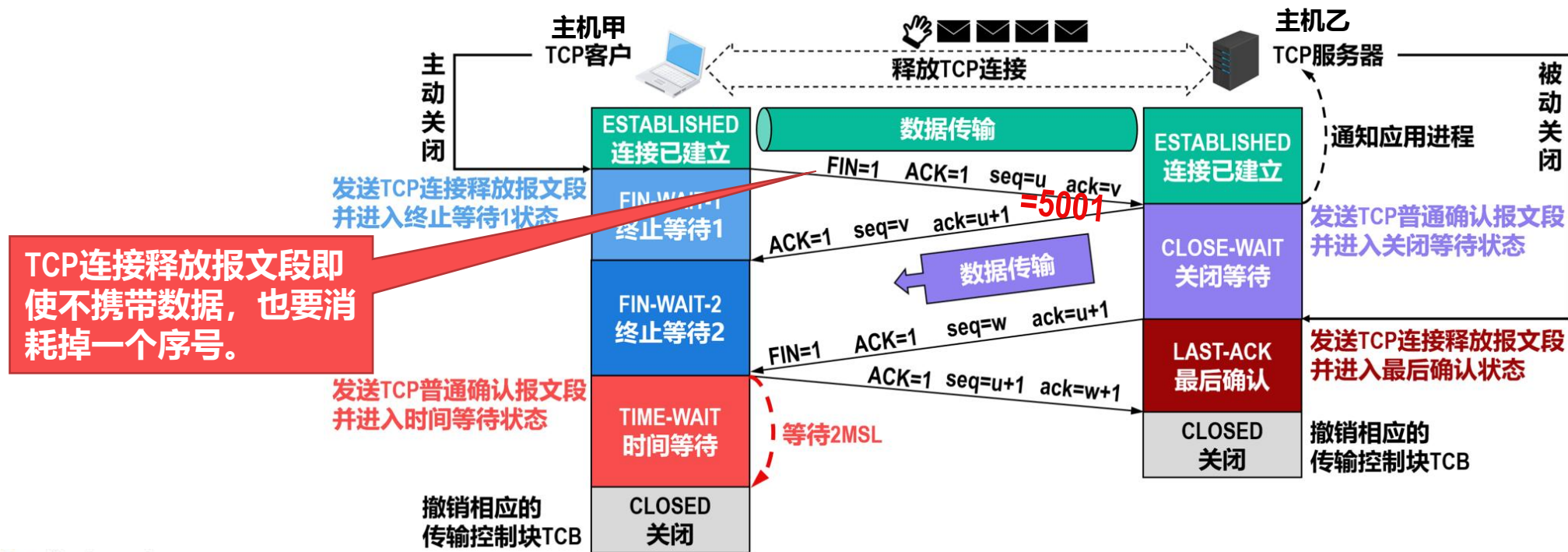
B. 4001

C. 4000

D. 3999

甲给乙发送的**第一个应用层数据字节**的TCP序号为1001，因为应用层数据作为数据载荷被封装在TCP报文段中。

甲在发送FIN段之前，给乙发送的**最后一个应用层数据字节**的TCP序号为5000。



02

“四报文挥手” 释放TCP连接

【2020年 题39】若主机甲与主机乙建立TCP连接时发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（**C**）。

解析

A. 4002

B. 4001

C. 4000

D. 3999

甲给乙发送的**第一个应用层数据字节**的TCP序号为1001，因为应用层数据作为数据载荷被封装在TCP报文段中。

甲在发送FIN段之前，给乙发送的**最后一个应用层数据字节**的TCP序号为5000。

综上所述，甲向乙已经发送了**字节序号为1001~5000共4000个字节的应用层数据**。

TCP保活计时器的作用

出现故障

TCP客户

ESTABLISHED
连接已建立基于已建立的TCP连接
进行可靠数据传输

TCP服务器

ESTABLISHED
连接已建立

如何发现？

不可能再发送数据

出现故障

TCP客户



TCP保活计时器的作用



TCP服务器

ESTABLISHED
连接已建立基于已建立的TCP连接
进行可靠数据传输ESTABLISHED
连接已建立

TCP报文段

不可能再发送数据

TCP探测报文段

TCP探测报文段

TCP探测报文段

启动保活计时器
(通常为2小时)

保活计时器到时

75秒后

75秒后

若一连发送10个探测报文段后仍无TCP客户进程的响应，TCP服务器进程就认为TCP客户进程所在主机出了故障，于是就关闭这个连接。

5.3.3 TCP的流量控制

01

TCP流量控制的基本概念

02

TCP的流量控制方法

01 TCP流量控制的基本概念

TCP客户



发送方

发送缓存

接收方

接收缓存



TCP服务器

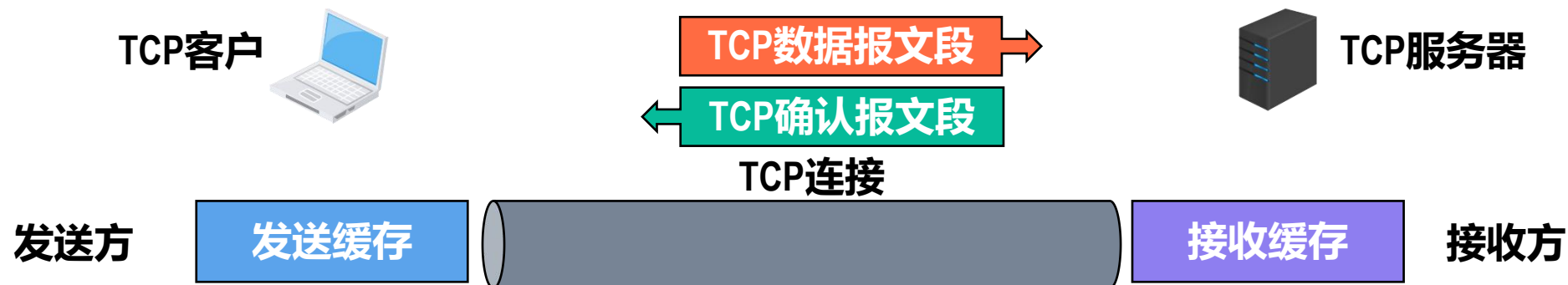
接收缓存

接收方

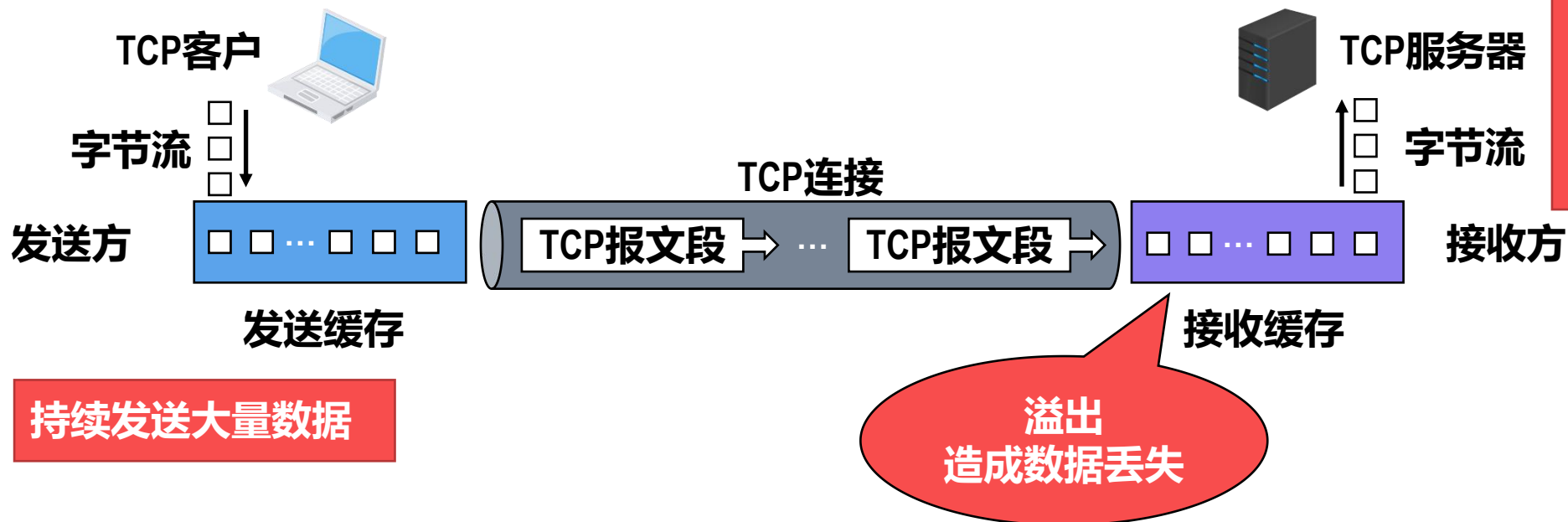
发送缓存

发送方

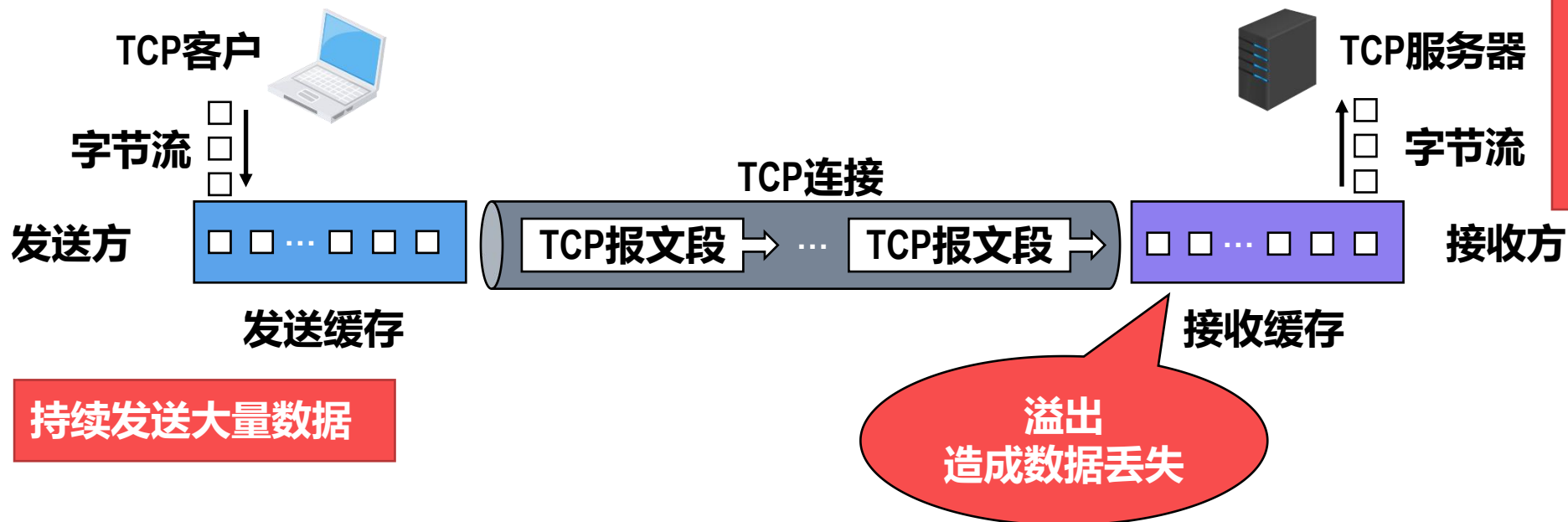
01 TCP流量控制的基本概念



01 TCP流量控制的基本概念

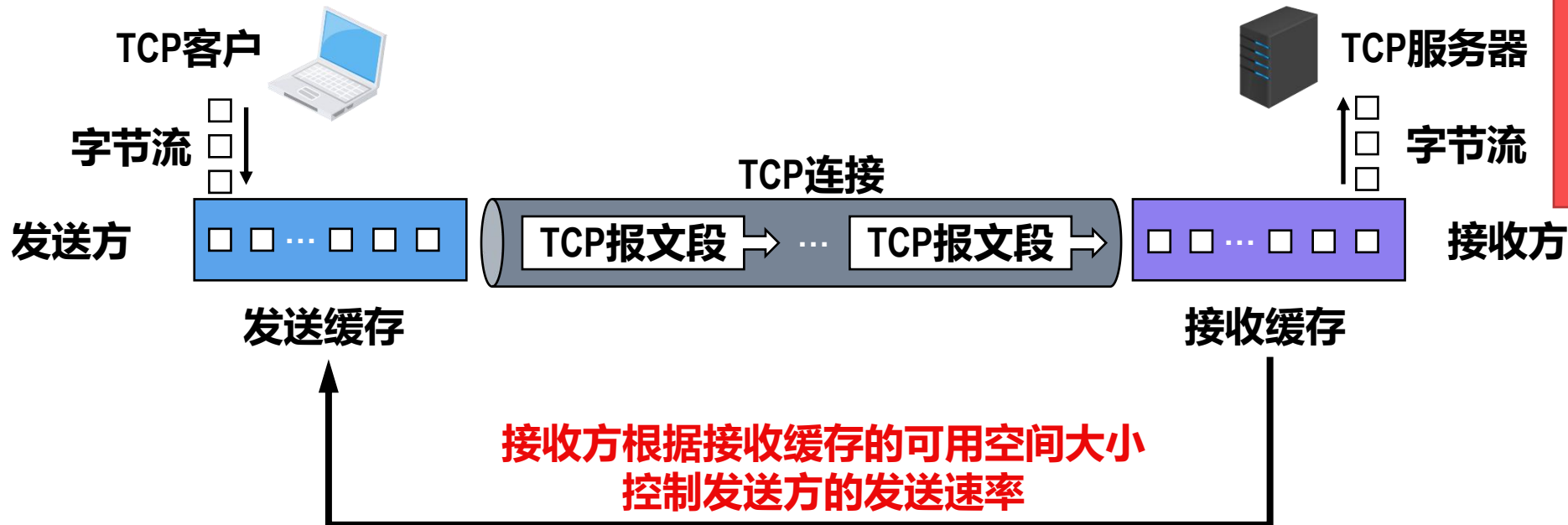


01 TCP流量控制的基本概念



■ TCP为应用程序提供了**流量控制**（Flow Control）机制，以**解决因发送方发送数据太快而导致接收方来不及接收，造成接收方的接收缓存溢出的问题。**

01 TCP流量控制的基本概念



应用程序可能正忙于其他任务，并不一定能够立刻取走数据。

■ TCP为应用程序提供了**流量控制**（Flow Control）机制，以**解决因发送方发送数据太快而导致接收方来不及接收，造成接收方的接收缓存溢出的问题。**

■ **流量控制的基本方法：接收方根据自己的接收能力（接收缓存的可用空间大小）控制发送方的发送速率。**

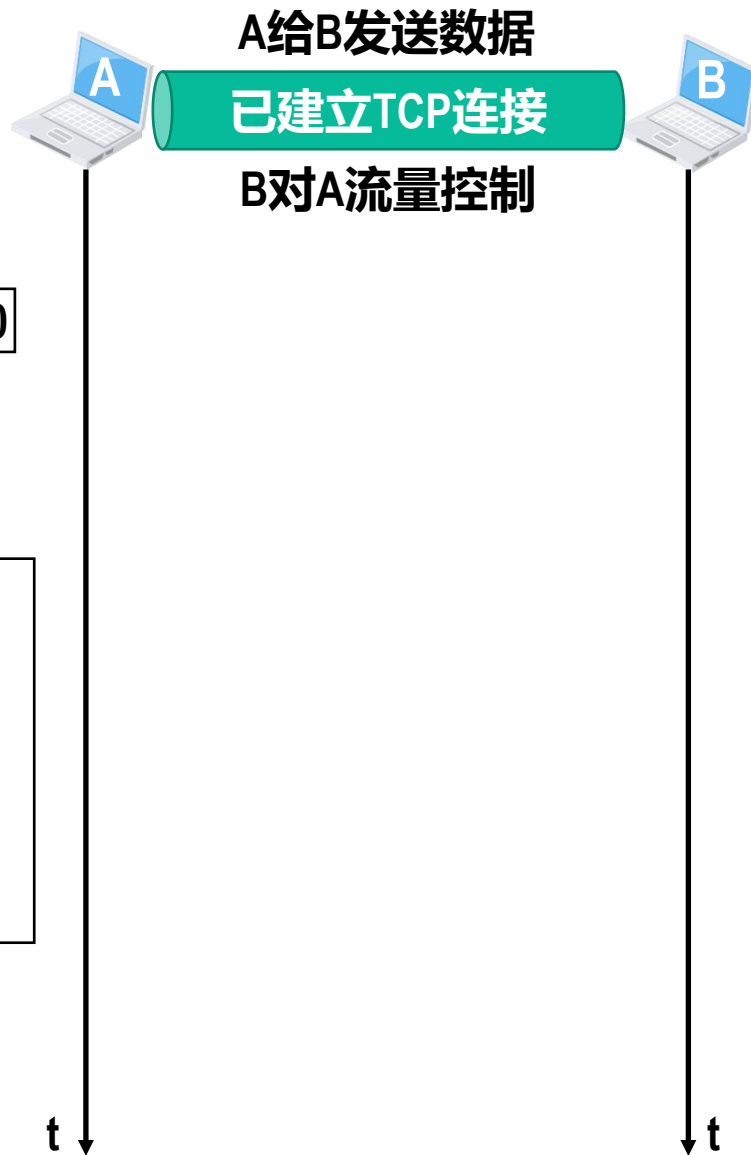
02 TCP的流量控制方法

swnd=400

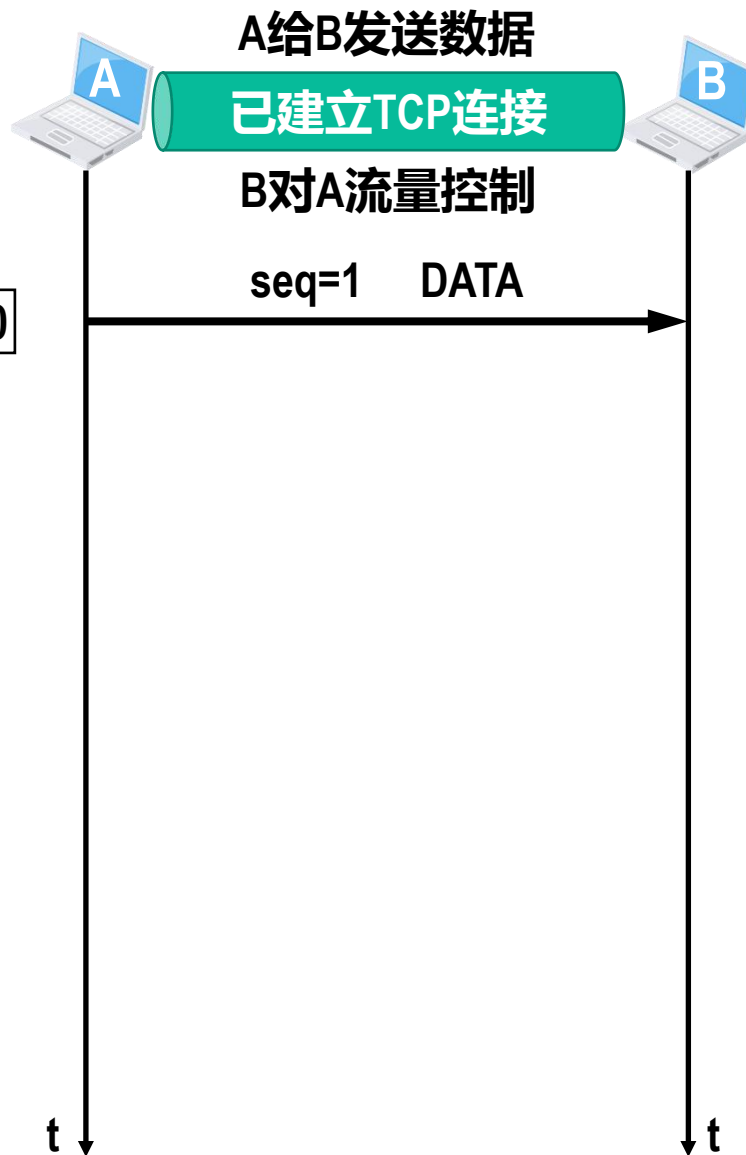
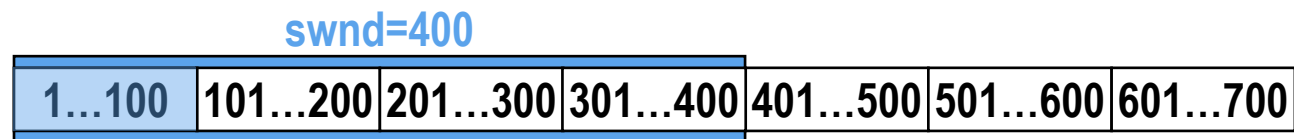
1...100	101...200	201...300	301...400	401...500	501...600	601...700
---------	-----------	-----------	-----------	-----------	-----------	-----------

假设：

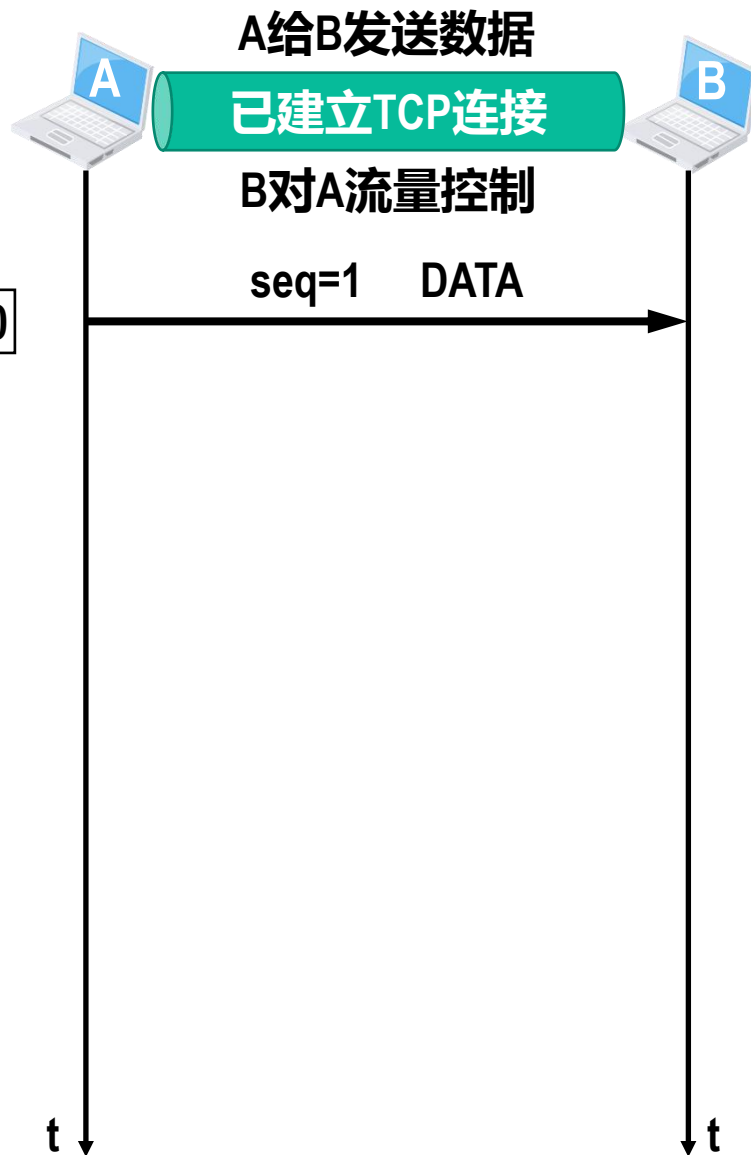
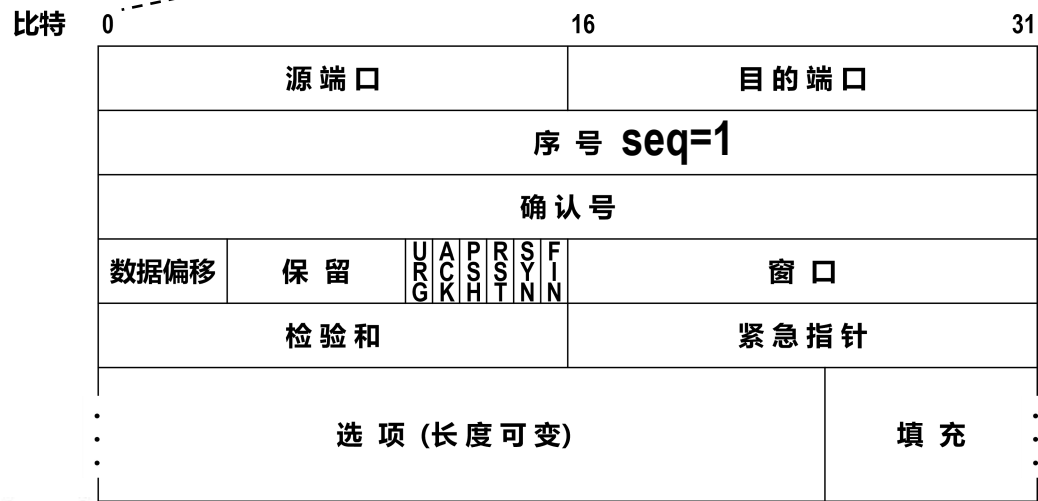
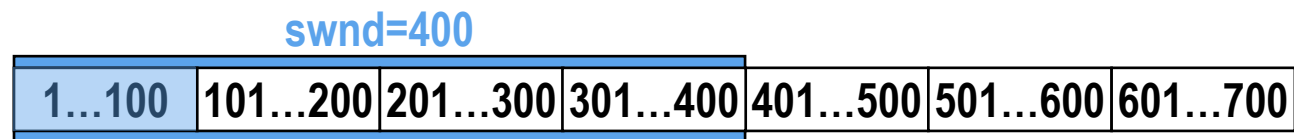
- ① 网络不会拥塞（不考虑TCP的拥塞控制）
- ② 在A和B建立TCP连接时，B告诉A：“我的接收窗口 rwnd=400”，因此A将自己的发送窗口swnd也设置为400。



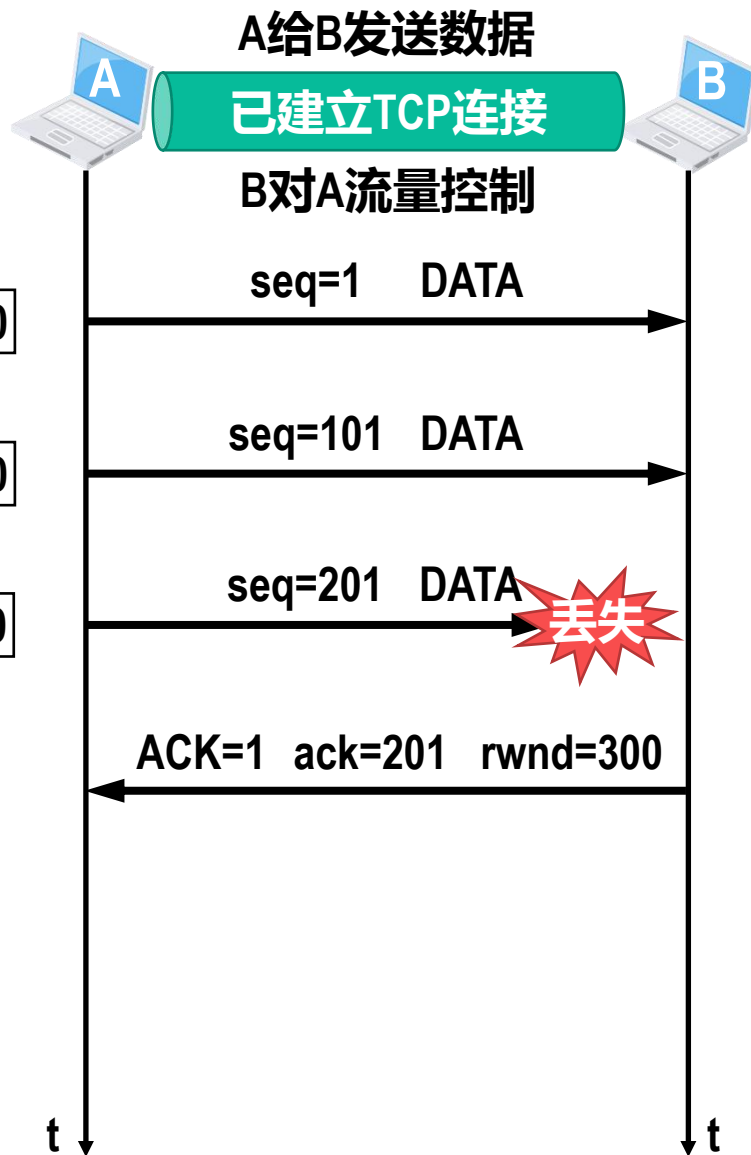
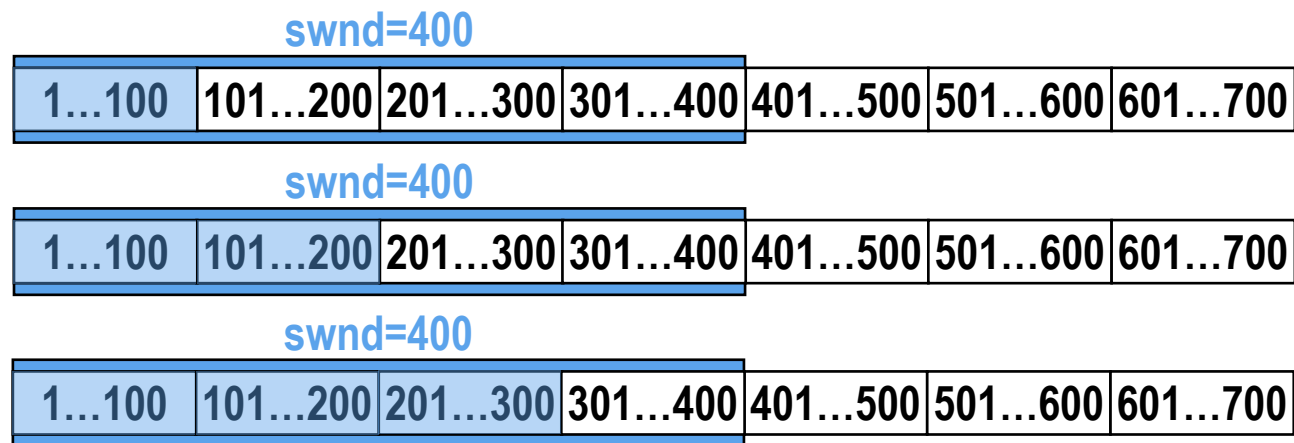
02 TCP的流量控制方法



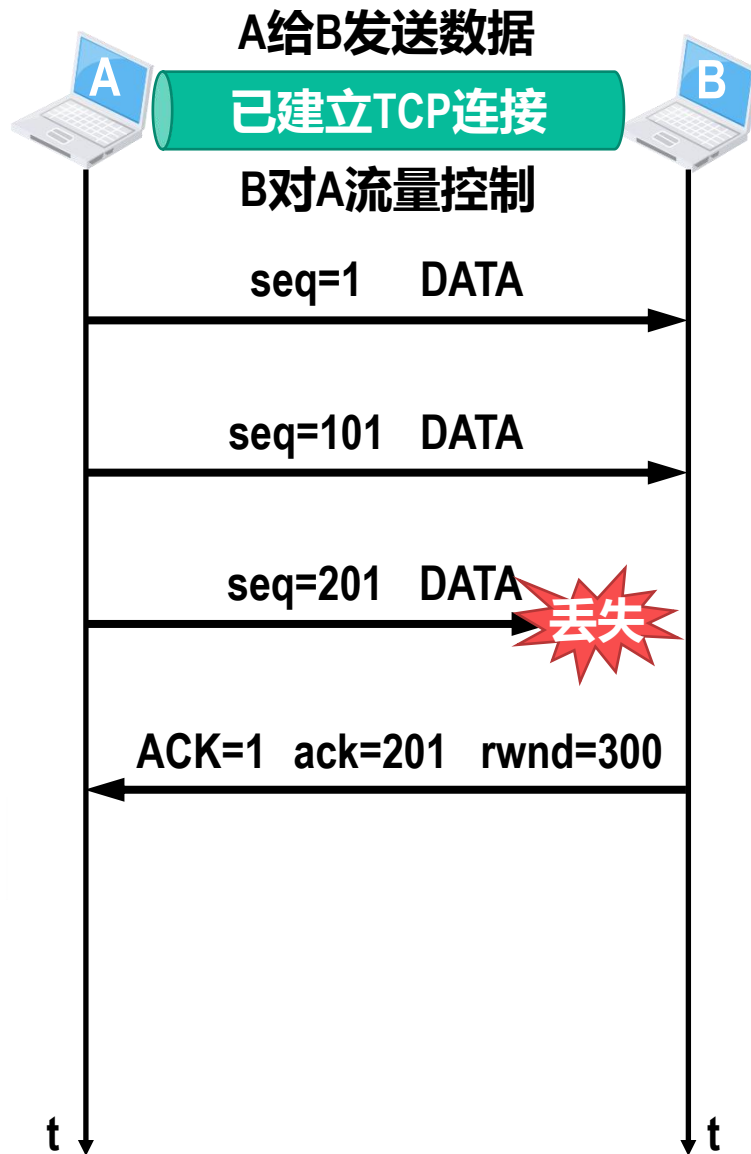
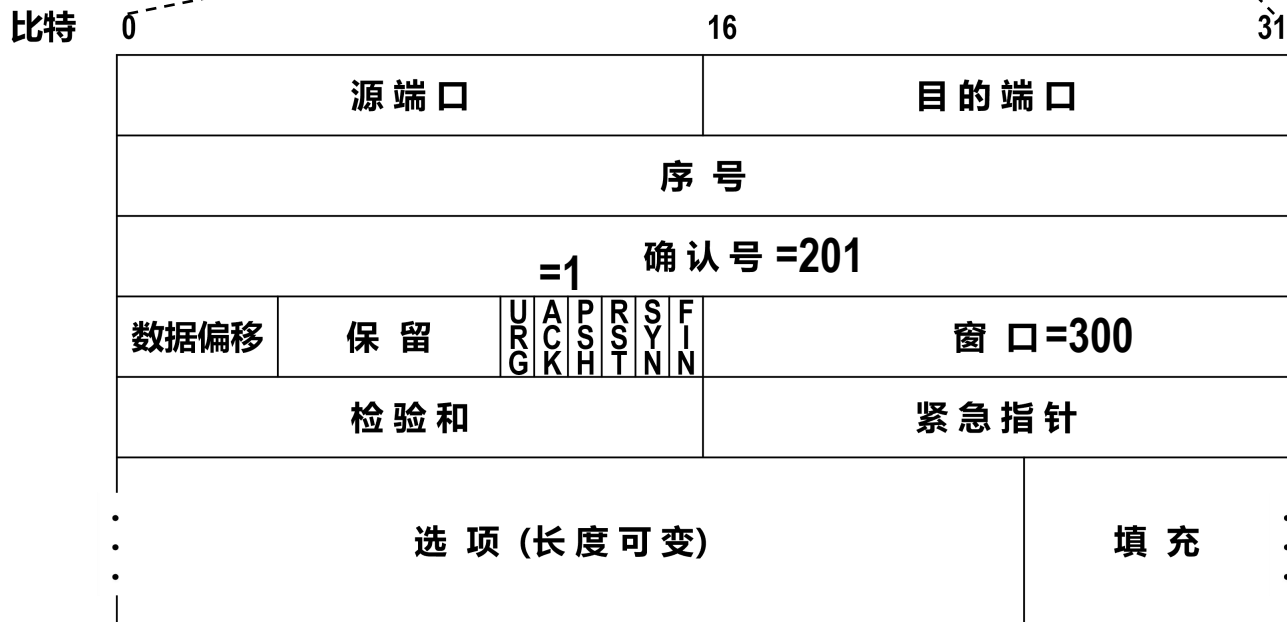
02 TCP的流量控制方法



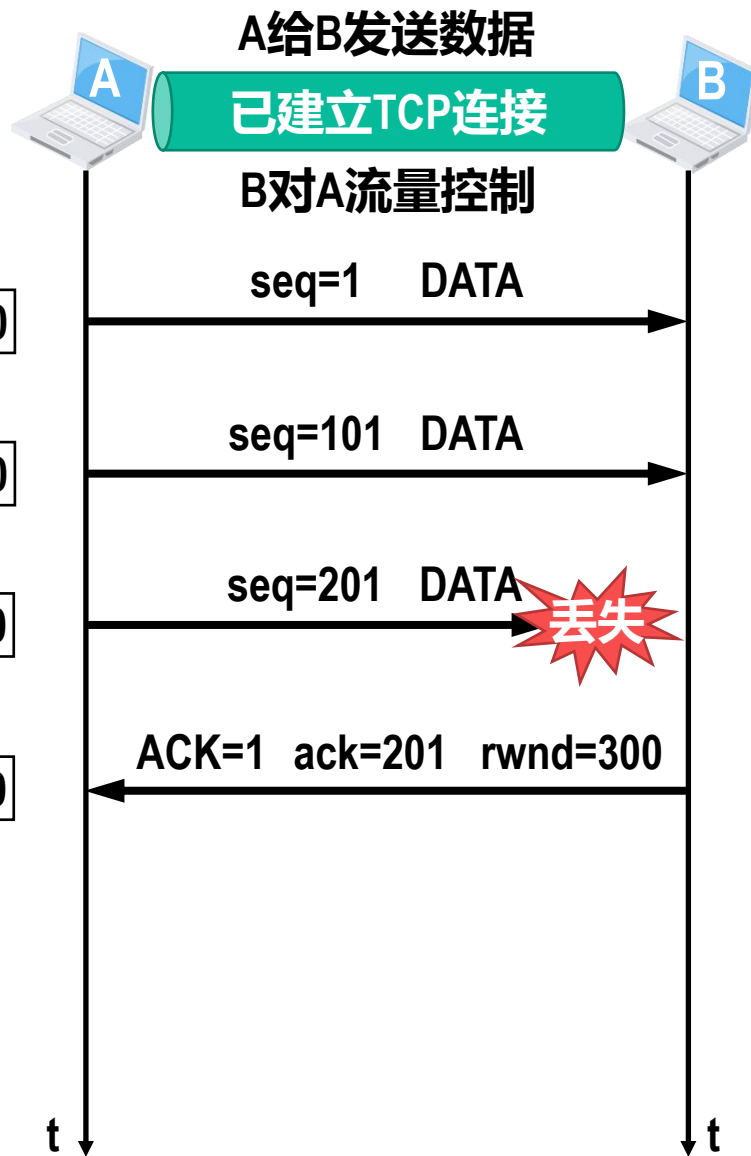
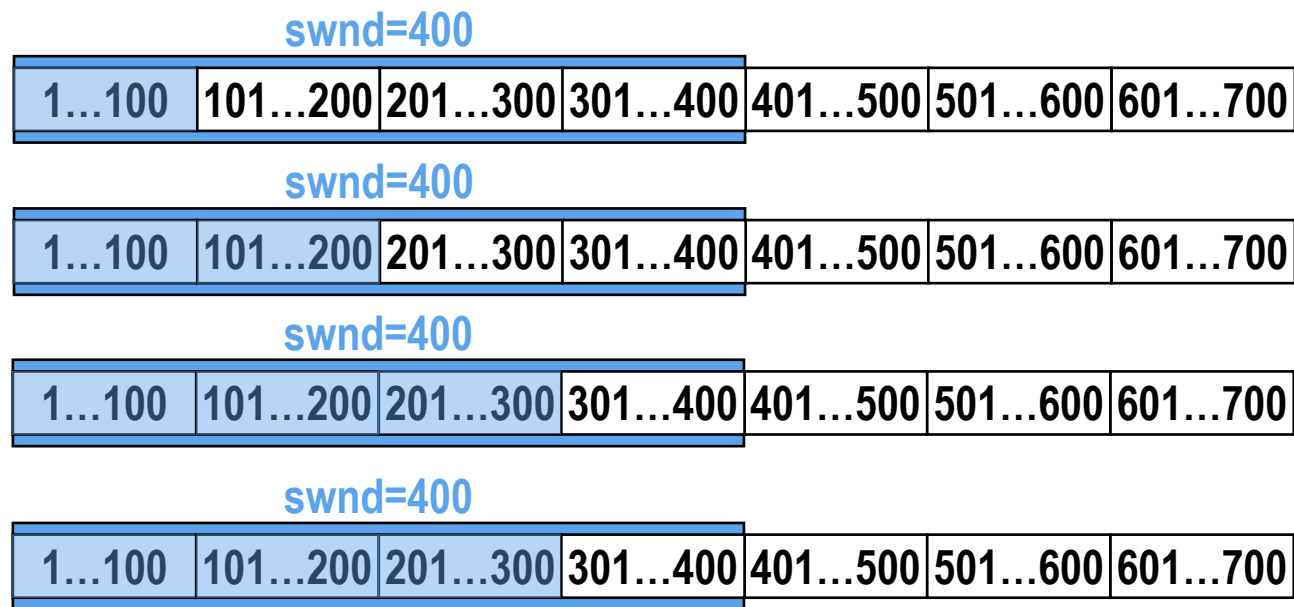
02 TCP的流量控制方法



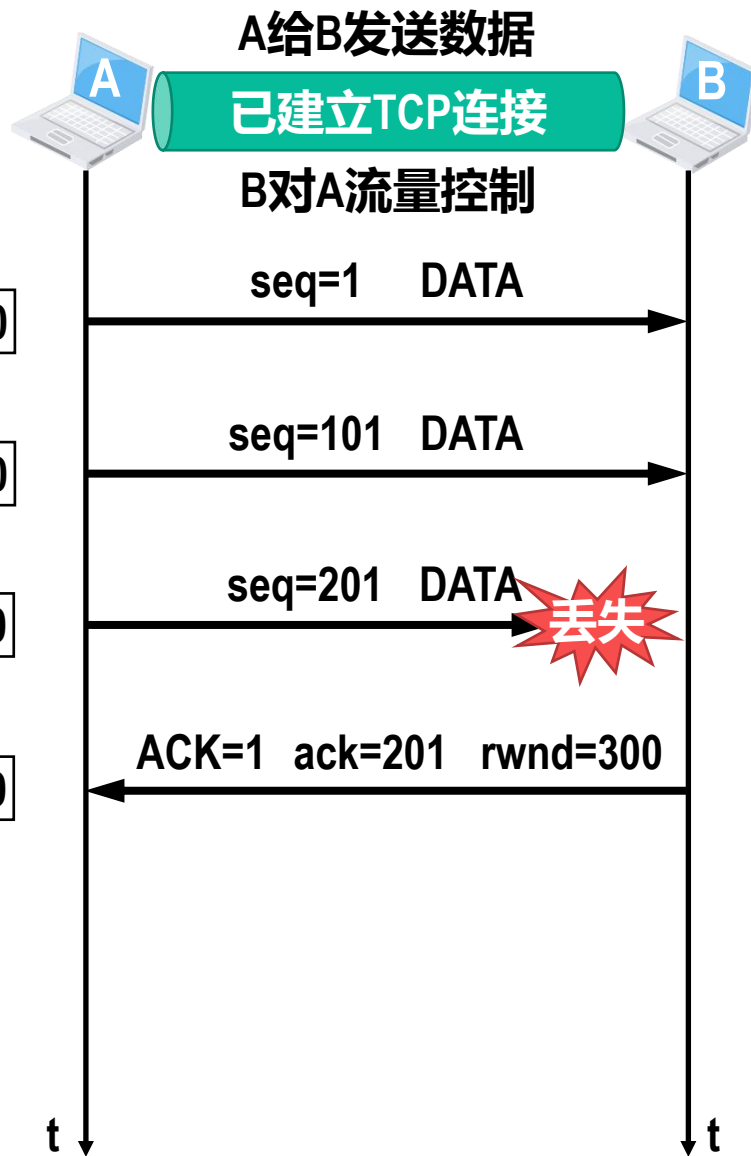
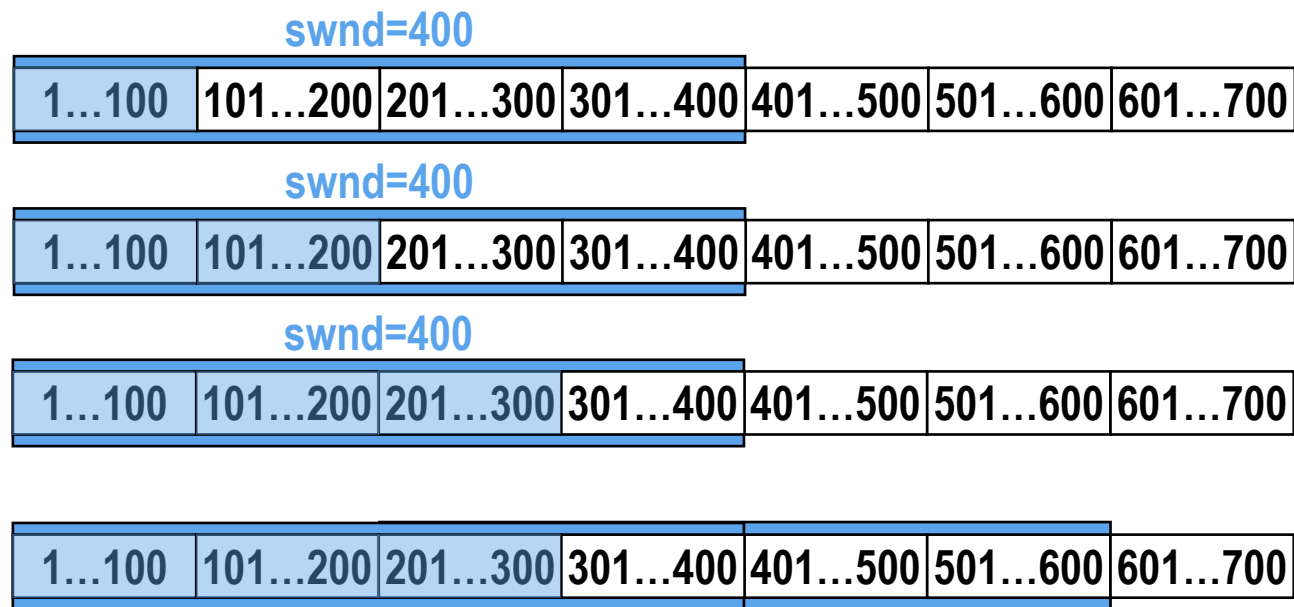
02 TCP的流量控制方法



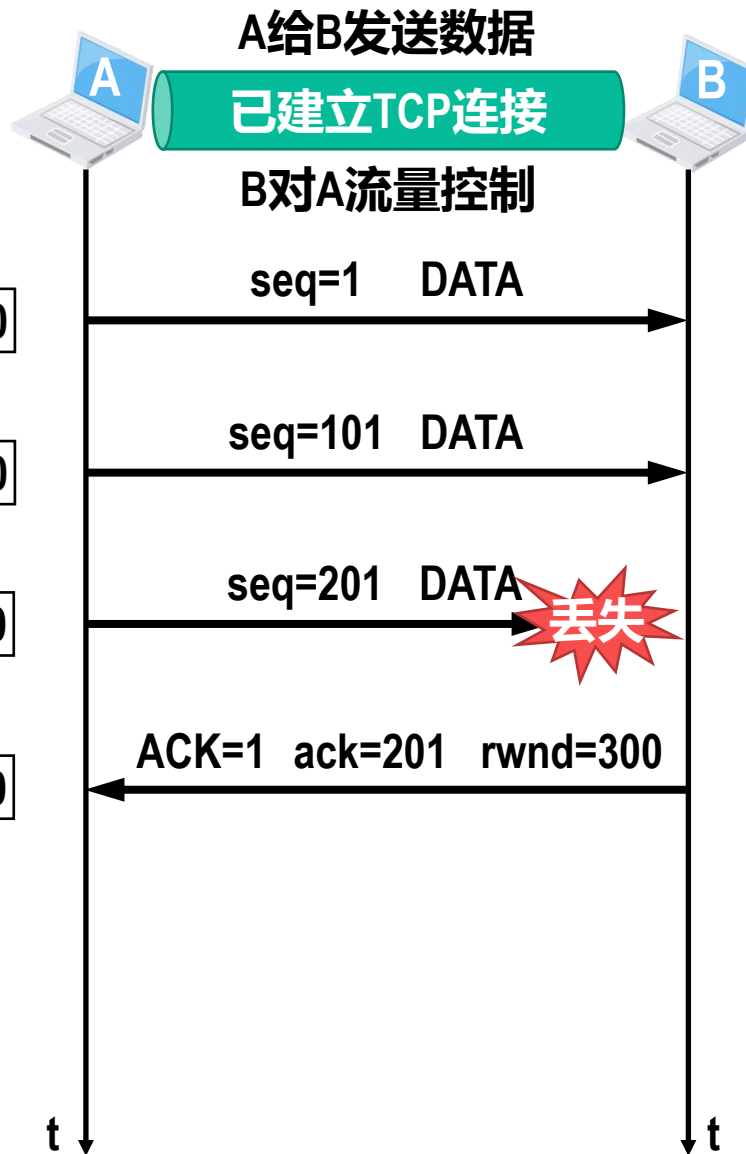
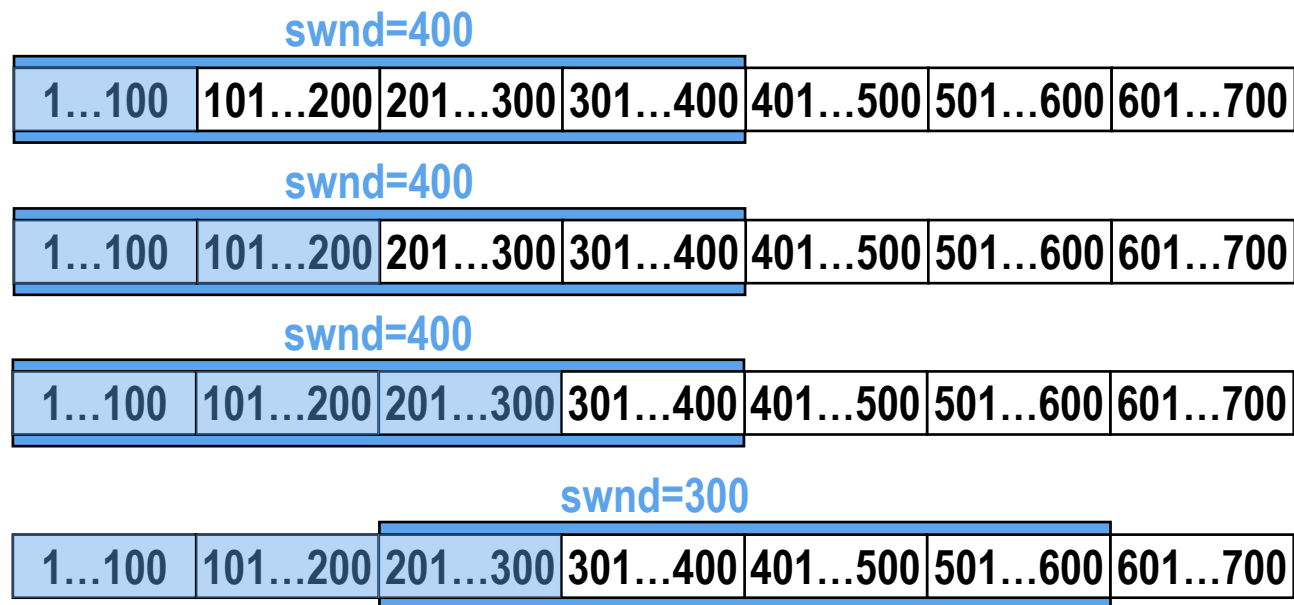
02 TCP的流量控制方法



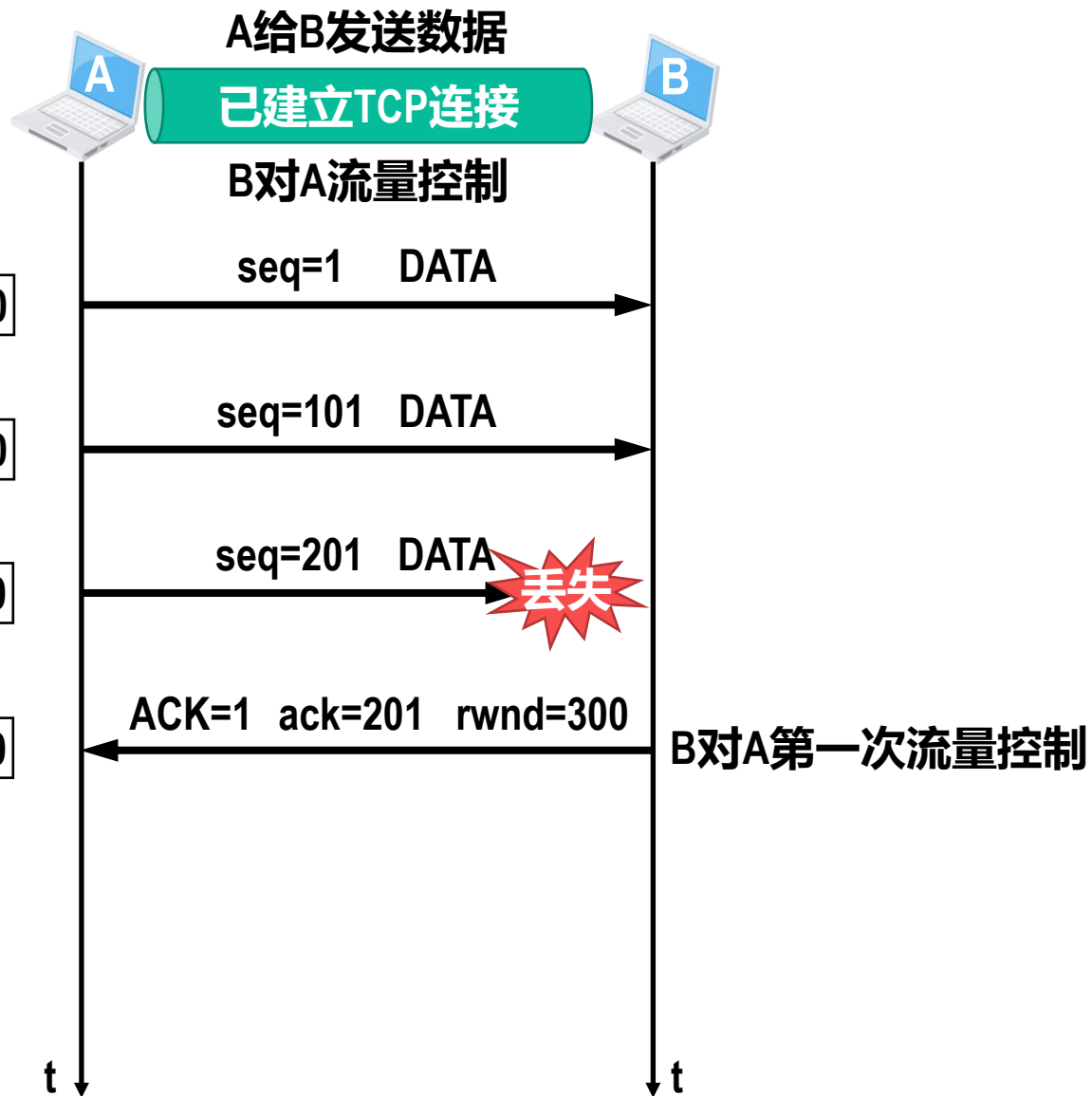
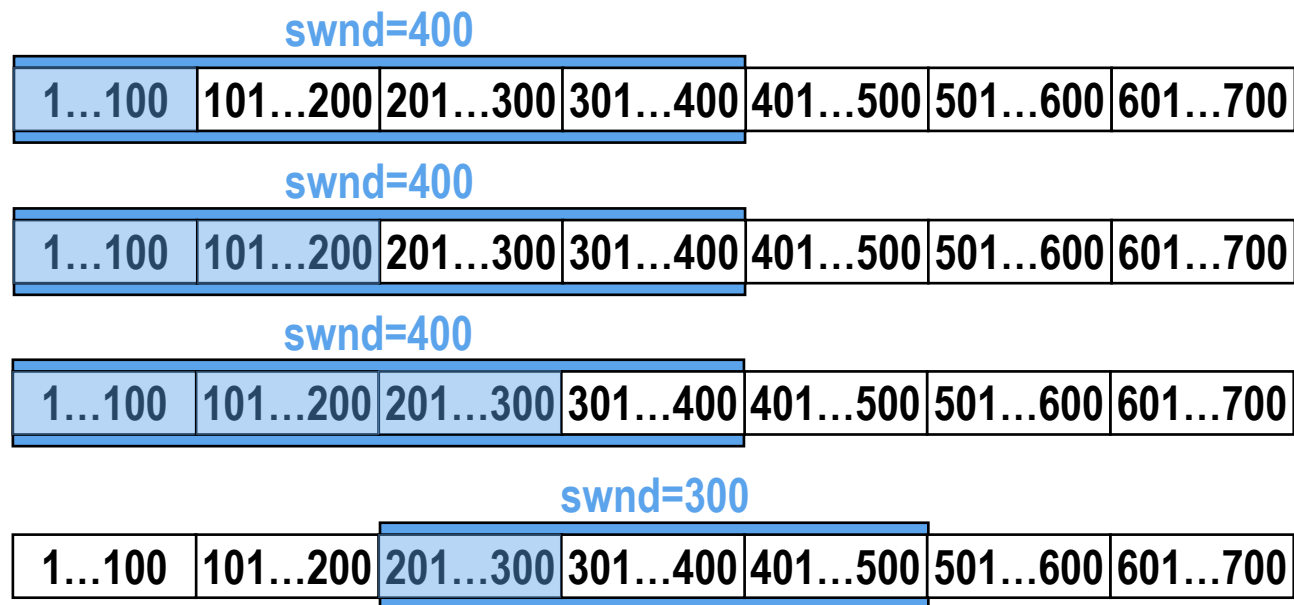
02 TCP的流量控制方法



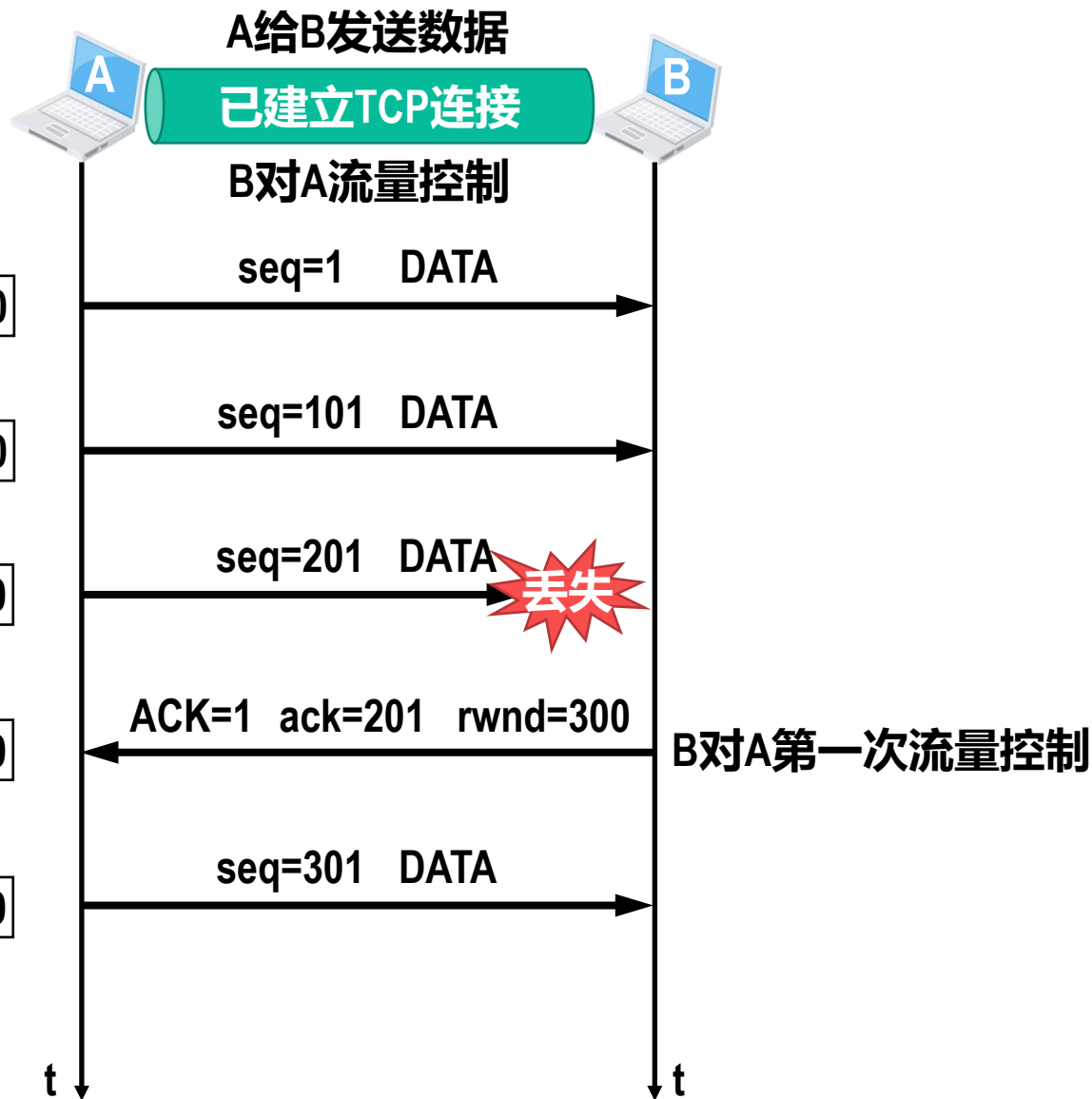
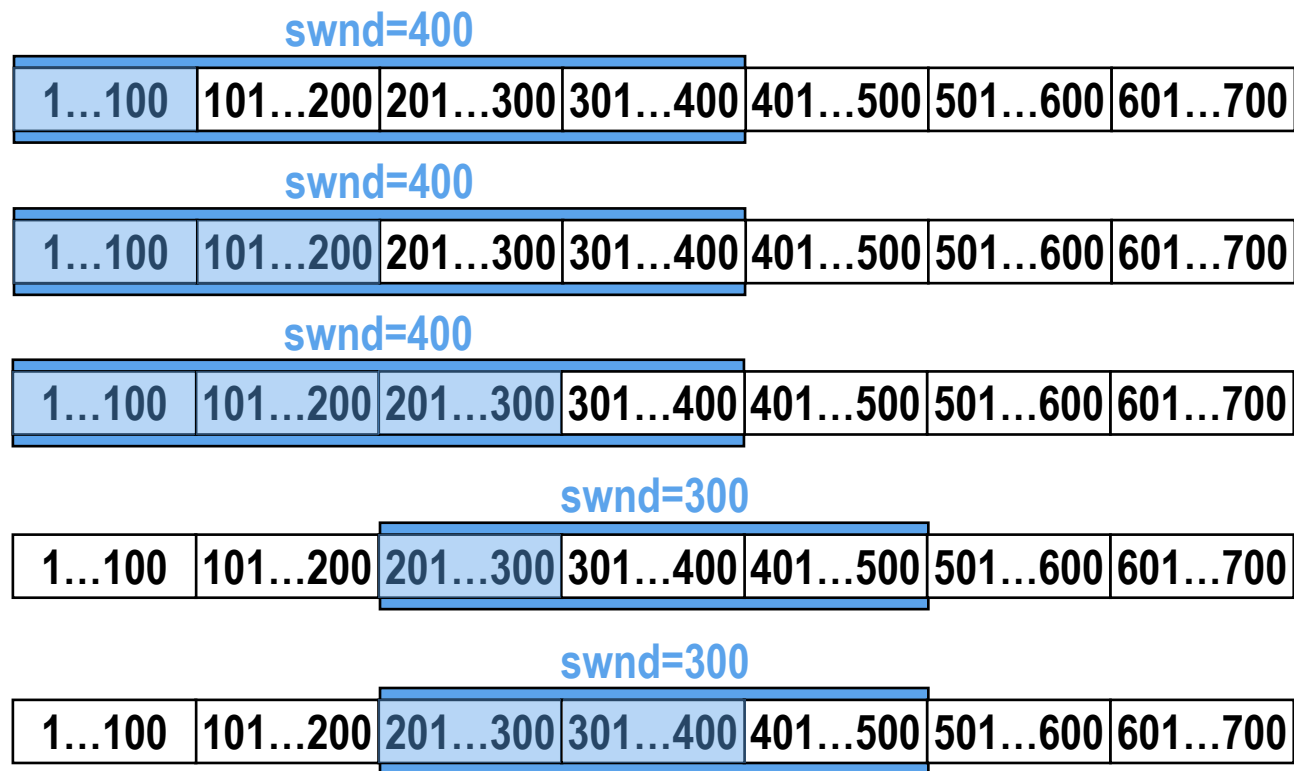
02 TCP的流量控制方法



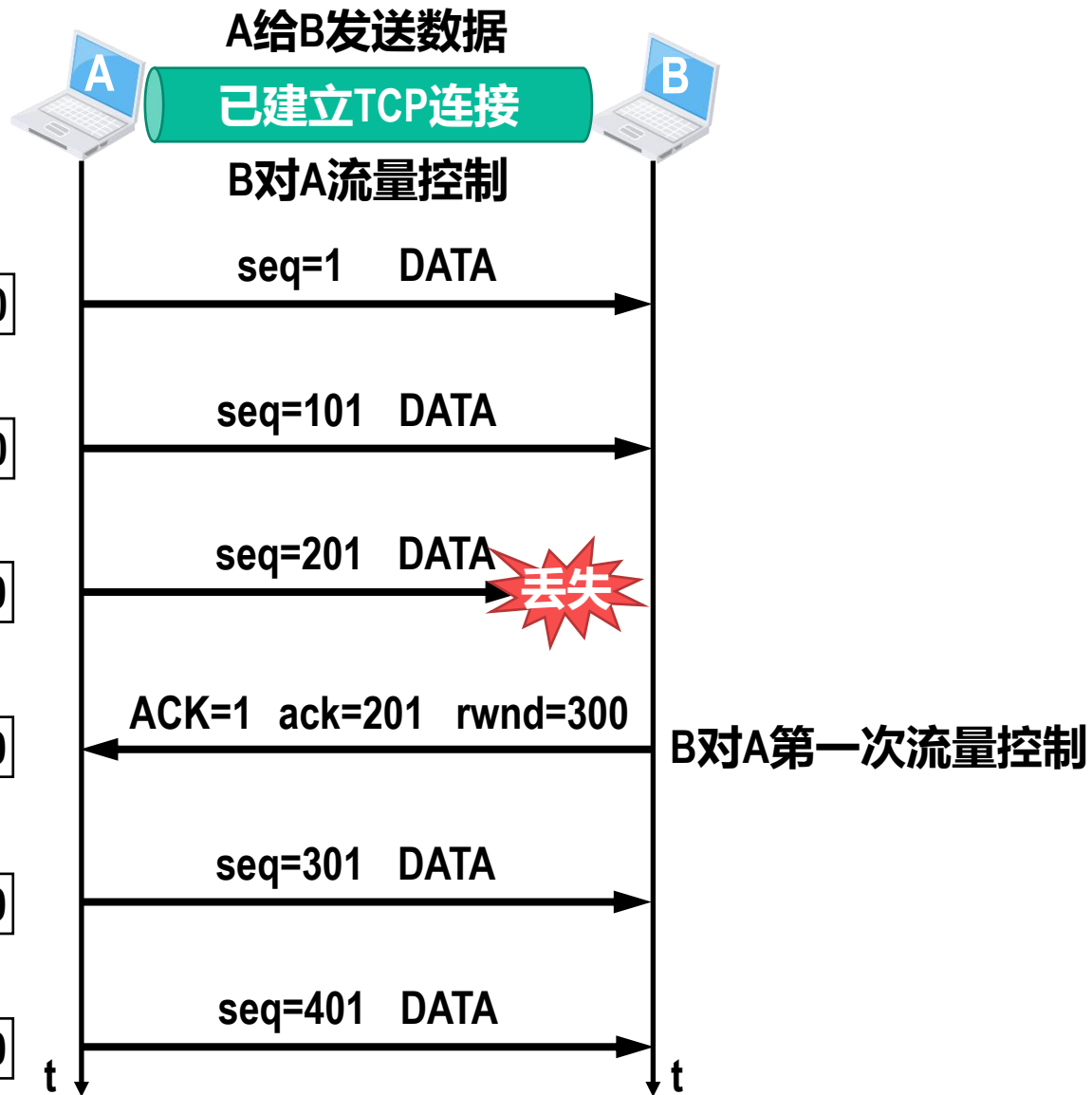
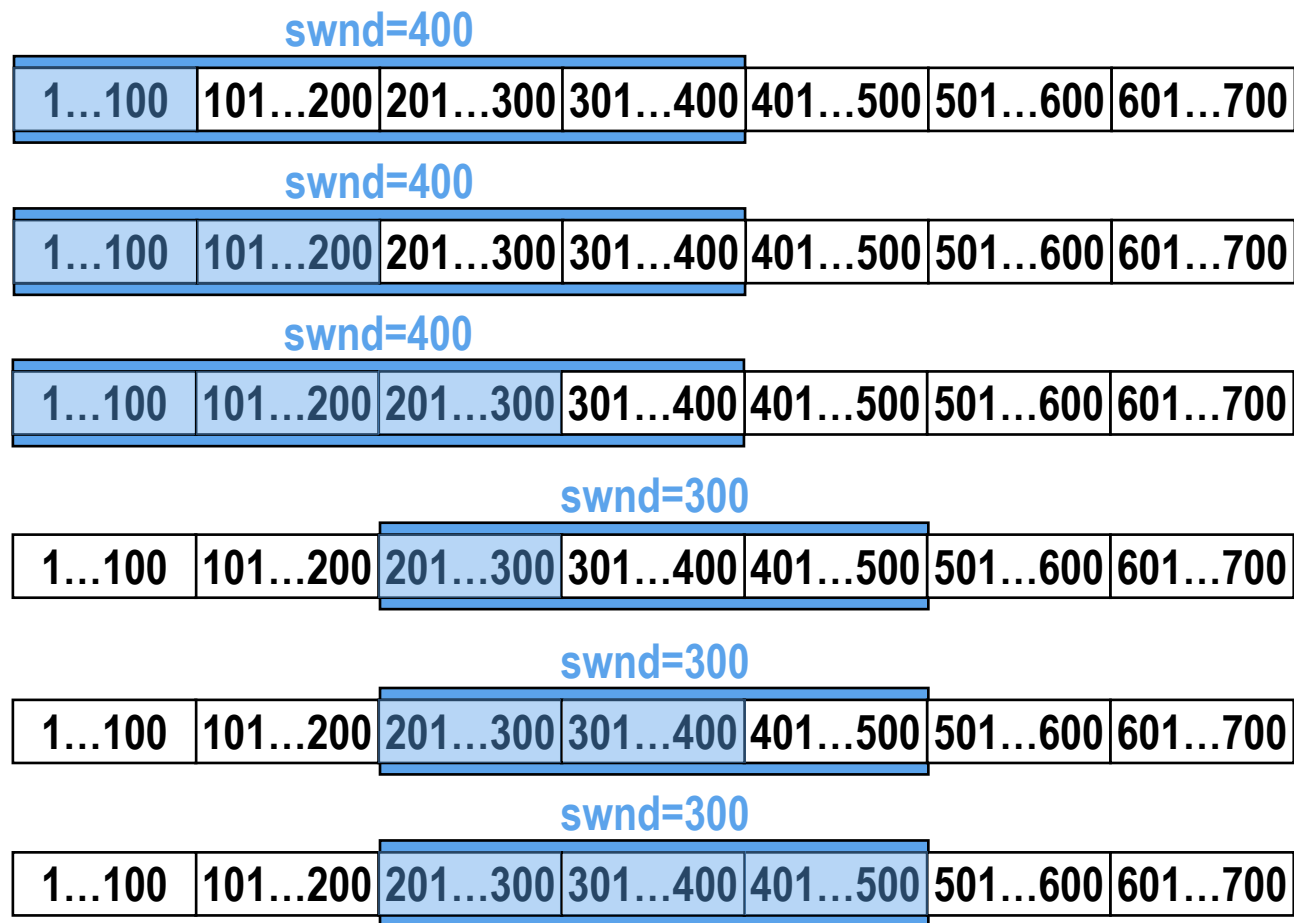
02 TCP的流量控制方法



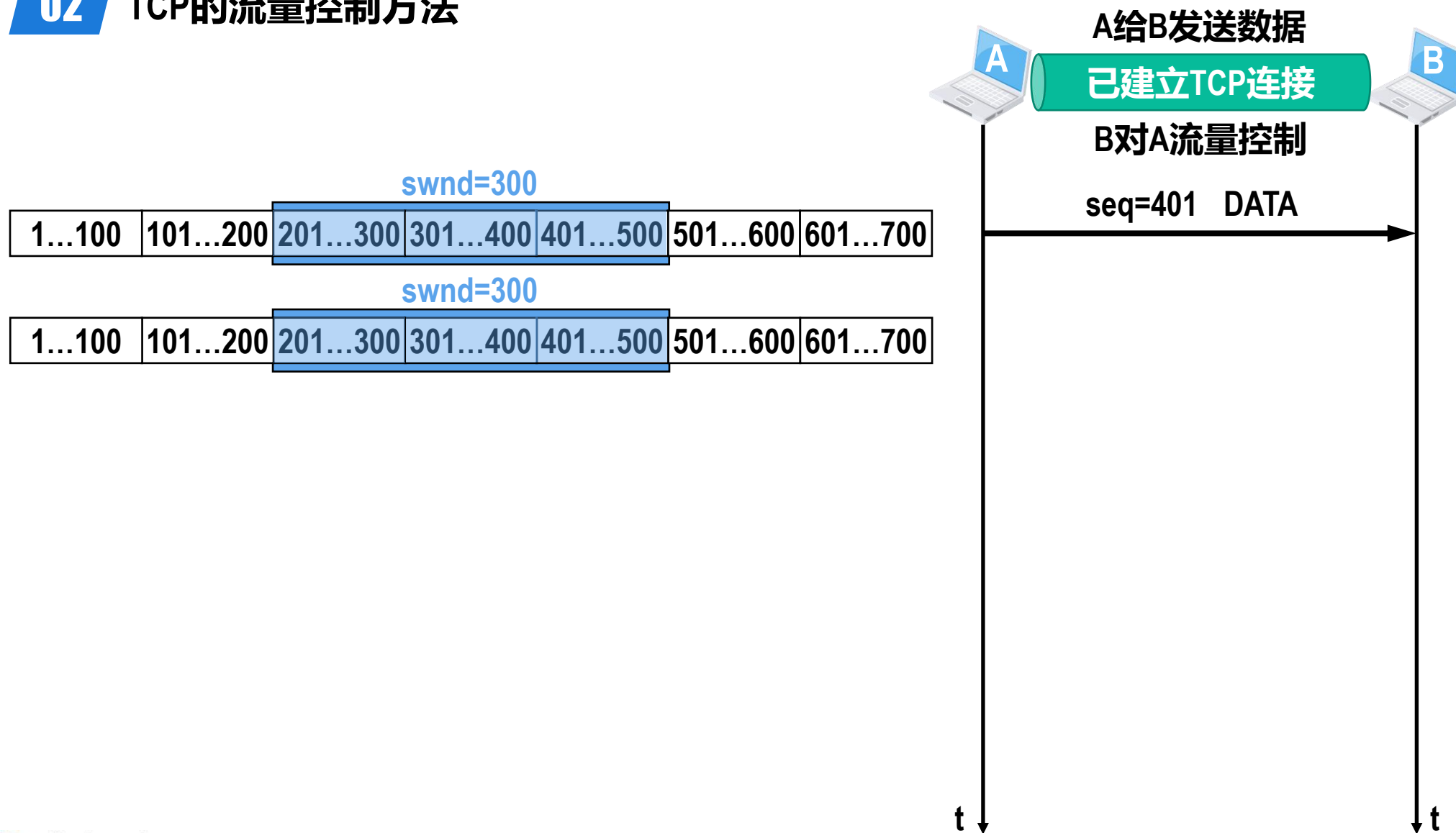
02 TCP的流量控制方法



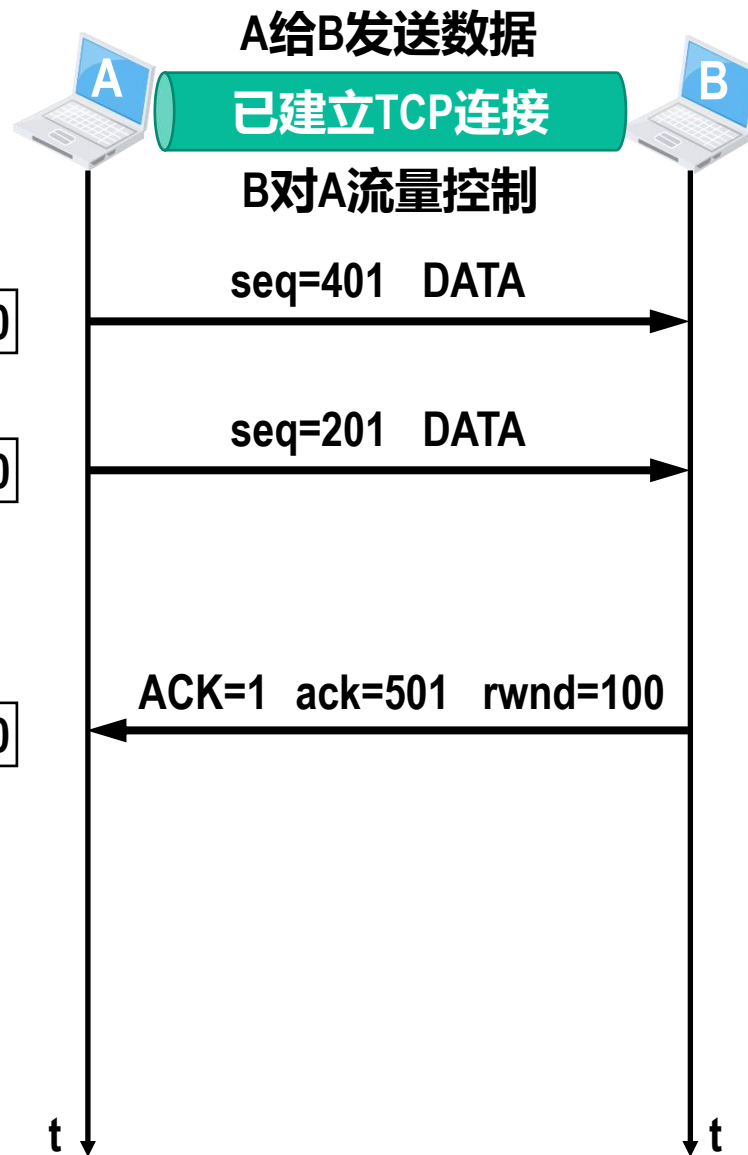
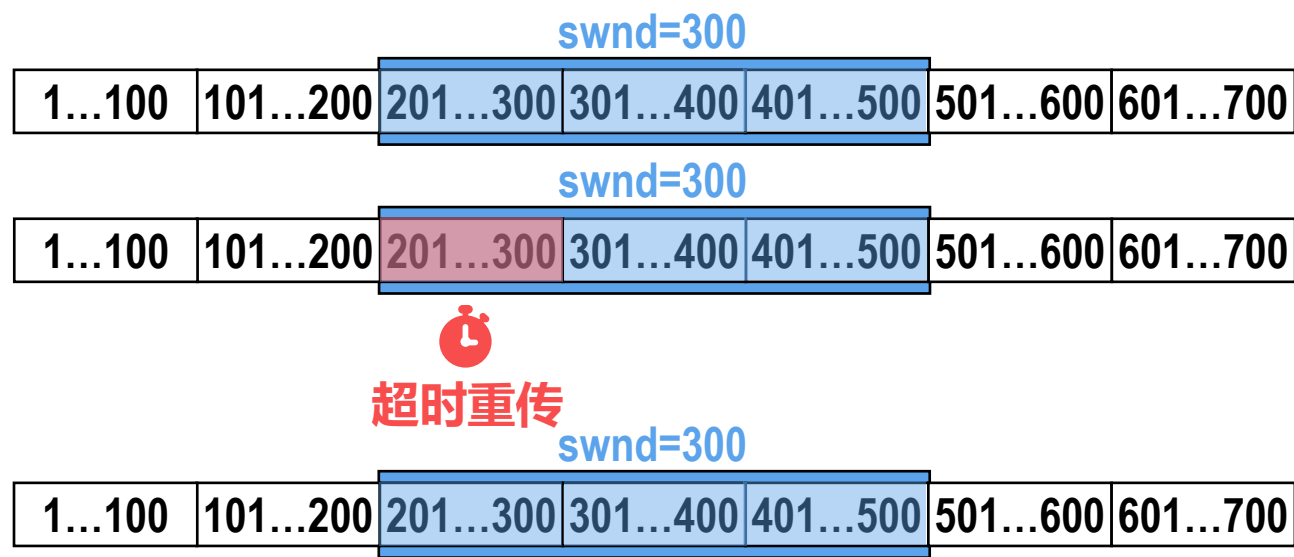
02 TCP的流量控制方法



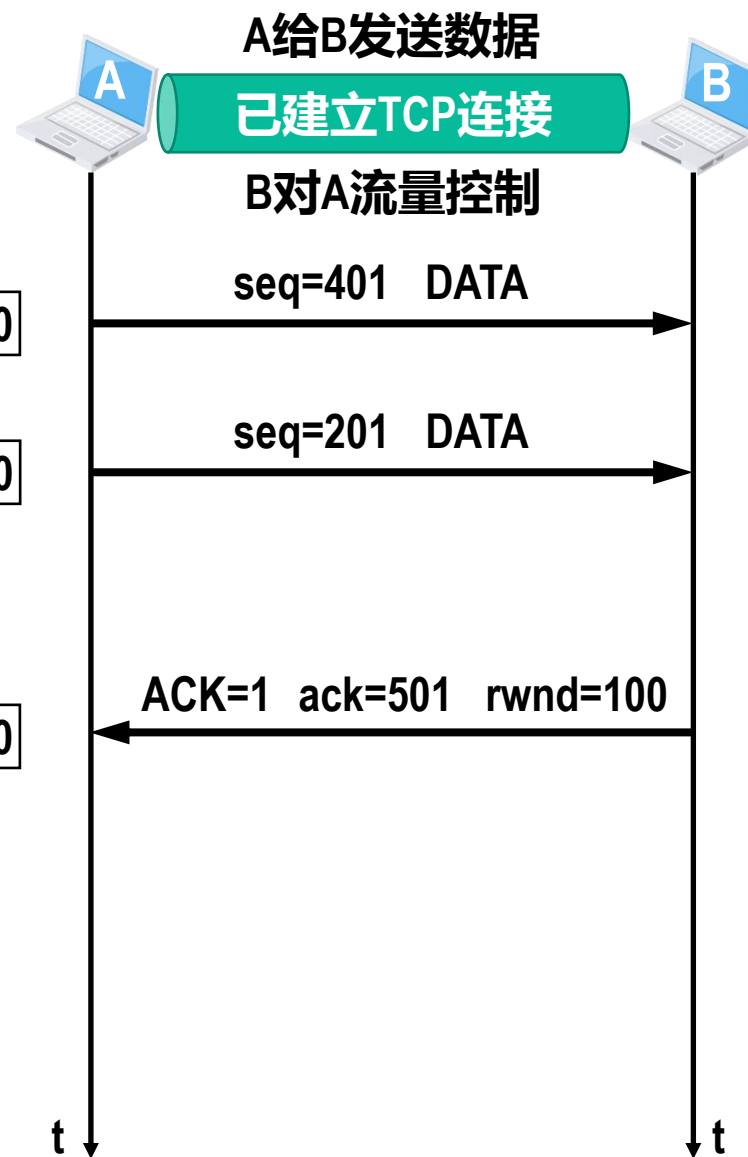
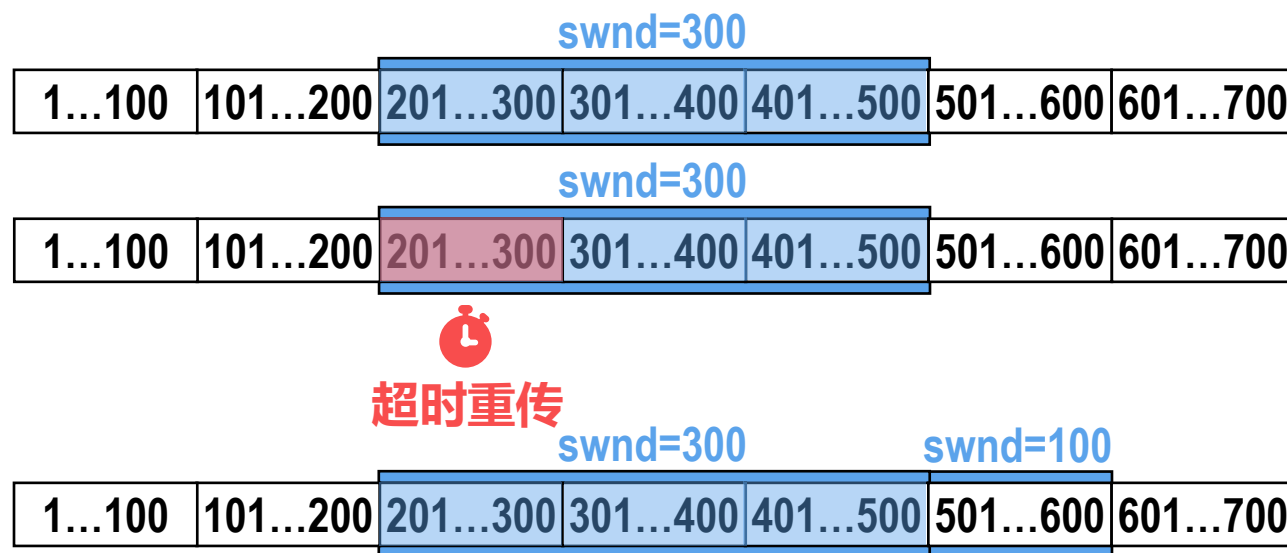
02 TCP的流量控制方法



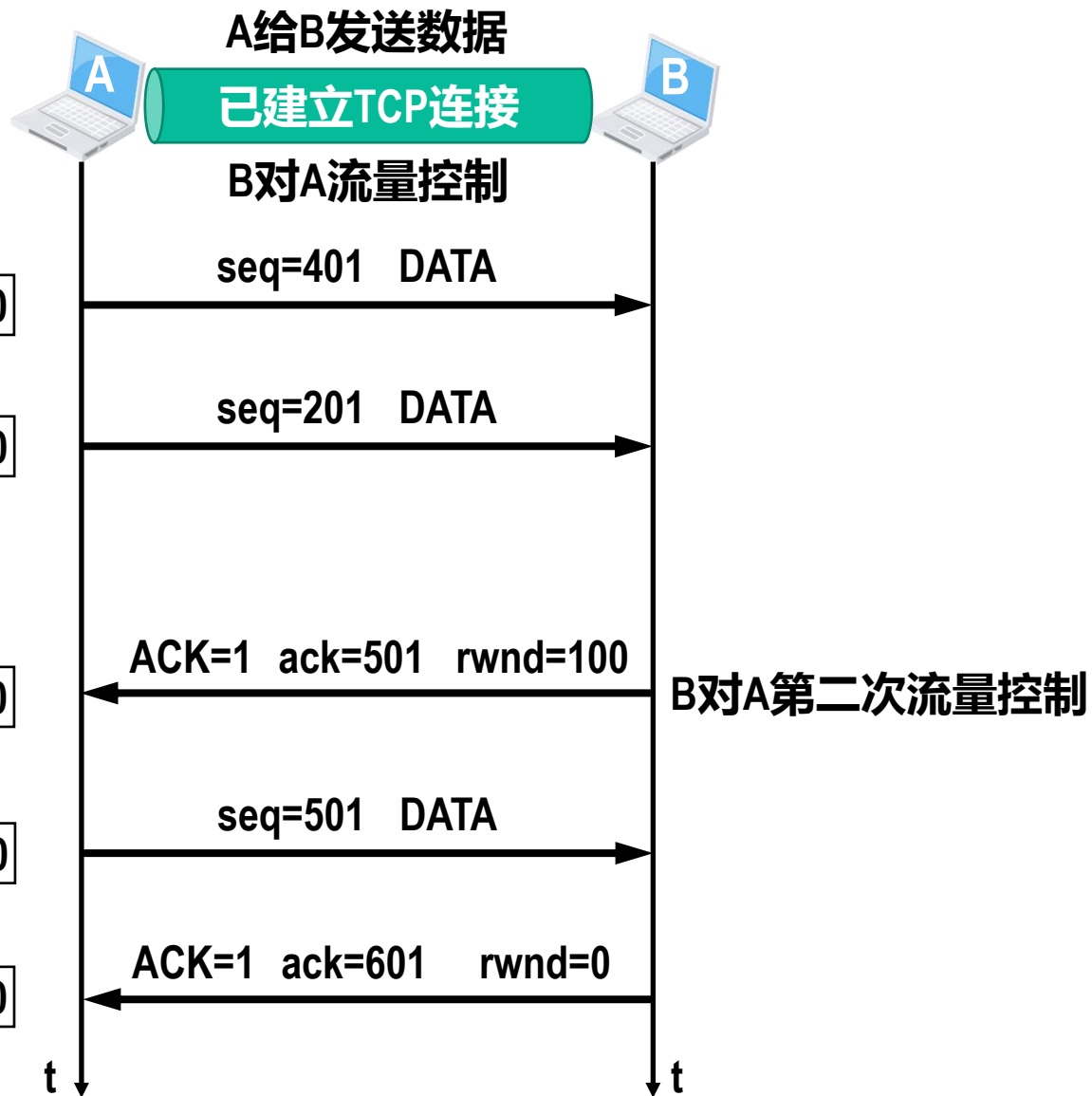
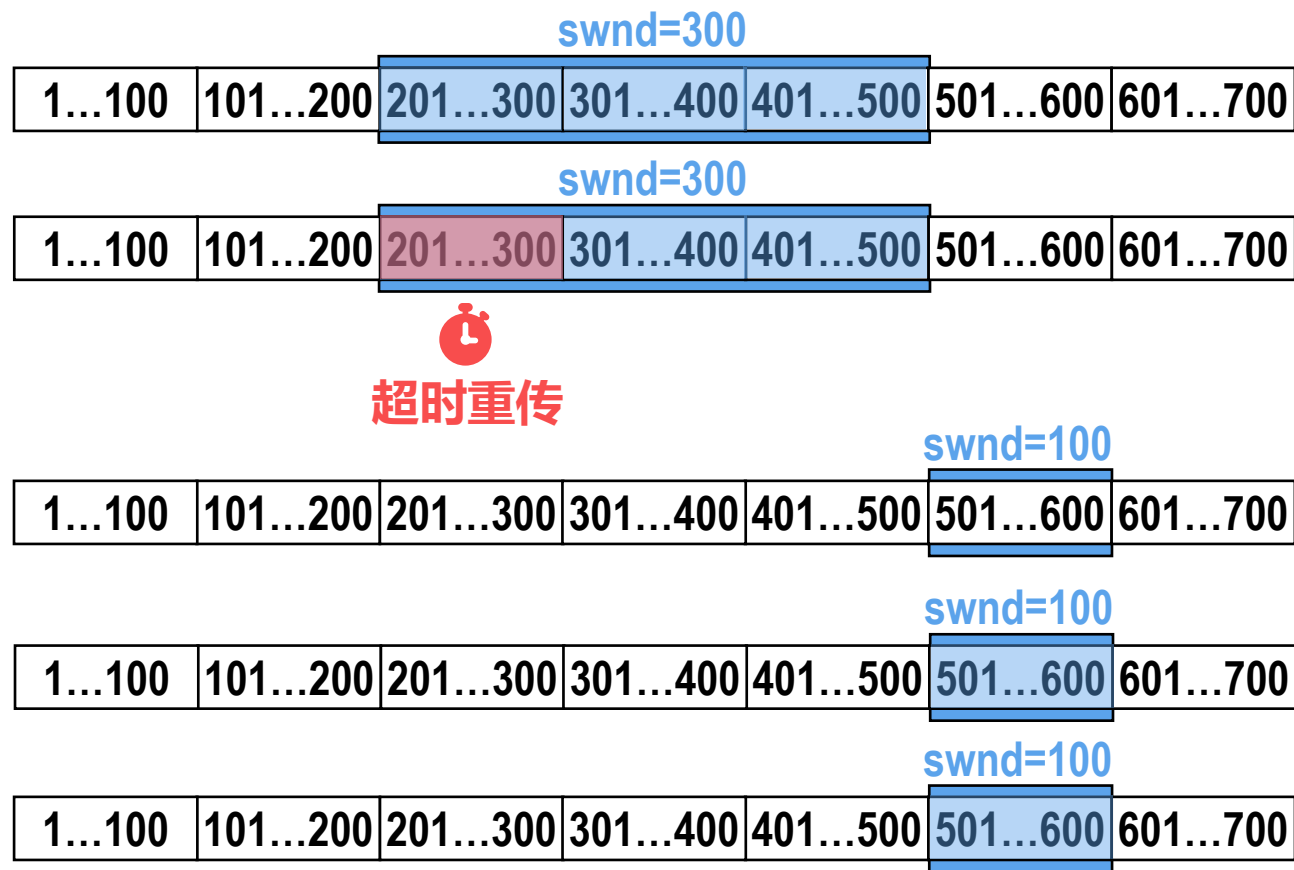
02 TCP的流量控制方法



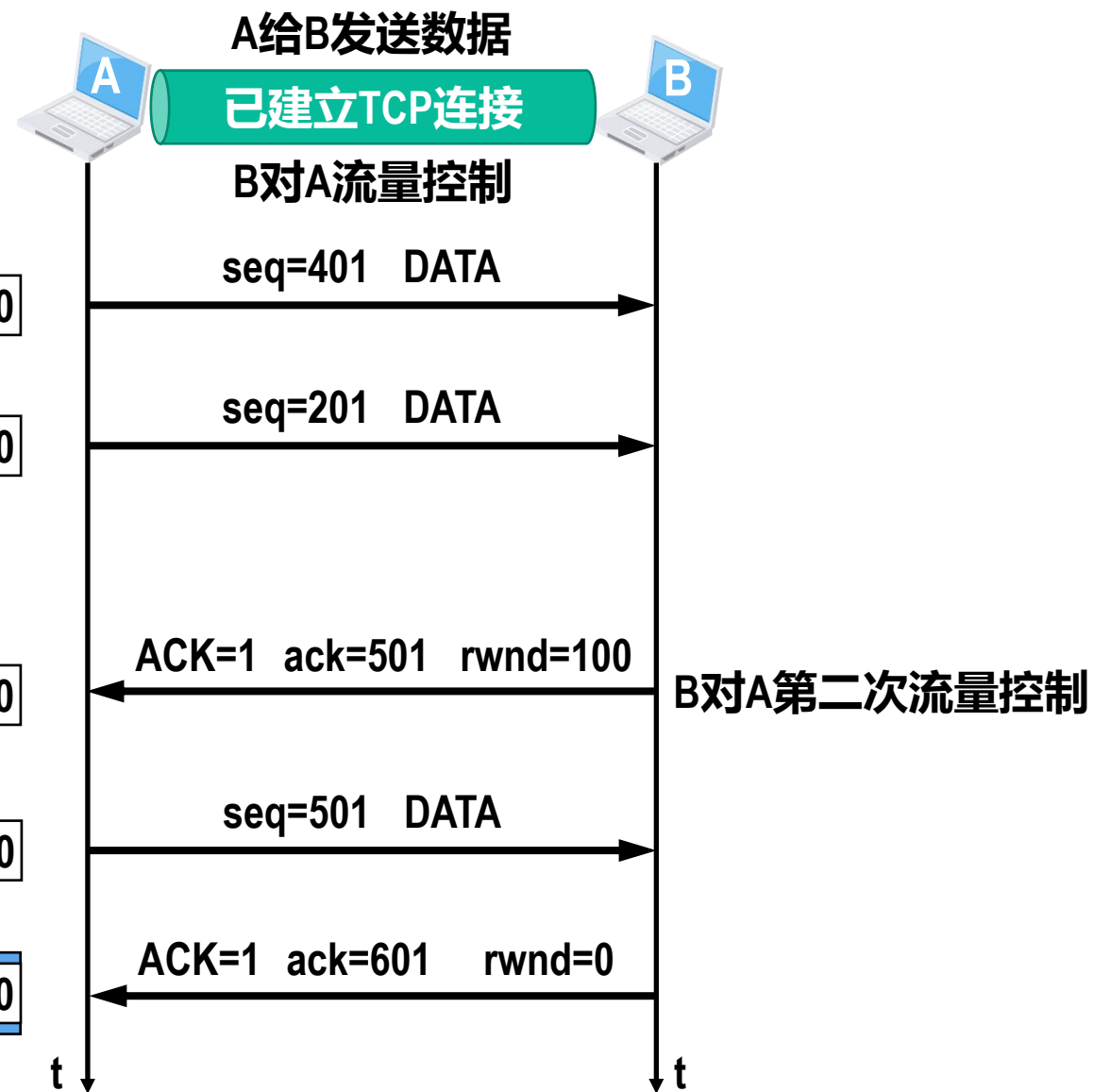
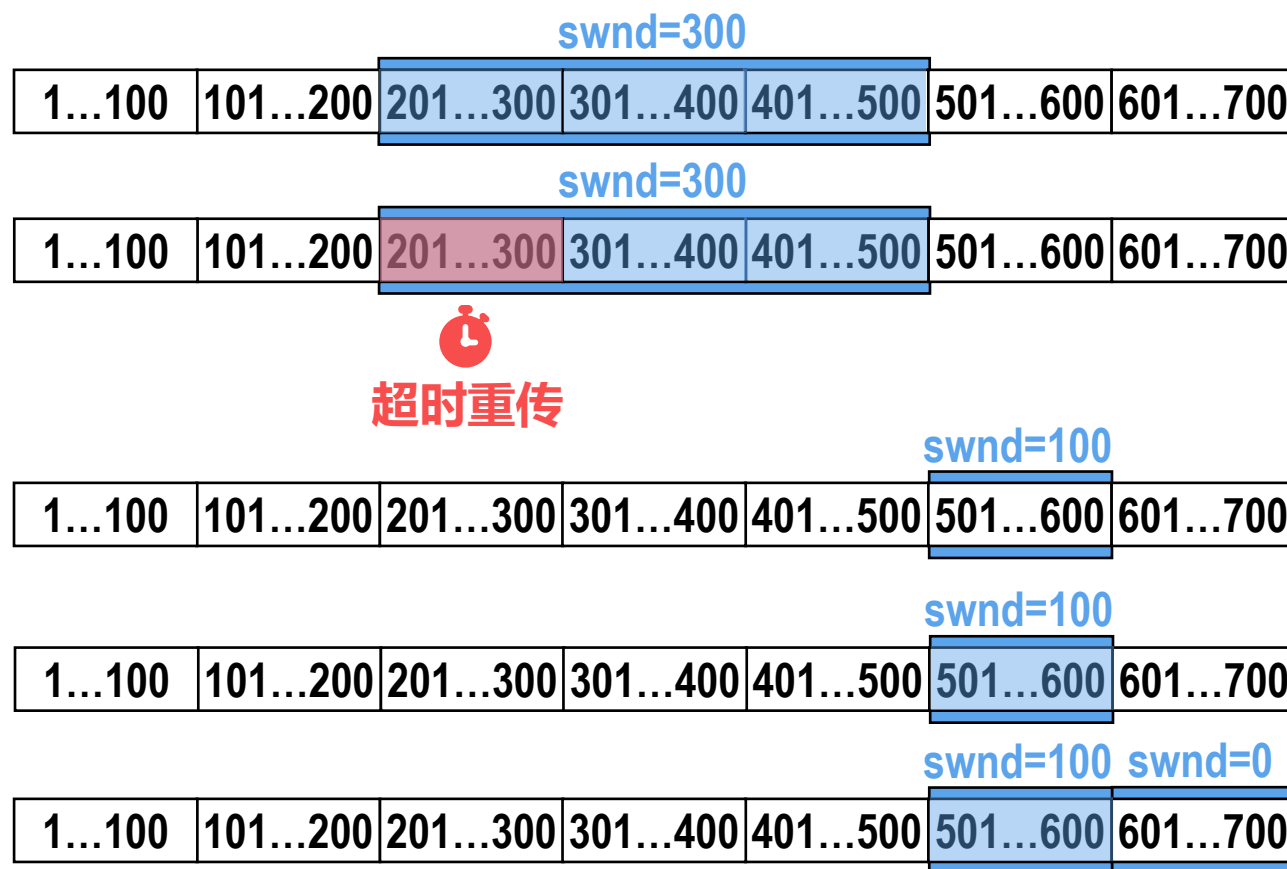
02 TCP的流量控制方法



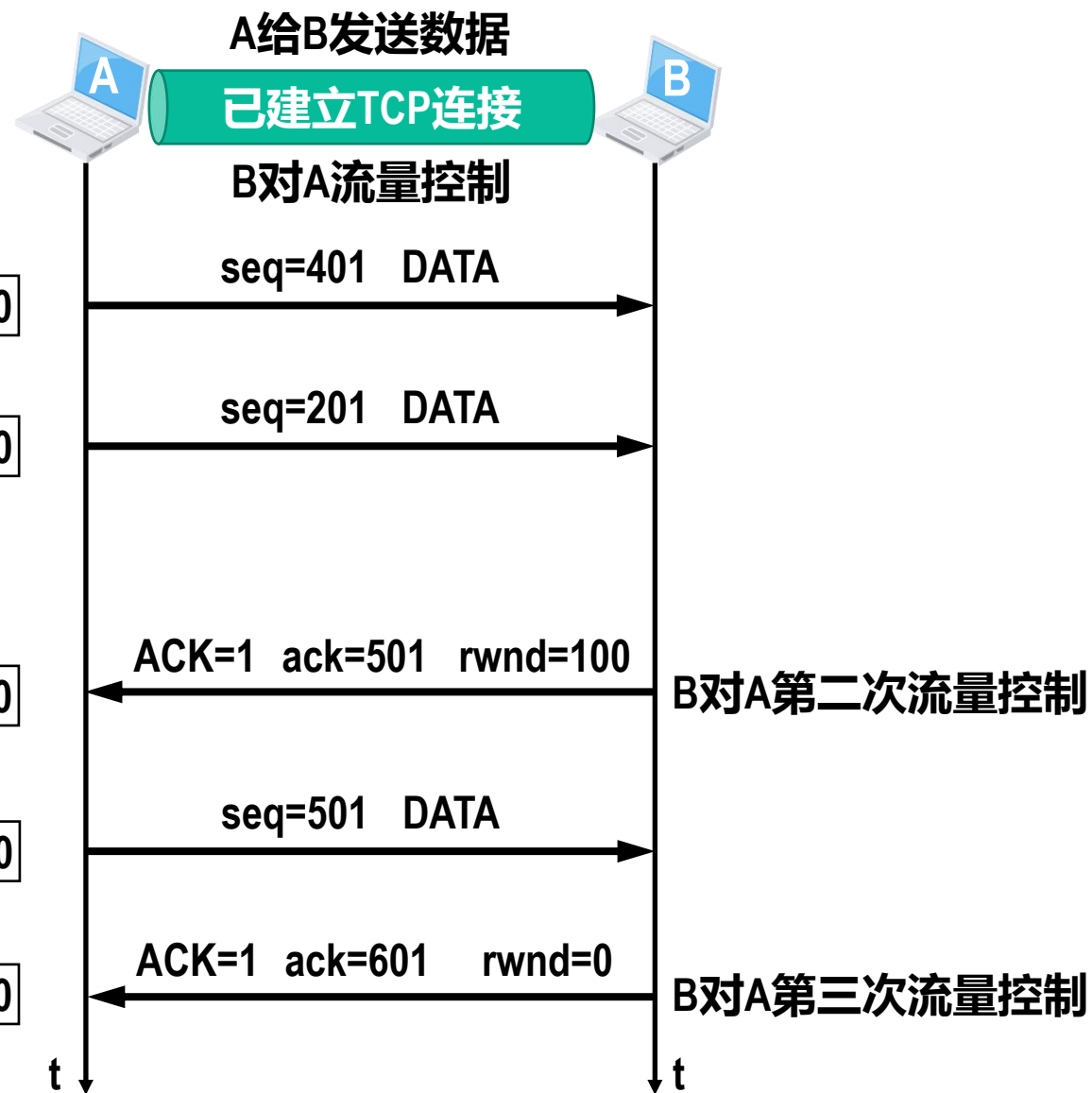
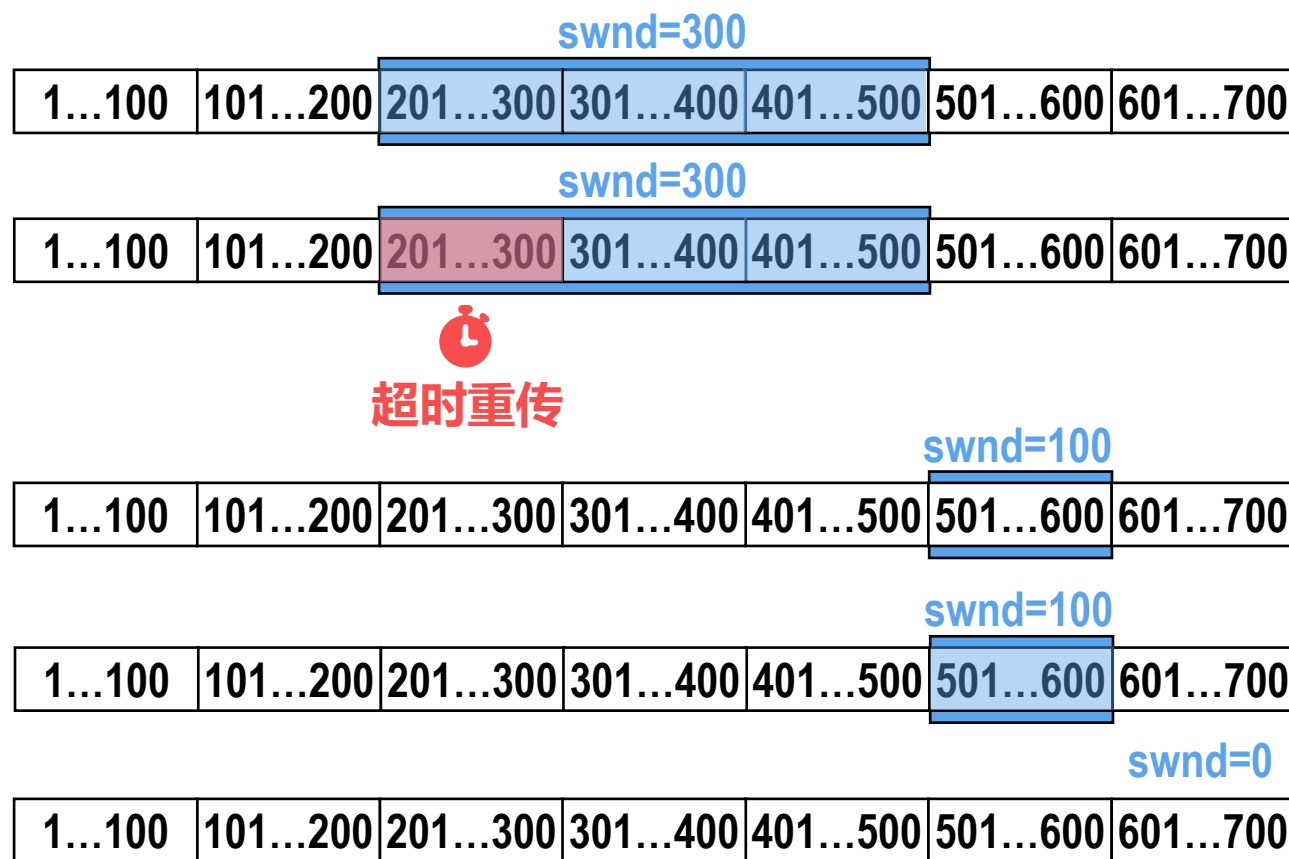
02 TCP的流量控制方法



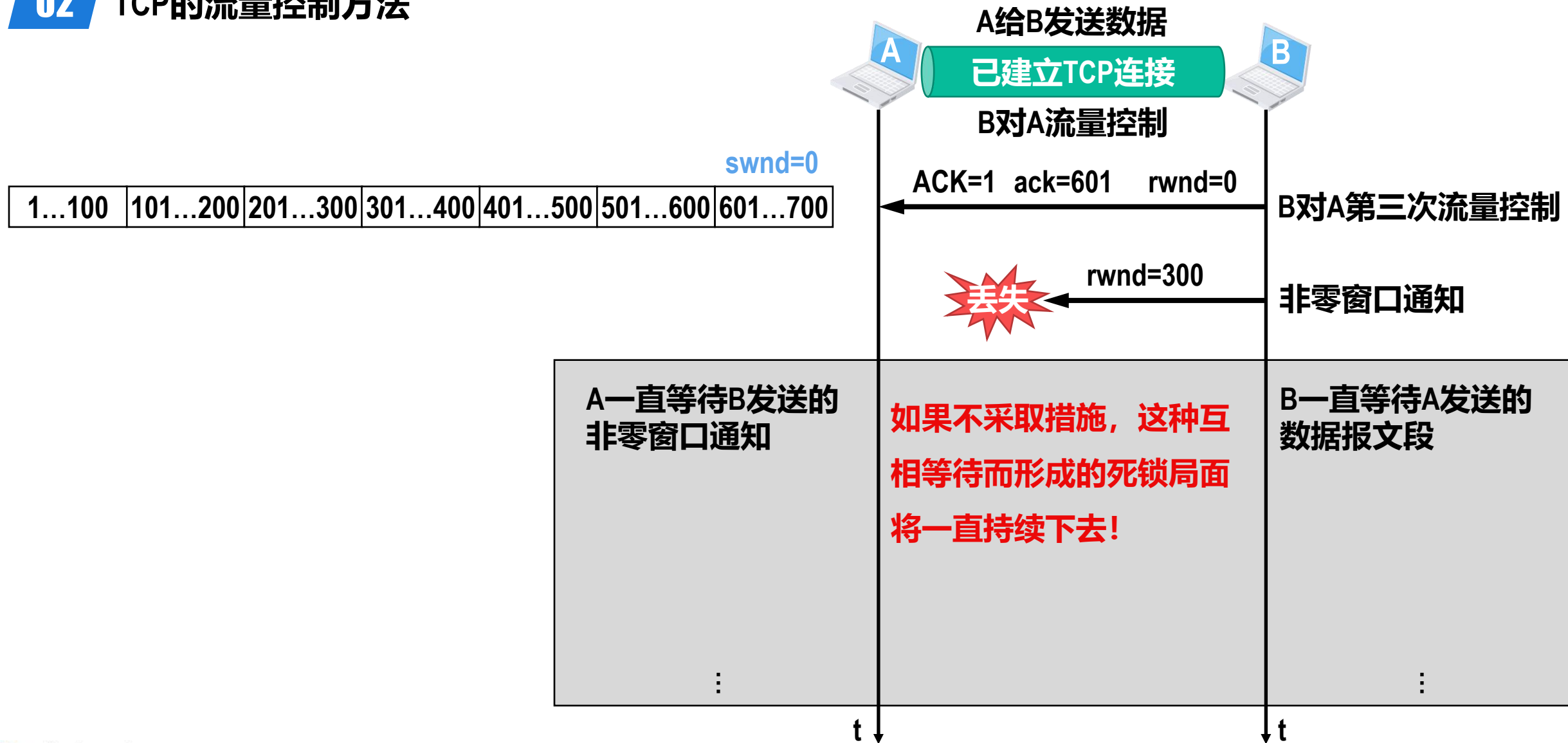
02 TCP的流量控制方法



02 TCP的流量控制方法



02 TCP的流量控制方法



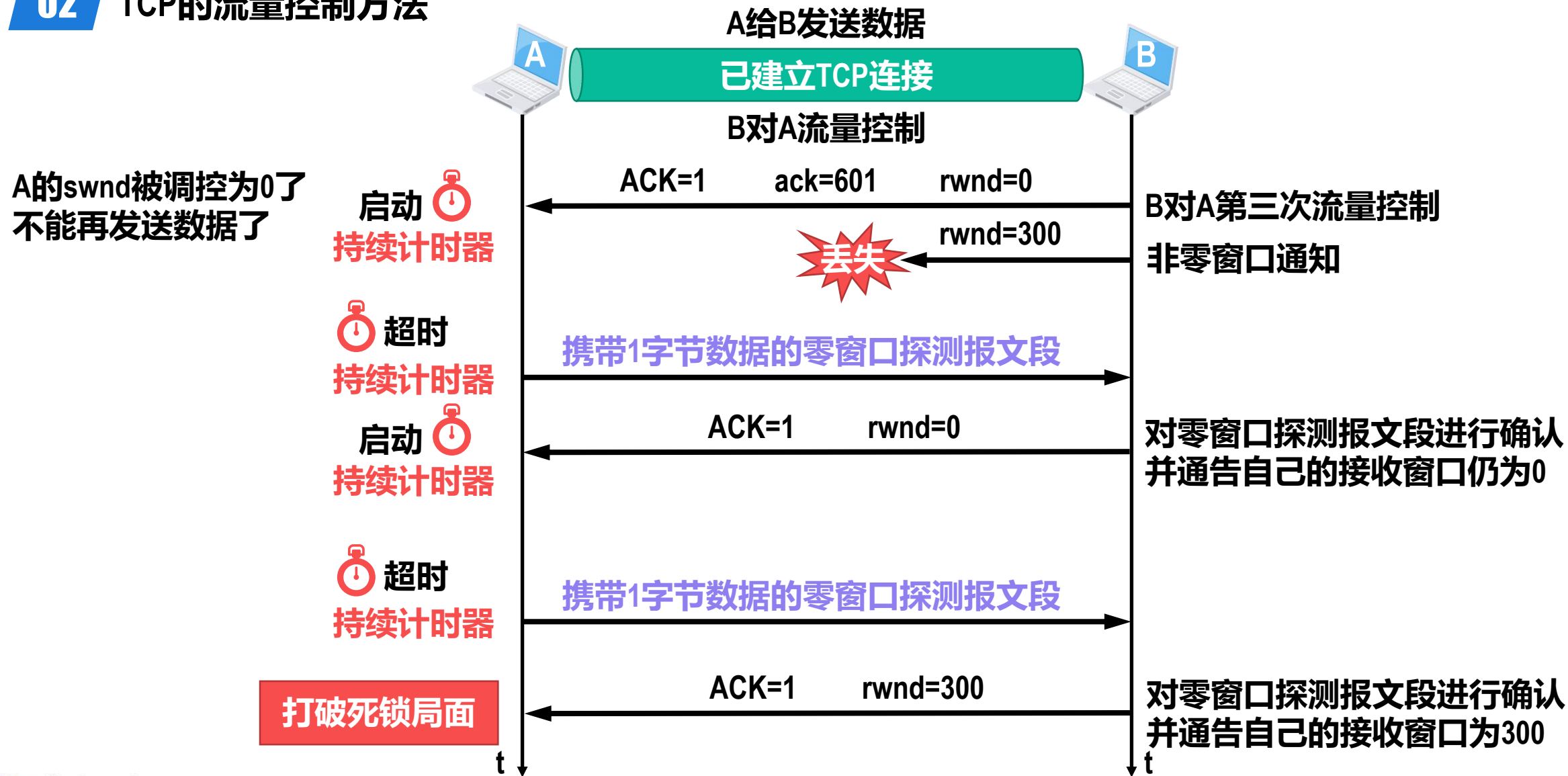
02 TCP的流量控制方法

为了打破由于非零窗口通知报文段丢失而引起的双方互相等待的死锁局面，TCP为每一个连接都设有一个**持续计时器**。

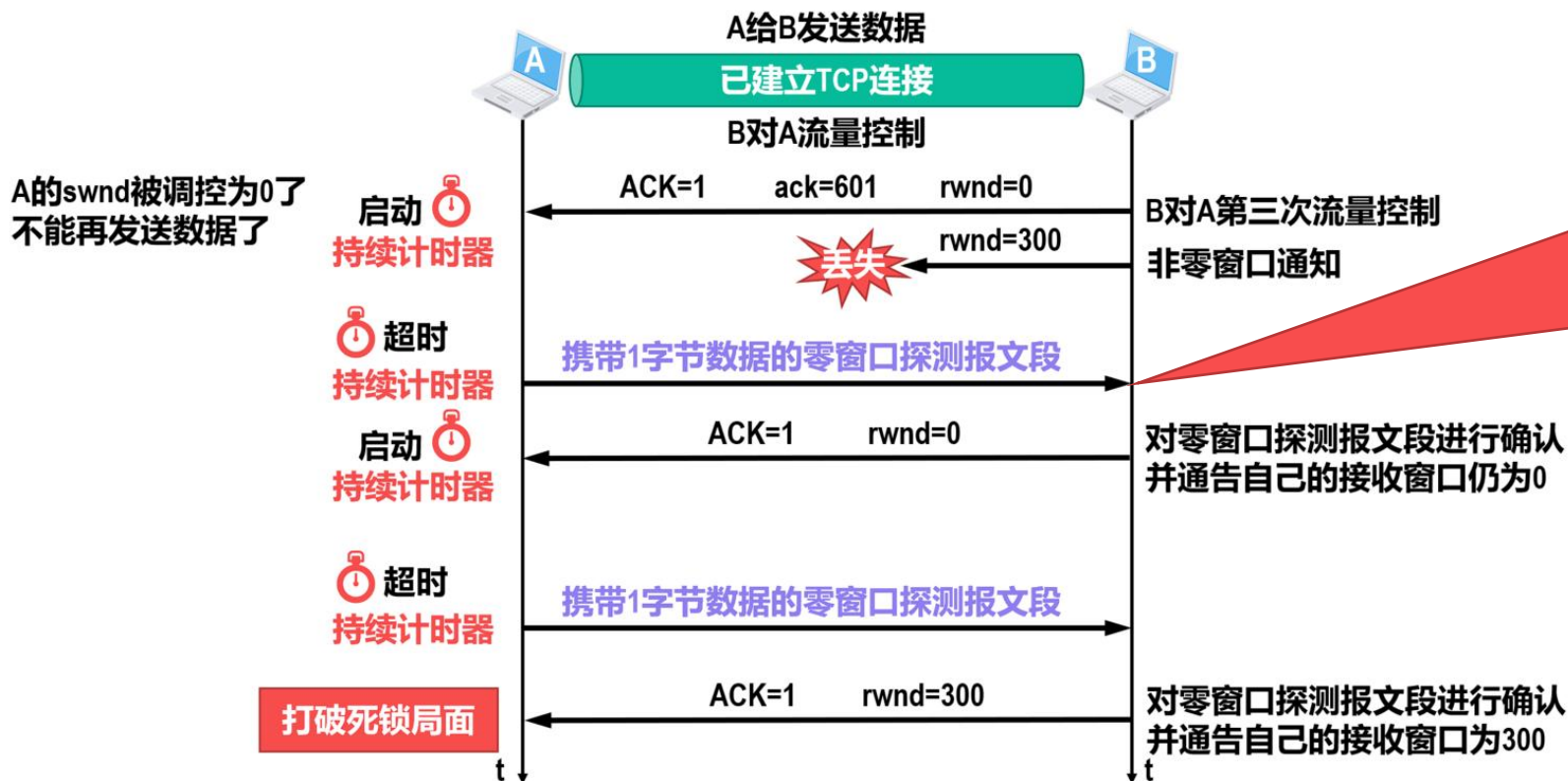
- 只要TCP连接的一方收到对方的零窗口通知，就启动持续计时器。
- 当持续计时器超时，就发送一个零窗口探测报文段，仅携带1字节的数据。
- 对方在确认这个零窗口探测报文段时，给出自己现在的接收窗口值。
- 如果接收窗口值仍然是0，那么收到这个报文段的一方就重新启动持续计时器。
- 如果接收窗口值不是0，那么死锁的局面就可以被打破了。



02 TCP的流量控制方法



02 TCP的流量控制方法

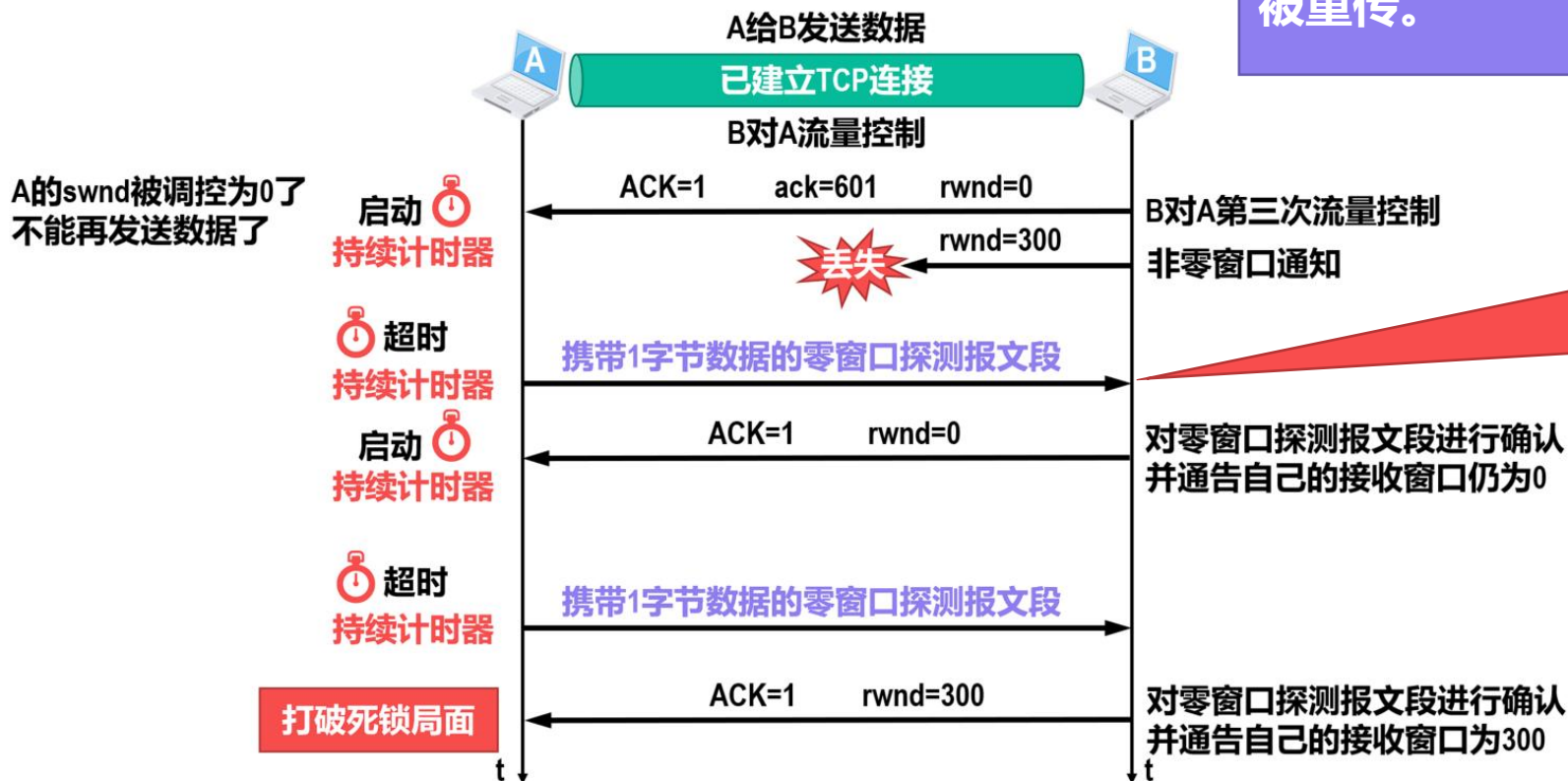


A发送的零窗口探测报文段到达B时，如果B此时的接收窗口值仍然为0，那么B根本就无法接受该报文段，又怎么会针对该报文段给A发回确认呢？

实际上TCP规定：即使接收窗口值为0，也必须接受零窗口探测报文段、确认报文段以及携带有紧急数据的报文段。



02 TCP的流量控制方法



回答是肯定的。因为零窗口探测报文段也有重传计时器，当重传计时器超时后，零窗口探测报文段会被重传。

如果零窗口探测报文段丢失了，还会打破死锁的局面吗？



02 TCP的流量控制方法

【2010年 题39】主机甲和主机乙之间建立了一个TCP连接，TCP最大段长度为1000字节。若主机甲的当前拥塞窗口为4000字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的第一个段的确认段，确认段中通告的接收窗口大小为2000字节，则此时主机甲还可以向主机乙发送的最大字节数是（**A**）。

A. 1000

B. 2000

C. 3000

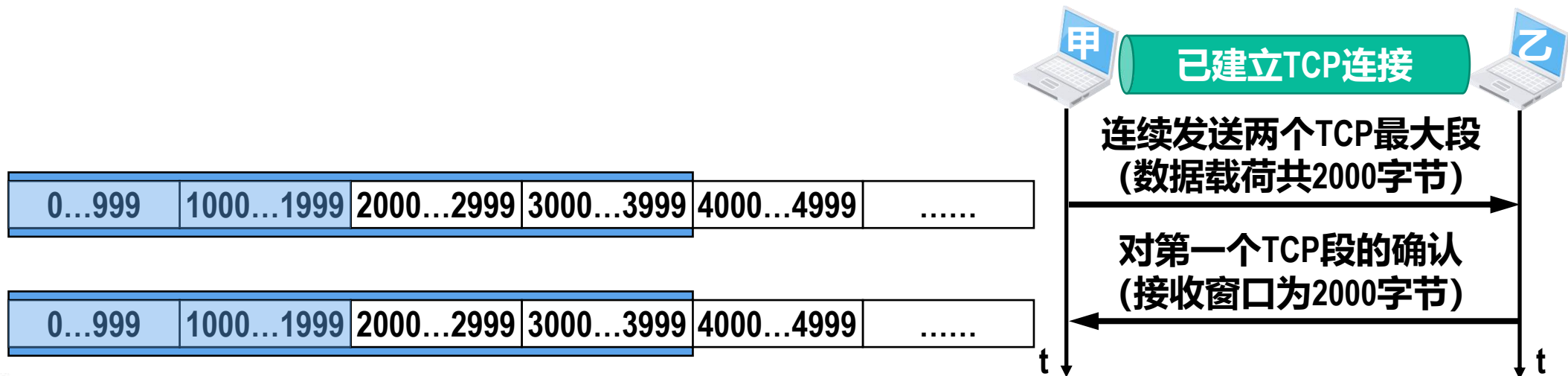
D. 4000

解析

TCP发送方的发送窗口值 = $\min[\text{TCP发送方的拥塞窗口值}, \text{TCP接收方的接收窗口值}]$

题目未给出TCP发送方的发送窗口值以及TCP接收方的接收窗口值，则取拥塞窗口值作为发送窗口值。

TCP最大报文段长度MSS，并不是指整个TCP报文段的长度，而是指TCP报文段的数据载荷的长度。



02 TCP的流量控制方法

【2010年 题39】主机甲和主机乙之间建立了一个TCP连接，TCP最大段长度为1000字节。若主机甲的当前拥塞窗口为4000字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的第一个段的确认段，确认段中通告的接收窗口大小为2000字节，则此时主机甲还可以向主机乙发送的最大字节数是（**A**）。

A. 1000

B. 2000

C. 3000

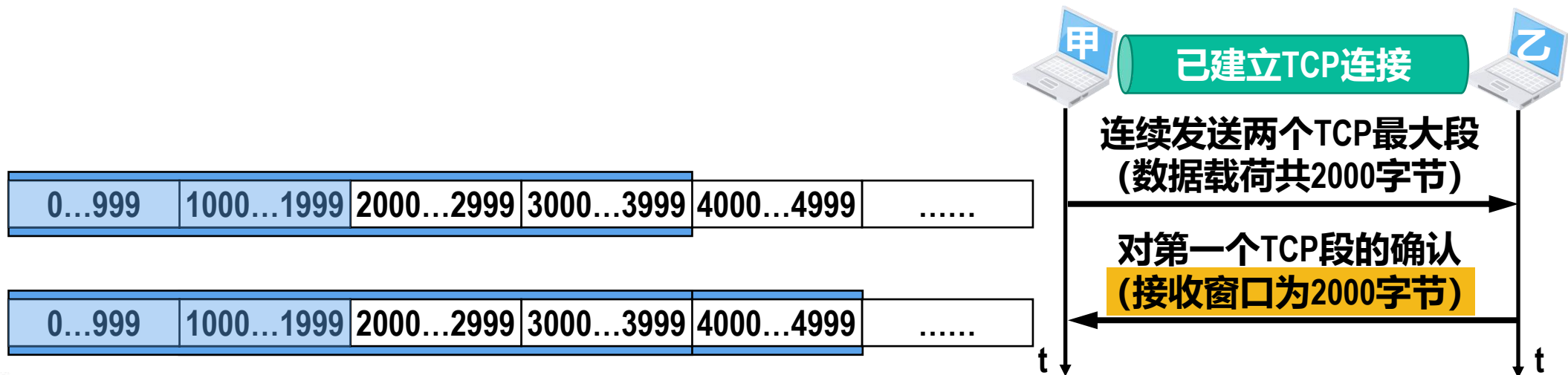
D. 4000

解析

TCP发送方的发送窗口值 = $\min$ [TCP发送方的拥塞窗口值, TCP接收方的接收窗口值]

题目未给出TCP发送方的发送窗口值以及TCP接收方的接收窗口值，则取拥塞窗口值作为发送窗口值。

TCP最大报文段长度MSS，并不是指整个TCP报文段的长度，而是指TCP报文段的数据载荷的长度。



02 TCP的流量控制方法

【2010年 题39】主机甲和主机乙之间建立了一个TCP连接，TCP最大段长度为1000字节。若主机甲的当前拥塞窗口为4000字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的第一个段的确认段，确认段中通告的接收窗口大小为2000字节，则此时主机甲还可以向主机乙发送的最大字节数是（**A**）。

A. 1000

B. 2000

C. 3000

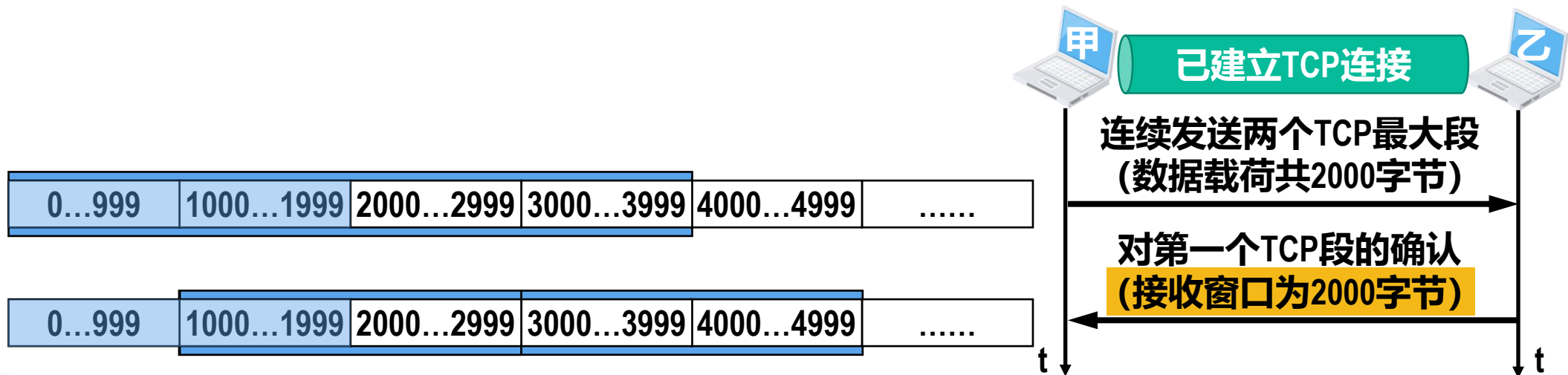
D. 4000

解析

TCP发送方的发送窗口值 = $\min[\text{TCP发送方的拥塞窗口值}, \text{TCP接收方的接收窗口值}]$

题目未给出TCP发送方的发送窗口值以及TCP接收方的接收窗口值，则取拥塞窗口值作为发送窗口值。

TCP最大报文段长度MSS，并不是指整个TCP报文段的长度，而是指TCP报文段的数据载荷的长度。



02 TCP的流量控制方法

【2010年 题39】主机甲和主机乙之间建立了一个TCP连接，TCP最大段长度为1000字节。若主机甲的当前拥塞窗口为4000字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的第一个段的确认段，确认段中通告的接收窗口大小为2000字节，则此时主机甲还可以向主机乙发送的最大字节数是（**A**）。

A. 1000

B. 2000

C. 3000

D. 4000

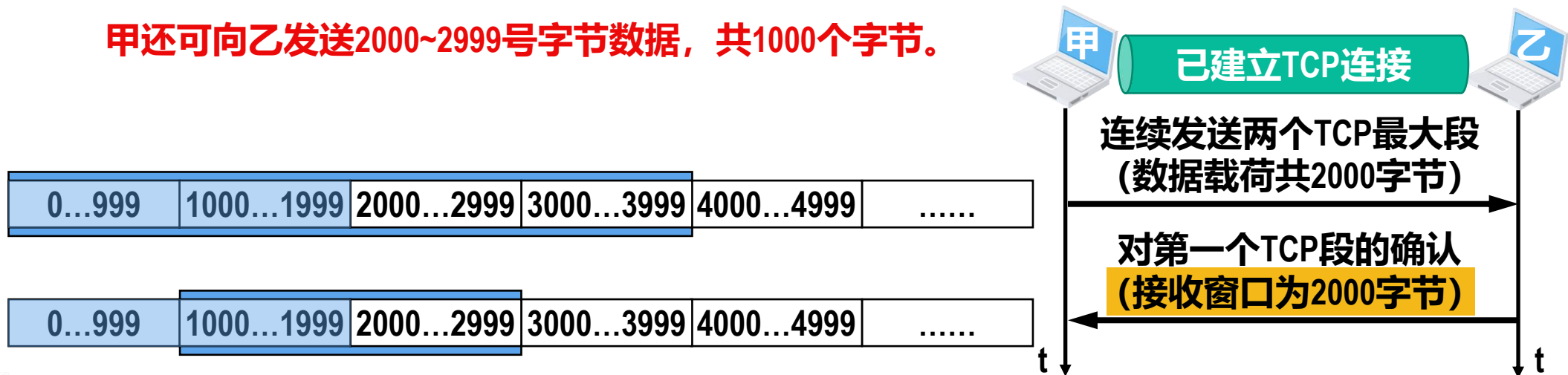
解析

TCP发送方的发送窗口值 = $\min$ [TCP发送方的拥塞窗口值, TCP接收方的接收窗口值]

题目未给出TCP发送方的发送窗口值以及TCP接收方的接收窗口值，则取拥塞窗口值作为发送窗口值。

TCP最大报文段长度MSS，并不是指整个TCP报文段的长度，而是指TCP报文段的数据载荷的长度。

甲还可向乙发送2000~2999号字节数据，共1000个字节。



5.3.4 TCP的拥塞控制

5.3.5 TCP拥塞控制与网际层拥塞控制的关系

01

拥塞控制的基本概念

02

拥塞控制的基本方法

03

TCP的四种拥塞控制方法

04

TCP拥塞控制与网际层拥塞控制的关系

01 拥塞控制的基本概念

拥塞

[yōng sāi] ✗

[yōng sè] ✓

鼻塞

[bí sāi] ✗

[bí sè] ✓

蛋挞

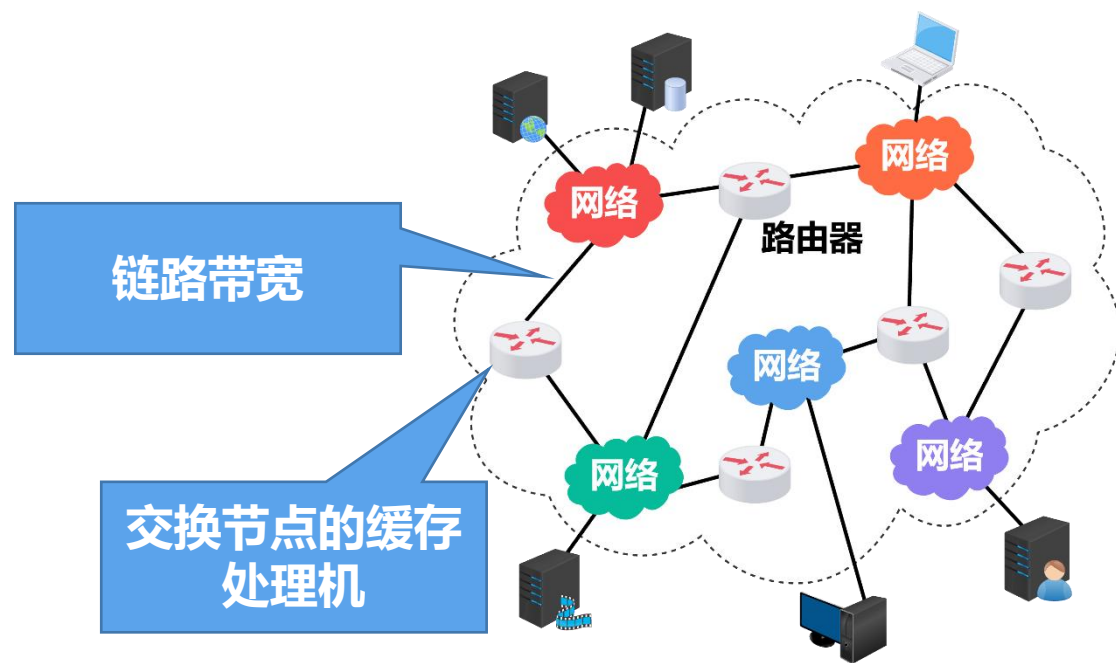
[dàn tǎ] ✗

[dàn tà] ✓

01 拥塞控制的基本概念

■ 在某段时间，若**对网络中某一资源的需求超过了该资源所能提供的可用部分**，网络性能就要变坏，这种情况就叫作**拥塞**（congestion）。

□ 计算机网络中的链路容量（带宽）、交换节点中的缓存和处理机等都是网络的资源。

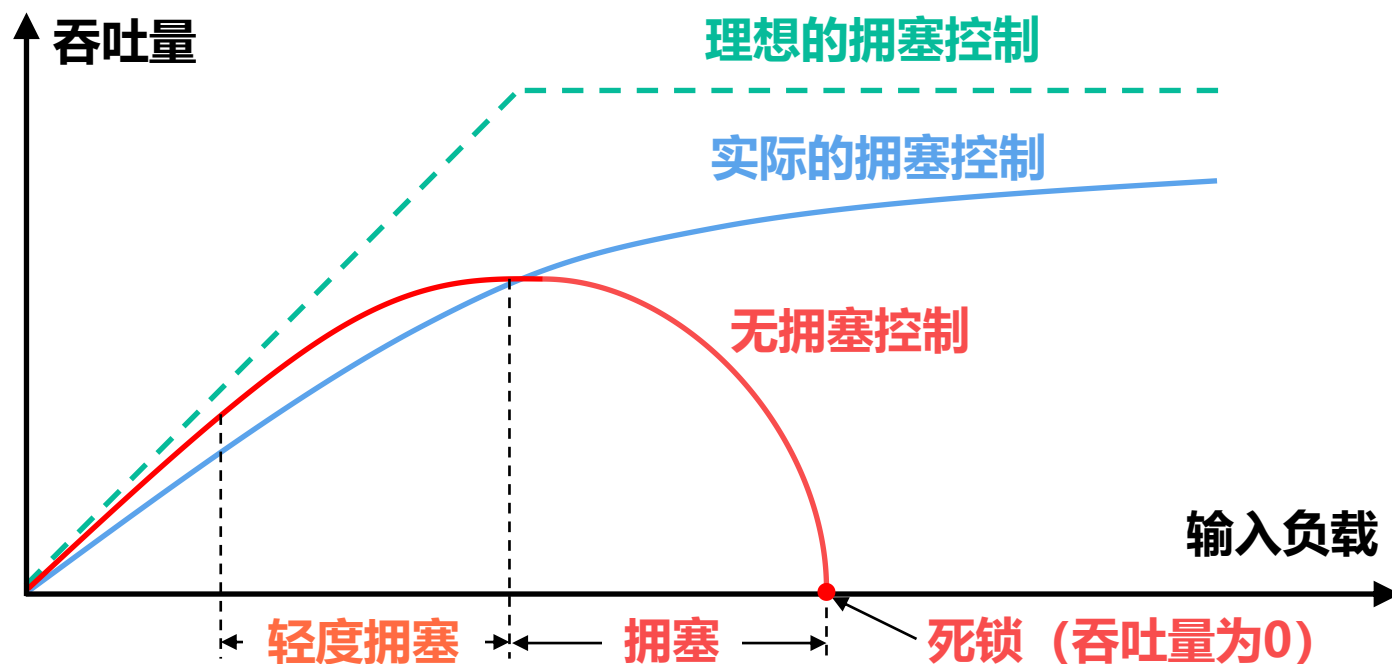


01 拥塞控制的基本概念

■ 在某段时间，若对网络中某一资源的需求超过了该资源所能提供的可用部分，网络性能就要变坏，这种情况就叫作**拥塞** (congestion)。

□ 计算机网络中的链路容量 (带宽)、交换节点中的缓存和处理机等都是网络的资源。

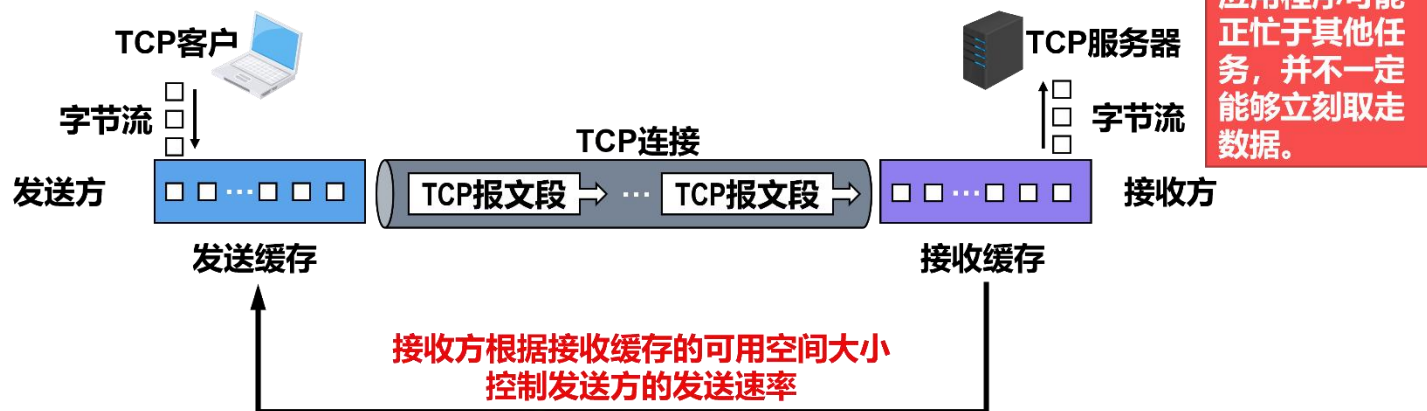
■ 若**出现拥塞而不进行控制**，整个网络的**吞吐量将随输入负荷的增大而下降**。



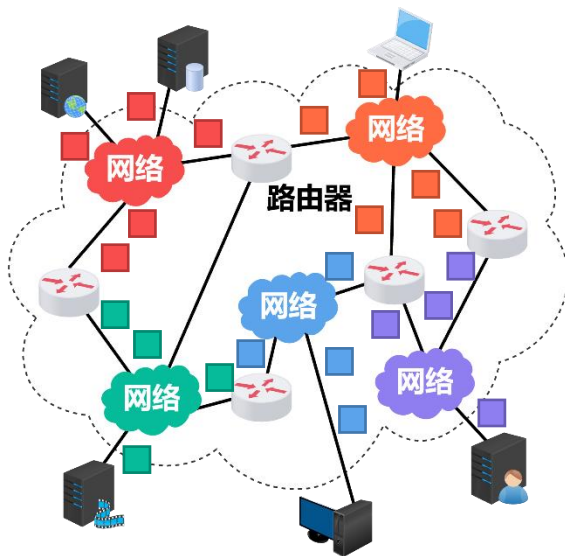
02 拥塞控制的基本方法

流量控制与拥塞控制的区别

流量控制



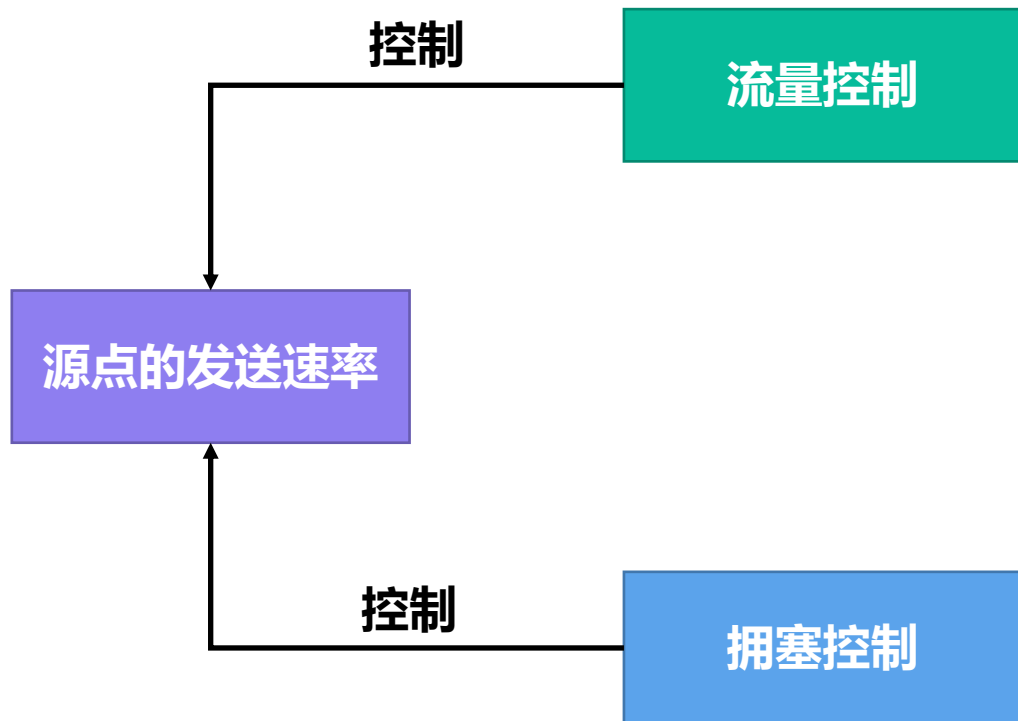
拥塞控制



防止过多的数据注入到网络中，
使网络能够承受现有的网络负荷。

02 拥塞控制的基本方法

流量控制与拥塞控制的区别



以**接收方的接收能力**控制发送方（源点）的发送速率
只与特定的点对点通信的发送方和接收方之间的流量有关

源点根据各方面因素，按拥塞控制算法**自行控制发送速率**
全局性问题，涉及网络中的所有的主机、路由器等

02 拥塞控制的基本方法

开环控制

- 试图用良好的设计来解决问题。
- 从一开始就保证问题不会发生。
- 一旦系统启动并运行起来，就不需要中途修正。

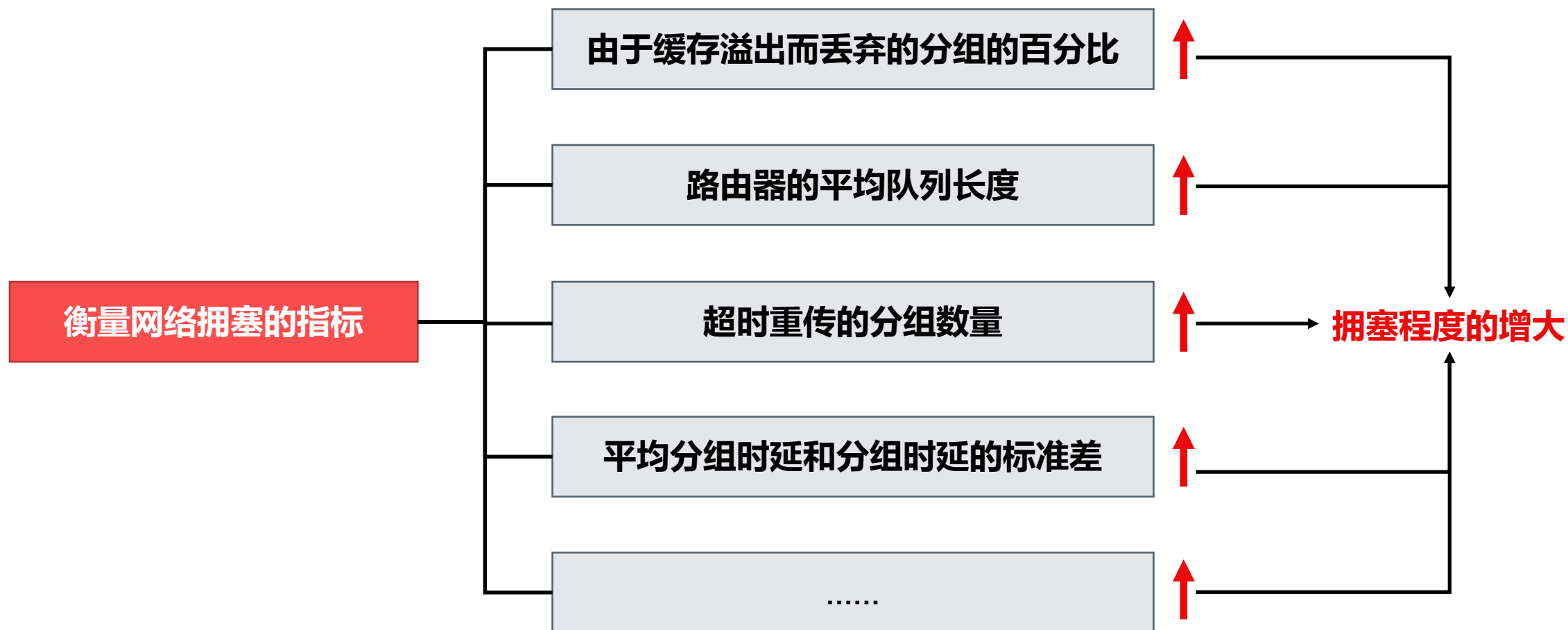
当网络的流量特征可以准确规定且性能要求可以事先获得时，适合使用开环控制。

闭环控制

- 基于反馈的控制方法，包括以下三个部分：
 - ① 监测网络拥塞在何时、何地发生。
 - ② 把拥塞发生的相关信息传送到可以采取行动的地方。
 - ③ 调整网络的运行以解决拥塞问题。

当网络的流量特征不能准确描述或者当网络不提供资源预留时，适合使用闭环控制。
因特网采用的就是闭环控制方法。

02 拥塞控制的基本方法



02 拥塞控制的基本方法

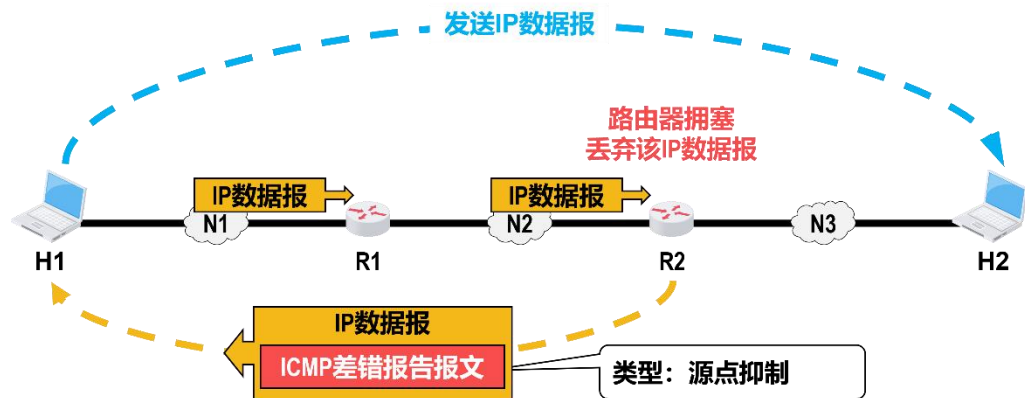
根据拥塞信息的反馈形式，可将闭环拥塞控制算法分为

显式反馈算法

从拥塞节点（即路由器）向源点提供关于网络中拥塞状态的显式反馈信息。

隐式反馈算法

源点自身通过对网络行为的观察（例如超时重传或往返时间RTT）来推断网络是否发生了拥塞。TCP采用的就是隐式反馈算法。



拥塞控制并不仅仅是运输层要考虑的问题。显式反馈算法就必须涉及网络层。虽然一些网络体系结构（如ATM网络）主要在网络层实现拥塞控制，但因特网主要利用隐式反馈在运输层实现拥塞控制。

02 拥塞控制的基本方法

进行拥塞控制是需要付出代价的

- 可能需要在节点之间**交换信息和各种命令**，以便选择拥塞控制的策略并实施控制，这样会产生额外开销。
- 可能需要**预留一些资源**用于特殊用户或特殊情况，这样就降低了网络资源的共享程度。

然而，为了确保网络性能的稳定，不会因为输入负载的增长而导致网络性能的恶化甚至出现崩溃，使用拥塞控制而付出一定的代价是值得的。

03 TCP的四种拥塞控制方法

慢开始
(slow-start)

拥塞避免
(congestion avoidance)

快重传
(fast retransmit)

快恢复
(fast recovery)

03 TCP的四种拥塞控制方法

慢开始
(slow-start)

拥塞避免
(congestion avoidance)

快重传
(fast retransmit)

快恢复
(fast recovery)

为了集中精力讨论拥塞控制算法的基本原理，假定如下条件

- ① **数据是单方向传送的**，而另一个方向只传送确认。
- ② 接收方总是有足够大的接收缓存空间，因而发送方的发送窗口的大小仅由网络的拥塞程度来决定，也就是**不考虑接收方对发送方的流量控制**。
- ③ 以**TCP最大报文段MSS**（即TCP报文段的数据载荷部分）的个数作为讨论问题的单位，而不是以字节为单位（尽管TCP是面向字节流的）。

03 TCP的四种拥塞控制方法

慢开始
(slow-start)

拥塞避免
(congestion avoidance)

快重传
(fast retransmit)

快恢复
(fast recovery)

为了集中精力讨论拥塞控制算法的基本原理，假定如下条件

- ① **数据是单方向传送的**，而另一个方向只传送确认。
- ② 接收方总是有足够大的接收缓存空间，因而发送方的发送窗口的大小仅由网络的拥塞程度来决定，也就是**不考虑接收方对发送方的流量控制**。
- ③ 以**TCP最大报文段MSS**（即TCP报文段的数据载荷部分）的个数作为讨论问题的单位，而不是以字节为单位（尽管TCP是面向字节流的）。

03 TCP的四种拥塞控制方法

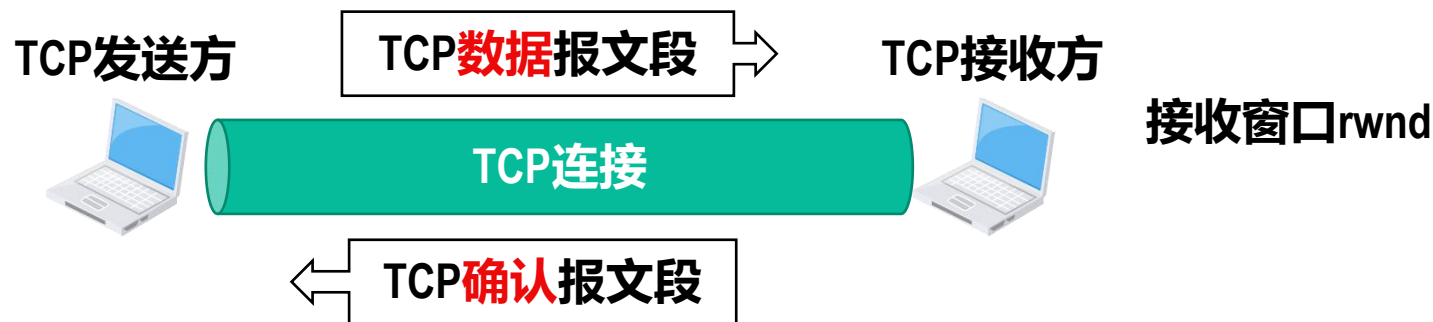
慢开始
(slow-start)

拥塞避免
(congestion avoidance)

快重传
(fast retransmit)

快恢复
(fast recovery)

发送窗口swnd
拥塞窗口cwnd



序号落入swnd的数据可以发送
取决于网络的拥塞程度和发送方所
采用的TCP拥塞控制算法

$$\text{swnd} = \min(\text{cwnd}, \text{rwnd})$$

取决于接收方可用的接收缓存的大小

不考虑拥塞控制
不考虑流量控制

$$\begin{aligned}\text{swnd} &= \text{rwnd} \\ \text{swnd} &= \text{cwnd}\end{aligned}$$

03 TCP的四种拥塞控制方法

慢开始
(slow-start)

拥塞避免
(congestion avoidance)

快重传
(fast retransmit)

快恢复
(fast recovery)

发送窗口swnd

拥塞窗口cwnd

慢开始门限sssthresh

不考虑流量控制

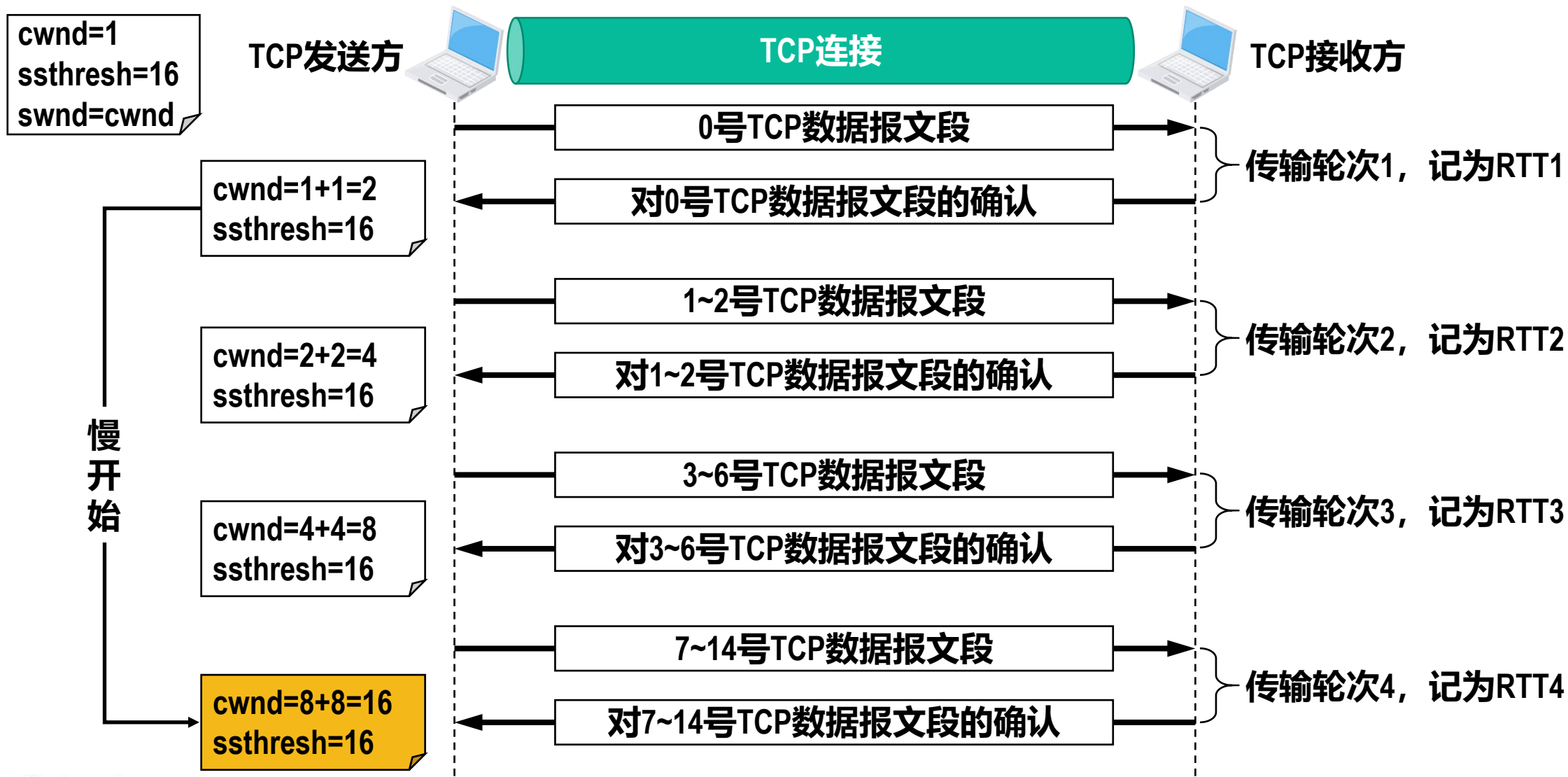


$$swnd = cwnd$$

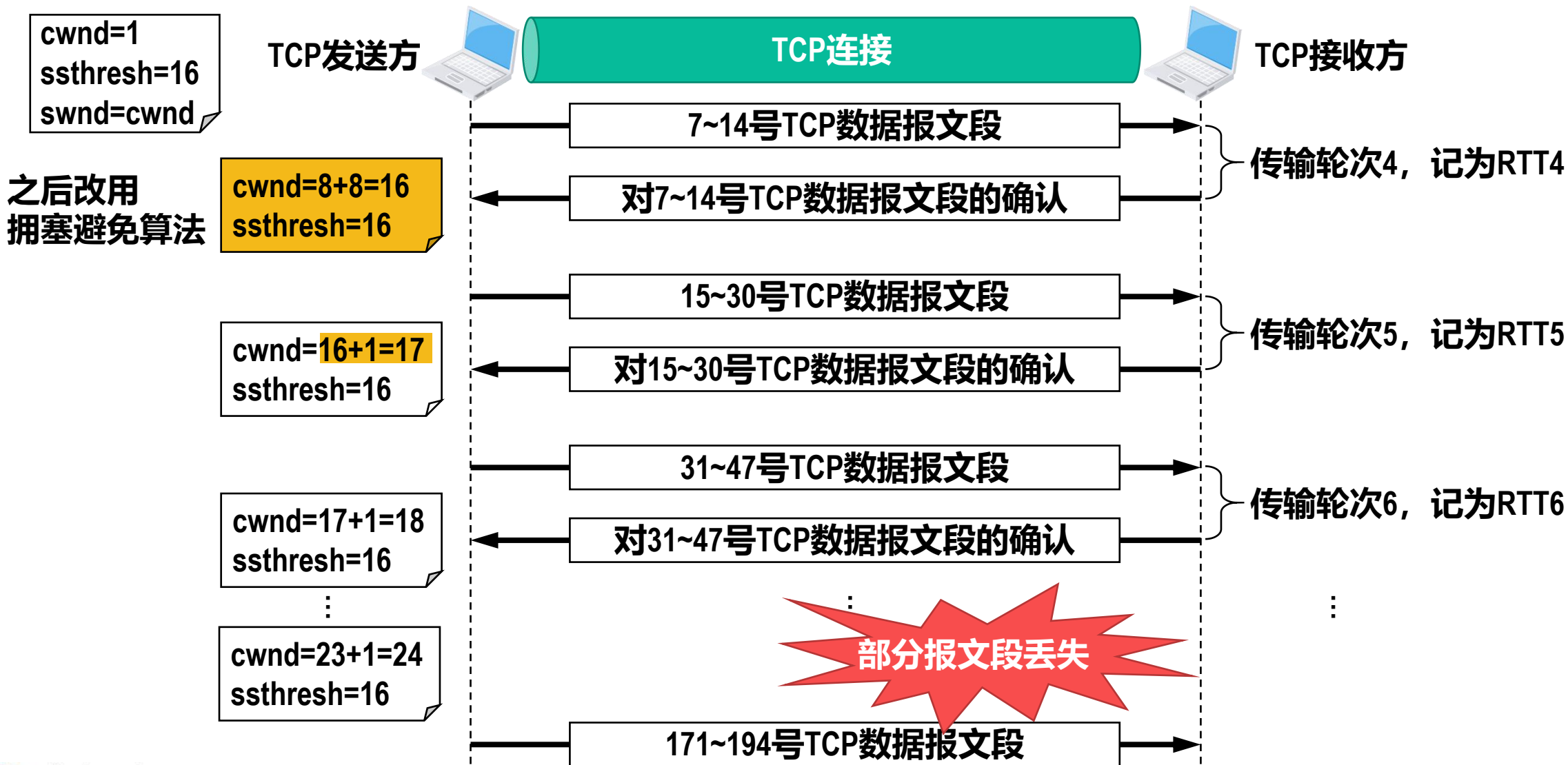
- cwnd的维护原则：只要网络没有**出现拥塞**，**拥塞窗口就再增大一些**，但只要**网络出现拥塞**，**拥塞窗口就减少一些**。
- 判断网络出现拥塞的依据：没有按时收到应当到达的TCP确认报文段而产生了**超时重传**。

- 当 $cwnd < sssthresh$ 时，使用慢开始算法。
- 当 $cwnd > sssthresh$ 时，停止使用慢开始算法而改用拥塞避免算法。
- 当 $cwnd = sssthresh$ 时，既可使用慢开始算法，也可使用拥塞避免算法。

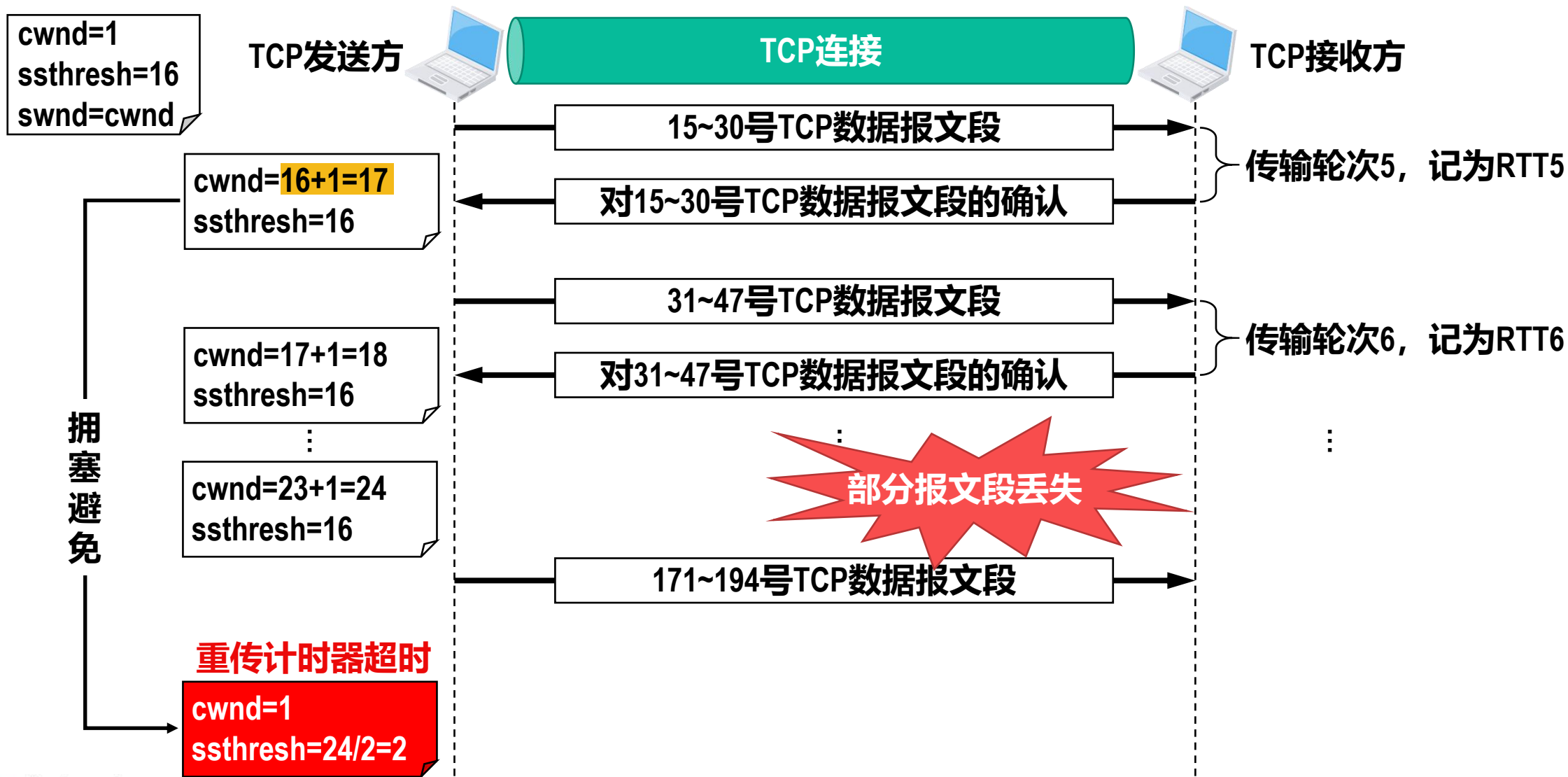
03 TCP的四种拥塞控制方法 —— 慢开始算法和拥塞避免算法



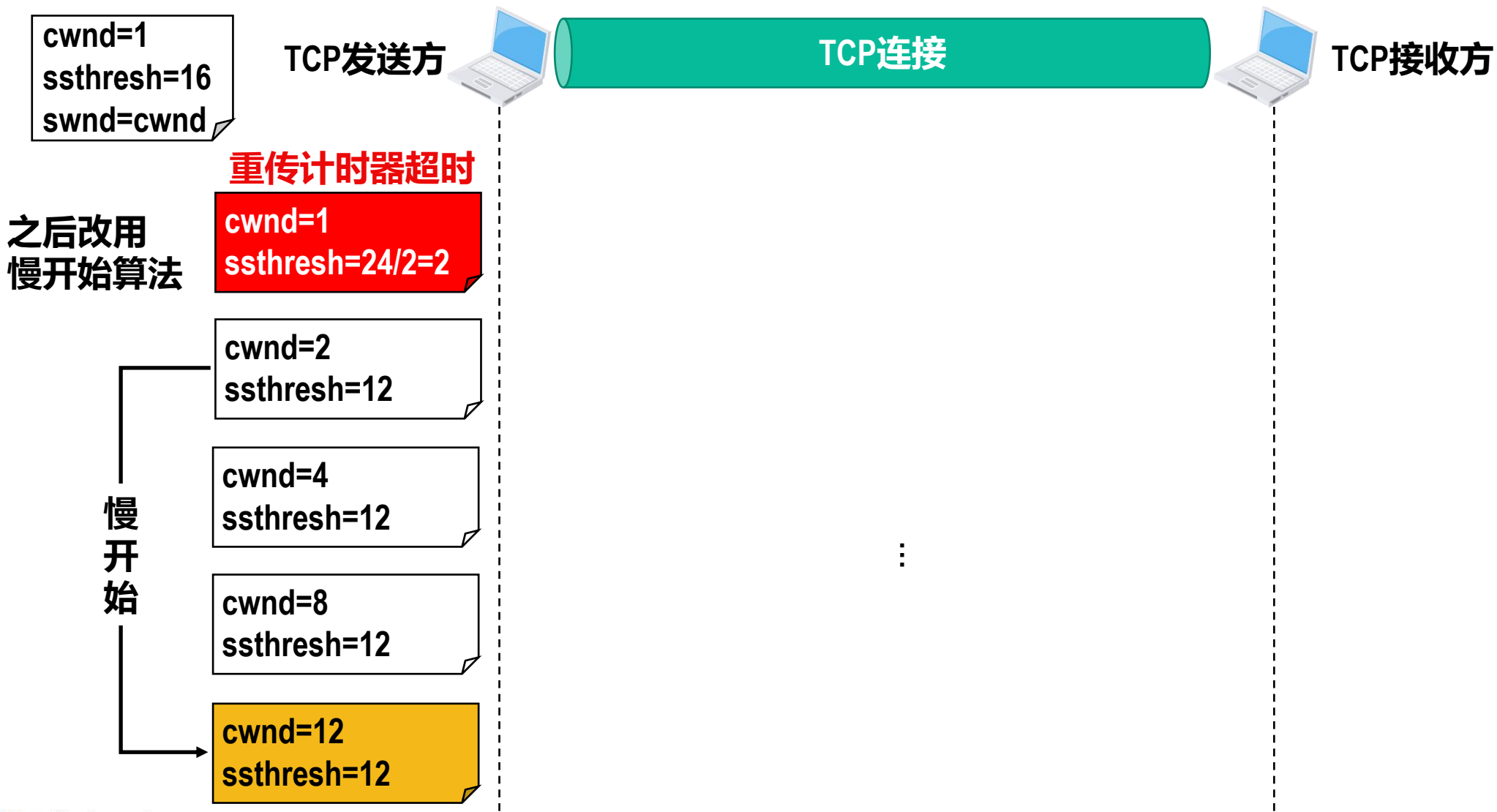
03 TCP的四种拥塞控制方法 —— 慢开始算法和拥塞避免算法



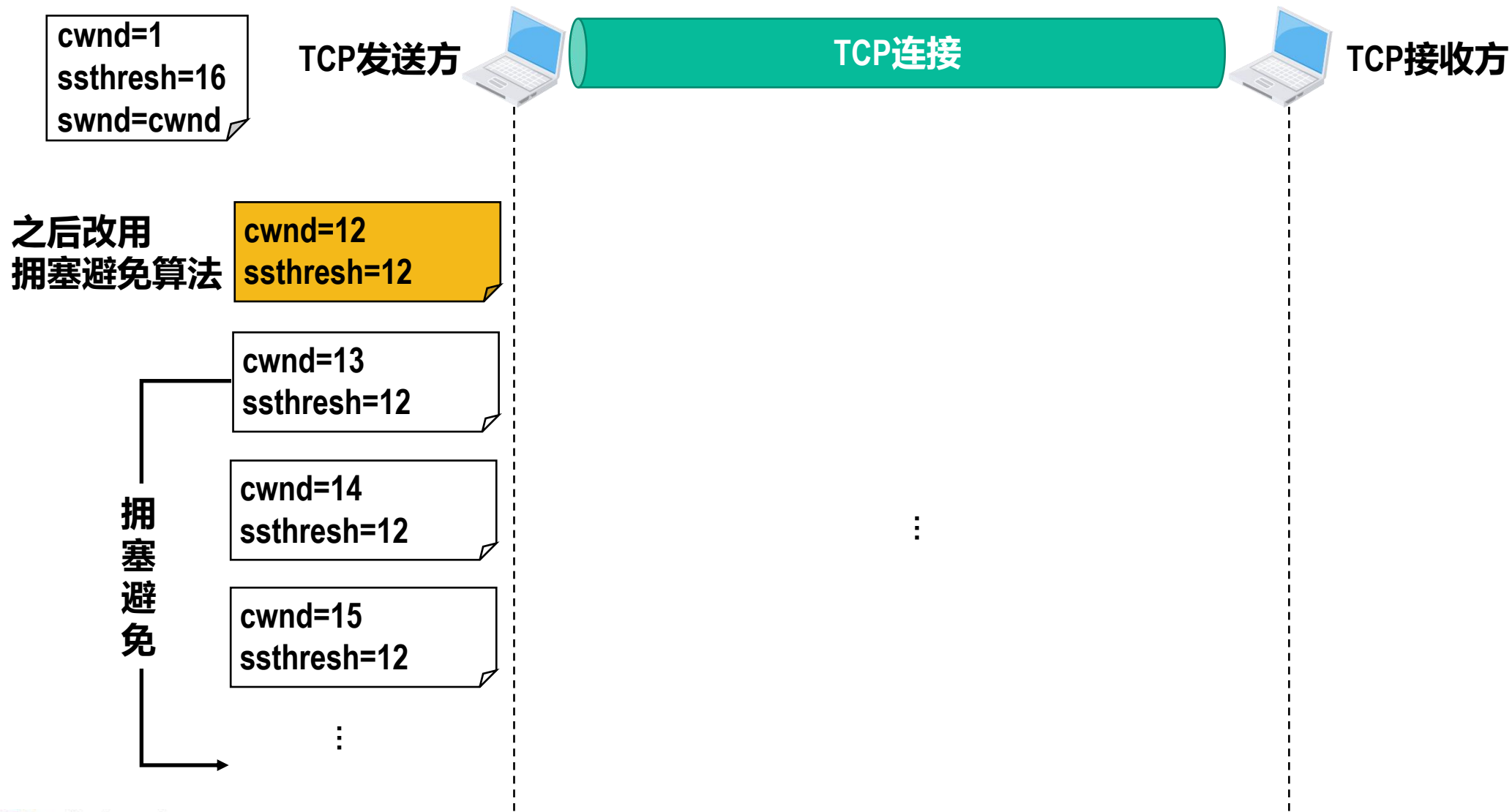
03 TCP的四种拥塞控制方法 —— 慢开始算法和拥塞避免算法



03 TCP的四种拥塞控制方法——慢开始算法和拥塞避免算法



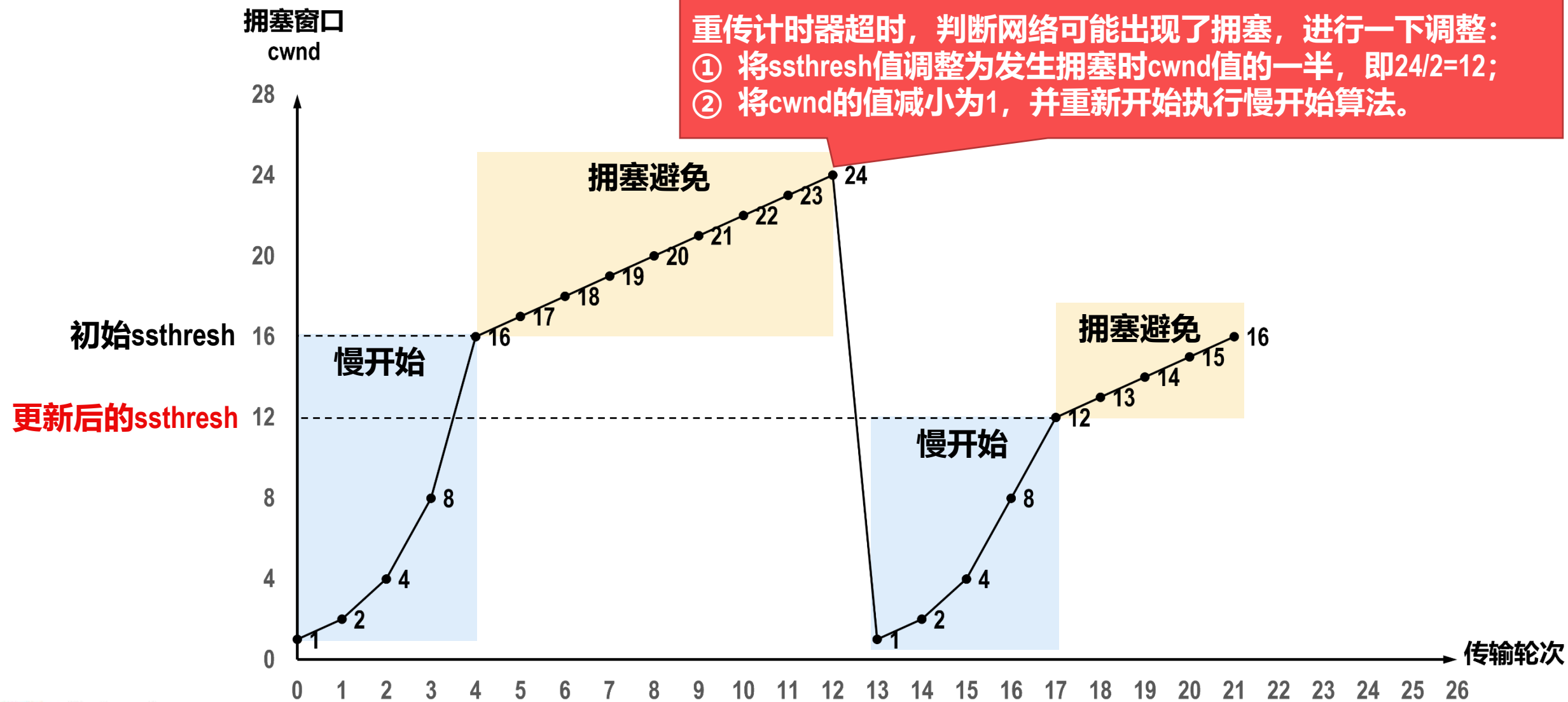
03 TCP的四种拥塞控制方法——慢开始算法和拥塞避免算法



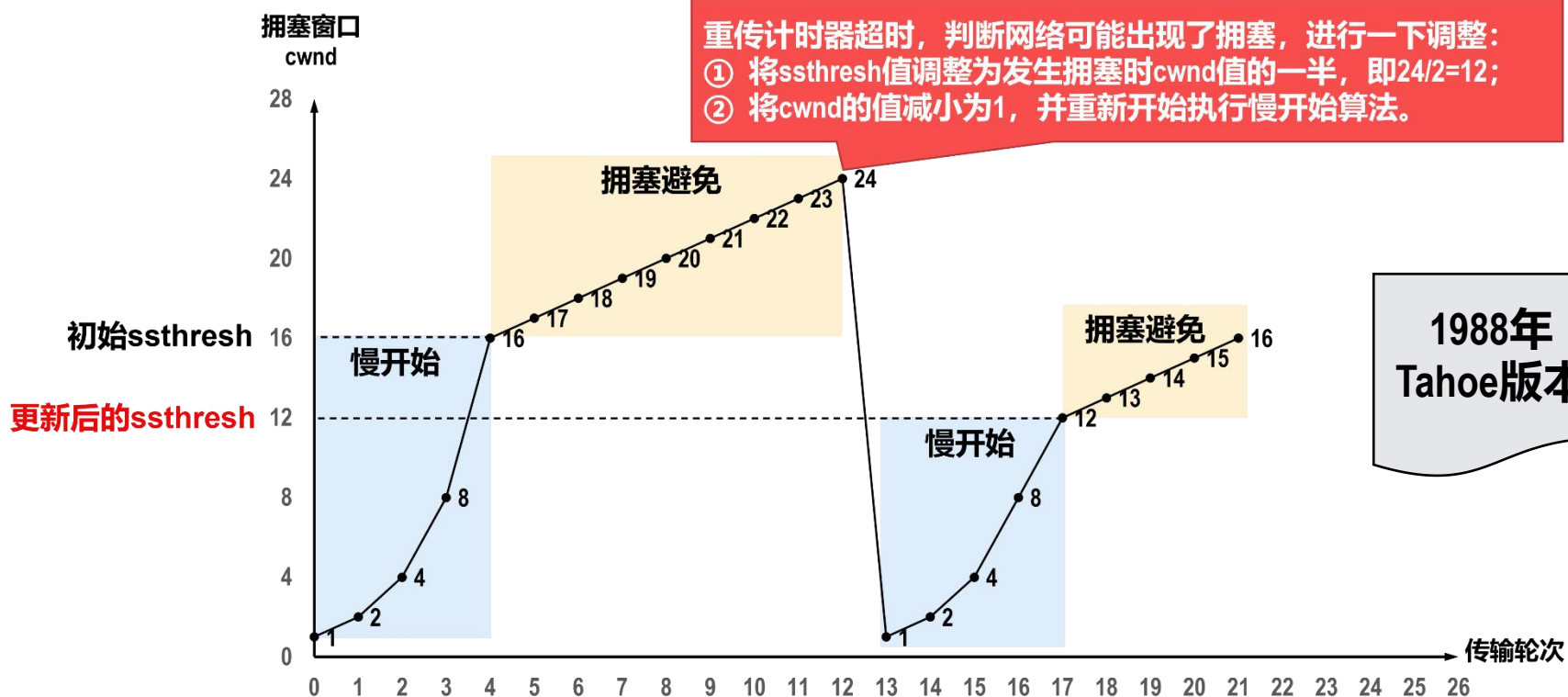
03 TCP的四种拥塞控制方法——慢开始算法和拥塞避免算法

重传计时器超时，判断网络可能出现了拥塞，进行一下调整：

- ① 将ssthresh值调整为发生拥塞时cwnd值的一半，即 $24/2=12$ ；
- ② 将cwnd的值减小为1，并重新开始执行慢开始算法。



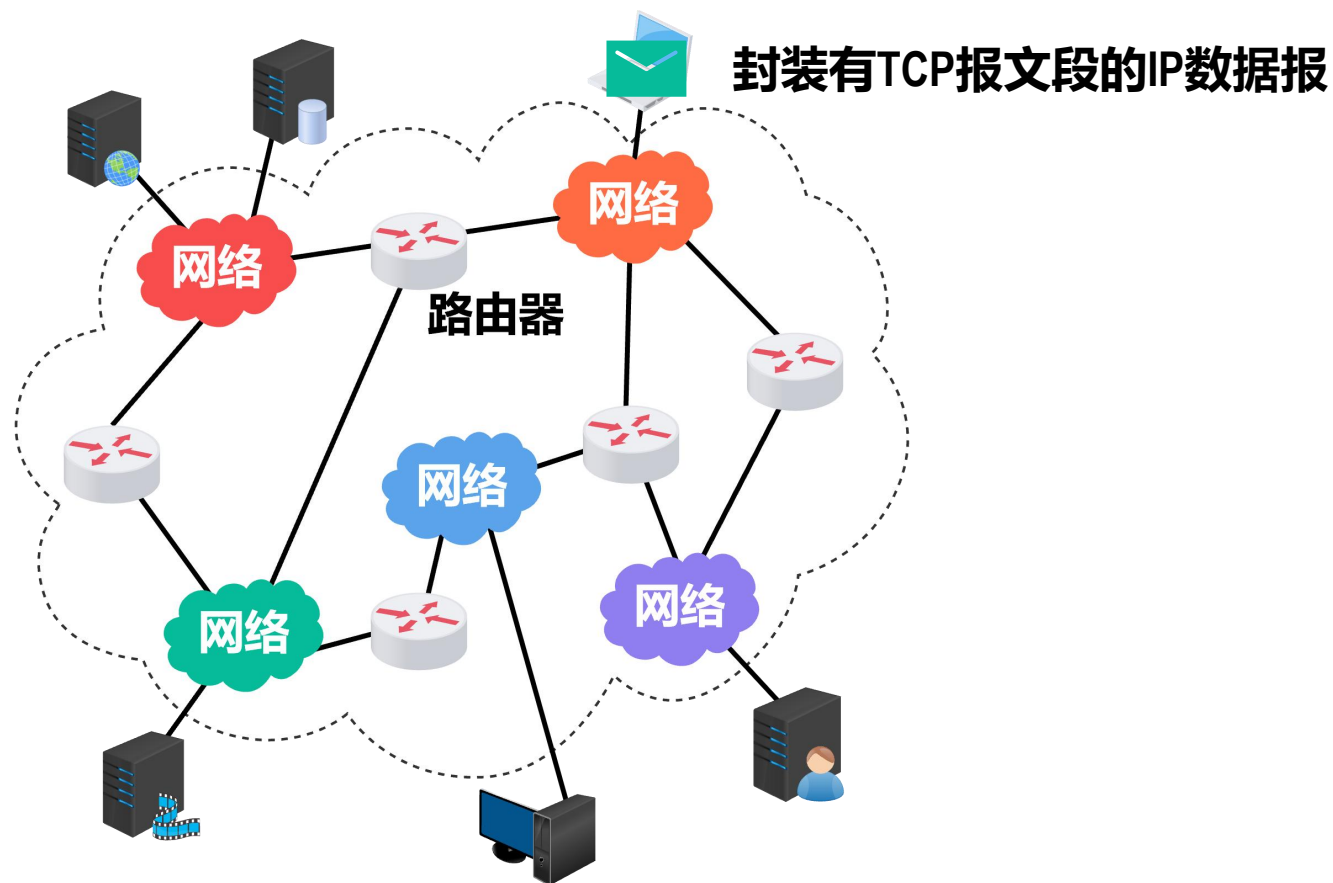
03 TCP的四种拥塞控制方法——慢开始算法和拥塞避免算法



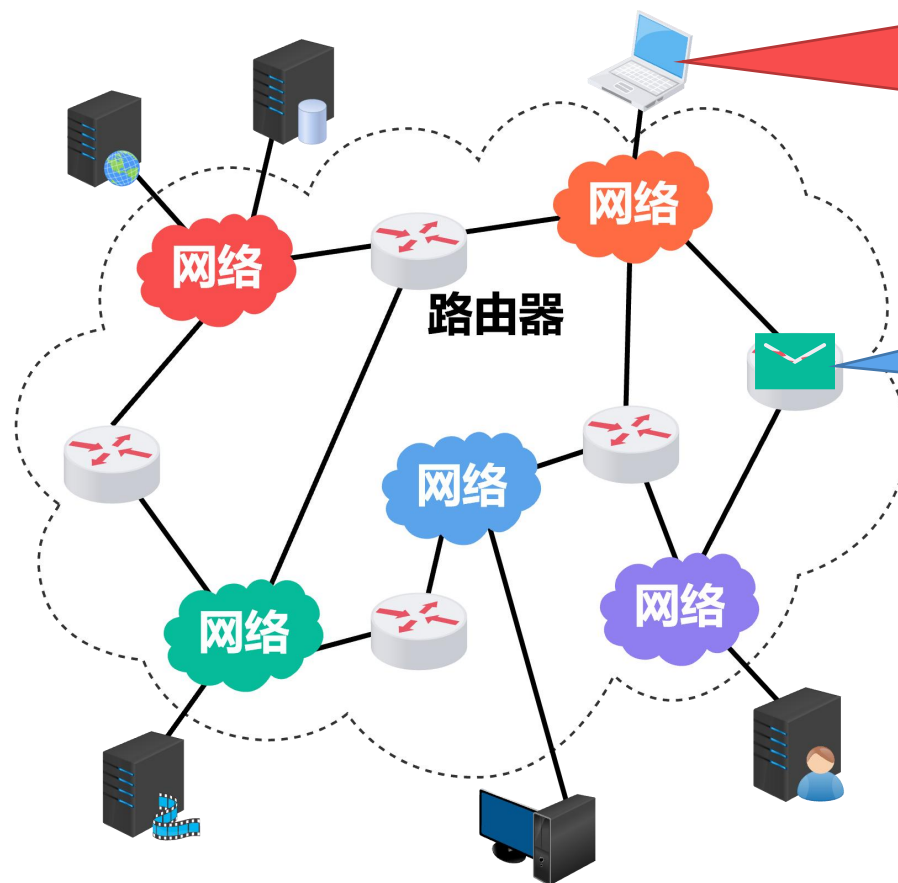
“慢开始”是指一开始向网络注入的报文段少，而并不是指拥塞窗口cwnd的值增长速度慢。

“拥塞避免”也并非指完全能够避免拥塞，而是指在拥塞避免阶段将cwnd值控制为按线性规律增长，使网络比较不容易出现拥塞。

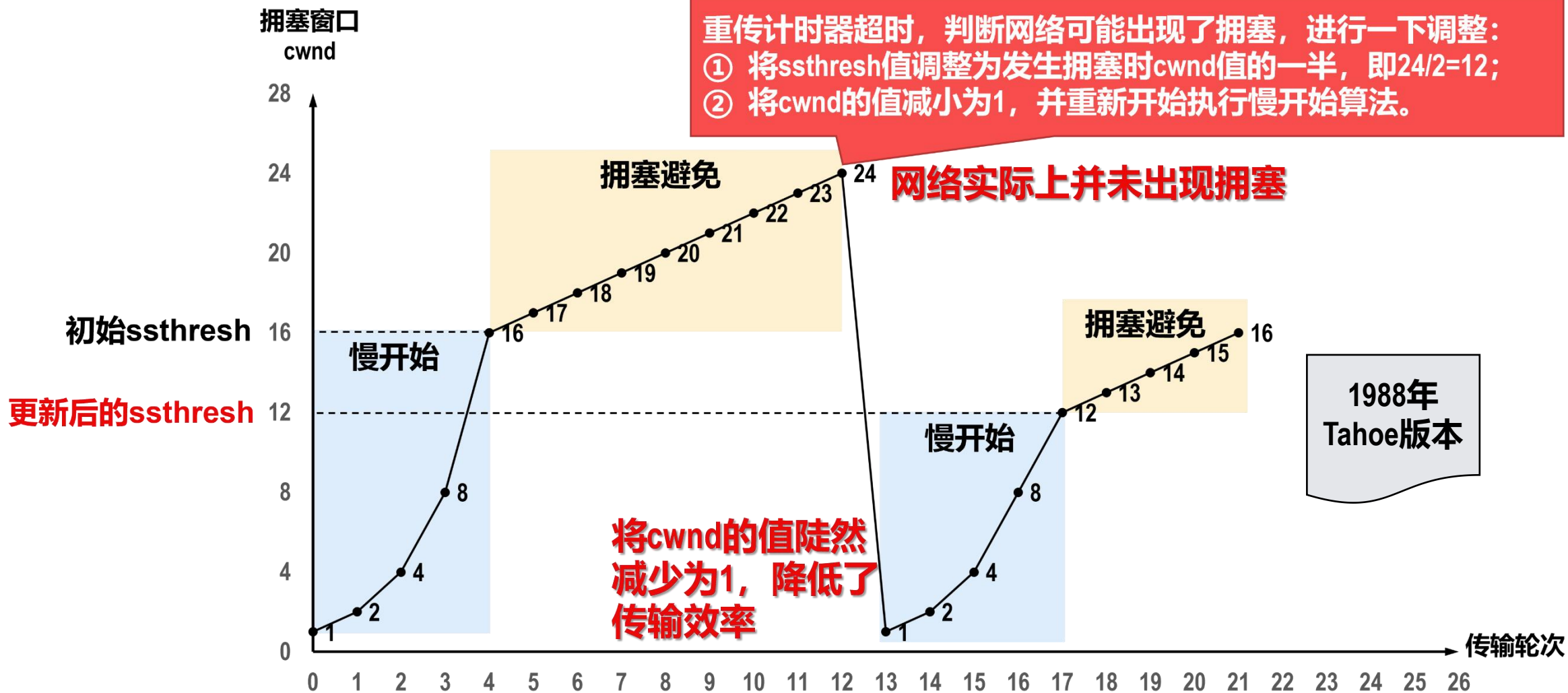
03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）



03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）



03 TCP的四种拥塞控制方法——快重传算法和快恢复算法 (改进TCP性能, 1990年Reno版本)



03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）

■ 采用快重传算法可以让发送方尽早知道发生了个别TCP报文段的丢失。

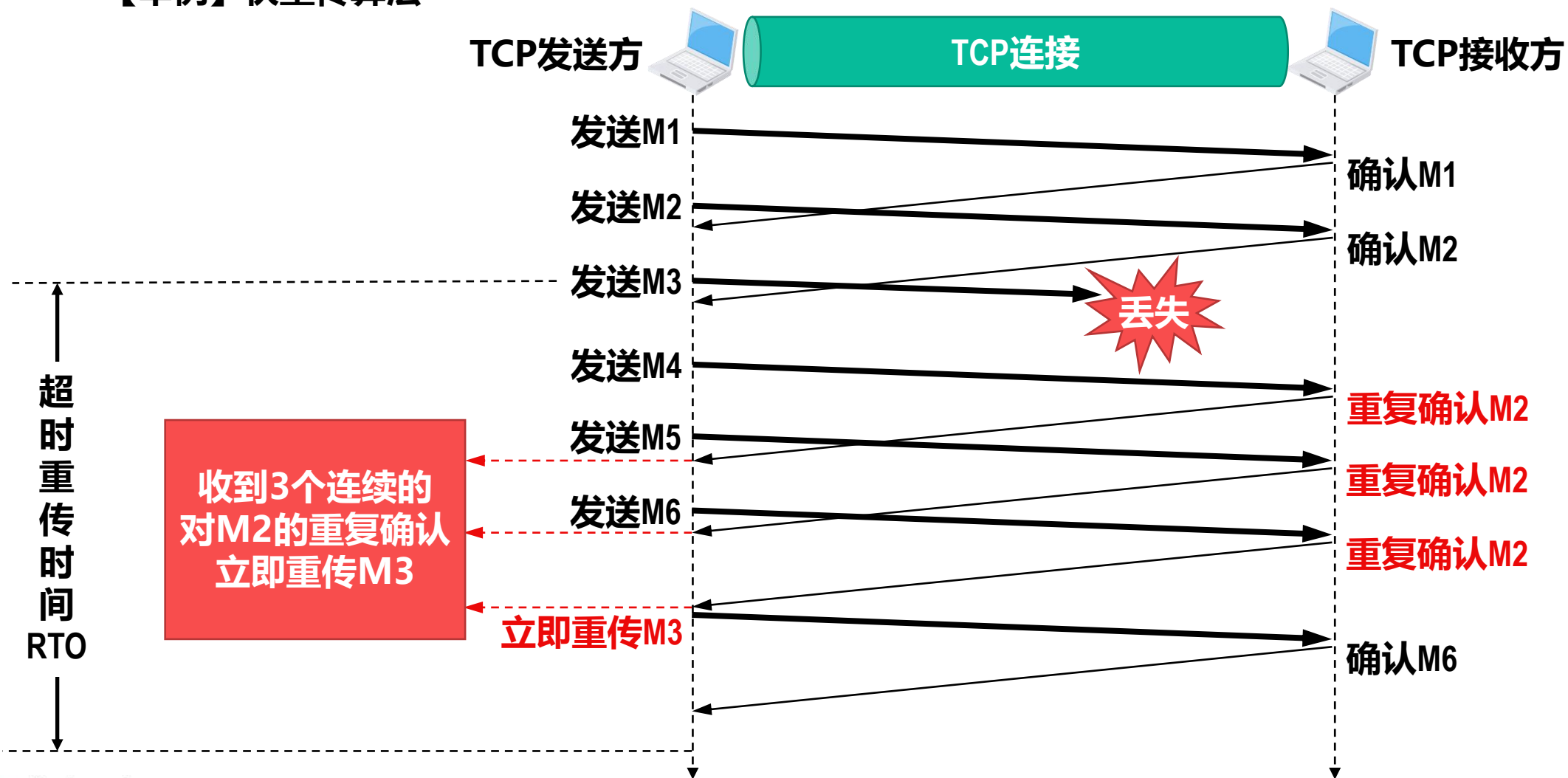
■ “快重传”是指**使发送方尽快（尽早）进行重传，而不是等重传计时器超时再重传。**

□ 这就要求**接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认，即使收到了失序的报文段也要立即发出对已收到的报文段的重复确认。**

□ **发送方一旦收到3个连续的重复确认，就将相应的报文段立即重传，而不是等该报文段的重传计时器超时再重传。**

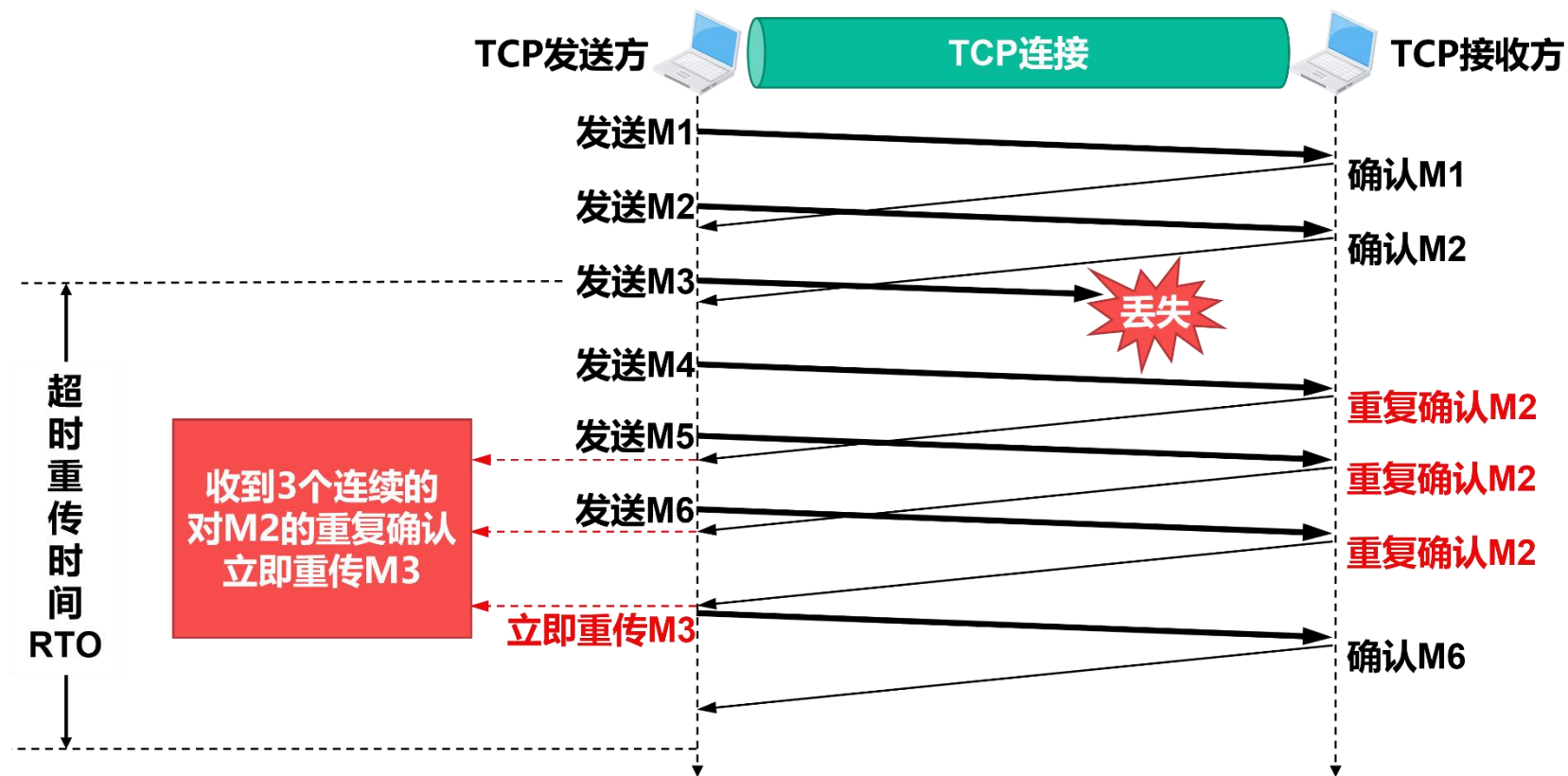
03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）

【举例】快重传算法



03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）

【举例】快重传算法



对于个别丢失的报文段，发送方不会出现超时重传，也就不会误认为出现了拥塞而错误地把拥塞窗口cwnd的值减为1。实践证明，使用快重传可以使整个网络的吞吐量提高约20%。

03 TCP的四种拥塞控制方法——快重传算法和快恢复算法（改进TCP性能，1990年Reno版本）

■ 与快重传算法配合使用的是快恢复算法，发送方一旦**收到3个重复确认**，就知道现在只是丢失了个别的报文段，于是不启动慢开始算法，而是**执行快恢复算法**。

- ☐ 发送方将慢开始门限ssthresh的值和拥塞窗口cwnd的值都调整为当前cwnd值的一半，并开始执行拥塞避免算法。
- ☐ 也有的快恢复实现是把快恢复开始时的cwnd值再增大一些，即 $cwnd = \text{新ssthresh} + 3$ 。
 - ☐ 既然发送方收到了3个重复的确认，就表明有3个数据报文段已经离开了网络。
 - ☐ 这3个报文段不再消耗网络资源而是停留在接收方的接收缓存中。
 - ☐ 可见现在网络中不是堆积了报文段而是减少了3个报文段，因此可以适当把cwnd值增大一些。

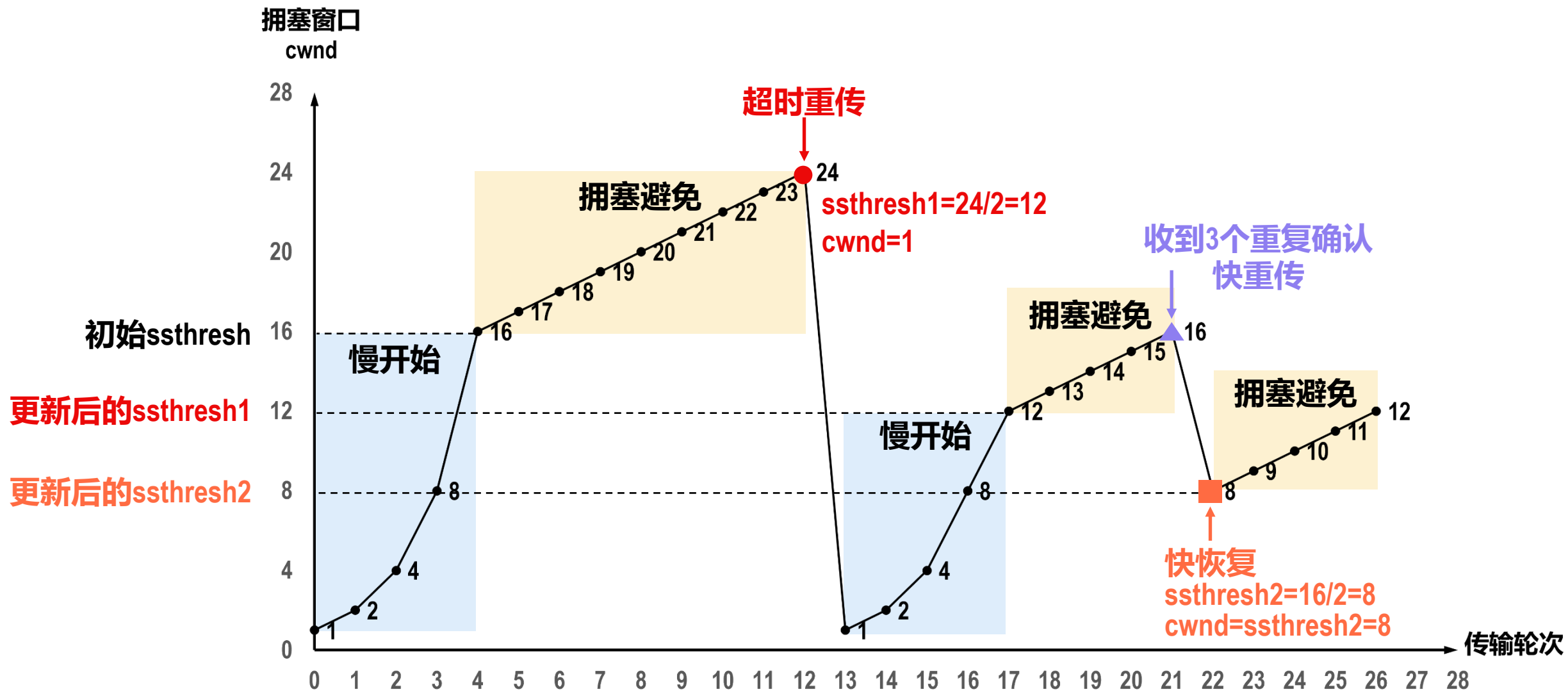
03 TCP的四种拥塞控制方法

慢开始

拥塞避免

快重传

快恢复



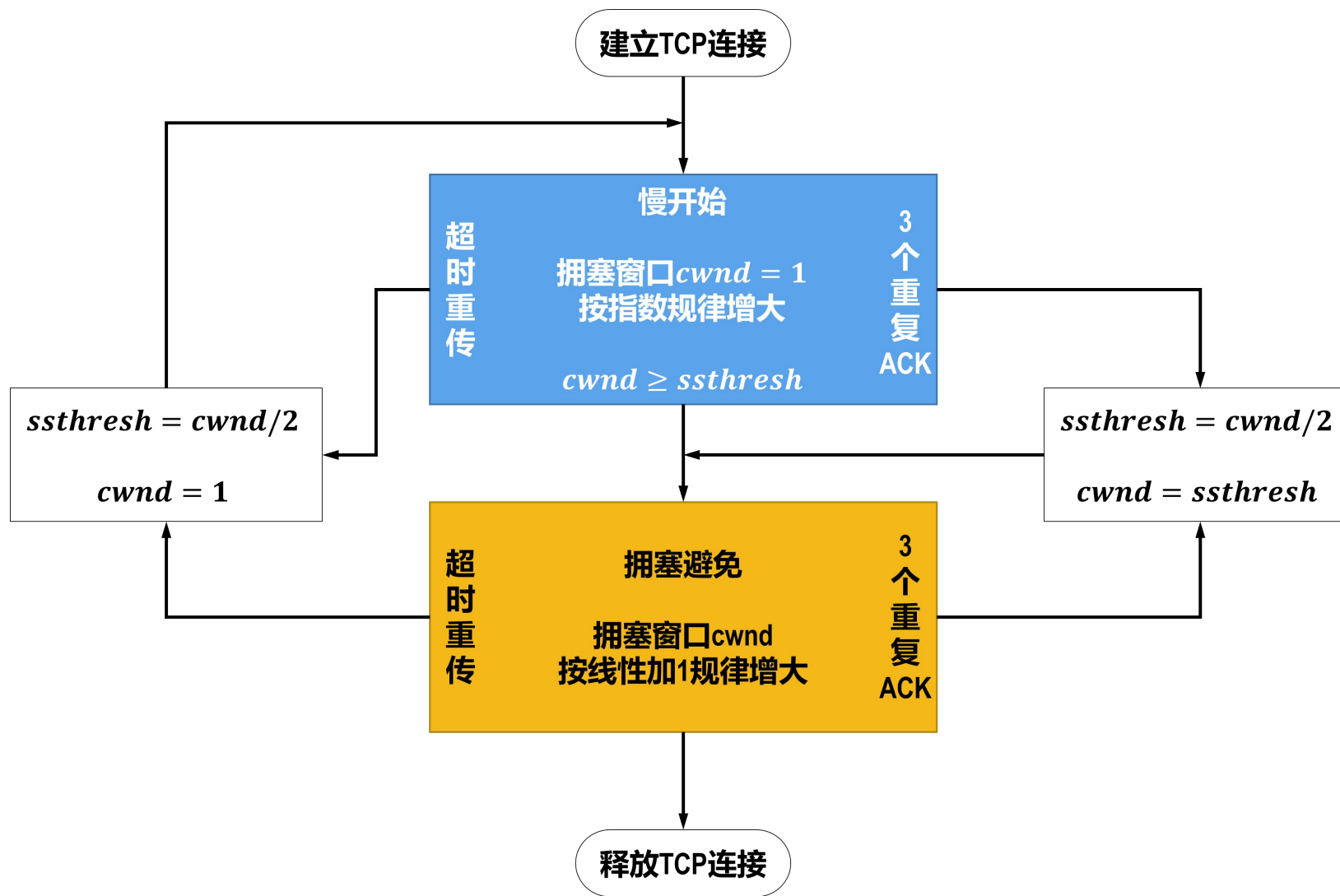
03 TCP的四种拥塞控制方法

慢开始

拥塞避免

快重传

快恢复



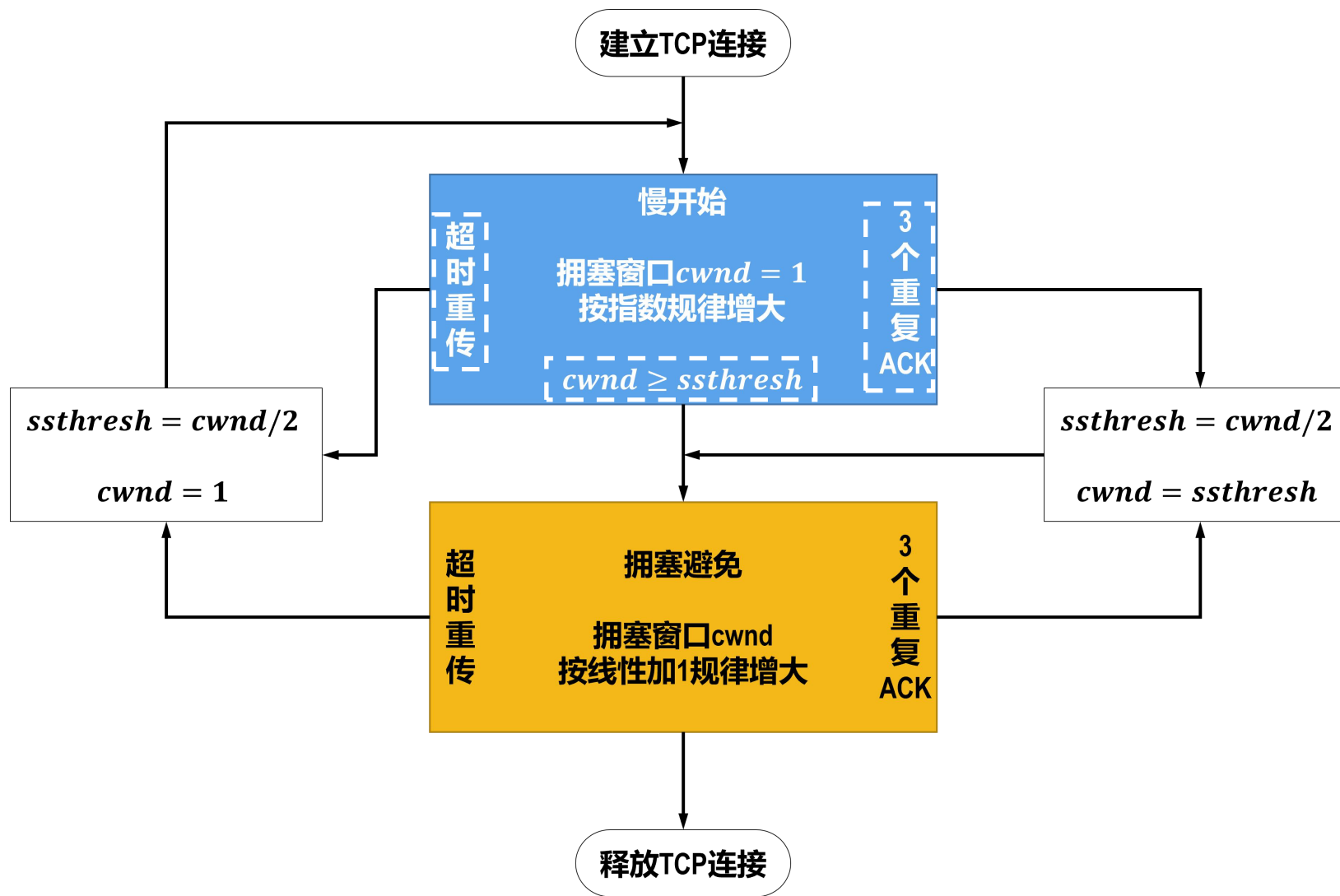
03 TCP的四种拥塞控制方法

慢开始

拥塞避免

快重传

快恢复



03 TCP的四种拥塞控制方法

【2009年 题39】一个TCP连接总是以1KB的最大段长发送TCP段，发送方有足够多的数据要发送。当拥塞窗口为16KB时发生了超时，如果接下来的4个RTT（往返时间）时间内的TCP段的传输都是成功的，那么当第4个RTT时间内发送的所有TCP段都得到肯定应答时，拥塞窗口大小是（C）。

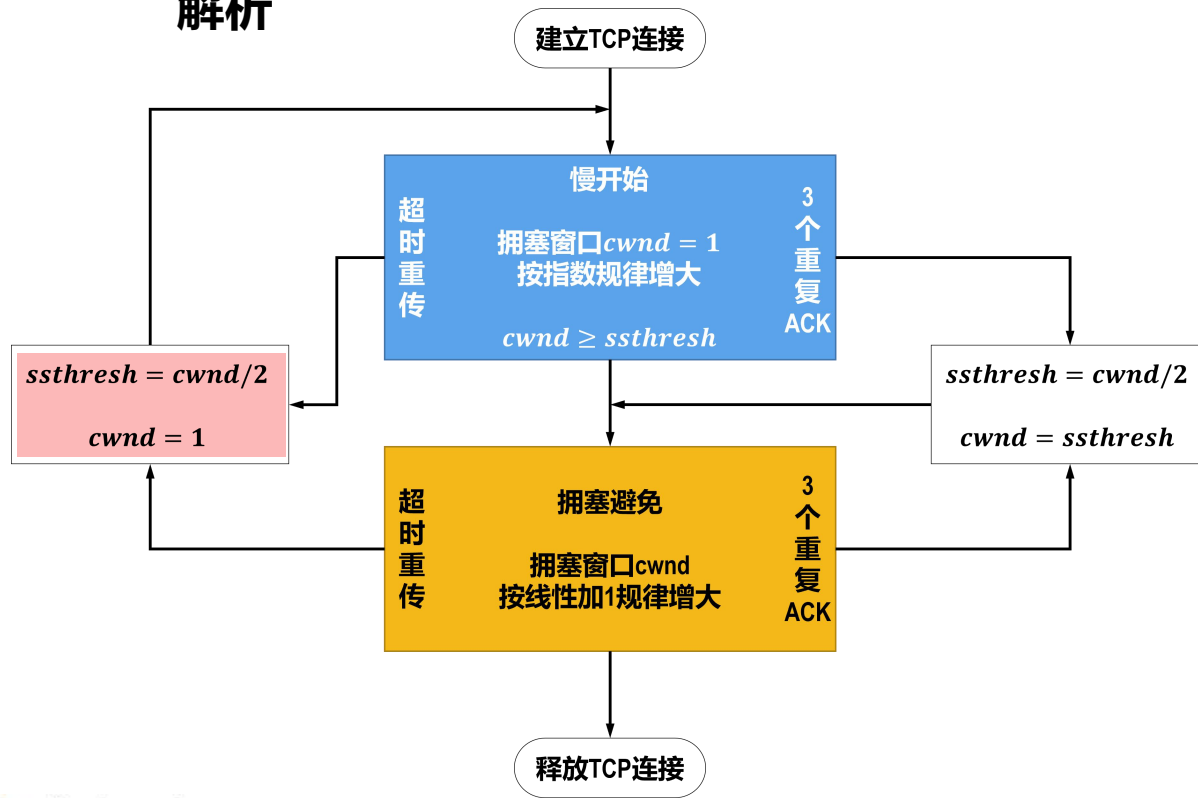
A. 7KB

B. 8KB

C. 9KB

D. 16KB

解析



出现超时重传时的处理

$$ssthresh = 16 / 2 = 8$$

$$cwnd = 1$$

开始慢开始算法

之后的

第1个传输轮次 (RTT) 结束

$$cwnd = 1 + 1 = 2$$

第2个传输轮次 (RTT) 结束

$$cwnd = 2 + 2 = 4$$

第3个传输轮次 (RTT) 结束

$$cwnd = 4 + 4 = 8$$

由于此时 $cwnd = 8 = ssthresh$ ，之后改为拥塞避免算法。

第4个传输轮次 (RTT) 结束

$$cwnd = 8 + 1 = 9$$

03 TCP的四种拥塞控制方法

【2014年 题38】主机甲和主机乙已建立了TCP连接，甲始终以MSS=1KB大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为10KB的确认段。若甲在t时刻发生超时时拥塞窗口为8KB，则从t时刻起，不再发生超时的情况下，经过10个RTT后，甲的发送窗口是 (A) 。

A. 10KB

B. 12KB 排除

C. 14KB 排除

D. 15KB 排除

解析

本题不知是出题人在给出的选项上有小小的失误，还是有意而为之，使得我们可以“秒杀”。

$$swnd = \min(\underset{\substack{\text{最大值} \\ 10KB}}{cwnd}, \underset{\substack{\text{无论} \\ \text{如何} \\ \text{变化}}}{rwnd})$$

03 TCP的四种拥塞控制方法

【2014年 题38】主机甲和主机乙已建立了TCP连接，甲始终以MSS=1KB大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为10KB的确认段。若甲在t时刻发生超时，拥塞窗口为8KB，则从t时刻起，不再发生超时的情况下，经过10个RTT后，甲的发送窗口是 (A)。

A. 10KB

B. 12KB 排除

C. 14KB 排除

D. 15KB 排除

解析

出现超时重传时的处理 { $ssthresh = 8 / 2 = 4$
 $cwnd = 1$
 开始慢开始算法

之后的

第1个传输轮次 (RTT) 结束 $cwnd = 1 + 1 = 2$

第2个传输轮次 (RTT) 结束 $cwnd = 2 + 2 = 4$

由于此时 $cwnd = 4 = ssthresh$ ，之后改为拥塞避免算法。

第3个传输轮次 (RTT) 结束 $cwnd = 4 + 1 = 5$

第4个传输轮次 (RTT) 结束 $cwnd = 5 + 1 = 6$

第5个传输轮次 (RTT) 结束 $cwnd = 6 + 1 = 7$

第6个传输轮次 (RTT) 结束 $cwnd = 7 + 1 = 8$

第7个传输轮次 (RTT) 结束 $cwnd = 8 + 1 = 9$

第8个传输轮次 (RTT) 结束 $cwnd = 9 + 1 = 10$

第9个传输轮次 (RTT) 结束 $cwnd = 10 + 1 = 11$

第10个传输轮次 (RTT) 结束 $cwnd = 11 + 1 = 12$

$$swnd = \min(cwnd, rwnd)$$

$$= \min(12, 10) = 10$$

03 TCP的四种拥塞控制方法

【2015年 题39】主机甲和主机乙新建一个TCP连接，甲的拥塞控制初始阈值为32KB，甲向乙始终以MSS=1KB大小的段发送数据，并一直有数据发送；乙为该连接分配16KB接收缓存，并对每个数据段进行确认，忽略段传输延迟。若乙收到的数据全部存入缓存，不被取走，则甲从连接建立成功时刻起，未发生超时的情况下，经过4个RTT后，甲的发送窗口是（**A**）。

A. 1KB

B. 8KB

C. 16KB

D. 32KB

解析

$$swnd = \min(cwnd, rwnd)$$

初始状态

$$swnd = \min(1, 16) = 1$$

第1个RTT结束

$$swnd = \min(1+1, 15) = 2$$

第2个RTT结束

$$swnd = \min(2+2, 13) = 4$$

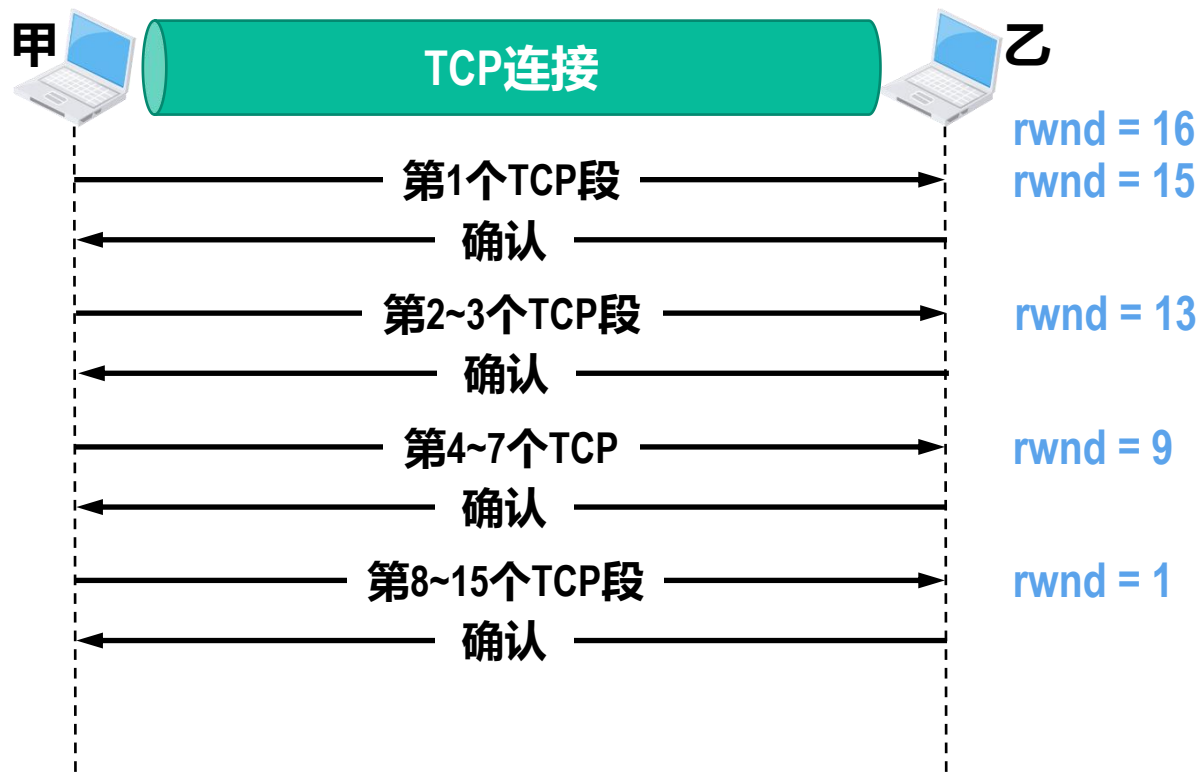
第3个RTT结束

$$swnd = \min(4+4, 9) = 8$$

第4个RTT结束

$$swnd = \min(8+8, 1) = 1$$

慢开始



03 TCP的四种拥塞控制方法

【2017年 题39】若甲向乙发起一个TCP连接，最大段长MSS=1KB，RTT=5ms，乙开辟的接收缓存为64KB，则甲从连接建立成功至发送窗口达到32KB，需经过的时间至少是（**A**）。

A. 25ms

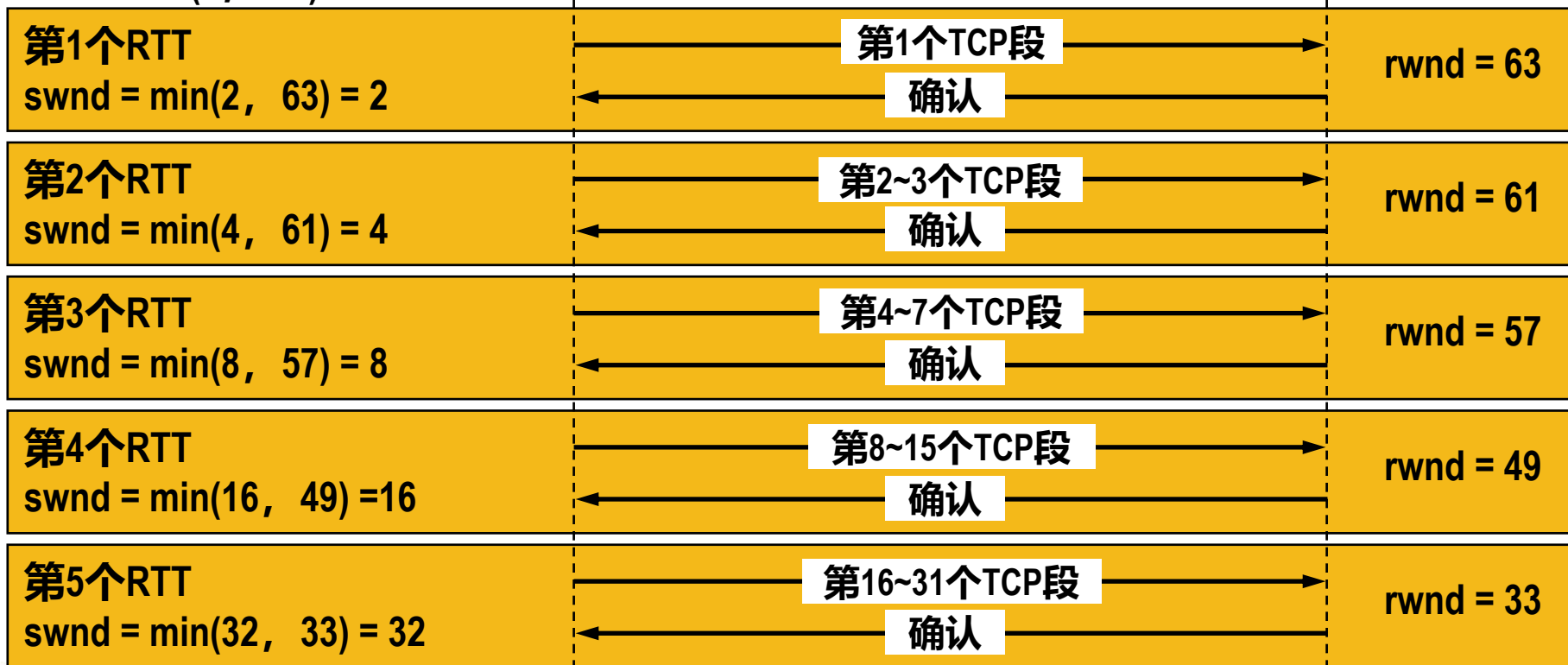
B. 30ms

C. 160ms

D. 165ms

解析

$\text{swnd} = \min(\text{cwnd}, \text{rwnd})$
 $\text{swnd} = \min(1, 64) = 1$

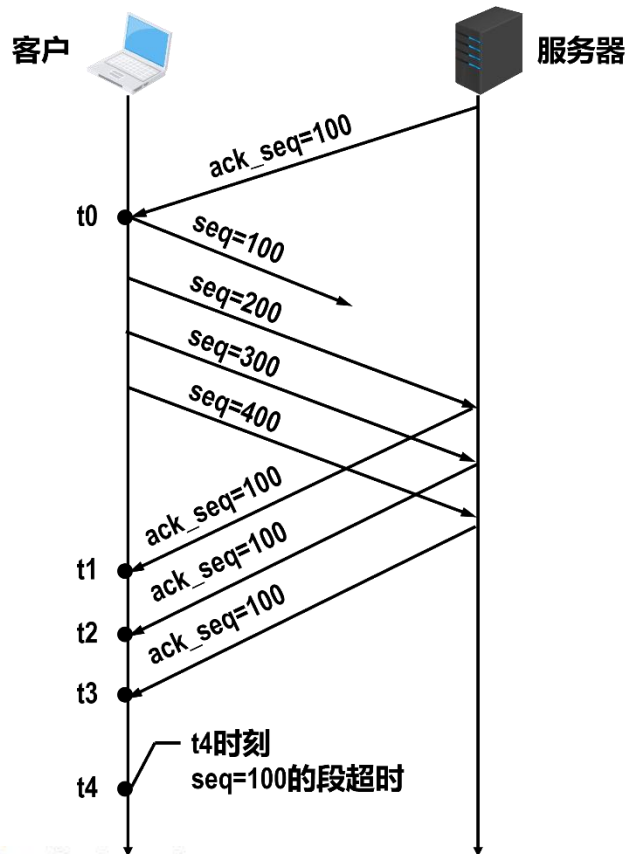


甲从连接建立成功至发送窗口swnd达到32KB，至少（即没有出现超时重传）经历5个RTT。

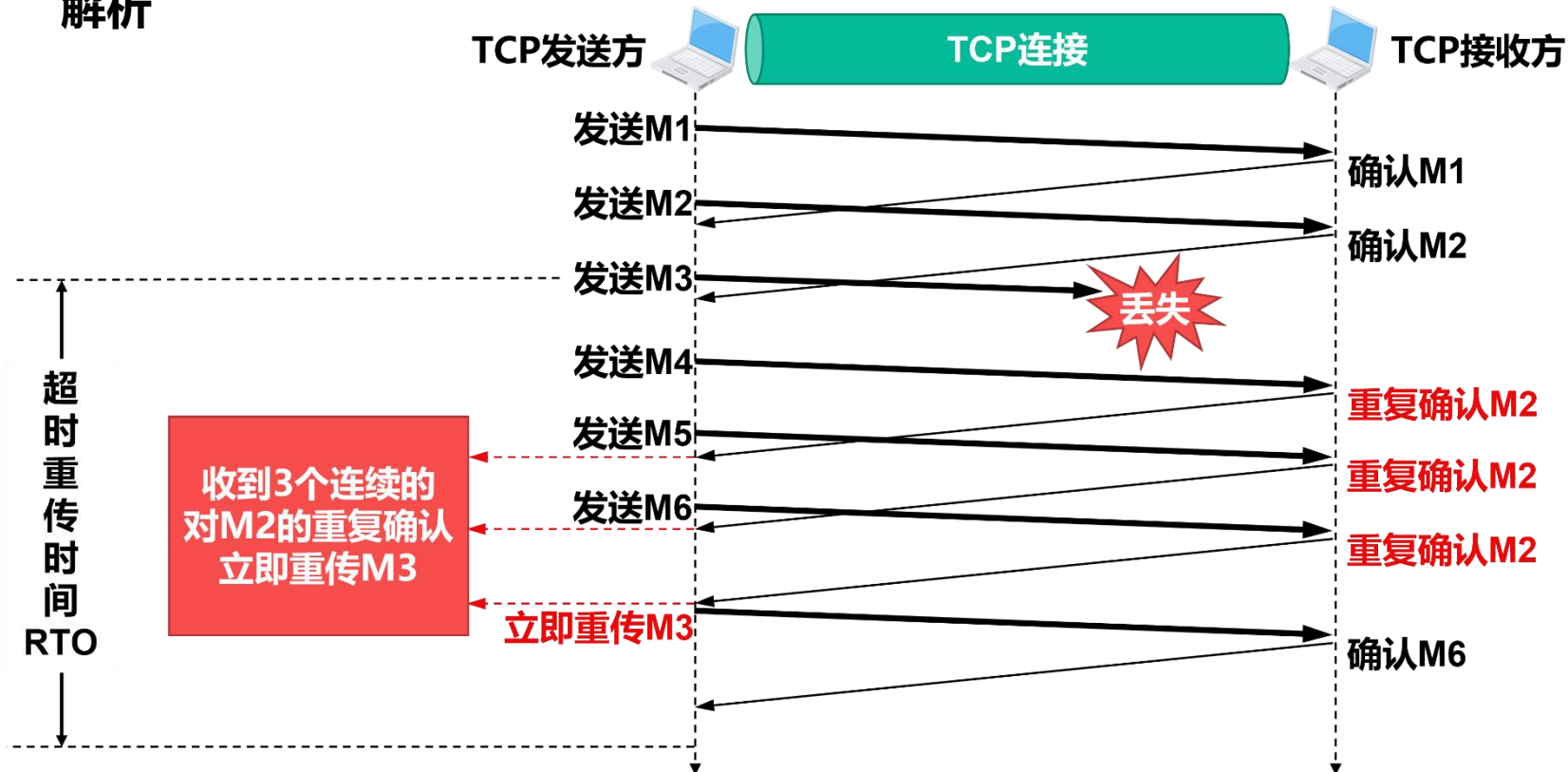
$$5\text{ms} \times 5 = 25\text{ms}$$

03 TCP的四种拥塞控制方法

【2019年 题38】若客户通过一个TCP连接向服务器发送数据的部分过程如下图所示。客户在 t_0 时刻第一次收到确认序列号 $\text{ack_seq}=100$ 的段，并发送序列号 $\text{seq}=100$ 的段，但发生丢失。若TCP支持快速重传，则客户重新发送 $\text{seq}=100$ 段的时刻是 (C)。

A. t_1 B. t_2 C. t_3 D. t_4 

解析



03 TCP的四种拥塞控制方法

【2020年 题38】若主机甲与主机乙已建立一条TCP连接，最大段长MSS为1KB，往返时间RTT为2ms，则在不出
现拥塞的前提下，拥塞窗口从8KB增长到20KB所需的最长时间是（C）。

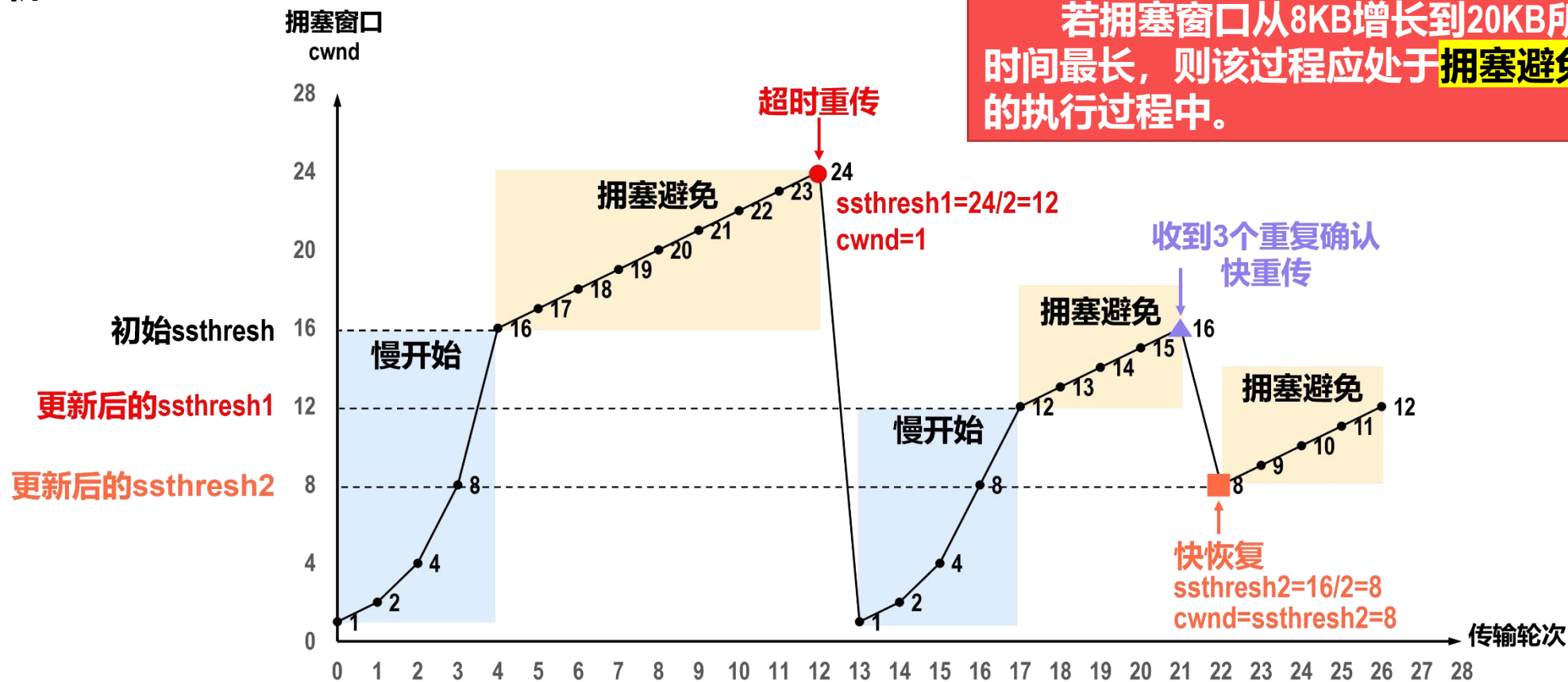
A. 4ms

B. 8ms

C. 24ms

D. 48ms

解析



03 TCP的四种拥塞控制方法

【2020年 题38】若主机甲与主机乙已建立一条TCP连接，最大段长MSS为1KB，往返时间RTT为2ms，则在不出
现拥塞的前提下，拥塞窗口从8KB增长到20KB所需的最长时间是（C）。

A. 4ms

B. 8ms

C. 24ms

D. 48ms

解析

若拥塞窗口从8KB增长到20KB所需的时间最长，则该过程应处于拥塞避免算法的执行过程中。

在上述过程中：

第1个传输轮次 (RTT) 结束 $cwnd = 8 + 1 = 9$

第2个传输轮次 (RTT) 结束 $cwnd = 9 + 1 = 10$

⋮

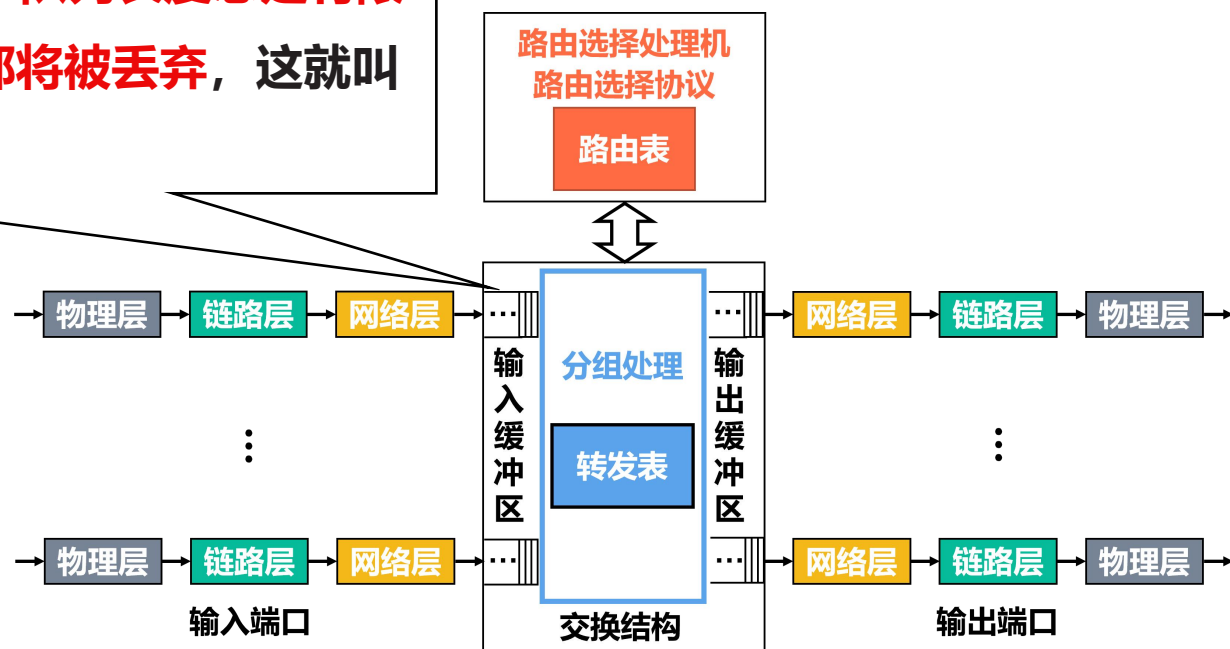
第12个传输轮次 (RTT) 结束 $cwnd = 19 + 1 = 20$

共经历了12个传输轮次 (RTT)，总耗时为 $2ms \times 12 = 24ms$

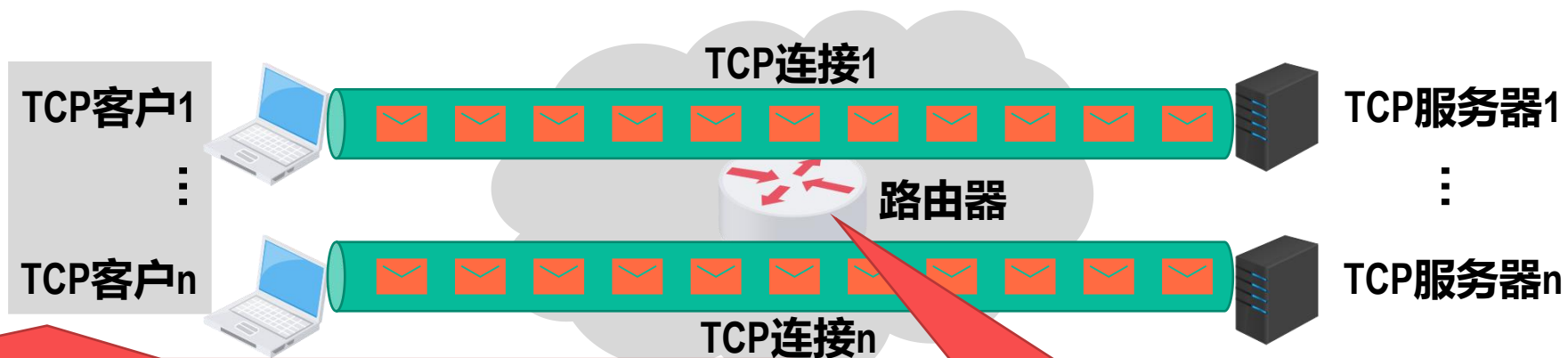
04 TCP拥塞控制与网际层拥塞控制的关系



路由器的**输入缓存**（可看作缓存队列，以下**简称为队列**）通常都按照“**先进先出FIFO**”的规则来处理到达的IP数据报。由于**队列长度总是有限的**，因此当**队列已满时**，之后**再到达的所有IP数据报都将被丢弃**，这就叫作**尾部丢弃策略**。



04 TCP拥塞控制与网际层拥塞控制的关系



② TCP报文段的这些发送方产生超时重传，这将导致它们将拥塞窗口cwnd的值陡降为1，因此发送窗口swnd的值也降低为1，并且进入TCP拥塞控制的慢开始阶段，称为全局同步。

① 接收缓存溢出而导致尾部丢弃

04 TCP拥塞控制与网际层拥塞控制的关系



③ 全网的通信量骤降，而在网络恢复正常后，其通信量又突然增大很多。



② TCP报文段的这些发送方产生超时重传，这将导致它们将拥塞窗口cwnd的值陡降为1，因此发送窗口swnd的值也降低为1，并且进入TCP拥塞控制的慢开始阶段，称为全局同步。

① 接收缓存溢出而导致尾部丢弃

04 TCP拥塞控制与网际层拥塞控制的关系



■ 为了避免网络中出现全局同步问题，在1998年提出了**主动队列管理**（Active Queue Management, **AQM**）。

- ☐ 所谓“主动”，就是在**路由器的队列长度达到某个阈值但还未满时就主动丢弃IP数据报**，而不是要等到路由器的队列已满时才不得不丢弃后面到达的IP数据报，这样就太被动了。
- ☐ 应当在**路由器队列长度达到某个值得警惕的数值时**，也就是**网络出现了某些拥塞征兆时**，就**主动丢弃到达的IP数据报来造成发送方的超时重传**，进而降低发送方的发送速率，因而有可能减轻网络的拥塞程度，甚至不出现网络拥塞。

■ 主动队列管理AQM可以有不同的实现方法，其中曾流行多年的就是**随机早期检测**（Random Early Detection, RED），也称为**随机早期丢弃**（Random Early Drop, RED 或 Random Early Discard, RED）。

- ☐ 路由器需要维护两个参数来实现RED：**队列长度最小门限和最大门限**。当每一个IP数据报到达路由器时，RED就按照规定的算法计算出当前的平均队列长度。

04 TCP拥塞控制与网际层拥塞控制的关系



■ 主动队列管理AQM可以有不同的实现方法，其中曾流行多年的就是随机早期检测（Random Early Detection, RED），也称为**随机早期丢弃**（Random Early Drop, RED 或 Random Early Discard, RED）。

- ☐ 路由器需要维护两个参数来实现RED：**队列长度最小门限**和**最大门限**。当每一个IP数据报到达路由器时，RED就按照规定的算法计算出当前的平均队列长度。
 - ☐ 若**平均队列长度小于最小门限**，则把新到达的IP数据报**存入**队列进行排队。
 - ☐ 若**平均队列长度大于最大门限**，则把新到达的IP数据报**丢弃**。
 - ☐ 若**平均队列长度在最小门限和最大门限之间**，则**按照某一丢弃概率 p 把新到达的IP数据报丢弃**（这体现了丢弃IP数据报的随机性）。

因特网工程任务组IETF曾经推荐在因特网中的路由器使用RED机制[RFC 2309]，但多年的实践证明，RED的使用效果并不理想。因此，在2015年公布的[RFC 7567]已经把[RFC 2309]列为“陈旧的”，并且不再推荐使用RED。然而，对路由器进行主动队列管理AQM仍然是必要的。现在已经有几种不同的算法来代替旧的RED，但都还在实验阶段。目前还没有一种算法能够成为IETF的标准，有兴趣的同学可以注意这方面的进展。

5.3.6 TCP可靠传输的实现

01

TCP可靠传输的实现

01 TCP可靠传输的实现

只考虑发送窗口swnd

发送方



字节序号

26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

TCP
数据
报文
段



基于
TCP
的
可靠
数据
传输

TCP
确
认
报
文
段



接收方

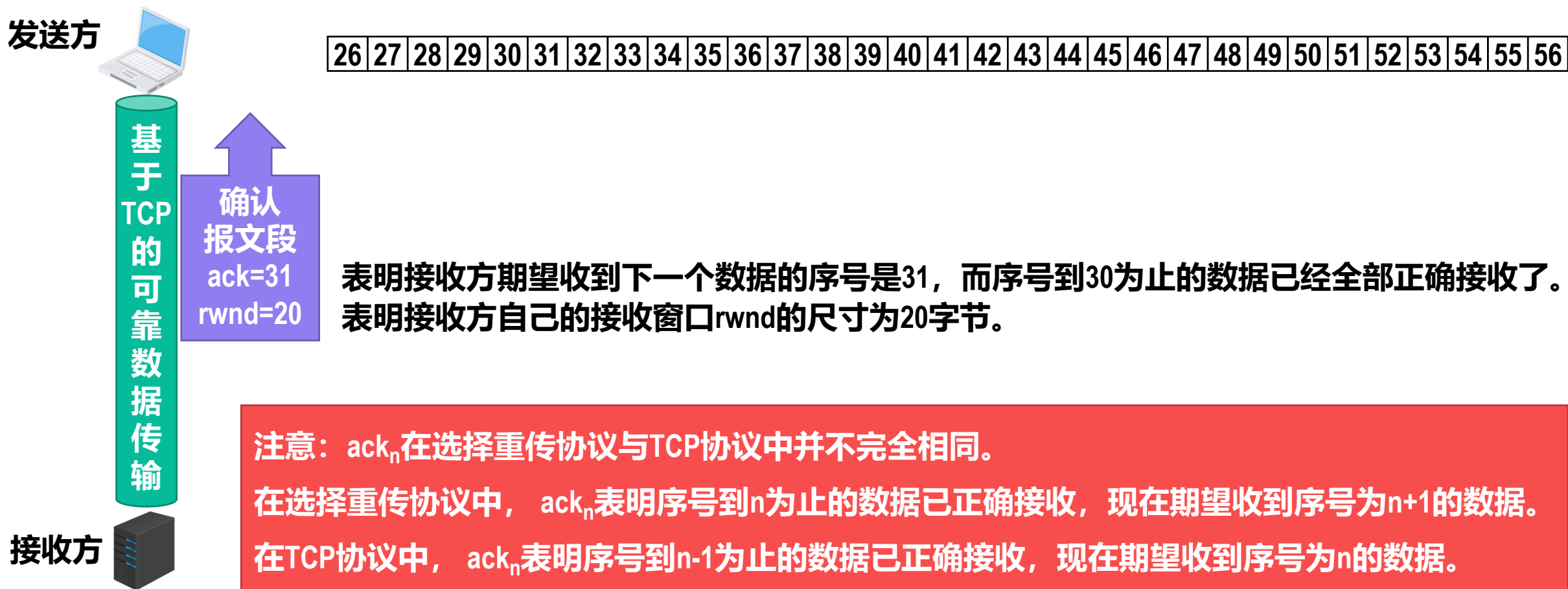


只考虑接收窗口rwnd

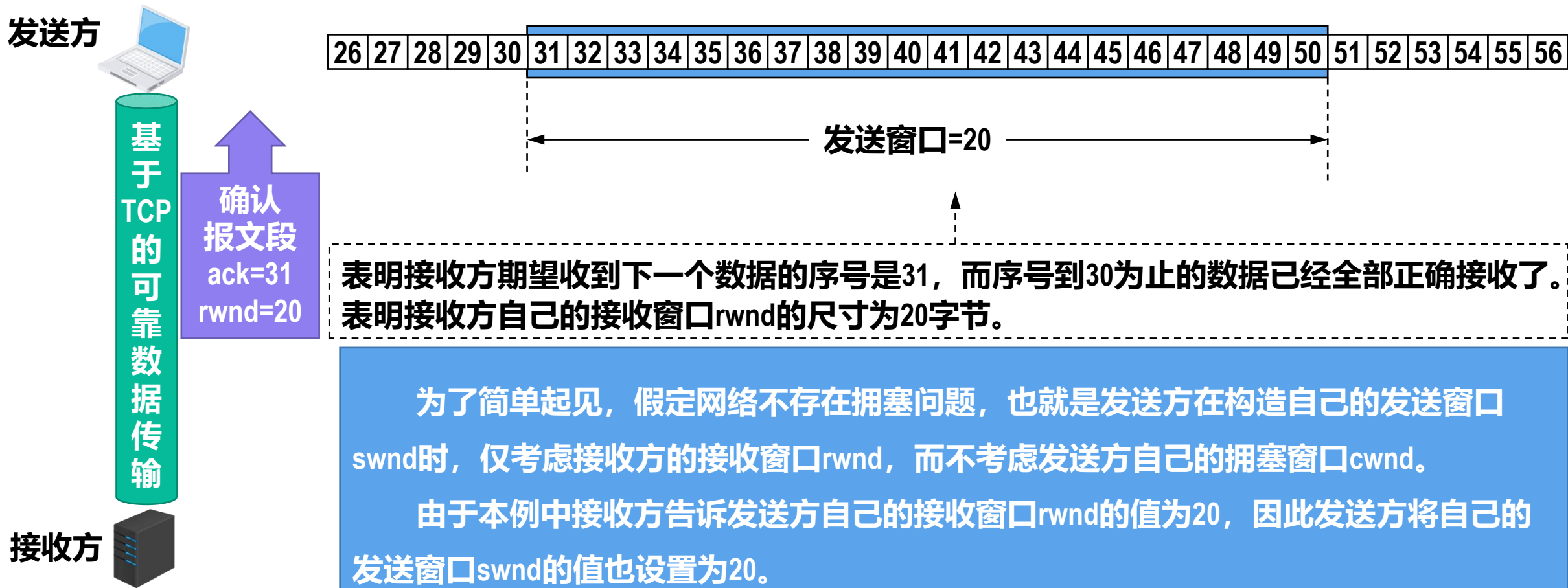
TCP的滑动窗口以字节为单位



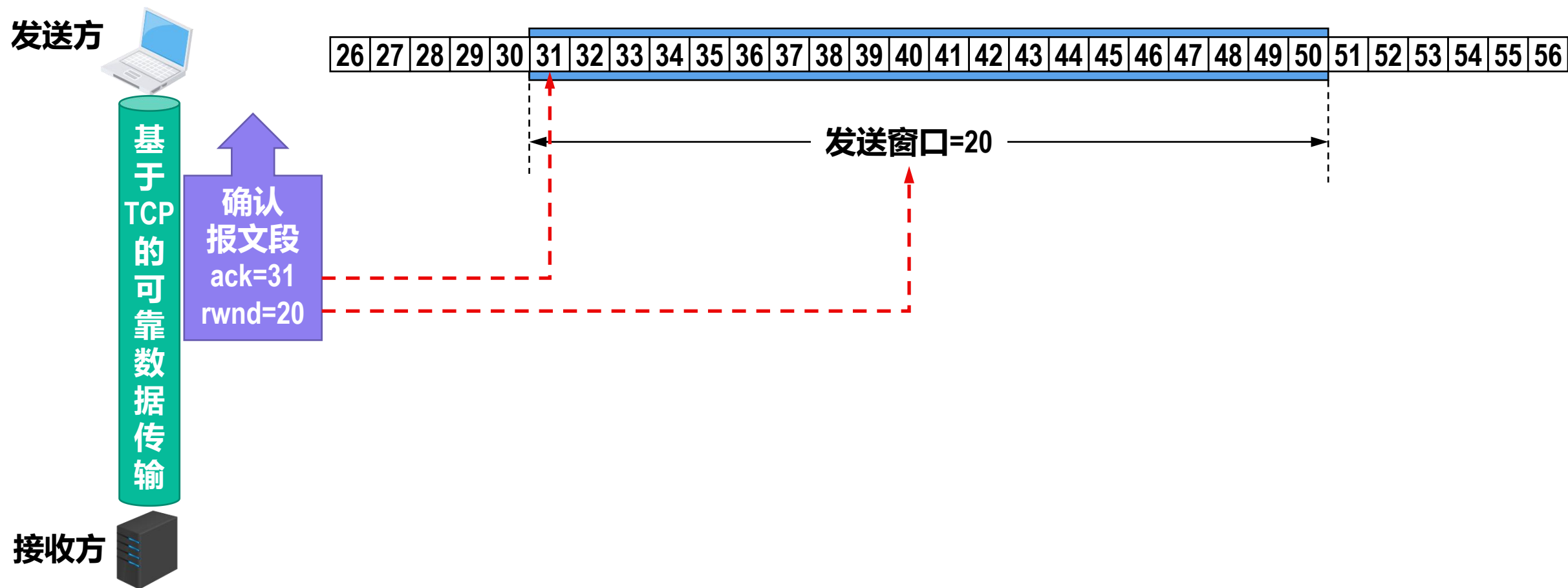
01 TCP可靠传输的实现



01 TCP可靠传输的实现



01 TCP可靠传输的实现



01 TCP可靠传输的实现

发送方

基于
TCP
的
可靠
数据
传输

接收方



发送窗口=20

- 发送方在没有收到接收方确认的情况下，可以把序号落入发送窗口内的数据依次全部发送出去。
- 凡是已经发送过的数据，在未收到确认之前都必须暂时保留，以便在超时重传时使用。

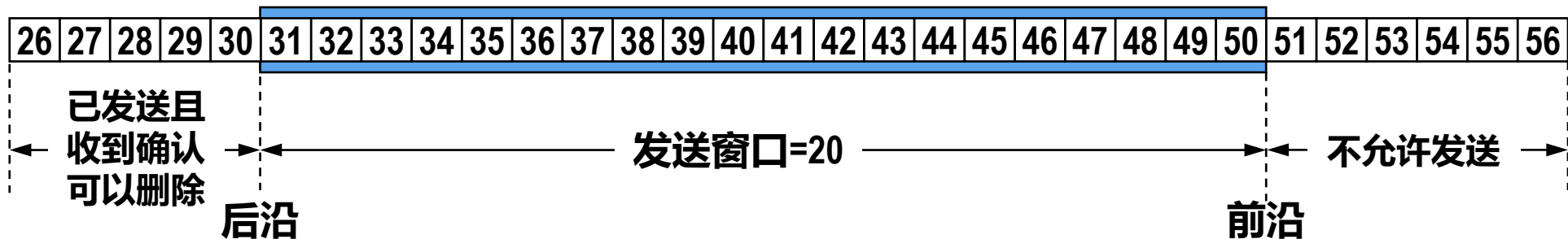
01 TCP可靠传输的实现

发送方



基于TCP的可靠数据传输

接收方



发送窗口后沿的移动情况:

- 不动 (没有收到新的确认)
- 前移→ (收到新的确认)

发送窗口前沿的移动情况:

- 前移→ (通常情况)
- 不动
 - 情况1: 没有收到新的确认, 接收方通知的窗口大小也没有改变。
 - 情况2: 收到了新的确认, 但接收方通知的窗口缩小了, 使发送窗口的前沿正好不动。
- 向后收缩 (接收方通知的窗口变小了)



01 TCP可靠传输的实现

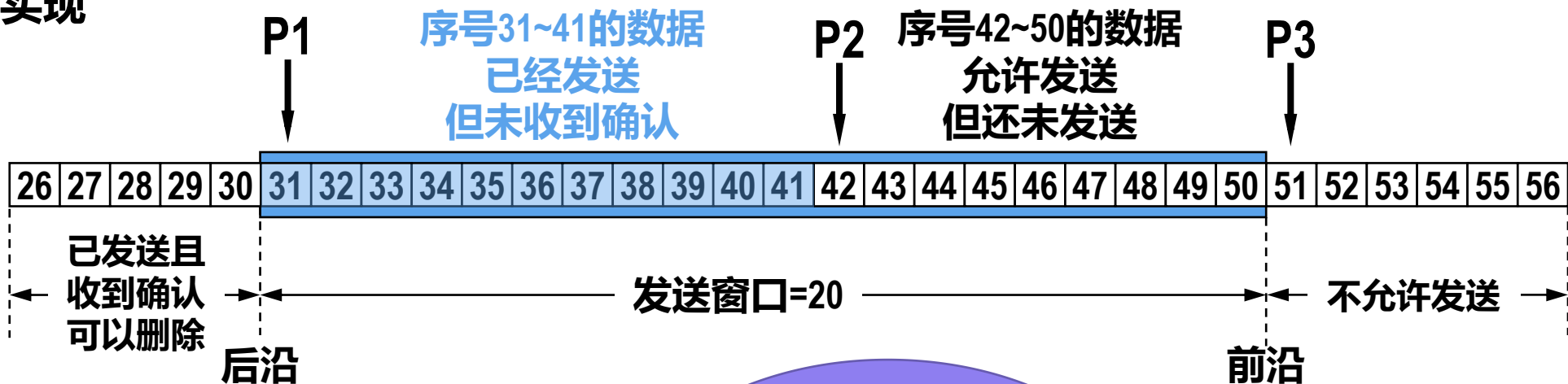
发送方



一些
数据报文段
共携带
31~41号字节

基于
TCP的
可靠
数据
传输

接收方



使用三个指针P1、P2、P3分别指向相应的字节序号：

- P1指向发送窗口内已发送但还未收到确认的第一个数据的序号。
- P2指向发送窗口内还未发送的第一个数据的序号。
- P3指向发送窗口前沿外的第一个数据的序号。

应该如何描述发送窗口的状态呢？换句话说，如果要编程实现滑动窗口机制，那么对于发送窗口的状态应该如何标记和维护呢？



01 TCP可靠传输的实现

发送方



一些

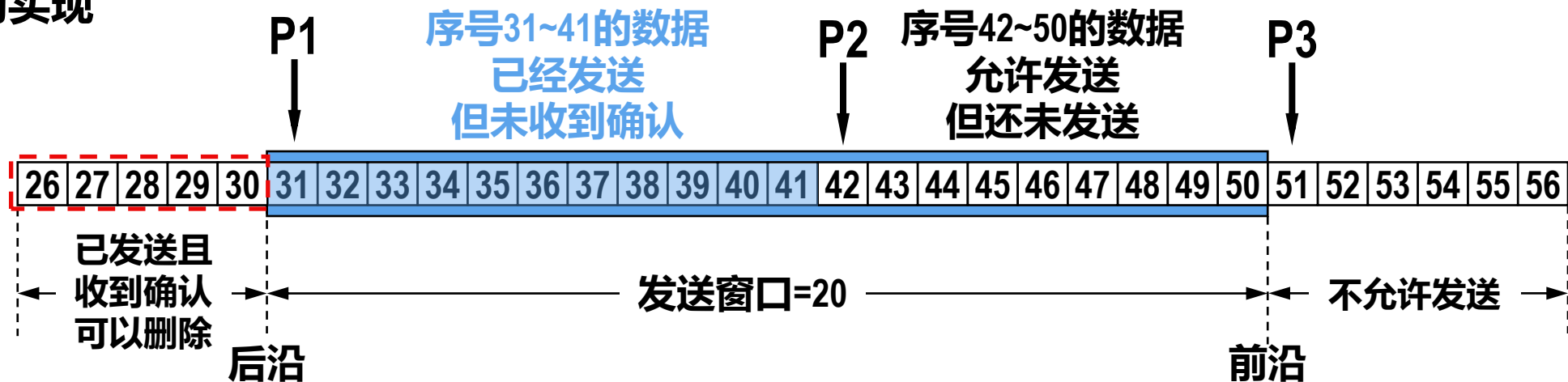
数据报文段

共携带

31~41号字节

基于TCP的可靠数据传输

接收方



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息:

- 小于P1的就是已发送并已收到确认的部分

01 TCP可靠传输的实现

发送方



一些

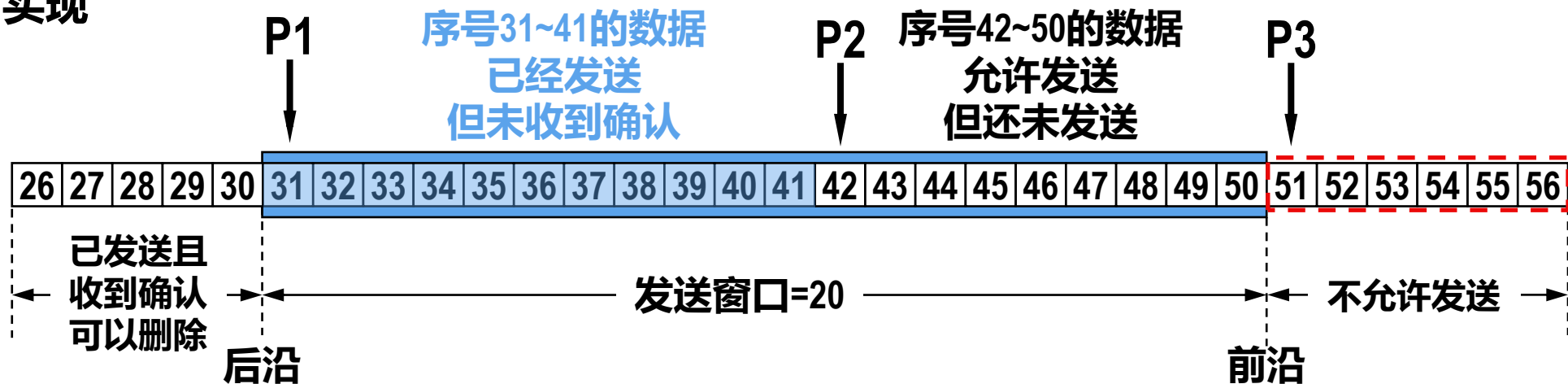
数据报文段

共携带

31~41号字节

基于TCP的可靠数据传输

接收方



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息:

- 小于P1的就是已发送并已收到确认的部分
- 大于等于P3的就是不允许发送的部分

01 TCP可靠传输的实现

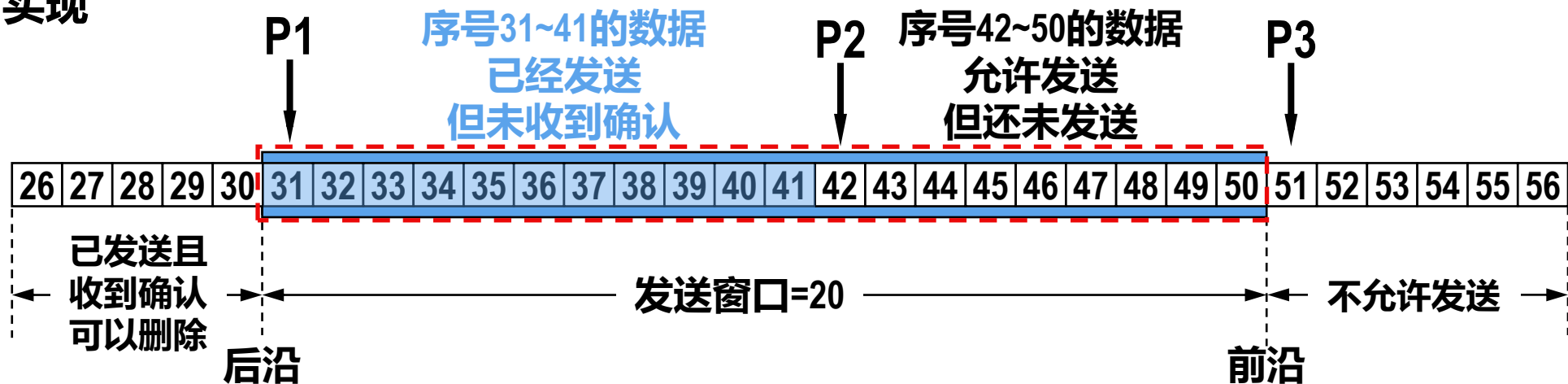
发送方



一些

数据报文段
共携带
31~41号字节基于
TCP的
可靠
数据
传输

接收方



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息:

- 小于P1的就是已发送并已收到确认的部分
- 大于等于P3的就是不允许发送的部分
- $P3 - P1 = \text{发送窗口尺寸}$

01 TCP可靠传输的实现

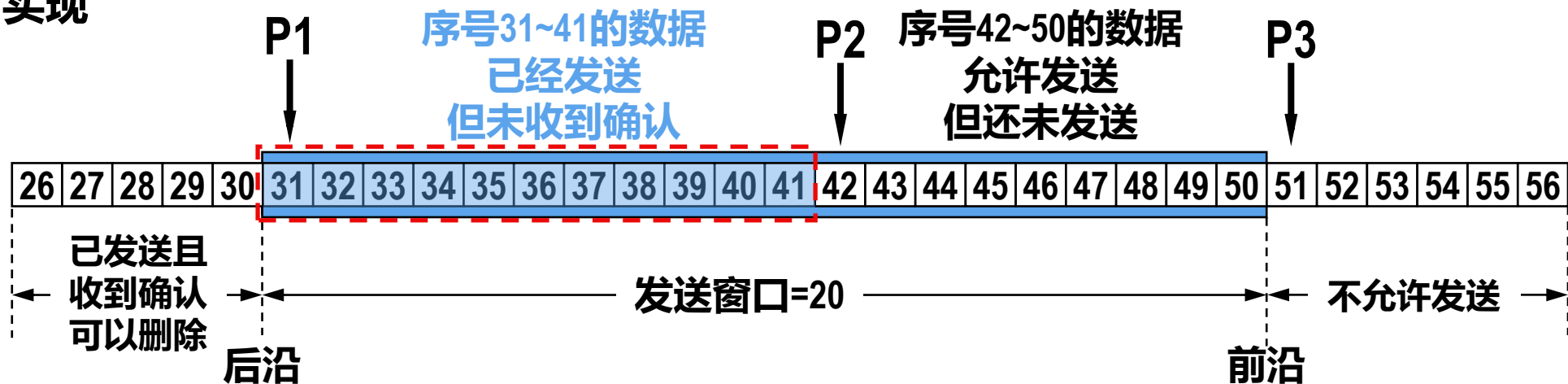
发送方



一些

数据报文段
共携带
31~41号字节基于
TCP的
可靠
数据
传输

接收方



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息:

- 小于P1的就是已发送并已收到确认的部分
- 大于等于P3的就是不允许发送的部分
- $P3 - P1 = \text{发送窗口尺寸}$
- $P2 - P1 = \text{已发送但还未收到确认的字节数量}$

01 TCP可靠传输的实现

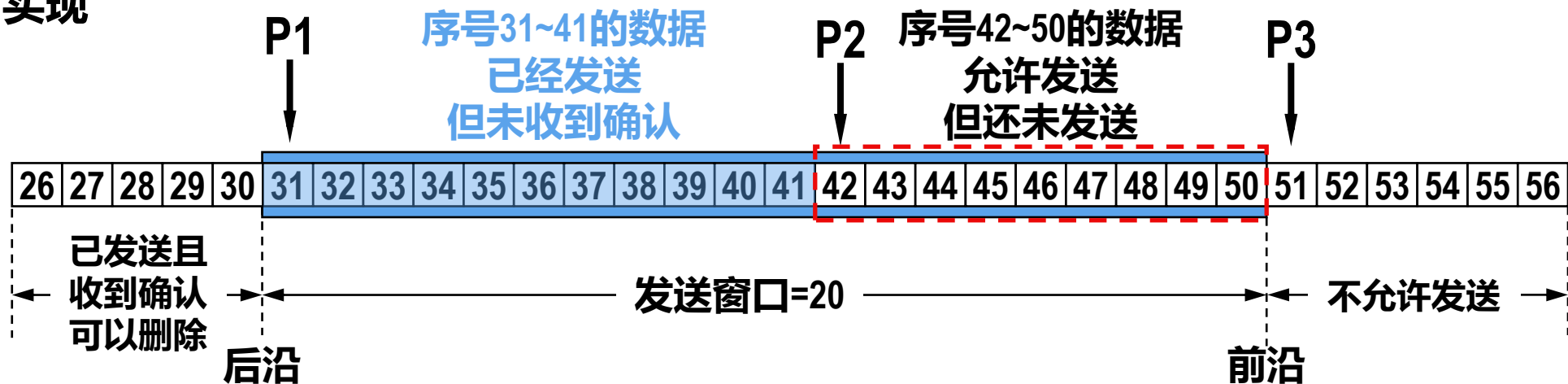
发送方



一些

数据报文段
共携带
31~41号字节基于
TCP的
可靠
数据
传输

接收方



这样就可以用P1、P2和P3这三个指针来描述发送窗口的相关信息:

- 小于P1的就是已发送并已收到确认的部分
- 大于等于P3的就是不允许发送的部分
- $P3 - P1$ = 发送窗口尺寸
- $P2 - P1$ = 已发送但还未收到确认的字节数量
- $P3 - P2$ = 允许发送但当前还未发送的字节数量 (又称为可用窗口或有效窗口)

01 TCP可靠传输的实现

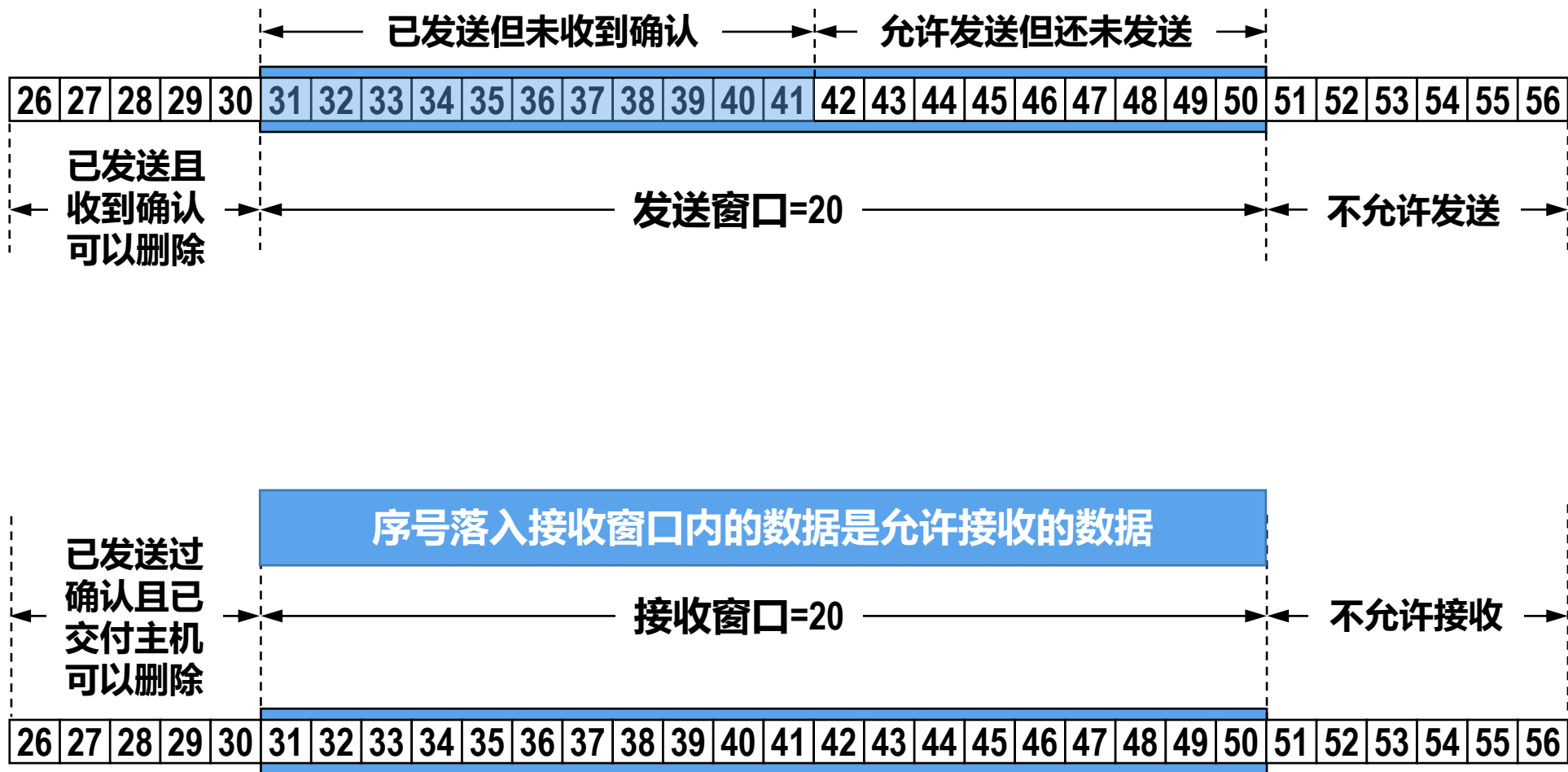
发送方



一些

数据报文段
共携带
31~41号字节基于
TCP的
可靠
数据
传输

接收方



01 TCP可靠传输的实现

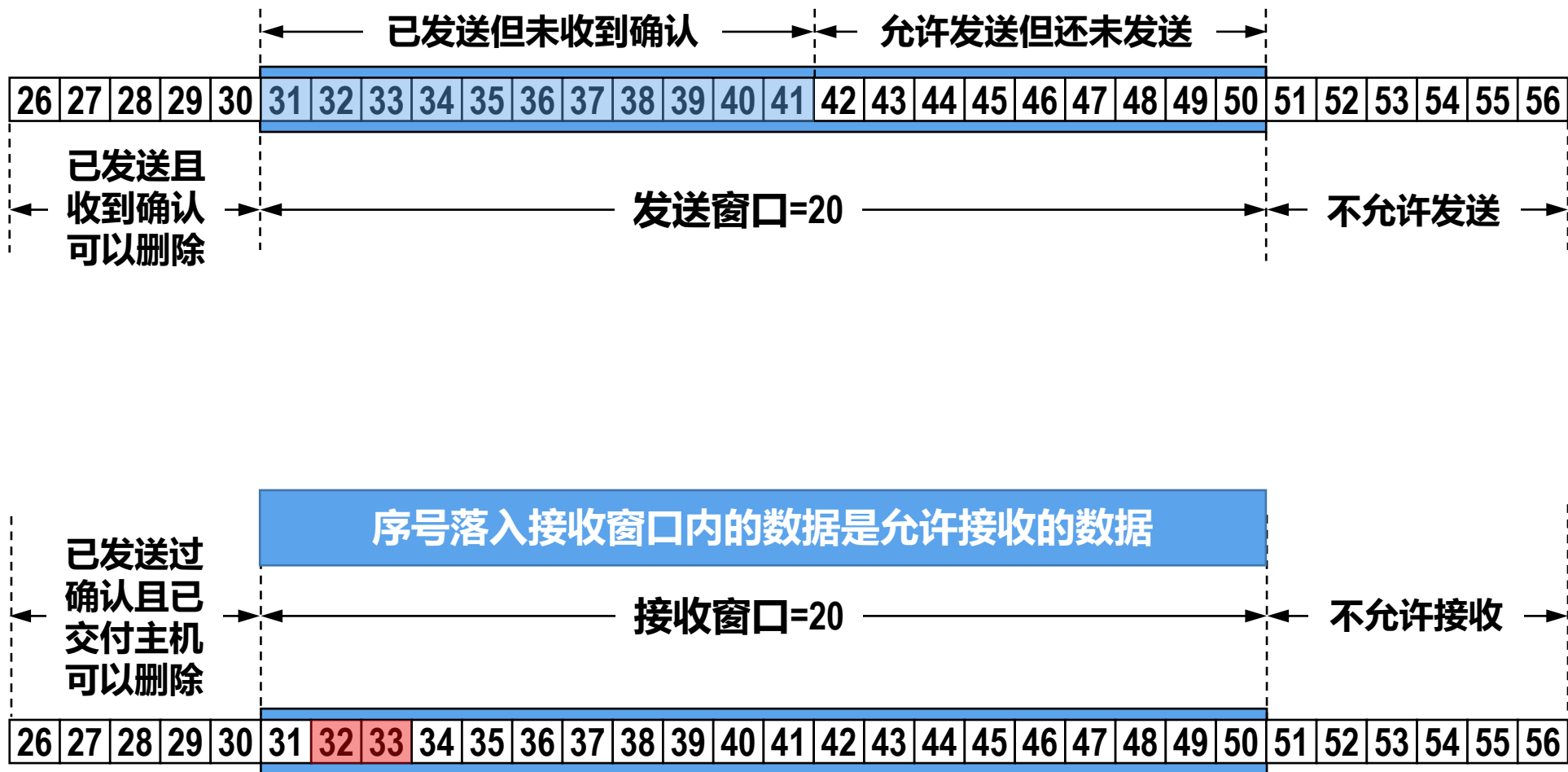
发送方



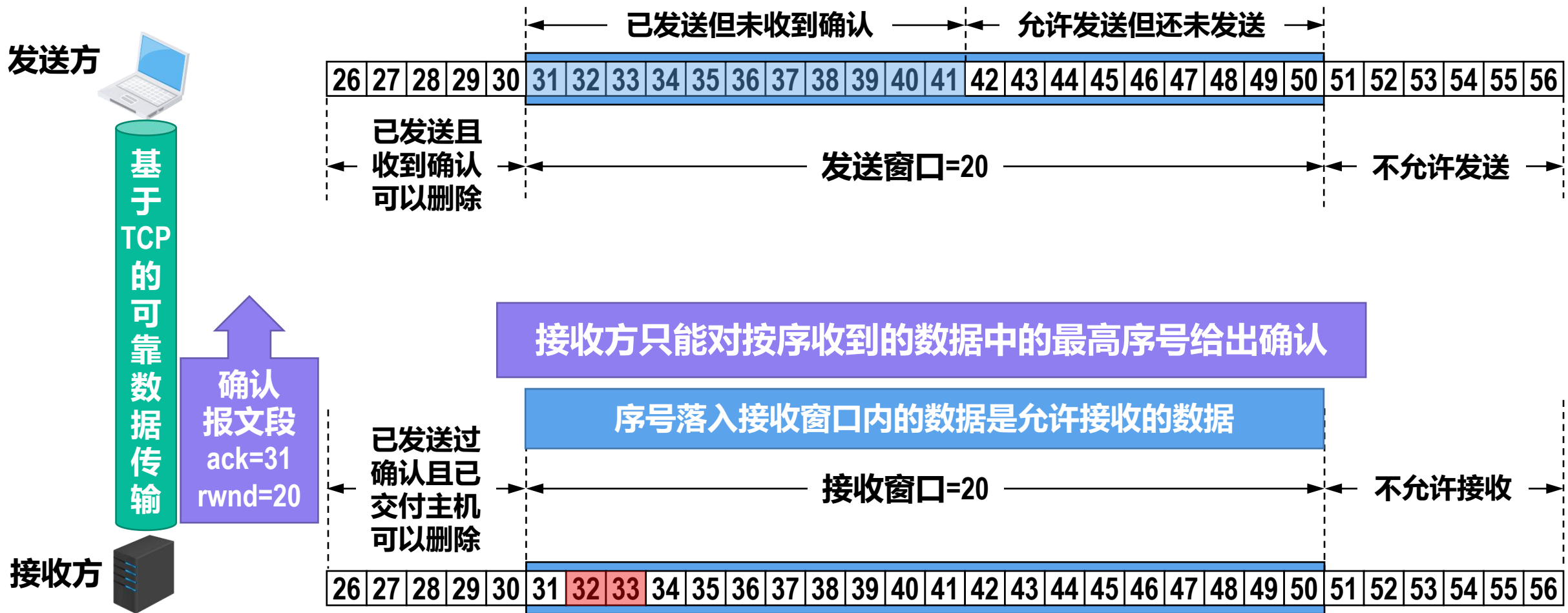
基于TCP的可靠数据传输

数据报文段
携带
32~33号字节

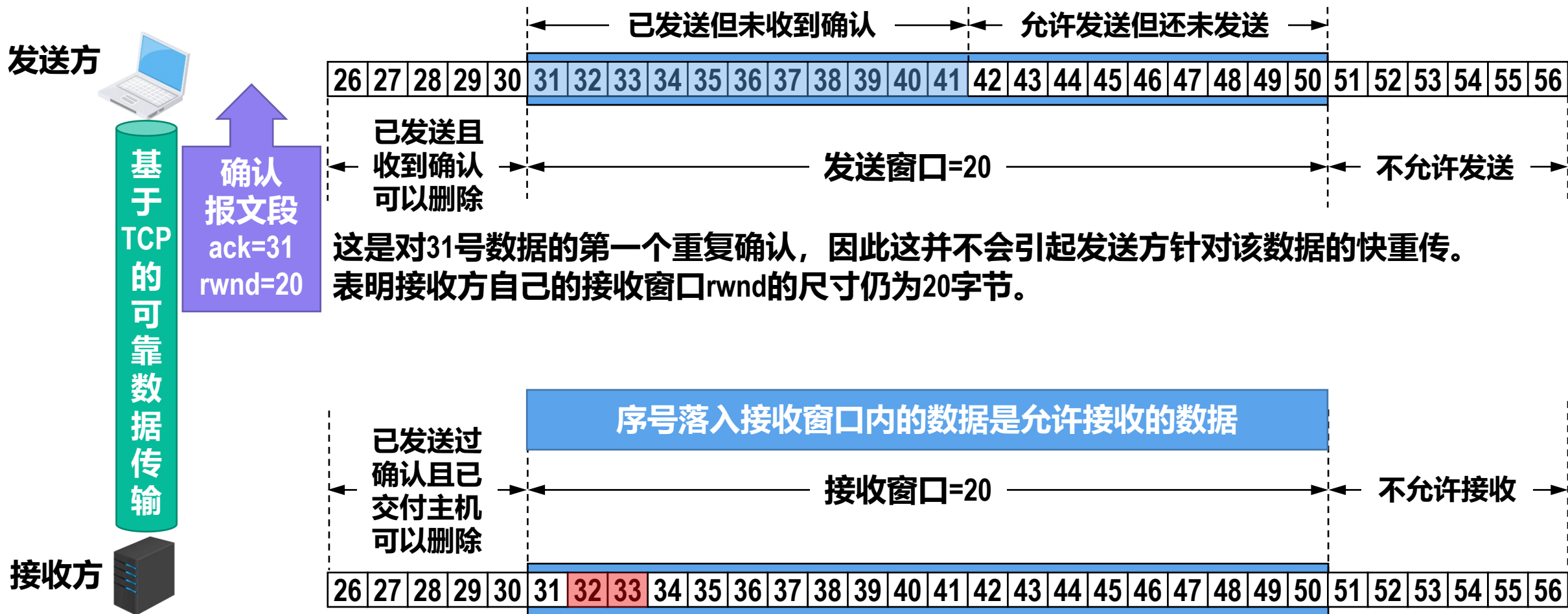
接收方



01 TCP可靠传输的实现



01 TCP可靠传输的实现



01 TCP可靠传输的实现

发送方



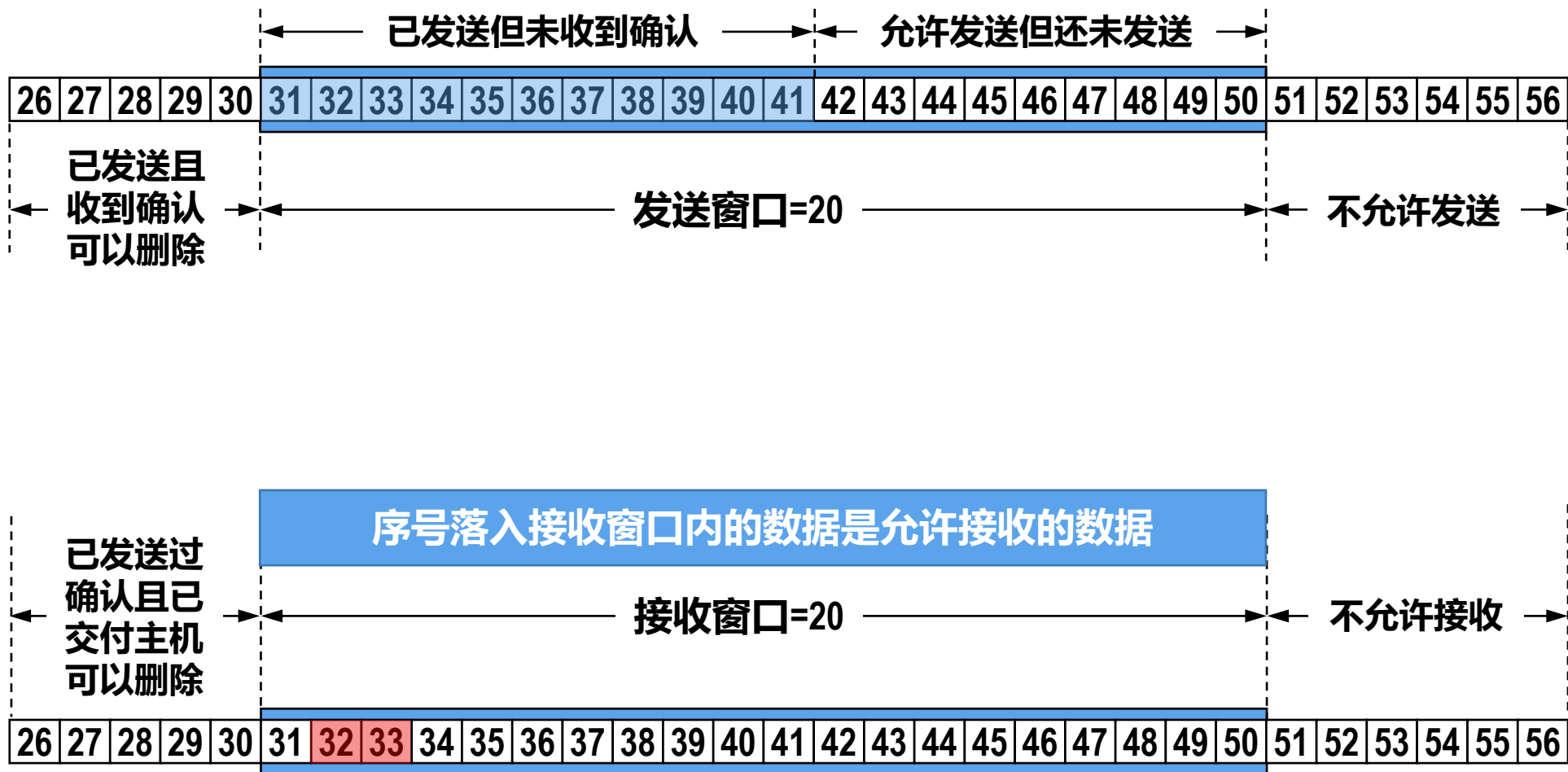
基于TCP的可靠数据传输

数据报文段

携带

31号字节

接收方



01 TCP可靠传输的实现

发送方



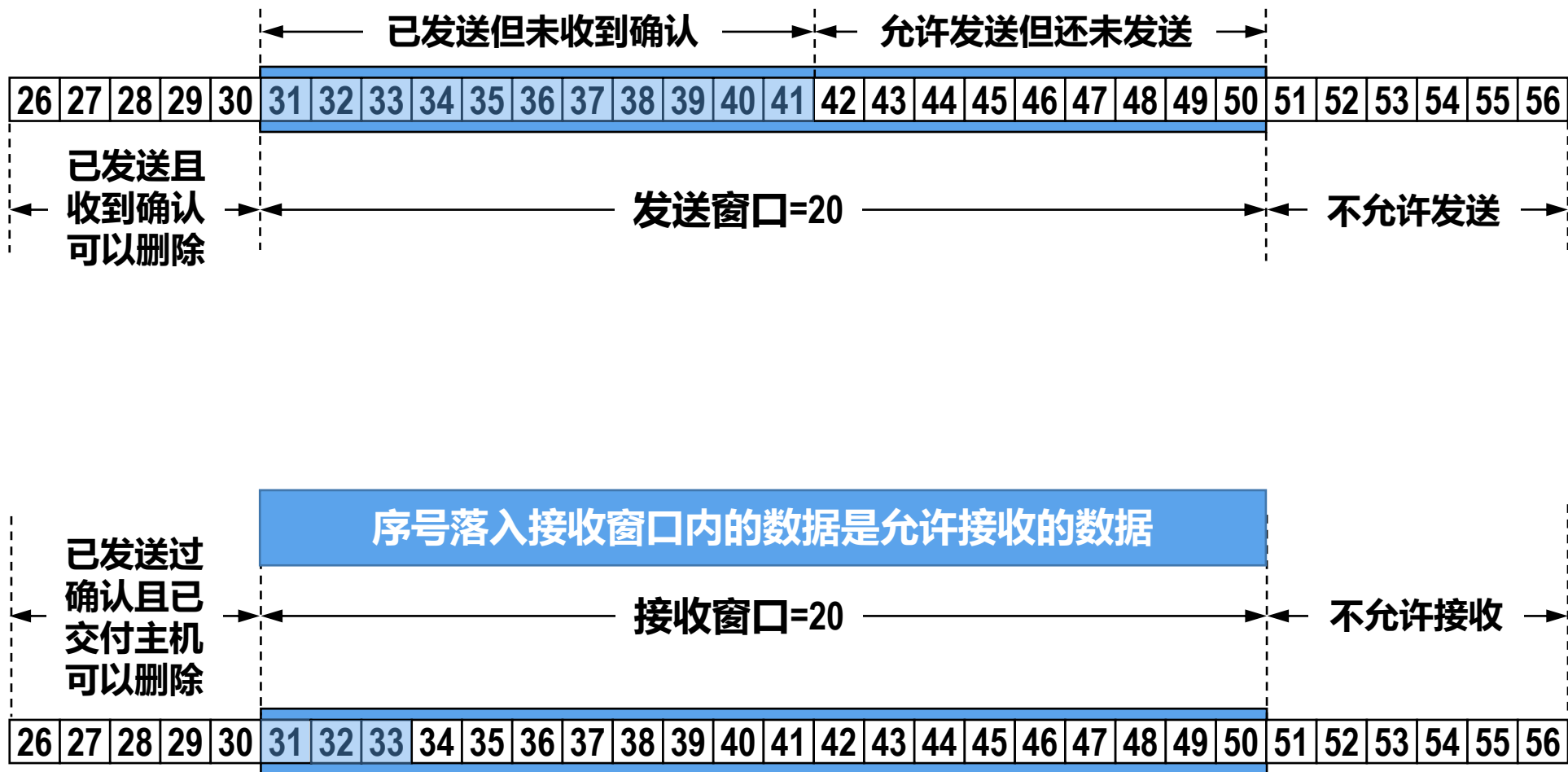
基于TCP的可靠数据传输

数据报文段

携带

31号字节

接收方



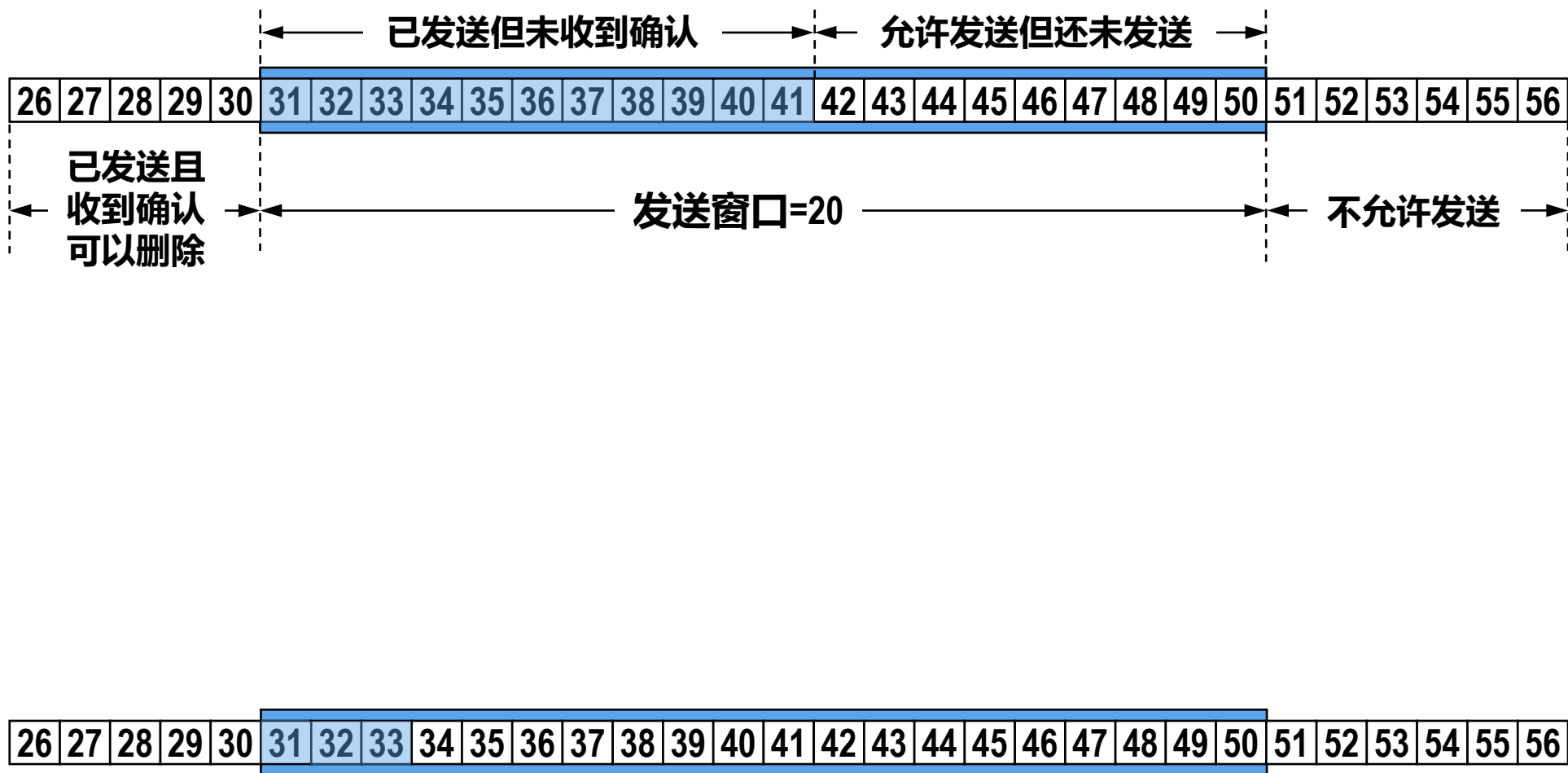
01 TCP可靠传输的实现

发送方



基于TCP的可靠数据传输

接收方



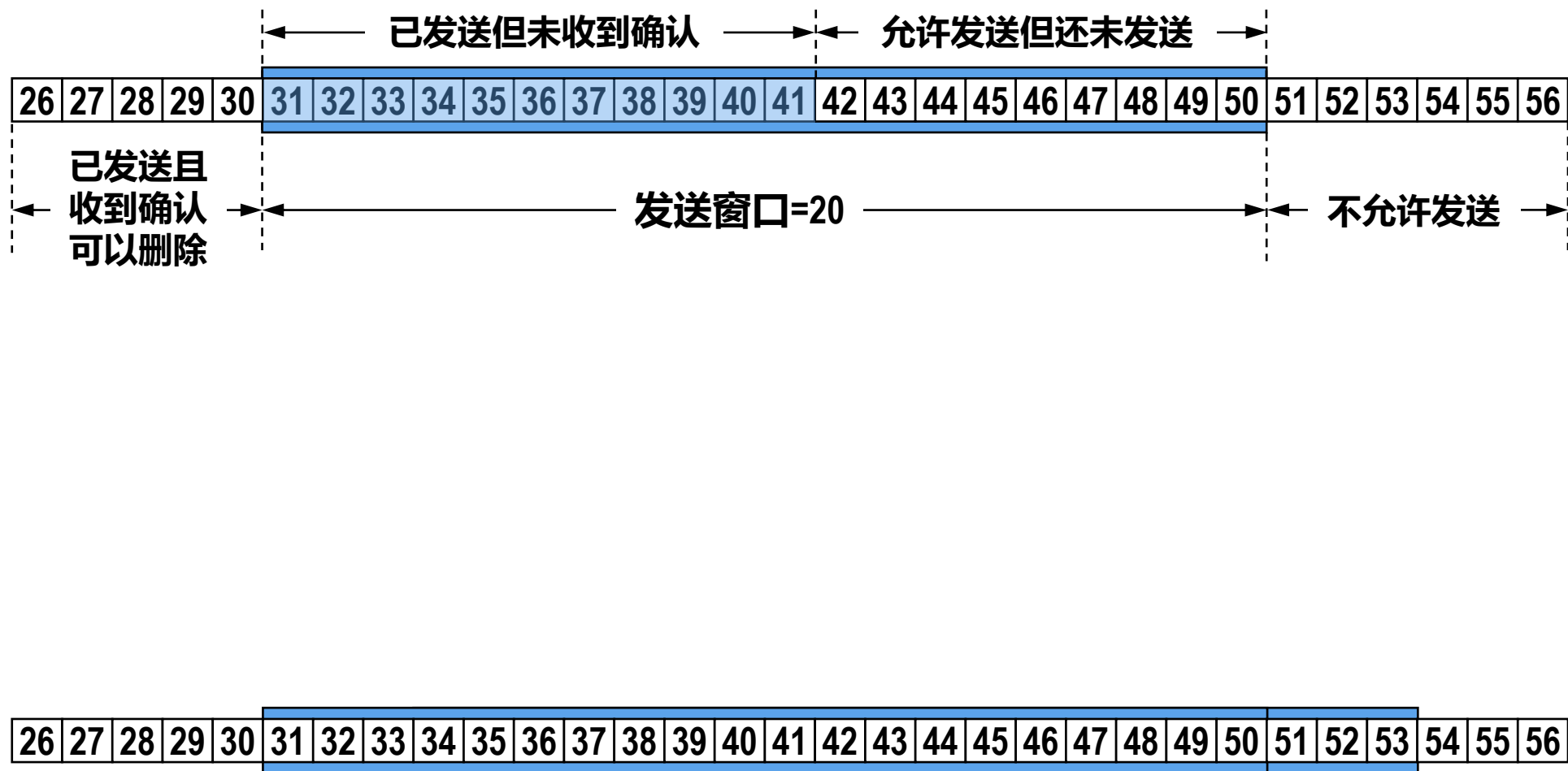
01 TCP可靠传输的实现

发送方

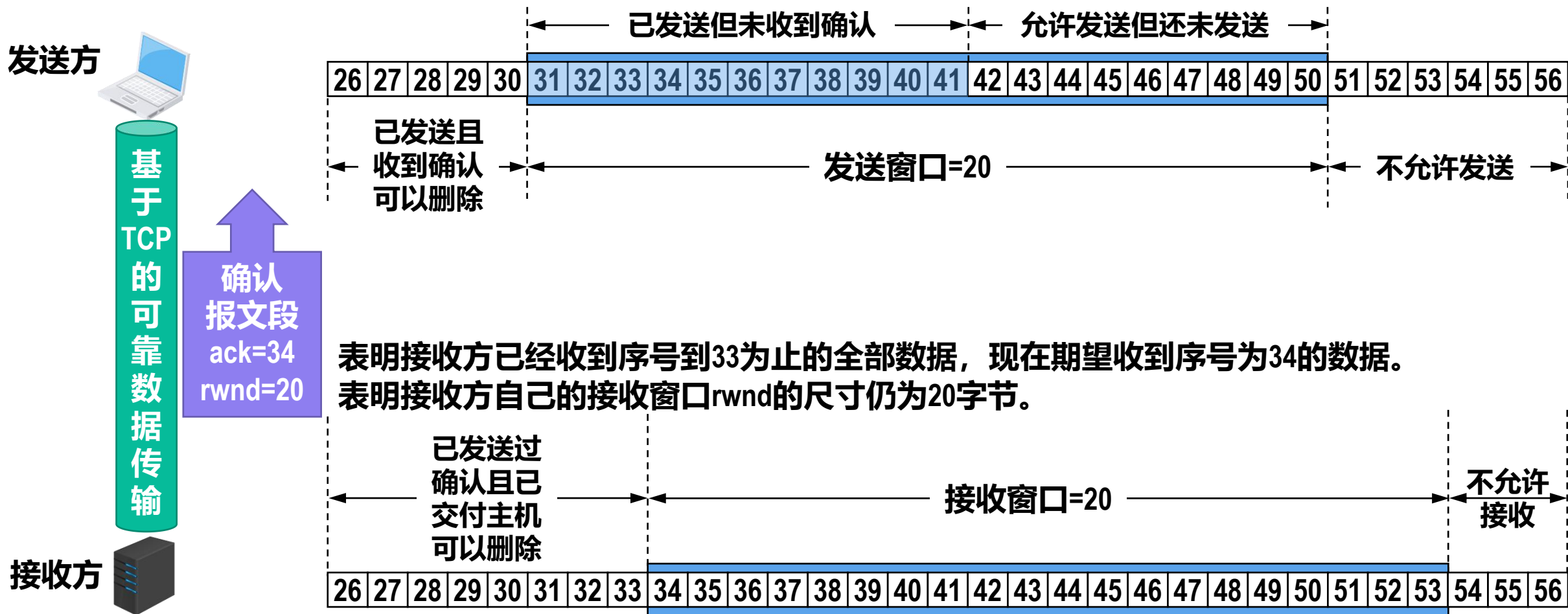


基于TCP的可靠数据传输

接收方



01 TCP可靠传输的实现



01 TCP可靠传输的实现

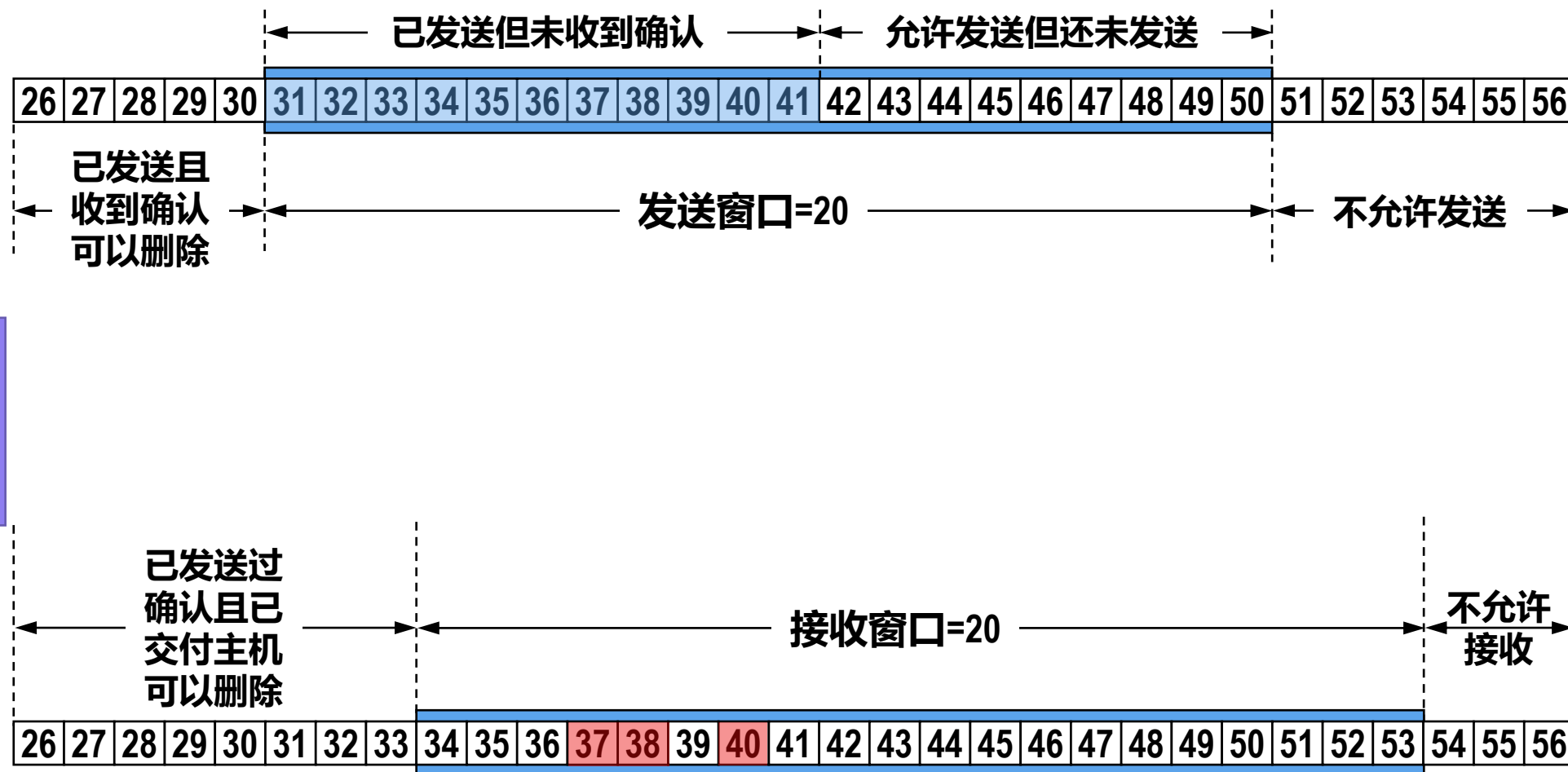
发送方



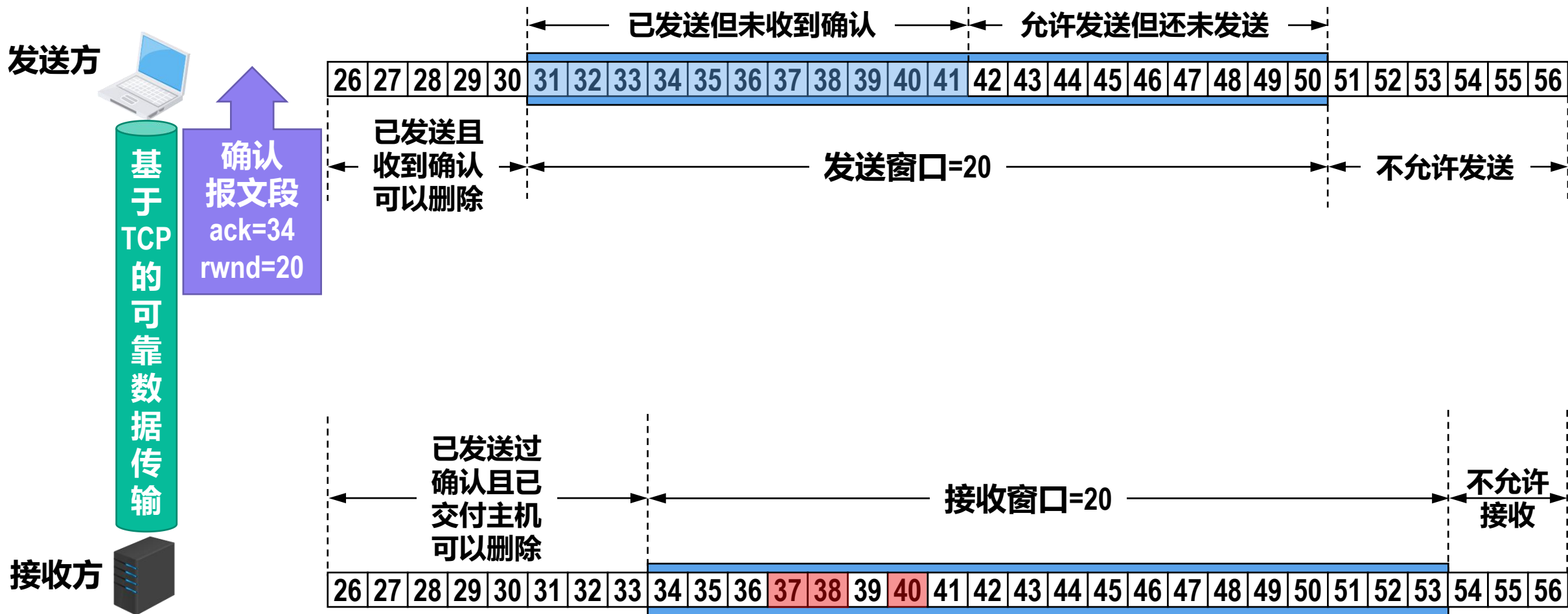
基于TCP的可靠数据传输

确认报文段
ack=34
rwnd=20一些数据报文段
共携带
37、38、40
号字节

接收方



01 TCP可靠传输的实现



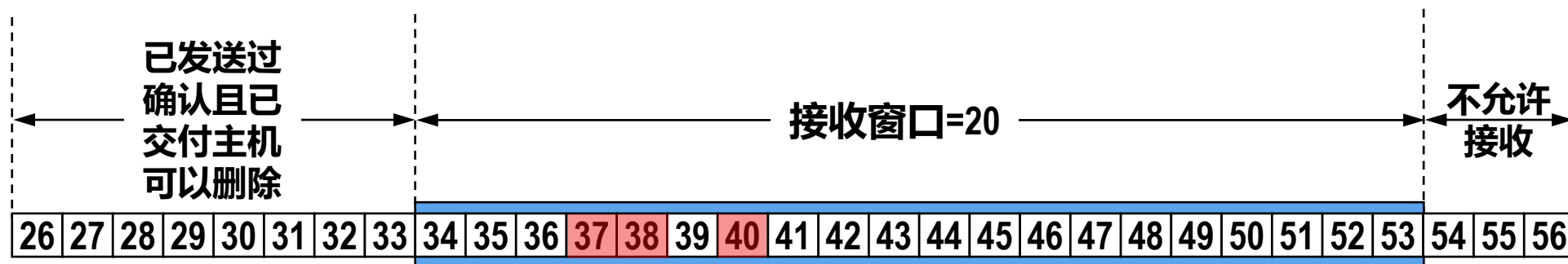
01 TCP可靠传输的实现

发送方



基于TCP的可靠数据传输

接收方



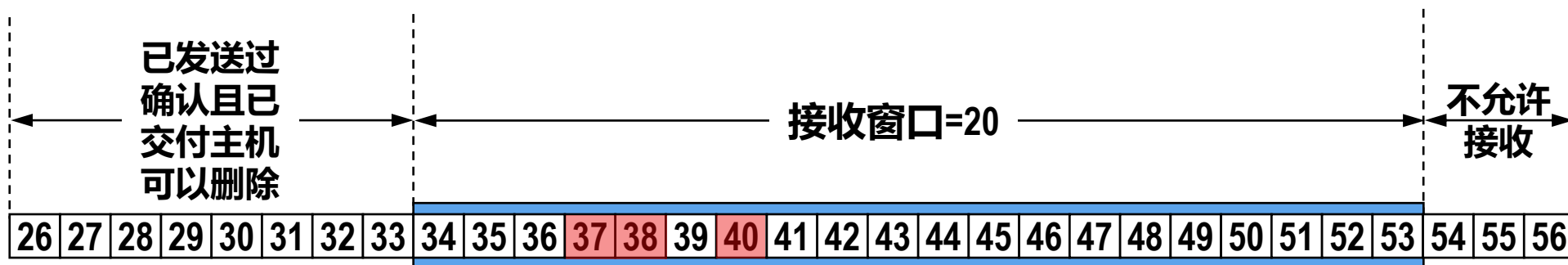
01 TCP可靠传输的实现

发送方



基于TCP的可靠数据传输

接收方



01 TCP可靠传输的实现

发送方



一些

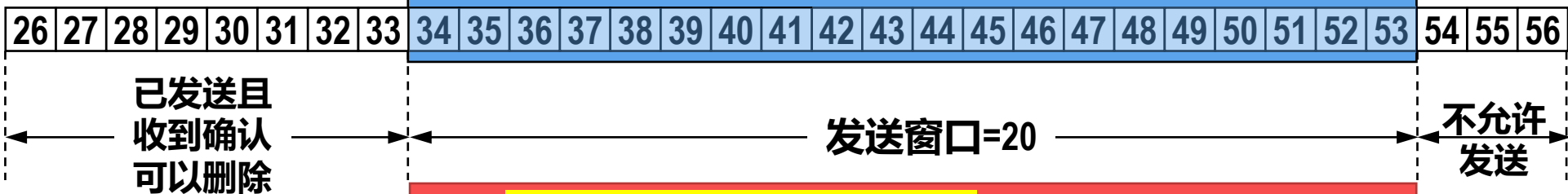
数据报文段
共携带
42~53号字节

基于TCP的可靠数据传输

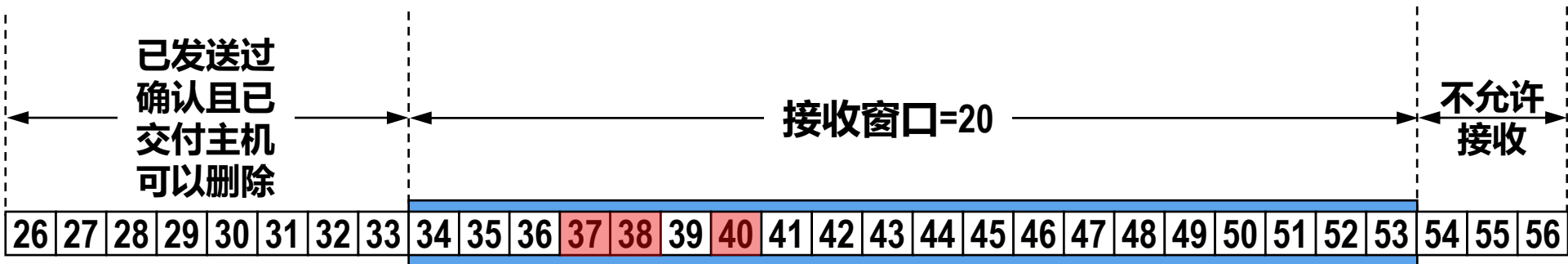
接收方



序号落在发送窗口内的已发送数据，如果迟迟收不到接收方的确认，则会产生**超时重传**。



发送窗口内的序号已经用完了，发送方在未收到接收方发来确认的情况下，**不能再发送新的数据**。



01 TCP可靠传输的实现

【2009年 题38】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了两个连续的TCP段，分别包含300字节和500字节的有效载荷，第一个段的序列号为200，主机乙正确接收到两个段后，发送给主机甲的确认序列号是（**D**）。

A. 500

B. 700

C. 800

D. 1000

【2013年 题39】主机甲与主机乙之间已建立一个TCP连接，双方持续有数据传输，且数据无差错与丢失。若甲收到1个来自乙的TCP段，该段的序号是1913、确认序号为2046、有效载荷为100字节，则甲立即发送给乙的TCP段的序号和确认序号分别是（**B**）。

A. 2046、2012

B. 2046、2013

C. 2047、2012

D. 2047、2013

【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1和第3个段，则主机乙发送给主机甲的确认序号是（**B**）。

A. 300

B. 500

C. 1200

D. 1400

已在之前的“5.3.1 TCP报文段的首部格式”这节微课视频中进行了解析

01 TCP可靠传输的实现

对于TCP可靠传输的实现，还需要做以下补充说明

- (1) 虽然发送方的发送窗口是根据接收方的接收窗口设置的，但在同一时刻，**发送方的发送窗口并不总是和接收方的接收窗口一样大**，这是因为：
 - 网络传送窗口值需要经历一定的时间滞后，并且这个时间还是不确定的。
 - 发送方还可能根据网络当时的拥塞情况适当减小自己的发送窗口尺寸。
- (2) **对于不按序到达的数据应如何处理，TCP并无明确规定。**
 - 如果接收方把不按序到达的数据一律丢弃，那么接收窗口的管理将会比较简单，但这样做对网络资源的利用不利，因为发送方会重复传送较多的数据。
 - TCP通常对不按序到达的数据先临时存放在接收窗口中，等到字节流中所缺少的字节收到后，再按序交付上层的应用进程。
- (3) TCP要求接收方**必须有累积确认**（这一点与选择重传协议不同）**和捎带确认机制**。这样可以减小传输开销。接收方可以在合适的时候发送确认，也可以在自己有数据要发送时把确认信息顺便捎带上。
 - **接收方不应过分推迟发送确认**，否则会导致发送方不必要的超时重传，这反而浪费了网络资源。TCP标准规定，确认推迟的时间不应超过0.5秒。若收到一连串具有最大长度的报文段，则必须每隔一个报文段就发送一个确认[RFC 1122]。
 - **捎带确认实际上并不经常发生**，因为大多数应用程序很少同时在两个方向上发送数据。
- (4) **TCP的通信是全双工通信**。通信中的每一方都在发送和接收报文段。因此，每一方都有自己的发送窗口和接收窗口。在谈到这些窗口时，一定要弄清楚是哪一方的窗口。

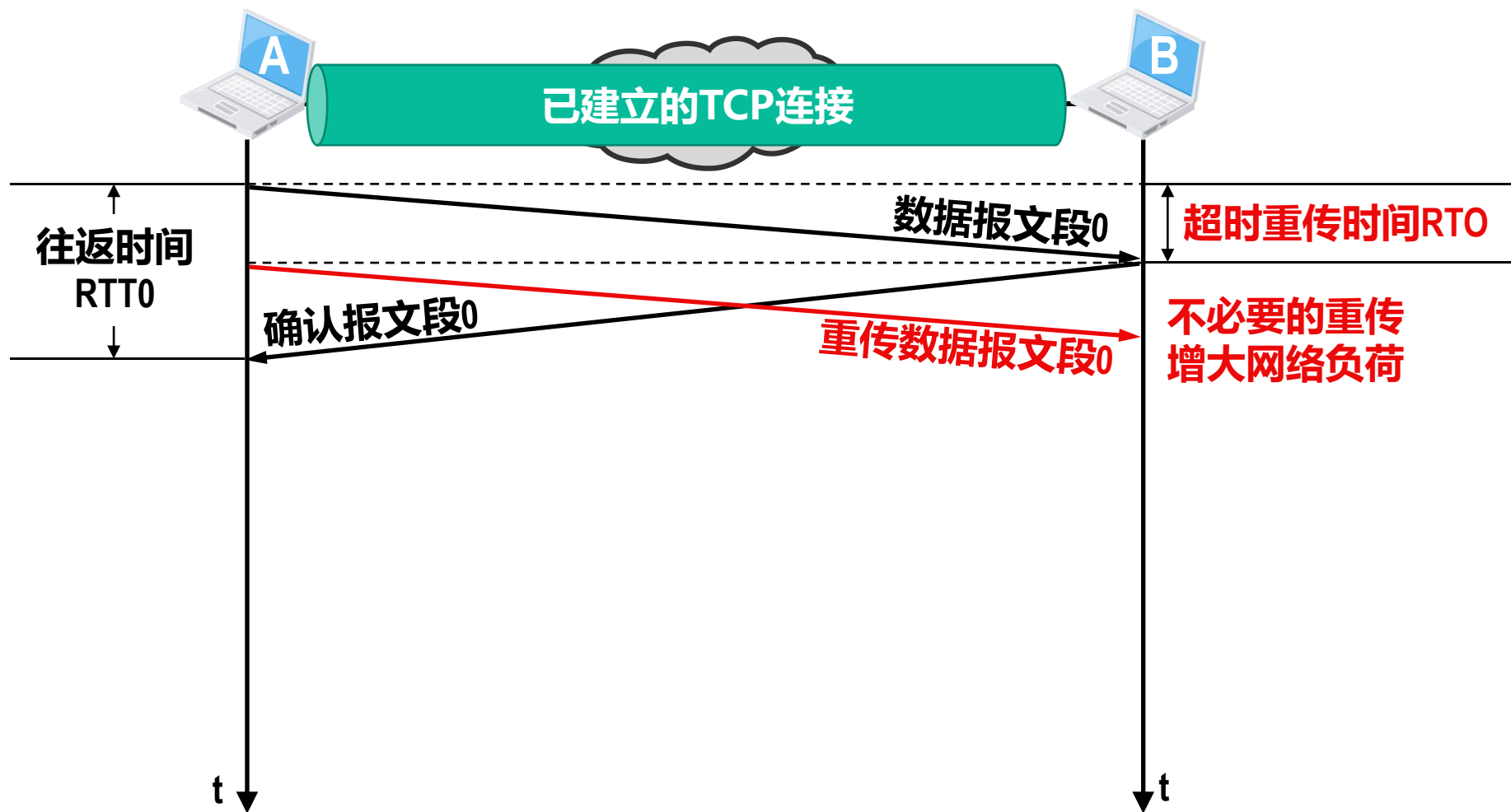
5.3.7 TCP超时重传时间的选择

01

TCP超时重传时间的选择

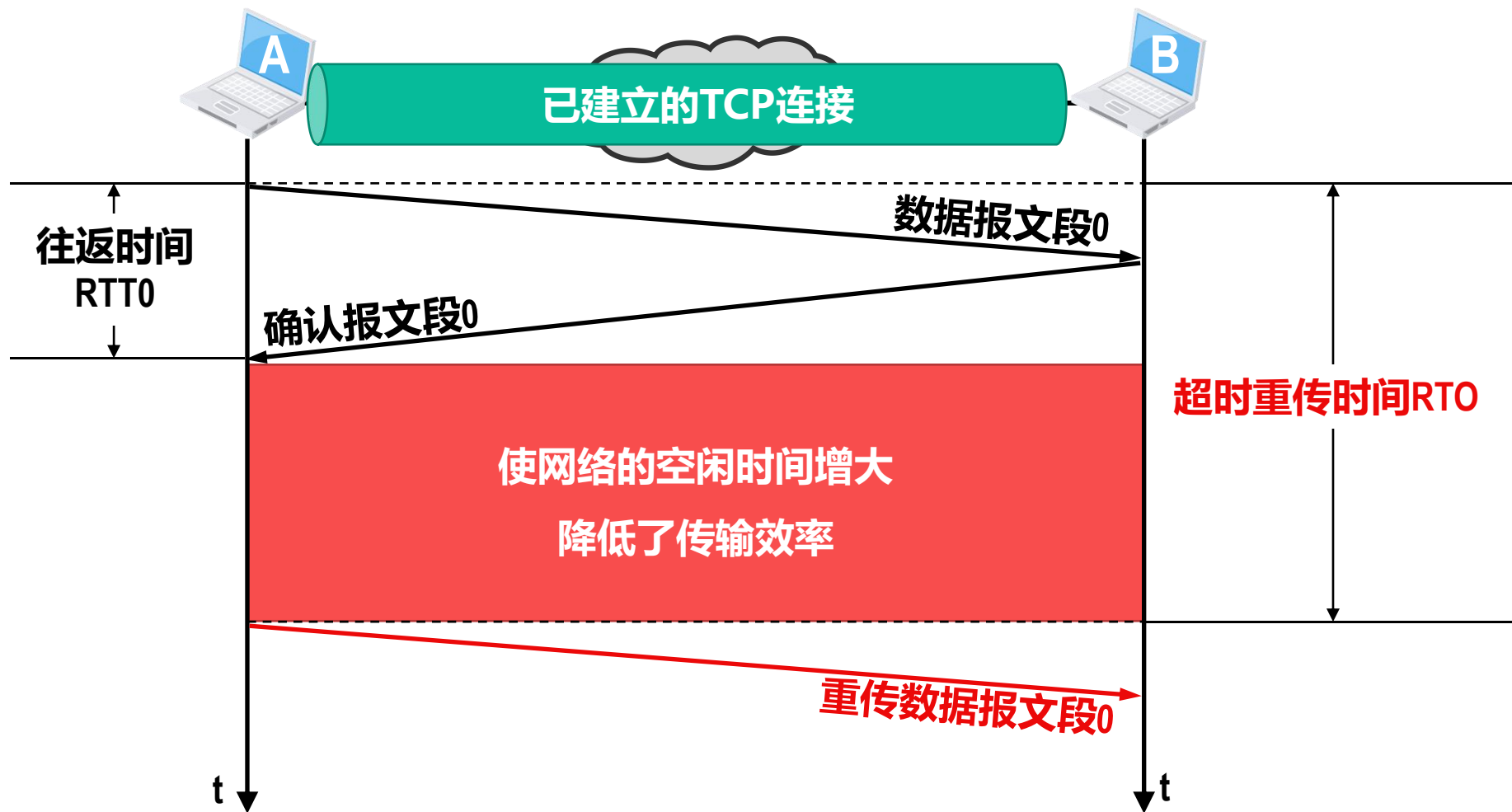
01 TCP超时重传时间的选择

TCP超时重传时间RTO的选择时TCP最复杂的问题之一



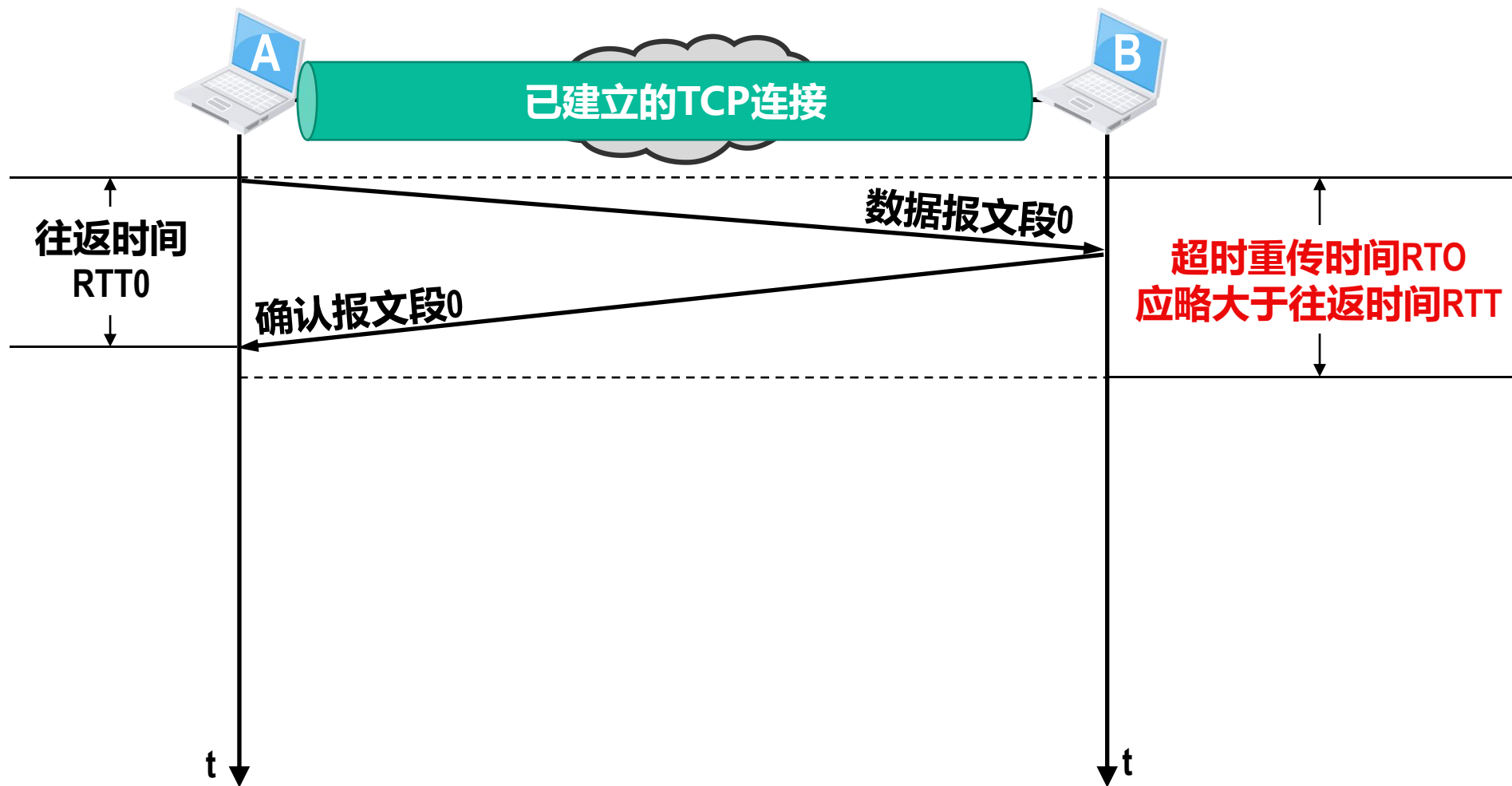
01 TCP超时重传时间的选择

TCP超时重传时间RTO的选择是TCP最复杂的问题之一



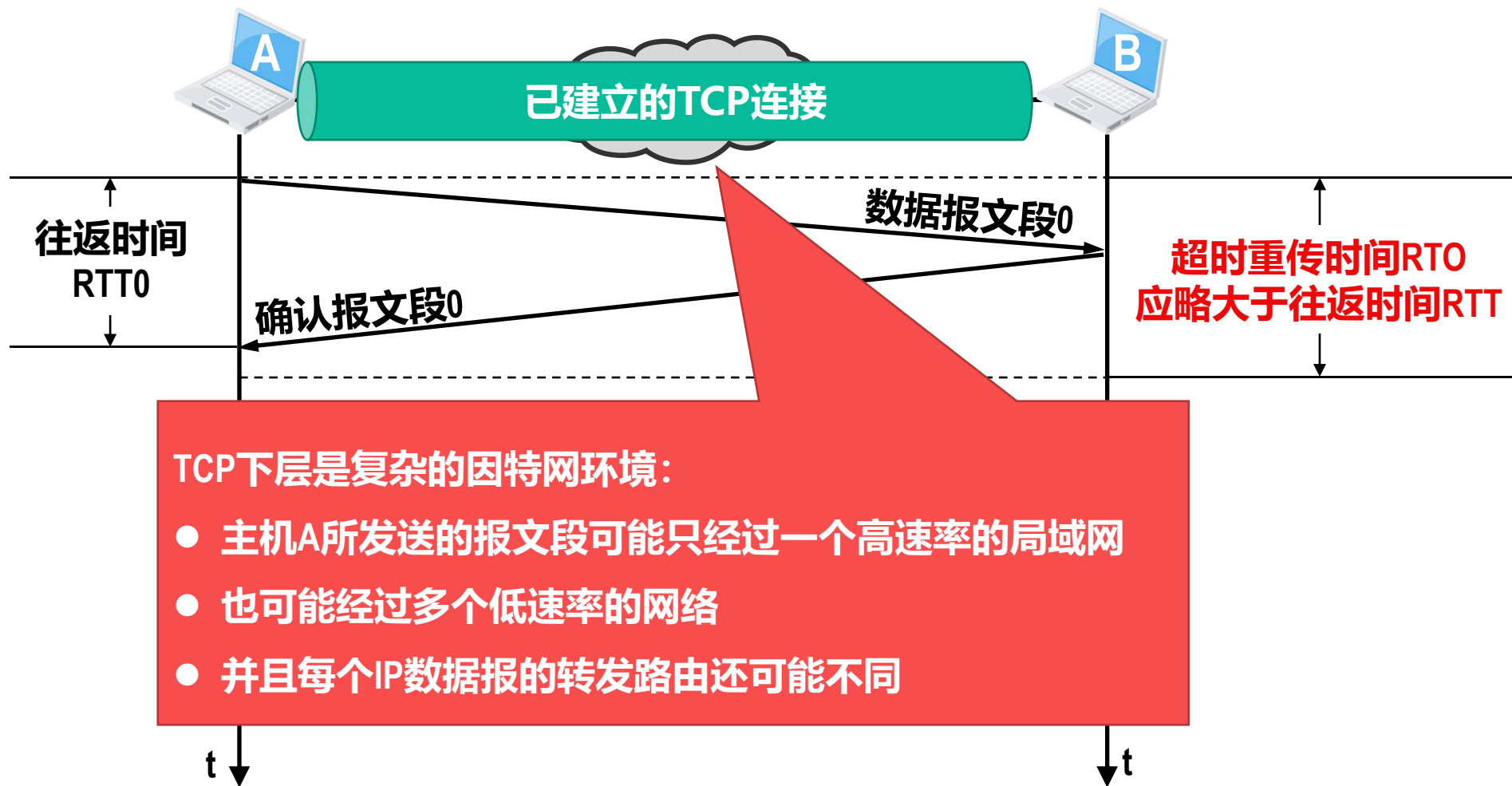
01 TCP超时重传时间的选择

TCP超时重传时间RTO的选择是TCP最复杂的问题之一



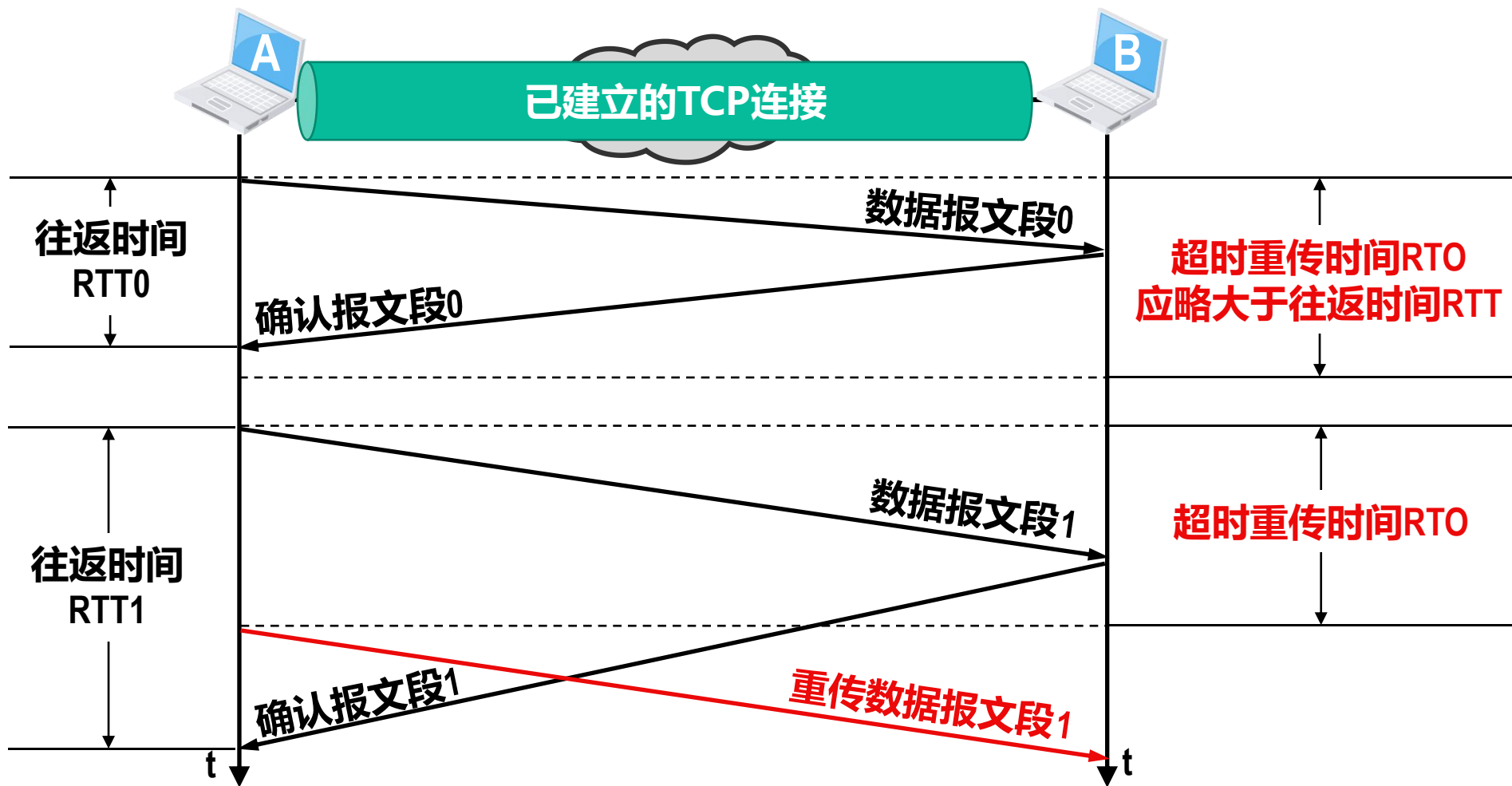
01 TCP超时重传时间的选择

TCP超时重传时间RTO的选择时TCP最复杂的问题之一



01 TCP超时重传时间的选择

TCP超时重传时间RTO的选择是TCP最复杂的问题之一



01 TCP超时重传时间的选择

不能直接使用略大于某次测量得到的往返时间RTT样本的值作为超时重传时间RTO。

但是，可以利用每次测量得到的RTT样本计算加权平均往返时间RTTs，这样可以得到比较平滑的往返时间。

$$RTT_{s1} = RTT_1$$

$$\text{新的}RTT_s = (1 - \alpha) \times \text{旧的}RTT_s + \alpha \times \text{新的}RTT\text{样本}$$

在上式中， $0 \leq \alpha < 1$

若 α 很接近于0，则新RTT样本对RTTs的影响不大；

若 α 很接近于1，则新RTT样本对RTTs的影响较大；

已成为建议标准的[RFC 6298]推荐的 α 值为1/8，即 0.125。

显然，超时重传时间RTO的值应略大于加权平均往返时间RTTs的值（而不是某个RTT样本的值）。

01 TCP超时重传时间的选择

显然，**超时重传时间RTO的值应略大于加权平均往返时间RTT_s的值**（而不是某个RTT样本的值）。

[RFC 6298]建议使用下式来计算超时重传时间RTO：

$$RTO = RTT_s + 4 \times RTT_D$$

加权平均往返时间RTT_s

$$RTT_{s1} = RTT_1$$

$$\text{新的}RTT_s = (1 - \alpha) \times \text{旧的}RTT_s + \alpha \times \text{新的}RTT\text{样本}$$

在上式中， $0 \leq \alpha < 1$ ，已成为建议标准的[RFC 6298]推荐的 α 值为1/8，即 0.125。

RTT偏差的加权平均RTT_D

$$RTT_{D1} = RTT_1 \div 2$$

$$\text{新的}RTT_D = (1 - \beta) \times \text{旧的}RTT_D + \beta \times |RTT_s - \text{新的}RTT\text{样本}|$$

在上式中， $0 \leq \beta < 1$ ，已成为建议标准的[RFC 6298]推荐的 β 值为1/4，即 0.25。

01 TCP超时重传时间的选择

显然，**超时重传时间RTO的值应略大于加权平均往返时间RTT_s的值**（而不是某个RTT样本的值）。

[RFC 6298]建议使用下式来计算超时重传时间RTO：

$$RTO = RTT_s + 4 \times RTT_D$$

加权平均往返时间RTT_s

$$RTT_{s1} = RTT_1$$

$$\text{新的}RTT_s = (1 - \alpha) \times \text{旧的}RTT_s + \alpha \times \text{新的}RTT\text{样本}$$

在上式中， $0 \leq \alpha < 1$ ，已成为建议标准的[RFC 6298]推荐的 α 值为1/8，即0.125。

RTT偏差的加权平均RTT_D

$$RTT_{D1} = RTT_1 \div 2$$

$$\text{新的}RTT_D = (1 - \beta) \times \text{旧的}RTT_D + \beta \times |RTT_s - \text{新的}RTT\text{样本}|$$

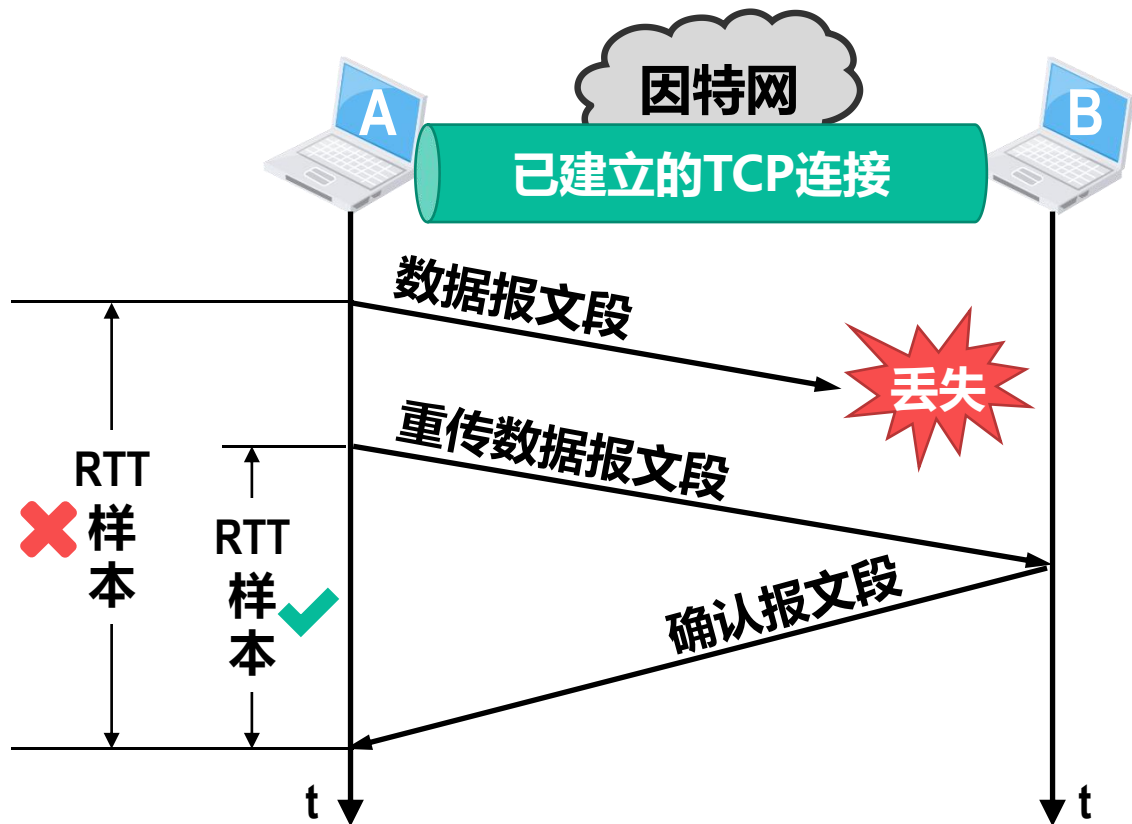
在上式中， $0 \leq \beta < 1$ ，已成为建议标准的[RFC 6298]推荐的 β 值为1/4，即0.25。

如果所测量到的RTT样本不正确，那么所计算出的RTT_s和RTT_D自然就不正确，进而所计算出的RTO也就不正确。

然而，RTT的测量确实是比较复杂的。

01 TCP超时重传时间的选择

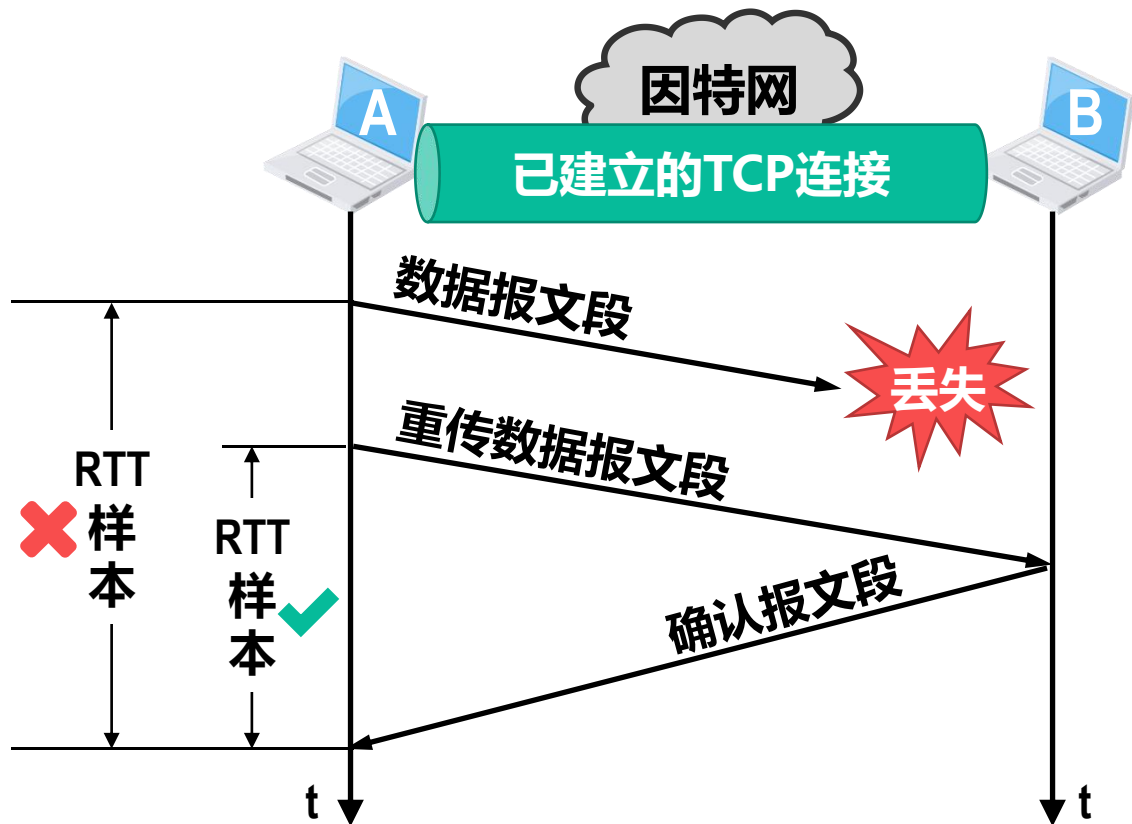
往返时间RTT的测量是比较复杂的



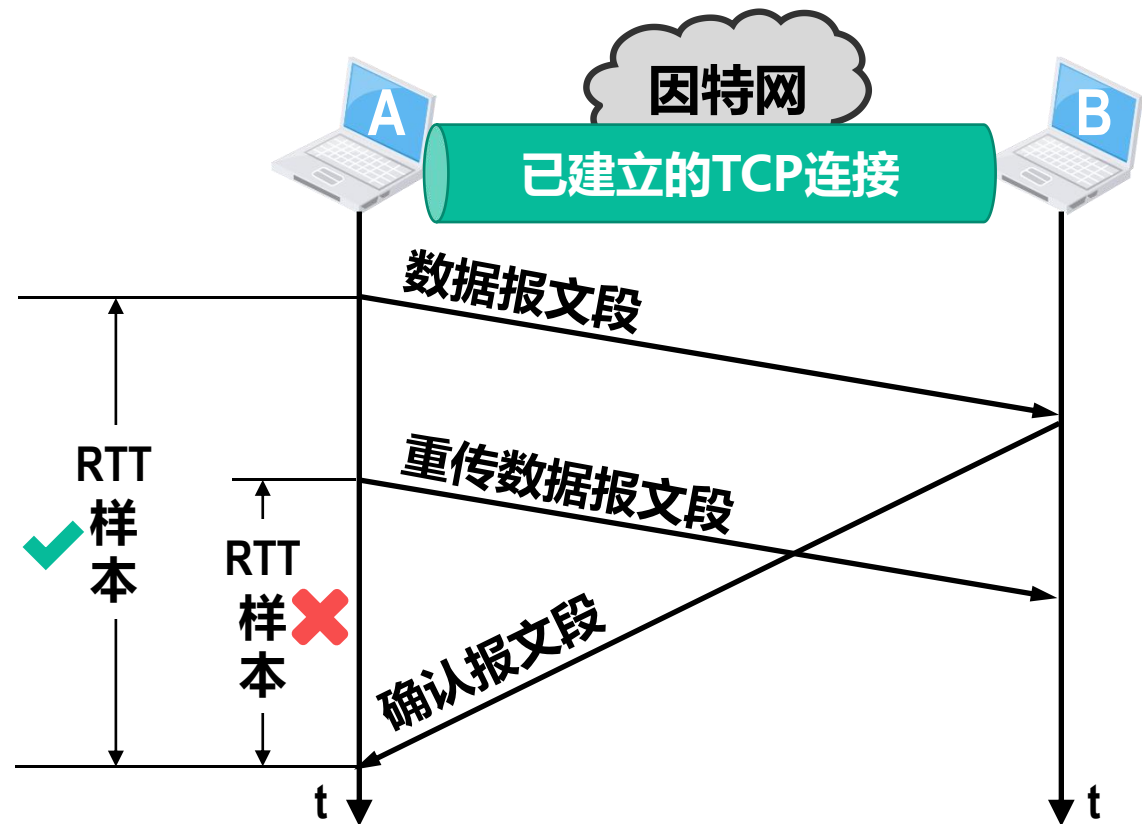
如果主机A误将该确认报文段当作对原数据报文段的确认，则所计算出的RTTs和RTO就会偏大，降低了传输效率。

01 TCP超时重传时间的选择

往返时间RTT的测量是比较复杂的



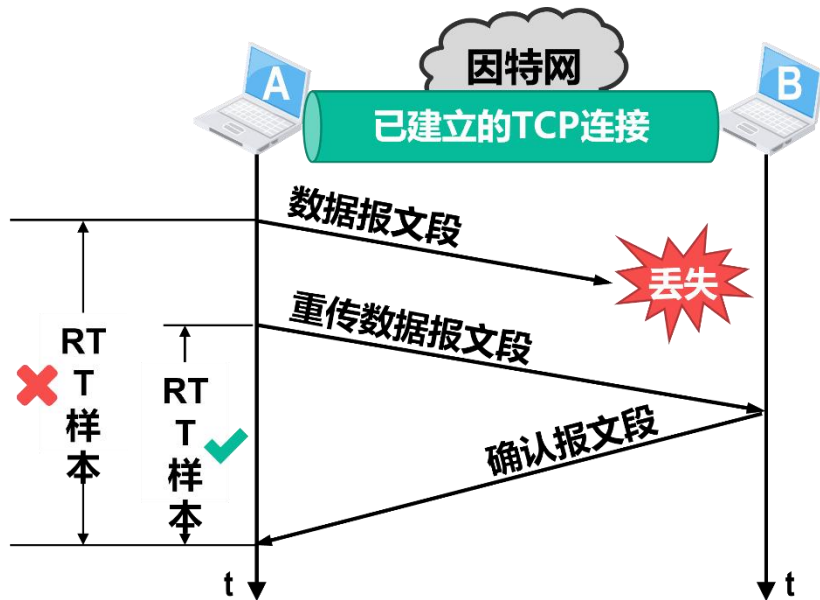
如果主机A误将该确认报文段当作对原数据报文段的确认，则所计算出的RTTs和RTO就会偏大，降低了传输效率。



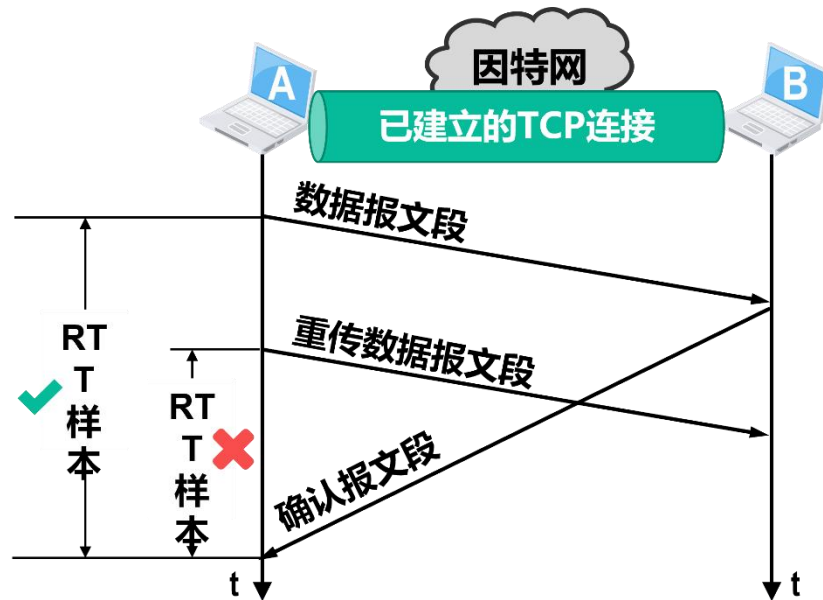
如果主机A误将该确认报文段当作对重传报文段的确认，则所计算出的RTTs和RTO就会偏小，这会导致后续报文段发生没有必要的重传，增大网络负荷。

01 TCP超时重传时间的选择

往返时间RTT的测量是比较复杂的



如果主机A误将该确认报文段当作对原数据报文段的确认, 则所计算出的RTTs和RTO就会偏大, 降低了传输效率。



如果主机A误将该确认报文段当作对重传报文段的确认, 则所计算出的RTTs和RTO就会偏小, 这会导致后续报文段发生没有必要的重传, 增大网络负荷。

通过上述两个例子可以看出：当发送方出现超时重传后，收到确认报文段时是无法判断出该确认到底是对原数据报文段的确认还是对重传数据报文段的确认，也就是无法准确测量出RTT，进而无法正确计算RTO。

01 TCP超时重传时间的选择

往返时间RTT的测量是比较复杂的

通过上述两个例子可以看出：当发送方出现超时重传后，收到确认报文段时是无法判断出该确认到底是对原数据报文段的确认还是对重传数据报文段的确认，也就是无法准确测量出RTT，进而无法正确计算RTO。

Karn
算法

在计算加权平均RTTs时，只要报文段重传了，就不采用其RTT样本。换句话说，出现重传时，不重新计算RTTs，进而RTO也不会重新计算。

新问题

报文段每重传一次，就把RTO增大一些。典型的做法是将新RTO的值取为旧RTO的2倍。

修正
Karn
算法

设想出现这样的情况：报文段的时延突然增大很多，并且之后很长一段时间都会保持这种时延（这可能是因为网络拓扑发生了变化）。因此在原来得出的RTO内，不会收到确认报文段，于是就重传报文段。但根据Karn算法，不考虑重传的报文段的RTT样本，因此RTO就无法更新，这会导致报文段反复被重传。

01 TCP超时重传时间的选择

【练习】假设要进行五次TCP往返时间RTT的测量，从第一次开始，依次测得RTT为30ms，26ms，32ms，24ms，而第五次测量RTT时出现了超时重传。设 $\alpha = \beta = 0.1$ ，请计算每次测量后所计算出的超时重传时间RTO。

解析

$$RTO = RTT_S + 4 \times RTT_D$$

加权平均往返时间 RTT_S

$$RTT_{S1} = RTT_1$$

$$\text{新的}RTT_S = (1 - \alpha) \times \text{旧的}RTT_S + \alpha \times \text{新的}RTT\text{样本}$$

在上式中， $0 \leq \alpha < 1$ ，已成为建议标准的[RFC 6298]推荐的 α 值为1/8，即 0.125。

RTT偏差的加权平均 RTT_D

$$RTT_{D1} = RTT_1 \div 2$$

$$\text{新的}RTT_D = (1 - \beta) \times \text{旧的}RTT_D + \beta \times |RTT_S - \text{新的}RTT\text{样本}|$$

在上式中， $0 \leq \beta < 1$ ，已成为建议标准的[RFC 6298]推荐的 β 值为1/4，即 0.25。

报文段发生重传，就不采用RTT样本计算RTO，而是把RTO增大一些。典型的做法是将新RTO的值取为旧RTO的2倍。

01 TCP超时重传时间的选择

【练习】 假设要进行五次TCP往返时间RTT的测量，从第一次开始，依次测得RTT为30ms，26ms，32ms，24ms，而第五次测量RTT时出现了超时重传。设 $\alpha = \beta = 0.1$ ，请计算每次测量后所计算出的超时重传时间RTO。

解析

根据题意可知， $RTT_1=30ms$ ， $RTT_2=26ms$ ， $RTT_3=32ms$ ， $RTT_4=24ms$ ，测量 RTT_5 时出现超时重传。

$$RTT_{S1} = RTT_1 = 30ms$$

$$RTT_{D1} = RTT_1 \div 2 = 15ms$$

$$\begin{aligned} RTT_{S2} &= (1 - \alpha) \times RTT_{S1} + \alpha \times RTT_2 \\ &= (1 - 0.1) \times RTT_{S1} + 0.1 \times RTT_2 \\ &= 0.9 \times 30ms + 0.1 \times 26ms = 29.6ms \end{aligned}$$

$$\begin{aligned} RTT_{D2} &= (1 - \beta) \times RTT_{D1} + \beta \times |RTT_{S2} - RTT_2| \\ &= (1 - 0.1) \times RTT_{D1} + 0.1 \times |RTT_{S2} - RTT_2| \\ &= 0.9 \times 15ms + 0.1 \times |29.6ms - 26ms| = 13.86ms \end{aligned}$$

$$\begin{aligned} RTT_{S3} &= (1 - 0.1) \times RTT_{S2} + 0.1 \times RTT_3 \\ &= 0.9 \times 29.6ms + 0.1 \times 32ms = 29.84ms \end{aligned}$$

$$\begin{aligned} RTT_{D3} &= (1 - 0.1) \times RTT_{D2} + 0.1 \times |RTT_{S3} - RTT_3| \\ &= 0.9 \times 13.86ms + 0.1 \times |29.84ms - 32ms| = \\ &= 12.69ms \end{aligned}$$

$$\begin{aligned} RTT_{S4} &= (1 - 0.1) \times RTT_{S3} + 0.1 \times RTT_4 \\ &= 0.9 \times 29.84ms + 0.1 \times 24ms = 29.256ms \end{aligned}$$

$$\begin{aligned} RTT_{D4} &= (1 - 0.1) \times RTT_{D3} + 0.1 \times |RTT_{S4} - RTT_4| \\ &= 0.9 \times 12.69ms + 0.1 \times |29.256ms - 24ms| = \\ &= 11.9466ms \end{aligned}$$

由于测量 RTT_5 时出现超时重传，因此不用计算 RTT_{S5} 。

由于测量 RTT_5 时出现超时重传，因此不用计算 RTT_{D5} 。

01 TCP超时重传时间的选择

【练习】 假设要进行五次TCP往返时间RTT的测量，从第一次开始，依次测得RTT为30ms，26ms，32ms，24ms，而第五次测量RTT时出现了超时重传。设 $\alpha = \beta = 0.1$ ，请计算每次测量后所计算出的超时重传时间RTO。

解析

根据题意可知， $RTT_1=30ms$ ， $RTT_2=26ms$ ， $RTT_3=32ms$ ， $RTT_4=24ms$ ，测量 RTT_5 时出现超时重传。

$$RTO = RTT_S + 4 \times RTT_D$$

$$RTT_{S1} = RTT_1 = 30ms$$

$$RTT_{D1} = RTT_1 \div 2 = 15ms$$

$$RTO_1 = RTT_{S1} + 4 \times RTT_{D1} = 30ms + 4 \times 15ms = 90ms$$

$$RTT_{S2} = 29.6ms$$

$$RTT_{D2} = 13.86ms$$

$$RTO_2 = RTT_{S2} + 4 \times RTT_{D2} = 29.6ms + 4 \times 13.86ms = 85.04ms$$

$$RTT_{S3} = 29.84ms$$

$$RTT_{D3} = 12.69ms$$

$$RTO_3 = RTT_{S3} + 4 \times RTT_{D3} = 29.84ms + 4 \times 12.69ms = 80.6ms$$

$$RTT_{S4} = 29.256ms$$

$$RTT_{D4} = 11.9466ms$$

$$RTO_4 = RTT_{S4} + 4 \times RTT_{D4} = 29.256ms + 4 \times 11.9466ms = 77.0424ms$$

由于测量 RTT_5 时出现超时重传，
因此不用计算 RTT_{S5} 和 RTT_{D5} 。

$$RTO_5 = 2 \times RTO_4 = 2 \times 77.0424ms = 154.0848ms$$

5.3.8 TCP的选择确认

01

TCP的选择确认

01 TCP的选择确认

■ 在之前介绍TCP的快重传和可靠传输时，TCP接收方**只能对按序收到的数据中的最高序号给出确认**。当发送方**超时重传**时，接收方之前**已收到的未按序到达的数据也会被重传**。



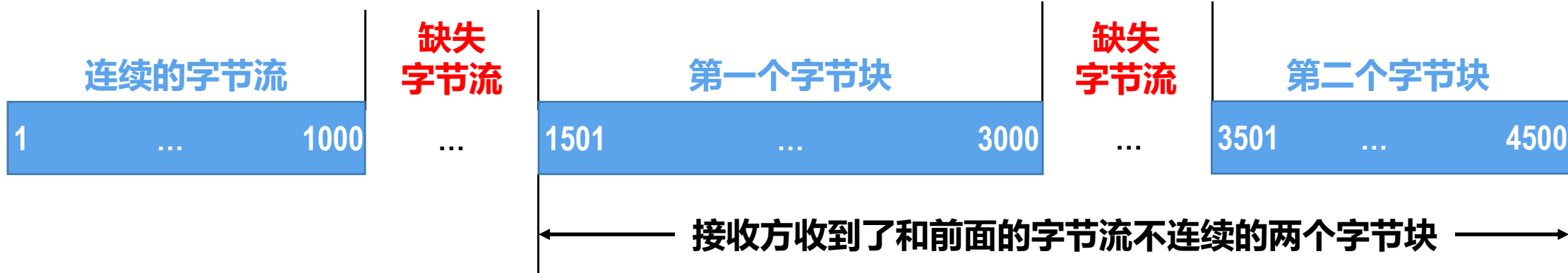
能否设法只传送缺少的数据而不重传已经正确到达、只是未按序到达的数据呢？

TCP可以使用**选择确认**（**Selective ACK, SACK**）

[RFC 2018]（建议标准）

01 TCP的选择确认

TCP接收方收到的对方发送过来的序号不连续的字节流



假设这些字节的序号都在接收窗口内，那么接收方就先收下这些数据，但要把这些信息准确地告诉发送方，使发送方不再重复发送这些数据。

01 TCP的选择确认

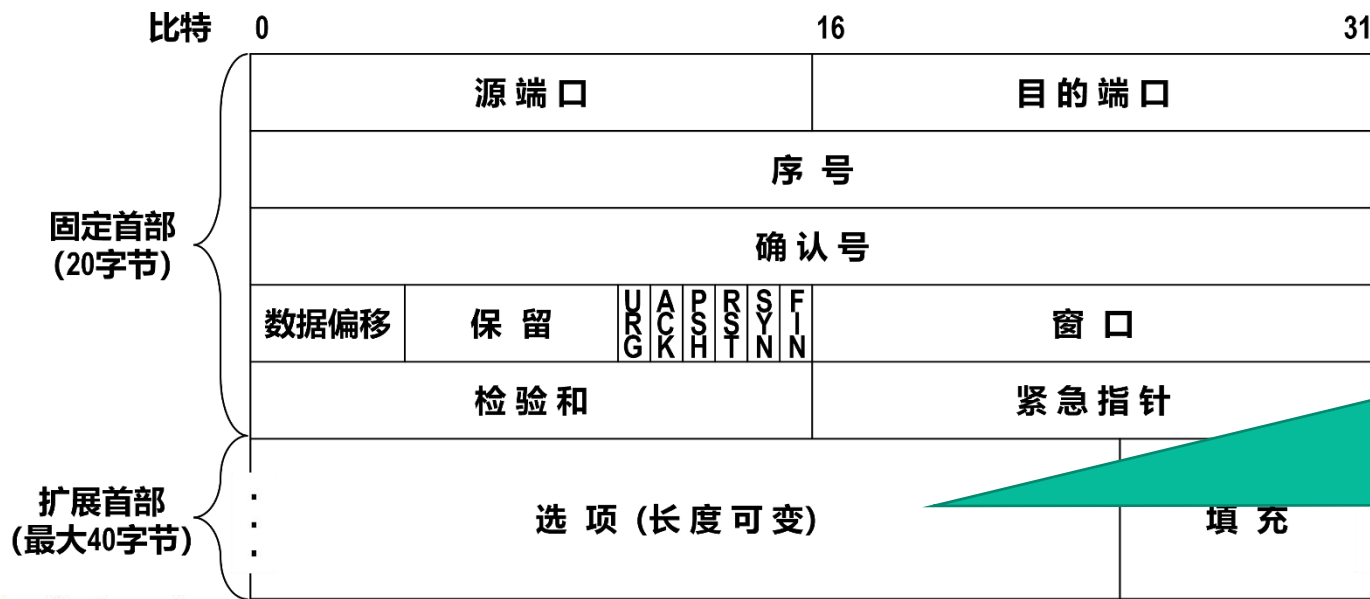
TCP接收方收到的对方发送过来的序号不连续的字节流



允许SACK选项 (1字节)
SACK选项长度 (1字节)
指明一个边界需要4字节 (因为序号为4字节)
一个字节块需要两个边界, 即8字节。
因此, 最多指明4个字节块的信息。

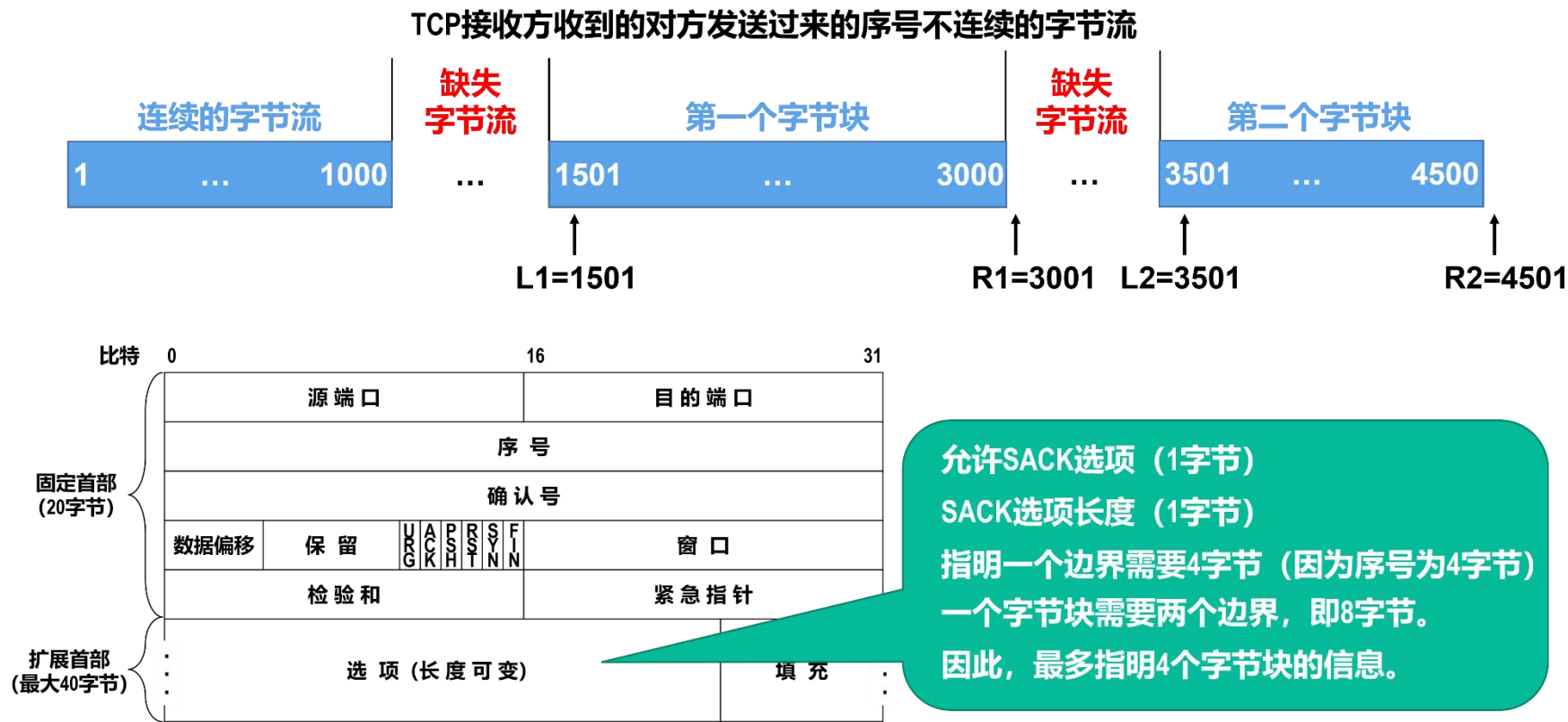
01 TCP的选择确认

TCP接收方收到的对方发送过来的序号不连续的字节流



允许SACK选项 (1字节)
SACK选项长度 (1字节)
指明一个边界需要4字节 (因为序号为4字节)
一个字节块需要两个边界, 即8字节。
因此, 最多指明4个字节块的信息。

01 TCP的选择确认



SACK相关文档并没有指明发送方应当怎样响应SACK。因此大多数的TCP实现还是重传所有未被确认的数据块。